# Password Manager

שם התלמיד: נועם שחם

ת"ז התלמיד: 326553393

שמות המנחים: ליאת סגל, עידן פלדברג, עודי רז.

תאריך הגשה: 31.5.2026



## תוכן עניינים

**Password Manager** ..... 1
**מבוא** ..... 8
ייזום ..... 8
תיאור ראשוני של הפרויקט ..... 8
הגדרת הלקוח ..... 8
מטרת הפרויקט ..... 8
סקירת פתרונות קיימים ..... 8
תיחום הפרויקט ..... 8
פירוט תיאור המערכת ..... 9
תיאור המערכת ..... 9
פירוט היכולות ..... 9
פירוט הבדיקות לפרויקט ..... 10
תכנון ולו"ז ביצוע הפרויקט ..... 11
אתגרים וסיכונים בביצוע הפרויקט ..... 11
**תיאור תחום הידע** ..... 12
רישום משתמשים חדשים לאפליקציה ..... 12
התחברות לאפליקציה ..... 12
רישום אתרים חדשים ..... 13
צפייה במידע השמור על האתרים ..... 13
שינוי סיסמה לאתרים ..... 14
מנגנון שכחתי סיסמה ..... 14
מניעת ניסיונות רצופים רבים להתחברות לאפליקציה ..... 15
**מבנה / ארכיטקטורה של הפרויקט** ..... 16
הארכיטקטורה של המערכת המוצעת ..... 16
תיאור החומרה ..... 16
שרטוט קשרי החומרה ..... 16
הטכנולוגיה הרלוונטית ..... 17
זרימת המידע במערכת ..... 17
הרצת לקוח ותהליך החלפת מפתחות הצפנת התקשורת ..... 17
רישום משתמשים חדשים לאפליקציה ..... 19
התחברות לאפליקציה ..... 20
זיהוי קול ..... 21

זיהוי פנים ..... 22
רישום אתרים חדשים ..... 23
צפייה במידע השמור על האתרים ..... 24
שינוי סיסמה לאתרים ..... 25
מנגנון שכחתי סיסמה ..... 26
אלגוריתמים מרכזיים בפרויקט ..... 27
הצפנה בתקשורת ..... 27
הצפנה בבסיס הנתונים ..... 27
זיהוי קול וזיהוי פנים ..... 28
תיאור סביבת הפיתוח ..... 30
תיאור פרוטוקול התקשורת ..... 30
מבנה ההודעה בפרוטוקול התקשורת ..... 30
הודעות מהלקוח לשרת ..... 30
הודעות מהשרת ללקוח ..... 34
מסכי המערכת ..... 39
מסך הבית ..... 39
מסך הרשמה ..... 40
מסך הזנת קוד לאימייל ..... 41
מסך הגדרת השאלות של המשתמש ..... 42
מסך הקמת זיהוי הקול ..... 43
מסך הקמת זיהוי הפנים ..... 44
מסך התחברות ..... 45
מסך בחירת פקטור ההזדהות השני ..... 46
מסך הזדהות עם זיהוי קול ..... 47
מסך הזדהות עם זיהוי פנים ..... 48
המסך הראשי (לאחר התחברות) ..... 49
חלונית הגדרות במסך הראשי ..... 50
מסך רישום אתר ..... 51
מסך העדפות ליצירת הסיסמה ..... 52
מסך מידע על אתר רשום ..... 53
מסך שינוי סיסמה לאתר רשום ..... 54
מסך שחזור אתר ..... 55
מסך הזנת תשובות לשאלות המשתמש ..... 56

מסך איפוס סיסמה ..... 57
מסך ניתוק מהשרת ..... 58
תרשים מסכי המערכת ..... 59
מבני הנתונים ..... 60
מסד הנתונים ..... 60
סקירת חולשות ואיומים ..... 65
שכבת האפליקציה ..... 65
שכבת התעבורה ..... 65
**מימוש הפרויקט** ..... 66
מודולים מיובאים ..... 66
מחלקות הפרוייקט ..... 67
ProtocolHandler ..... 67
ServerProtocolHandler ..... 68
ClientProtocolHandler ..... 68
ConnectionManager ..... 68
ServerConnectionManager ..... 69
ClientConnectionManager ..... 69
Server ..... 70
DataBase ..... 72
UserManager ..... 74
EmailManager ..... 75
AudioManager ..... 75
VoiceAuthenticator ..... 76
VideoManager ..... 76
FaceRecognizer ..... 77
PasswordHandler ..... 77
Client ..... 77
UI ..... 80
SignupWindow ..... 84
AuthSelectWindow ..... 85
ChangePasswordWindow ..... 85
CodeLoginWindow ..... 86

HomeFrame ..... 86
LoginWindow ..... 86
PasswordPreferencesWindow ..... 87
ResetPasswordWindow ..... 87
SecurityQuestionsAuthWindow ..... 88
SecurityQuestionsSetupWindow ..... 88
SettingsMenuFrame ..... 88
SiteInfoWindow ..... 89
SiteRecoveryWindow ..... 89
SiteRegistrationWindow ..... 90
ToolTip ..... 90
VideoCaptureWindow ..... 90
VoiceRecordingWindow ..... 91
תרשים קשרים בין מחלקות הפרויקט ..... 92
אלגוריתמים וקטעי קוד מיוחדים ..... 93
הצפנה בתקשורת ..... 93
הצפנה בבסיס הנתונים ..... 95
זיהוי פנים ..... 99
זיהוי קול ..... 100
מסמך בדיקות למערכת ..... 102
בדיקת הרשמה לאפליקציה ..... 102
בדיקת התחברות לאפליקציה ..... 102
בדיקת רישום אתרים חדשים עבור משתמש ..... 102
בדיקת צפייה במידע על אתר רשום ..... 102
בדיקת הגדרת העדפות ליצירת סיסמה לאתר ..... 103
בדיקת שינוי סיסמה לאתר רשום ..... 103
בדיקת מחיקת אתר רשום ..... 103
בדיקת שחזור אתר שנמחק ..... 104
בדיקת שדה חיפוש האתרים הרשומים ..... 104
בדיקת הוספת אמצעי ההתחברות הביומטריים מחלונית ההגדרות ..... 104
בדיקת ממשק המשתמש במצב dark mode ..... 104
בדיקת תהליך איפוס הסיסמה ..... 105

בדיקת ההצפנה בתקשורת ..... 105
בדיקת אבטחת בסיס הנתונים ..... 105
בדיקת הגנת DoS ..... 105
**מדריך למשתמש** ..... 107
קבצי המערכת ..... 107
התקנת המערכת ..... 108
הסביבה הנדרשת ..... 108
הכלים הנדרשים ..... 108
נתונים התחלתיים ..... 108
רשת ..... 109
ארכיטקטורה מינימלית נדרשת להרצת המערכת ..... 109
הפעלת המערכת ..... 109
**רפלקציה** ..... 110
**ביבליוגרפיה** ..... 111
**נספחים** ..... 112
הסבר על הצפנת RSA ..... 112
הסבר על אלגוריתם PBKDF2HMAC ..... 112
הסבר על cosine similarity ..... 112
קוד הפרויקט ..... 113
Protocol Handler ..... 113
ServerProtocolHandler ..... 115
ClientProtocolHandler ..... 116
ConnectionManager ..... 116
ServerConnectionManager ..... 117
ClientConnectionManager ..... 120
Server ..... 121
DataBase ..... 132
UserManager ..... 144
EmailManager ..... 146
AudioManager ..... 147
VoiceAuthenticator ..... 149
VideoManager ..... 150

151 ..... FaceRecognizer
152 ..... PasswordHandler
154 ..... Client
163 ..... UI
180 ..... SignupWindow
186 ..... AuthSelectWindow
189 ..... ChangePasswordWindow
191 ..... CodeLoginWindow
194 ..... HomeFrame
195 ..... LoginWindow
197 ..... PasswordPreferencesWindow
201 ..... ResetPasswordWindow
204 ..... SequirityQuestionsAuthWindow
206 ..... SequrityQuestionsSetupWindow
208 ..... SettingsMenuFrame
210 ..... SiteInfoWindow
212 ..... SiteRecoveryWindow
214 ..... SiteRegistrationWindow
216 ..... ToolTip
217 ..... VideoCaptureWindow
220 ..... VoiceRecordingWindow
223 ..... client_status
223 ..... color_palletes

# מבוא

## ייזום

### תיאור ראשוני של הפרויקט

מנהל סיסמאות - אפליקציה נגישה ששומרת למשתמש את הסיסמאות שלו ויוצרת סיסמאות חזקות לפי אתר באופן בטוח ומאובטח.

בחרתי את הנושא הזה לפרויקט מכיוון שהוא דורש מערכות הצפנה והגנת סייבר מורכבות מכיוון שהוא עוסק בסיסמאות רבות ובמידע רגיש של המשתמשים.

### הגדרת הלקוח

האפליקציה מיועדת לשימוש על ידי אנשים פרטיים.

### מטרת הפרויקט

האפליקציה תאפשר למשתמש להשתמש בסיסמאות ארוכות ובטוחות יותר שקשה לזכור אותן ללא עזרה ורישום. הסיסמאות יהיו מרוכזות במקום אחד, קלות למציאה, ומאובטחות על ידי אמצעי הזדהות מתקדמים.

### סקירת פתרונות קיימים

הרעיון של מנהל סיסמאות הוא לא דבר חדש. הדוגמה הבולטת ביותר למנהל סיסמאות קיים הוא מנהל הסיסמאות של גוגל: מנהל הסיסמאות של גוגל מאחסן את הסיסמאות של המשתמש, נותן אפשרות ליצירת סיסמה חזקה ומשלים אוטומטית את הסיסמאות בשדות של אתרים.
האפליקציה שלי דומה מאוד למנהל הסיסמאות של גוגל, אבל היא לא כוללת מילוי אוטומטי של שדות סיסמאות באתרים מכיוון שלא כתבתי את הפרויקט כ – browser extension. במקום, המילוי האוטומטי, האפליקציה שלי מתמקדת יותר באבטחת המידע ואמצעי הזדהות ביומטריים שאין למנהל הסיסמאות של גוגל.

### תיחום הפרויקט

הפרויקט עוסק ב- :

- **תקשורת** – תקשורת מבוססת סוקטים ופרוטוקול TCP בין הלקוחות לשרת עם פרוטוקול תקשורת משלי. השרת הוא מרובה לקוחות בעזרת select.
- **מערכת הפעלה** – יש גישה למערכת הקבצים, שימוש במיקרופון ובמצלמה, שימוש ב – clipboard. יש שימוש ב threading.
- **אבטחה** – המידע שעובר בתקשורת מוצפן באמצעות הצפנה היברידית (RSA + AES), המידע בבסיס הנתונים hashed והמידע הרגיש מוצפן עם AES.
- **ממשק משתמש** – יש ממשק משתמש אינטראקטיבי ומעוצב ב custom tkinter הכולל מסכים רבים.
- **AI וטכנולוגיות הזדהות** – שימוש במודלי AI של זיהוי פנים וקול. שליחת קוד הזדהות באימייל.

# פירוט תיאור המערכת

## תיאור המערכת

הפרויקט שלי הוא מנהל סיסמאות. המטרה של האפליקציה היא לאחסן סיסמאות חזקות (ארוכות עם הרבה תווים שונים) עבור המשתמש במקום אחד מאובטח ונגיש. המשתמש לא יצטרך לזכור בעצמו את הסיסמאות לכל אתר – הוא יוכל לחפש בקלות את הסיסמה לאתר מסוים ולהעתיק אותה ל clipboard ישירות מהאפליקציה.
כדי להיכנס לאפליקציית מנהל הסיסמאות שלי, המשתמש צריך לעבור אימות דו שלבי ( Multi Factor Authentication). השלב הראשון הוא הזדהות רגילה עם שם משתמש וסיסמה. אם המשתמש שכח את הסיסמה שלו, יש לו אפשרות ללחוץ על "שכחתי סיסמה", לענות על שאלות הזדהות ולאפס את הסיסמה שלו לאפליקציה. השלב השני בהזדהות הוא בחירה בין הזדהות עם קוד שנשלח לאימייל, זיהוי פנים או זיהוי קול.
אחרי שהמשתמש מתחבר, הוא יכול לרשום אתרים חדשים שאליהם הוא ישמור את הסיסמאות, לצפות בסיסמאות ושמות משתמש שמורים לאתרים שכבר רשומים, להחליף סיסמה לאתר שמור, ולמחוק אתר מהרשימה עם אפשרות לשחזר אותו.
מכיוון שאתרים מסוימים לא מקבלים תווים מסוימים, או שהם דורשים אורך מינימלי, המשתמש יכול לקבוע את ההגדרות שלו ליצירת הסיסמאות לאתרים באמצעות מחולל הסיסמאות שהכנתי.

## פירוט היכולות

- רישום משתמשים חדשים לאפליקציה. אפשר להקים את ההתחברות הביומטרית בזמן הרישום או לאחר ההתחברות במסך ההגדרות.
- התחברות לאפליקציה לפי משתמש, עם כמה פקטורי הזדהות אפשריים ושילוב ביניהם (Multi Factor Authentication):
  1. שם משתמש וסיסמה (כולל בדיקה בזמן הרישום האם הסיסמה לאפליקציה מספיק חזקה).
  2. הודעה לאימייל עם קוד חד פעמי.
  3. הזדהות ביומטרית לפי זיהוי פנים או קול.
- רישום אתרים חדשים ויצירת סיסמה חדשה אליהם.
- מערכת יצירת סיסמה חזקה לכל אתר באופן אוטומטי לפי דרישות המשתמש (אורך מינימלי, תווים מיוחדים, מספרים...).
- אפשרות לצפות בסיסמאות ושמות המשתמש של המשתמש לכל האתרים השמורים באפליקציה ולהעתיק אותם ל .clipboard הסיסמאות יהיו מוצגות בצורה נסתרת (כוכביות) עם אפשרות להציג אותן.
- שינוי סיסמה לאתרים הרשומים.
- הצפנה של מעבר הנתונים בין הלקוח לשרת.
- Database מאובטח ומוצפן המכיל את פרטי המשתמשים והאתרים הרשומים.
- מימוש מנגנון "שכחתי סיסמה" להתחברות לאפליקציה בעזרת האימייל או שאלות שרק המשתמש יודע לענות עליהן.
- מניעת ניסיונות רצופים רבים להתחברות לאפליקציה על ידי נעילה זמנית אחרי מספר ניסיונות כושלים בפרק זמן קצר.

## פירוט הבדיקות לפרויקט

- בדיקת הרשמה לאפליקציה – בדיקה שמשתמש חדש יכול להירשם לאפליקציה, עם כל חלקי ההרשמה (שם משתמש, סיסמה, אימייל ווידוא של האימייל, הגדרת שאלות אבטחה של המשתמש, הקמת זיהוי קול ופנים בהצלחה), ובדיקה של ההתחברות האוטומטית לאפליקציה לאחר ההרשמה.
- בדיקת התחברות לאפליקציה – בדיקה שהלקוח יכול להתחבר למשתמש קיים באפליקציה עם שם משתמש וסיסמה, וכל פעם עם פקטור הדהות שני אחר מבין הבאים:
  - בדיקה ראשונה – התחברות עם קוד חד פעמי לאימייל.
  - בדיקה שנייה – התחברות עם זיהוי קול.
  - בדיקה שלישית – התחברות עם זיהוי פנים.

  כולל גם בדיקת התחברות עם הזנת פרטי התחברות לא נכונים ובדיקת הודעות השגיאה המוצגות למשתמש.
- בדיקת רישום אתרים חדשים עבור משתמש – בדיקה שהמשתמש יכול לרשום אתרים חדשים לאפליקציה, והם יופיעו ברשימת האתרים הרשומים של המשתמש.
- בדיקת צפייה במידע על אתר רשום – בדיקה שהמשתמש יכול לבחור אתר מרשימת האתרים הרשומים ולצפות במידע הרשום על האתר, ושהמידע שיופיע יהיה נכון ומדויק.
- בדיקת הגדרת העדפות ליצירת סיסמה לאתר – בדיקה שהמשתמש יכול להגדיר העדפות ליצירת הסיסמה, ושהסיסמה שתיווצר לאתר באמת תתאים להעדפות שהמשתמש קבע.
- בדיקת שינוי סיסמה לאתר רשום – בדיקה שהמשתמש יכול לבחור אתר מרשימת האתרים הרשומים ולשנות את הסיסמה השמורה אליו.
- בדיקת מחיקת אתר רשום – בדיקה שהמשתמש יכול לבחור אתר ולמחוק אותו מרשימת האתרים הרשומים.
- בדיקת שחזור אתר שנמחק – בדיקה שהמשתמש יכול לשחזר אתר שנמחק מרשימת האתרים הרשומים.
- בדיקת שדה חיפוש האתרים הרשומים – בדיקה שהמשתמש יכול לחפש אתר מסוים בשדה החיפוש במסך הראשי ושהאתר / אתרים המתאימים ורק הם יופיעו ברשימת האתרים הרשומים.
- בדיקת הוספת אמצעי ההתחברות הביומטריים מחלונית ההגדרות – בדיקה שהמשתמש יכול להוסיף זיהוי קול / זיהוי פנים מחלונית ההגדרות במסך הראשי אם הוא לא הגדיר אותם בזמן הרישום לאפליקציה. צריך לבדוק גם שאמצעי ההתחברות שנוספו באמת תקינים והמשתמש יכול להתחבר באמצעותם לאפליקציה.
- בדיקת ממשק המשתמש במצב dark mode – בדיקה שממשק המשתמש נראה תקין ועובד כרגיל גם כאשר הוא מוצג ב – dark mode.
- בדיקת תהליך איפוס הסיסמה – בדיקה שהמשתמש יכול להתחיל את תהליך איפוס הסיסמה עם כפתור שכחתי סיסמה במסך ההתחברות, שהוא יכול לענות על שאלות האבטחה שהוא הגדיר בזמן ההרשמה, ושהוא יכול לאפס את הסיסמה שלו. לבדוק גם שהסיסמה באמת שונתה ושהמשתמש יכול להתחבר באמצעות הסיסמה החדשה.
- בדיקת ההצפנה בתקשורת – בדיקה שהנתונים העוברים בתקשורת מוצפנים ואי אפשר להאזין ולגנוב מידע רגיש.

- בדיקת אבטחת בסיס הנתונים – בדיקה שהנתונים הרגישים בבסיס הנתונים מוצפנים או hashed, ואי אפשר לגנוב מידע רגיש של המשתמשים מבסיס הנתונים.
- בדיקת הגנת DoS – בדיקה שהשרת מנתק את הלקוח אם הוא עבר את מגבלת הבקשות לשרת בפרק זמן קצר.

## תכנון ולו"ז ביצוע הפרויקט

1. כתיבת פרוטוקול תקשורת
2. כתיבת תקשורת בין שרת ולקוח
3. פיתוח הפעולות הבסיסיות בצד הלקוח ובצד השרת (התחברות עם שם משתמש וסיסמה, רישום אתר, צפייה במידע השמור על האתרים, שינוי סיסמה לאתר רשום)
4. כתיבת ממשק המשתמש הראשוני
5. תכנון טבלאות בסיס הנתונים וכתיבת בסיס הנתונים עם הגנה מ – SQL Injection
6. פיתוח הצפנה היברידית של RSA+AES בתקשורת בין השרת ללקוח
7. מימוש הצפנות ו – hashing בבסיס הנתונים
8. פיתוח מערכת העדפות ליצירת סיסמה חזקה לאתרים רשומים
9. פיתוח התחברות דו שלבית לאפליקציה עם קוד חד פעמי לאימייל
10. פיתוח הקלטת קול והתחברות דו שלבית לאפליקציה עם זיהוי קול
11. פיתוח שאלות האבטחה ומנגנון שכחתי סיסמה
12. פיתוח צילום וידאו והתחברות דו שלבית לאפליקציה עם זיהוי פנים
13. הוספת עוד פעולות למשתמש (חיפוש אתר, מחיקת אתר, שחזור אתר, הגדרות והוספת פקטורי הזדהות נוספים גם לאחר ההרשמה)
14. עדכון ושיפור ממשק המשתמש, הוספת dark mode
15. פיתוח מנגנון הגבלת מספר הנסיונות להתחברות לאפליקציה
16. פיתוח מנגנון הגנה מ DoS
17. פיתוח התחברות דו שלבית לאפליקציה עם sms – לא הוצא לפועל בגלל שדורש תשלום לשירותים ובגלל שהיכולת דומה מאוד להתחברות עם אימייל שכבר יש.

## אתגרים וסיכונים בביצוע הפרויקט

האתגרים המרכזיים להתמודדות בפרויקט הם סוגיות האבטחה בגלל שהפרויקט עוסק במידע רגיש של המשתמשים. כפי שפרטתי בתוכן הפרויקט, דרכי ההתמודדות עם סוגיות אבטחת המידע הן הצפנת המידע במעבר, הצפנת המידע המאוחסן בבסיס הנתונים ואבטחת הכניסה למערכת באמצעות פקטורים מרובים (Multi Factor Authentication).

בנוסף, בגלל שהמשתמשים מפקידים בידי השרת את כל הסיסמאות שלהם, הוא צריך להיות עמיד ולא לקרוס כדי שהמשתמשים יוכלו להתחבר לאתרים שלהם בכל זמן נתון. כדי להבטיח את זה, עטפתי את רוב היכולות של השרת ב exception handling, ופיתחתי מנגנון הגנה מ- DoS ועומס על השרת.

# תיאור תחום הידע

## רישום משתמשים חדשים לאפליקציה

**מהות היכולת**: קליטת הפרטים של המשתמש (שם משתמש, סיסמה, כתובת אימייל), אימות של האימייל, יצירה של השאלות האישיות של המשתמש וקבלת התשובות שלו, ואפשרות להקים את ההתחברות באמצעות הנתונים הביומטריים.

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך הרשמה, מסך אימות קוד אימייל, מסך יצירת שאלות הזדהות, מסך הקלטת קול ומסך הקלטת וידאו)
- קליטת נתונים
- בדיקת תקינות וחוזק של הסיסמה
- שליחת קוד לאימייל של המשתמש מהשרת ואימות האימייל
- הפקת דגימת הקול / נתוני הפנים של המשתמש
- שליחה לשרת
- הצפנה ופענוח בתקשורת
- בדיקת נתונים וכתיבה אל בסיס הנתונים בשרת
- יצירת מפתח הצפנה ופענוח חדש עבור המשתמש
- הצפנה ו hash בבסיס הנתונים בשרת
- קבלת תשובה מהשרת
- הצגת שגיאה למשתמש או התחברות

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים, מנהל אימייל, מנהל וידאו, מזהה פנים, מנהל אודיו, מאמת קול.

## התחברות לאפליקציה

**מהות היכולת**: התחברות של משתמשים לאפליקציה עם שם משתמש, סיסמה ופקטור הזדהות נוסף (קוד לאימייל, זיהוי פנים או זיהוי קול).

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך התחברות, מסך בחירת פקטור הזדהות נוסף, מסך אימות קוד אימייל, מסך הקלטת קול ומסך הקלטת וידאו)
- קליטת נתונים
- מניעת ניסיונות רצופים רבים להתחברות
- שליחת קוד לאימייל של המשתמש מהשרת
- הפקת דגימת הקול / נתוני הפנים של המשתמש והשוואתם לנתונים הקיימים
- שליחה לשרת
- הצפנה ופענוח בתקשורת

- אימות נתונים מול בסיס הנתונים בשרת
- פענוח בבסיס הנתונים בשרת
- קבלת תשובה מהשרת
- הצגת שגיאה למשתמש או התחברות

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים, מנהל אימייל, מנהל וידאו, מזהה פנים, מנהל אודיו, מאמת קול.

## רישום אתרים חדשים

**מהות היכולת**: רישום של אתר חדש למערכת על ידי משתמש. עבור אותו האתר במערכת ישמרו שם המשתמש והסיסמה המתאימים לו, וכן ההעדפות של המשתמש ליצירת הסיסמה עבור האתר המסוים. המשתמש יכול להעתיק את הסיסמה החדשה שנוצרת ל – clipboard.

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך ראשי עם כפתור רישום אתר, מסך רישום אתר, מסך העדפות יצירת הסיסמה)
- קליטת נתונים
- יצירת סיסמה חזקה לפי דרישות המשתמש
- שליחה לשרת
- הצפנה ופענוח בתקשורת
- הכנסת הנתונים אל בסיס הנתונים בשרת
- הצפנה של המידע בבסיס הנתונים בשרת
- קבלת תשובה מהשרת
- הצגת התשובה למשתמש
- העתקה ל-clipboard

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים, אחראי סיסמאות (מייצר סיסמה על פי ההעדפות).

## צפייה במידע השמור על האתרים

**מהות היכולת**: צפייה בשמות המשתמש והסיסמאות השמורים לאתרים. הסיסמאות מוצגות בצורה נסתרת עם אפשרות להציג ולהסתיר אותן. בנוסף, יש אפשרות להעתיק את הסיסמאות ל – clipboard.

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך ראשי עם כפתור צפייה במידע, מסך מידע על האתר)
- שליחה לשרת
- הצפנה ופענוח בתקשורת
- פענוח וקבלת הנתונים מבסיס הנתונים בשרת
- קבלת המידע מהשרת

- הצגת המידע למשתמש
- העתקה ל-clipboard

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים.

## שינוי סיסמה לאתרים

**מהות היכולת**: שינוי הסיסמה לאתרים הרשומים. מכיוון שיש אתרים הדורשים שינוי סיסמה כל תקופה מסוימת, צריך שתהיה יכולת לשנות את הסיסמה. אפשר לשנות מחדש את ההעדפות ליצירת הסיסמה החדשה, ויש אפשרות להעתיק את הסיסמה החדשה שנוצרת ל-clipboard.

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך ראשי עם כפתור שינוי סיסמה לאתר, מסך שינוי סיסמה, מסך העדפות יצירת הסיסמה)
- קליטת נתונים
- יצירת סיסמה חזקה לפי דרישות המשתמש
- שליחה לשרת
- הצפנה ופענוח בתקשורת
- עדכון הנתונים בבסיס הנתונים
- הצפנה של המידע בבסיס הנתונים
- קבלת תשובה מהשרת
- הצגת התשובה למשתמש
- העתקה ל-clipboard

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים, אחראי סיסמאות (מייצר סיסמה על פי ההעדפות).

## מנגנון שכחתי סיסמה

**מהות היכולת**: האפליקציה אמורה לשמור במקום אחד את כל הסיסמאות של המשתמש, אבל מה אם המשתמש שוכח את הסיסמה שלו לאפליקציה עצמה? מנגנון השכחתי סיסמה נועד כדי לפתור את הבעיה הזו, אבל באופן שלא פוגע באבטחה. המשתמש יכול להכניס את שם המשתמש שלו וללחוץ על "שכחתי סיסמה", ולאחר מכן במקום להכניס את הסיסמה שלו, הוא עונה על שאלות אישיות פרטיות שרק הוא יודע, ועובר עוד פקטור הזדהות נוסף (קוד לאימייל, זיהוי פנים או זיהוי קול). לאחר ההזדהות, המשתמש יכול להכניס סיסמה חדשה לאפליקציה שתשמר לפעמים הבאות.

**אוסף יכולות נדרשות**:

- ממשק משתמש (מסך התחברות, מסך תשובות לשאלות הזדהות, מסך בחירת פקטור הזדהות נוסף, מסך אימות קוד אימייל, מסך הקלטת קול, מסך הקלטת וידאו ומסך איפוס סיסמה)
- קליטת נתונים
- מניעת ניסיונות רצופים רבים להתחברות

- שליחת קוד לאימייל של המשתמש מהשרת
- הפקת דגימת הקול / נתוני הפנים של המשתמש והשוואתם לנתונים הקיימים
- שליחה לשרת
- הצפנה ופענוח בתקשורת
- אימות נתונים מול בסיס הנתונים בשרת
- הצפנה ופענוח בבסיס הנתונים בשרת
- קבלת תשובה מהשרת
- הצגת תשובות מהשרת למשתמש
- הכנסת סיסמה חדשה לבסיס הנתונים ושינוי מפתח ההצפנה והפענוח של המשתמש בהתאם לסיסמה החדשה
- התחברות בסוף התהליך

**אובייקטים נחוצים**: ממשק משתמש, תקשורת, הצפנה ופענוח, בסיס נתונים, מנהל אימייל, מנהל וידאו, מזהה פנים, מנהל אודיו, מאמת קול.

## מניעת ניסיונות רצופים רבים להתחברות לאפליקציה

**מהות היכולת**: מניעת ניסיונות רצופים רבים להתחברות לאפליקציה על ידי נעילה זמנית של האפליקציה עבור המשתמש אחרי מספר ניסיונות כושלים בפרק זמן קצר. נועד כדי למנוע brute force עבור פריצה למשתמשים.

**אוסף יכולות נדרשות**:

- שליחה לשרת
- הצפנה ופענוח בתקשורת
- מבנה נתונים בשרת ששומר ניסיונות התחברות כושלים של משתמשים בפרק זמן קצר
- בדיקה בשרת וחסימה זמנית של המשתמש אם הוא עבר את מספר הניסיונות הכושלים המותרים
- קבלת תשובה מהשרת

**אובייקטים נחוצים**: תקשורת, הצפנה ופענוח.

# מבנה / ארכיטקטורה של הפרויקט

## הארכיטקטורה של המערכת המוצעת

### תיאור החומרה

**שרת**: המרכז של האפליקציה, מנהל את המשתמשים ומקבל מהם פקודות. השרת מתקשר עם בסיס הנתונים, שולף ממנו מידע ועורך בו שינויים, והוא מתקשר עם הלקוחות בעזרת תקשורת סוקטים בפרוטוקול TCP, וממלא את פקודותיהם.

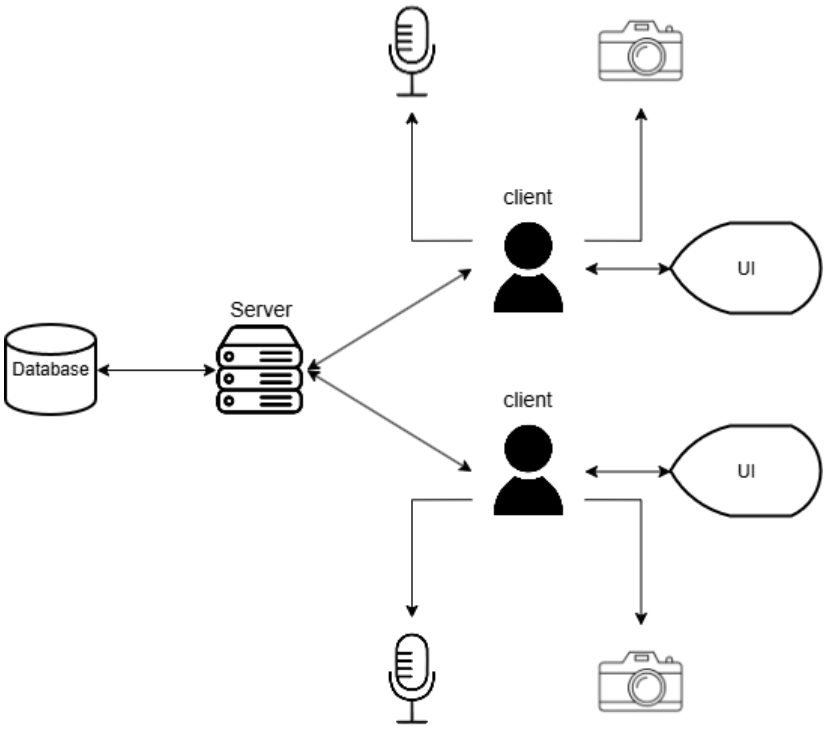
**בסיס נתונים**: מאגר של כל נתוני המשתמשים של האפליקציה. המידע בבסיס הנתונים נשמר בין ריצות של השרת ולכן הוא אמין ומאובטח יותר מהזיכרון של השרת. דרך פעולות בבסיס הנתונים, השרת כותב ושולף ממנו מידע.

**לקוחות**: הלקוח הוא אוסף היכולות של המשתמשים. הוא נמצא בתקשורת עם ממשק המשתמש וממנו הוא מקבל קלט מהמשתמש, הוא נמצא בתקשורת עם השרת שאליו הוא שולח פקודות מהמשתמש, ומקבל ממנו תשובות שהוא מציג בממשק המשתמש. הלקוח בנוסף משתמש ברכיבי קלט כמו המיקרופון והמצלמה עבור ההזדהות הביומטרית.

**ממשק משתמש**: ממשק המשתמש הוא מה שמוצג בפועל למשתמש של האפליקציה, ומה שמאפשר קלט ישיר ממנו. ממשק המשתמש בקשר עם הלקוח, מעביר אליו את הקלט והפקודות של המשתמש ומקבל ממנו את התשובה שאותה הוא צריך להציג למשתמש. ממשק המשתמש גם מנהל את החלונות הקטנים שנפתחים על גבי החלון הראשי של האפליקציה (Top level windows).

**אמצעי קלט**: מיקרופון ומצלמה של המשתמשים. הלקוח מפעיל אותם אם זיהוי קול או זיהוי פנים נבחרו בתור הפקטור השני להזדהות, או בעת ההקמה של ההתחברות הביומטרית.

### שרטוט קשרי החומרה



## הטכנולוגיה הרלוונטית

**שפת תכנות**: הפרויקט נכתב בשפת פייתון, ונעשה גם שימוש ב SQL עבור בסיס הנתונים.

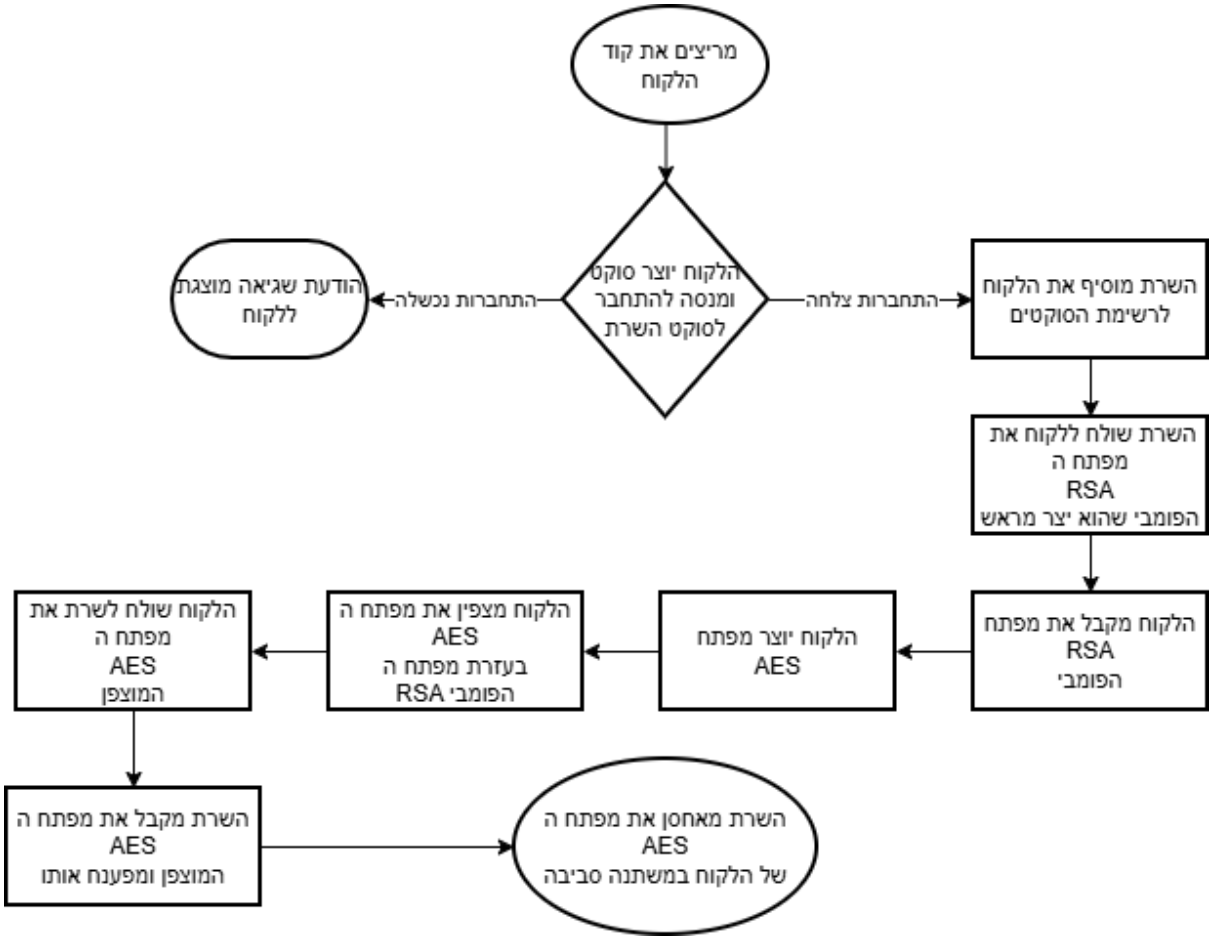
**מערכת הפעלה**: מערכת ההפעלה במחשב בו כתבתי את הפרויקט היא Window 11.

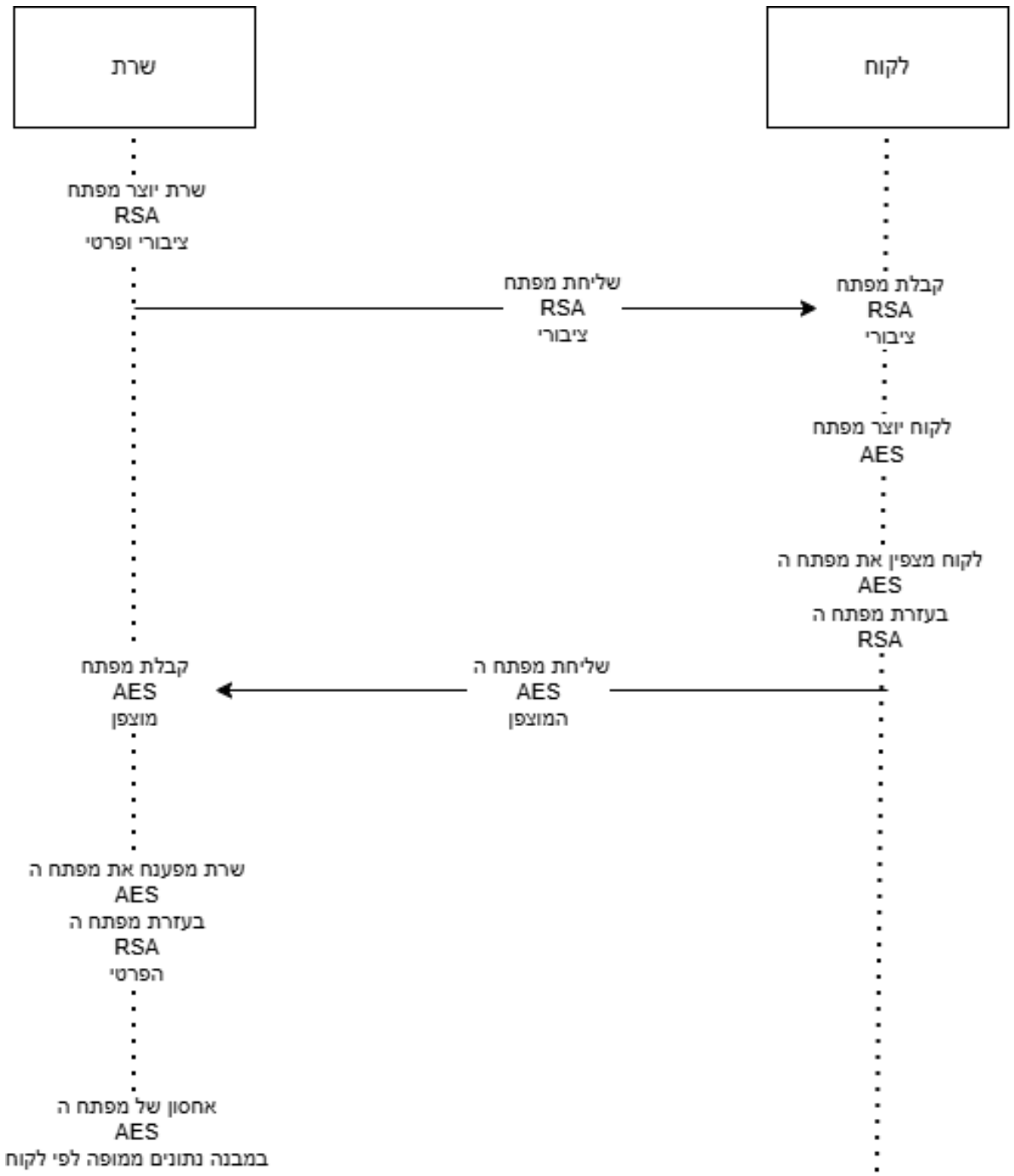
**תקשורת**: התקשורת נעשית באמצעות סוקטים בעזרת פרוטוקול TCP. פרוטוקול TCP הוא פרוטוקול תקשורת מאובטח ואמין עבור תקשורת סוקטים, והוא מבטיח שמידע לא יאבד בדרך ויגיע מסודר ובשלמותו. הפרויקט שלי דורש יציבות ואמינות יותר ממהירות, ולכן בחרתי להשתמש בפרוטוקול TCP ולא UDP.

**תחומי עניין נוספים**: הפרויקט עושה שימוש במיקרופון ובמצלמה של המשתמשים.

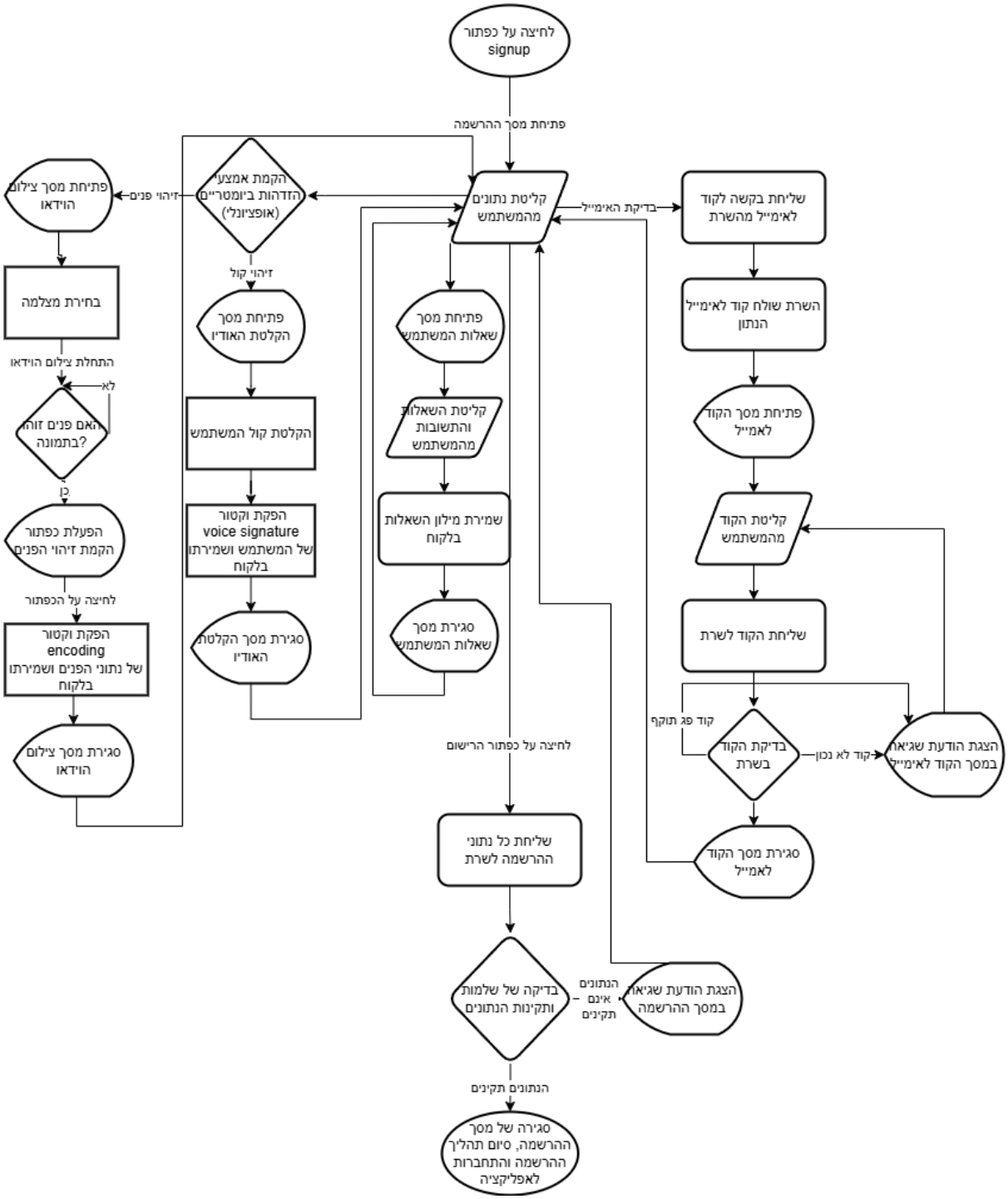
## זרימת המידע במערכת

### הרצת לקוח ותהליך החלפת מפתחות הצפנת התקשורת

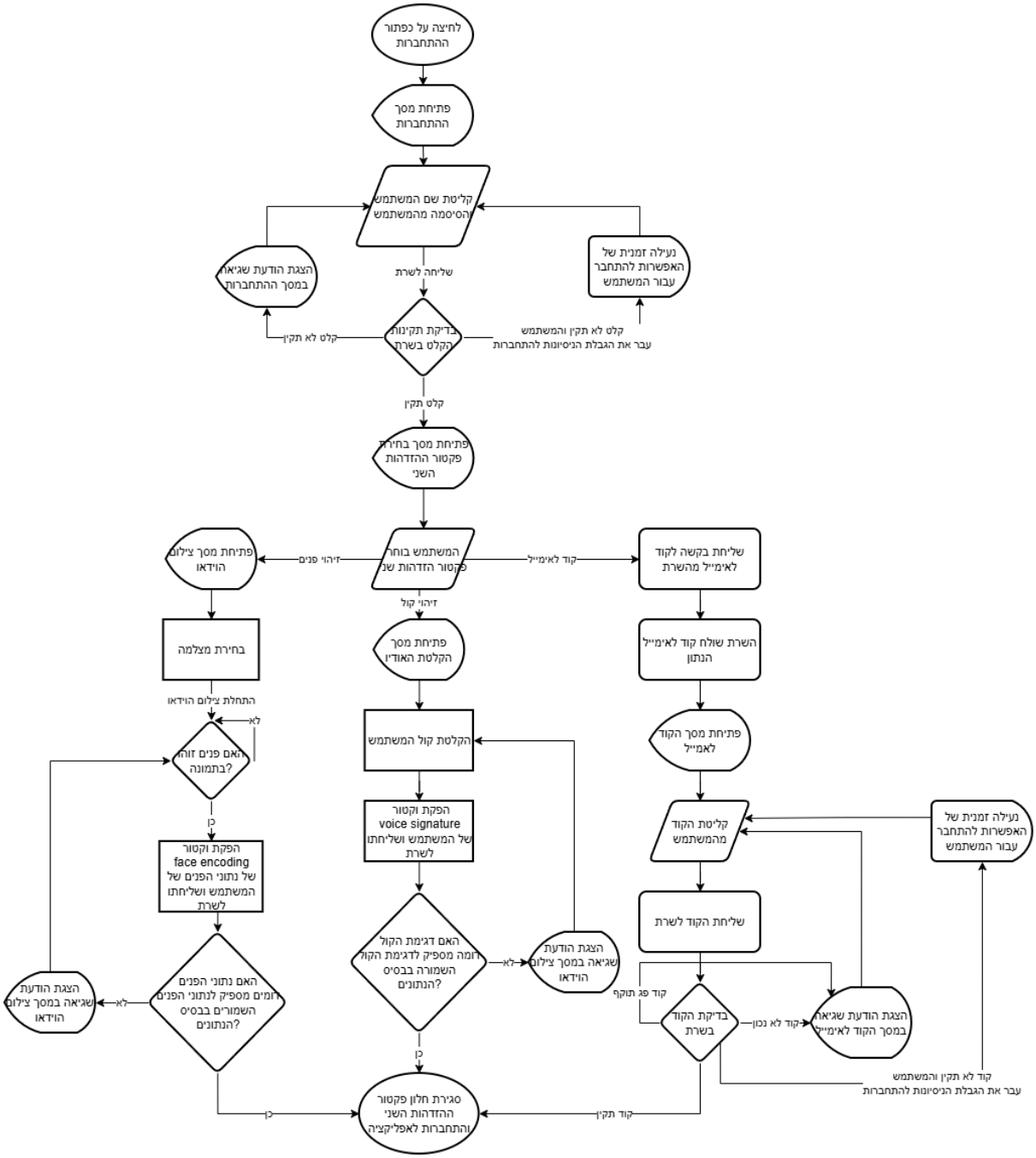



שרת
לקוח
שרת יוצר מפתח
RSA
ציבורי ופרטי
שליחת מפתח
RSA
ציבורי
קבלת מפתח
RSA
ציבורי
לקוח יוצר מפתח
AES
לקוח מצפין את מפתח ה
AES
בעזרת מפתח ה
RSA
שליחת מפתח ה
AES
המוצפן
קבלת מפתח
AES
מוצפן
שרת מפענח את מפתח ה
AES
בעזרת מפתח ה
RSA
הפרטי
אחסון של מפתח ה
AES
במבנה נתונים ממופה לפי לקוח

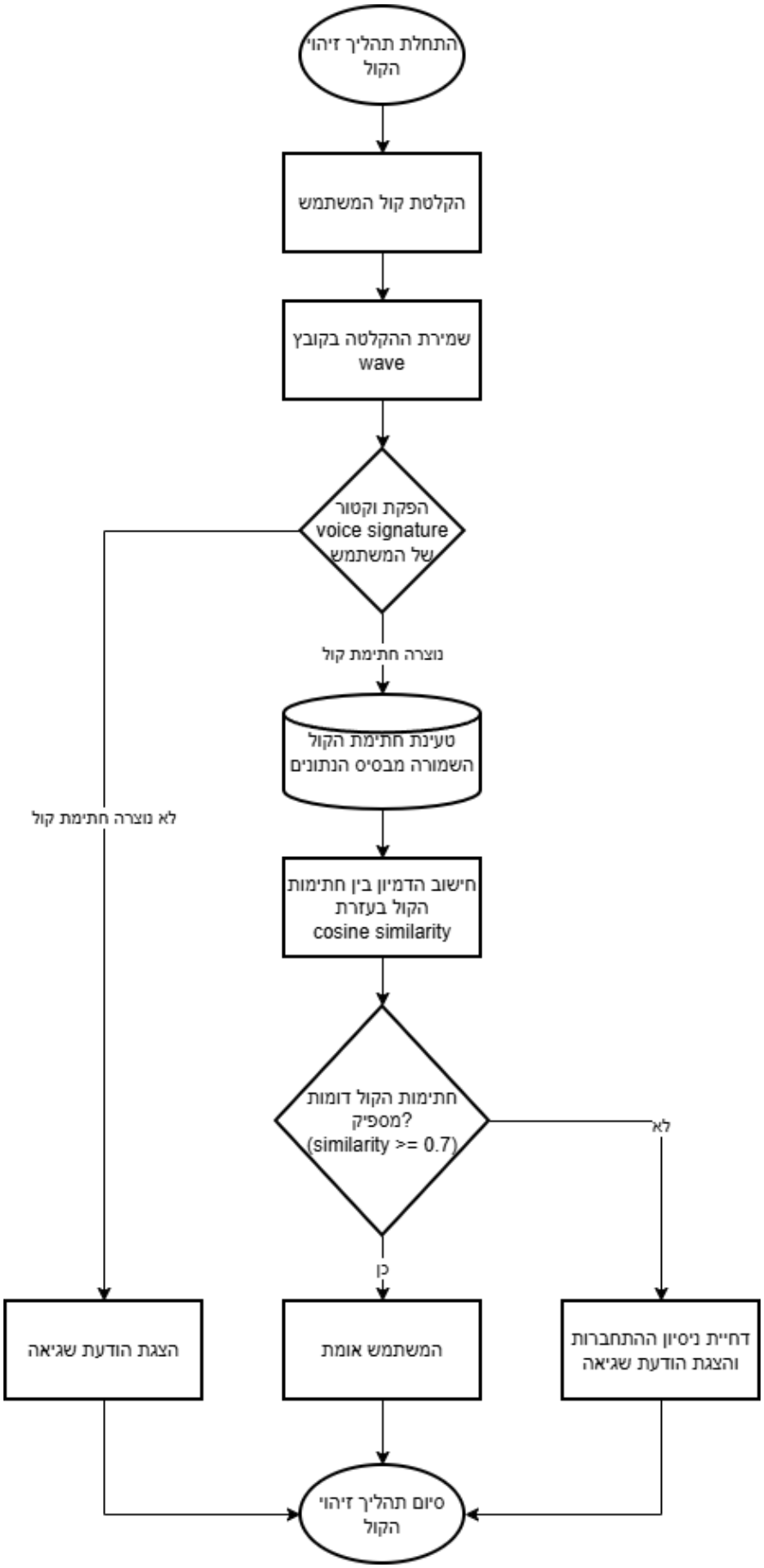
## רישום משתמשים חדשים לאפליקציה



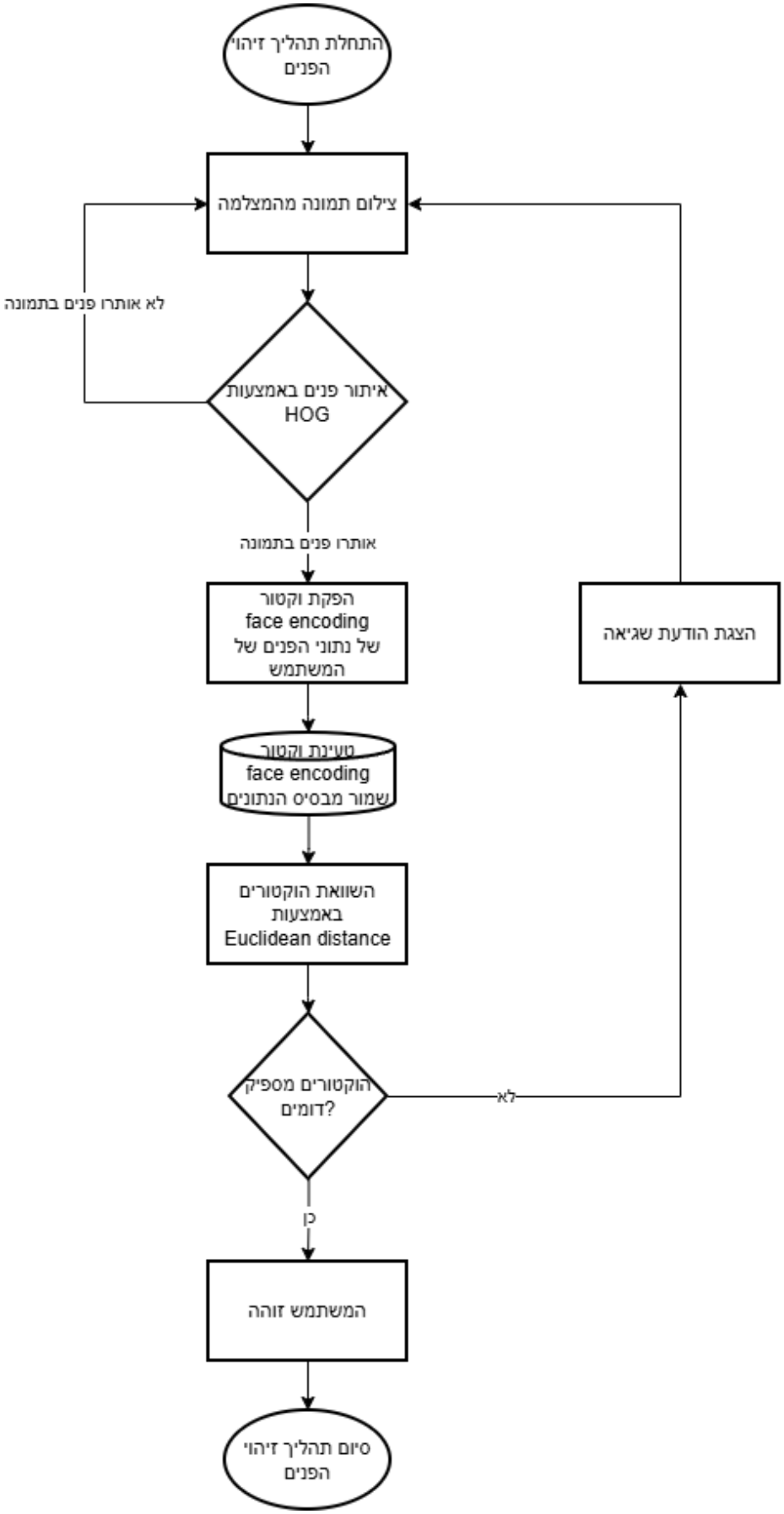
## התחברות לאפליקציה



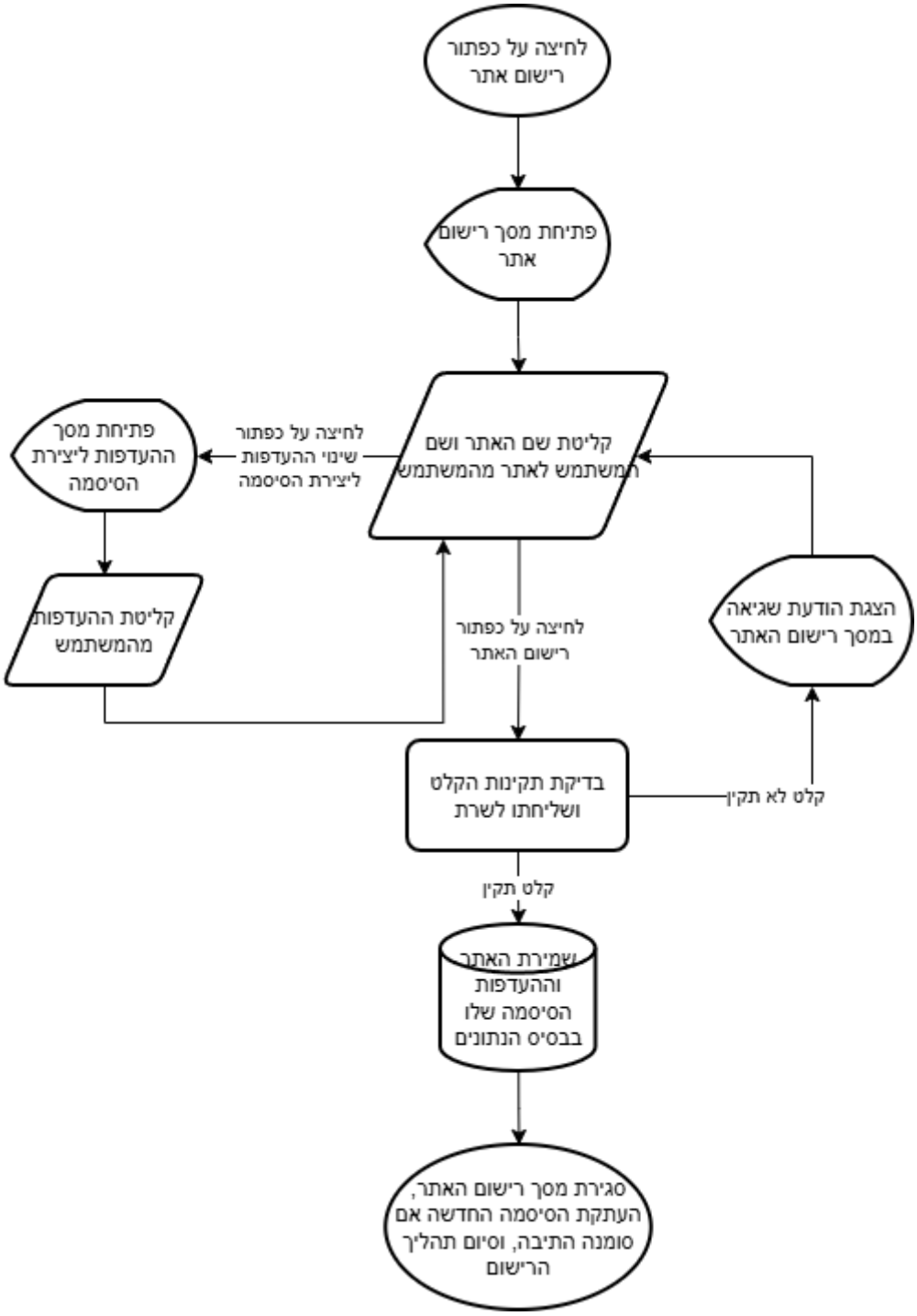
## זיהוי קול


התחלת תהליך זיהוי הקול
הקלטת קול המשתמש
שמירת ההקלטה בקובץ wave
הפקת וקטור voice signature של המשתמש
נוצרה חתימת קול
טעינת חתימת הקול השמורה מבסיס הנתונים
חישוב הדמיון בין חתימות הקול בעזרת cosine similarity
חתימות הקול דומות מספיק? (similarity >= 0.7)
לא
כן
לא נוצרה חתימת קול
הצגת הודעת שגיאה
המשתמש אומת
דחיית ניסיון ההתחברות והצגת הודעת שגיאה
סיום תהליך זיהוי הקול

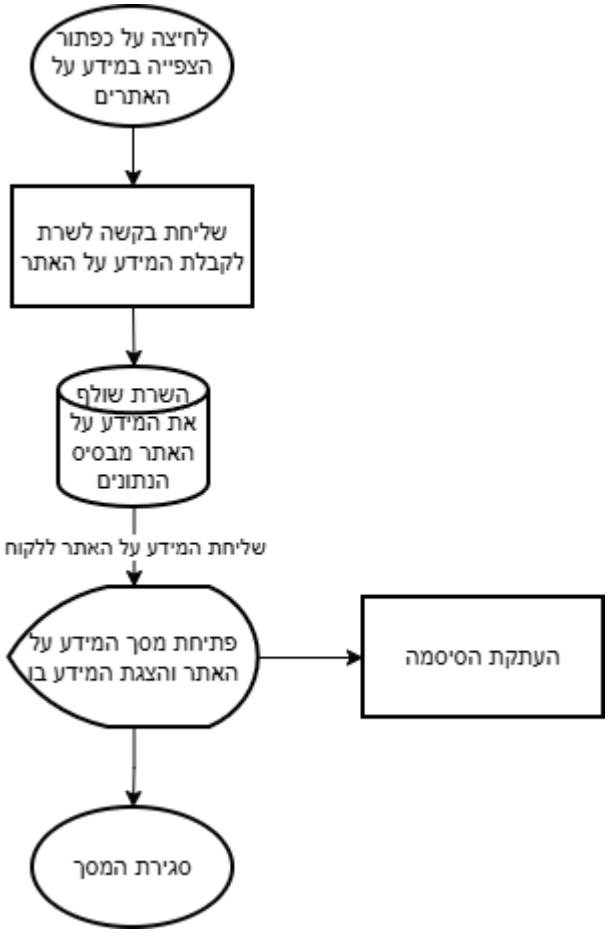
## זיהוי פנים


התחלת תהליך זיהוי הפנים
צילום תמונה מהמצלמה
לא אותרו פנים בתמונה
איתור פנים באמצעות HOG
אותרו פנים בתמונה
הפקת וקטור face encoding של נתוני הפנים של המשתמש
טעינת וקטור face encoding שמור מבסיס הנתונים
השוואת הוקטורים באמצעות Euclidean distance
הוקטורים מספיק דומים?
לא
כן
הצגת הודעת שגיאה
המשתמש זוהה
סיום תהליך זיהוי הפנים

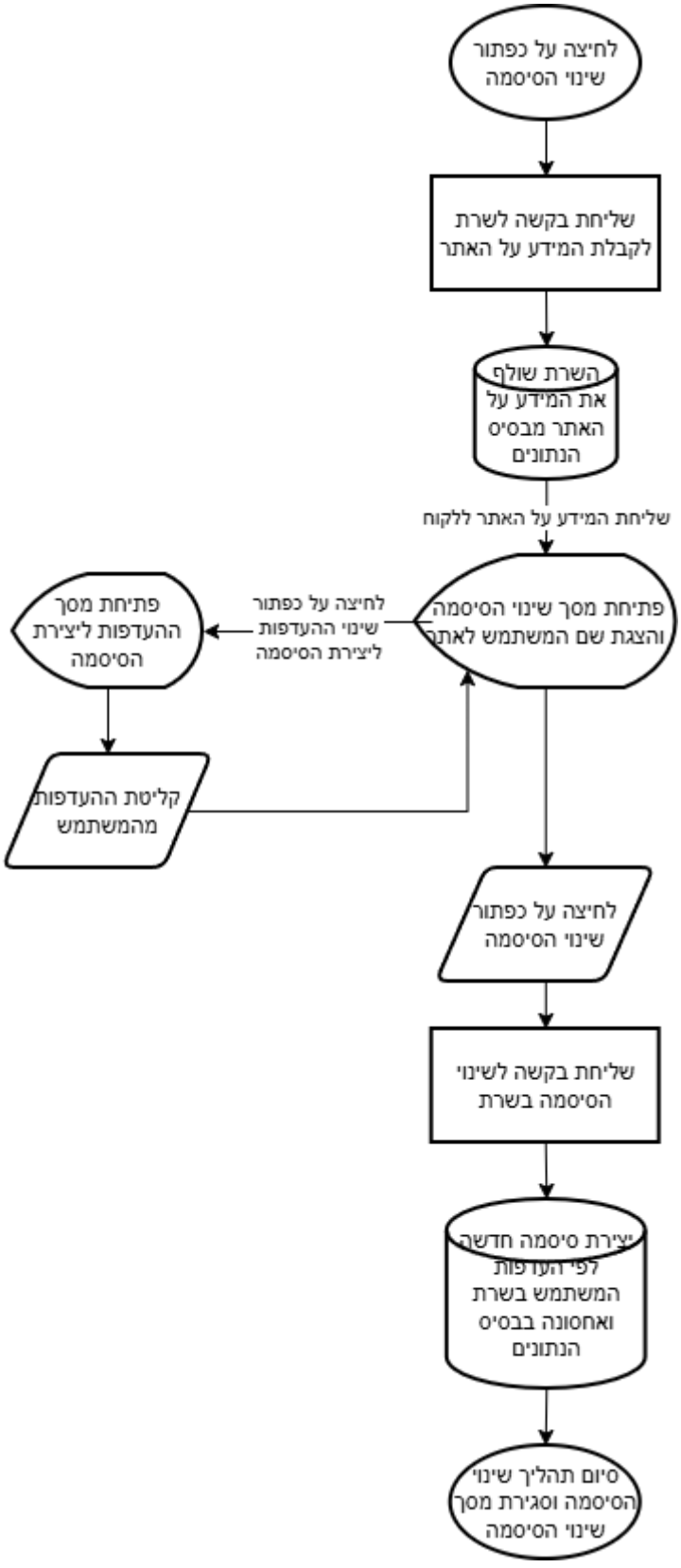
## רישום אתרים חדשים



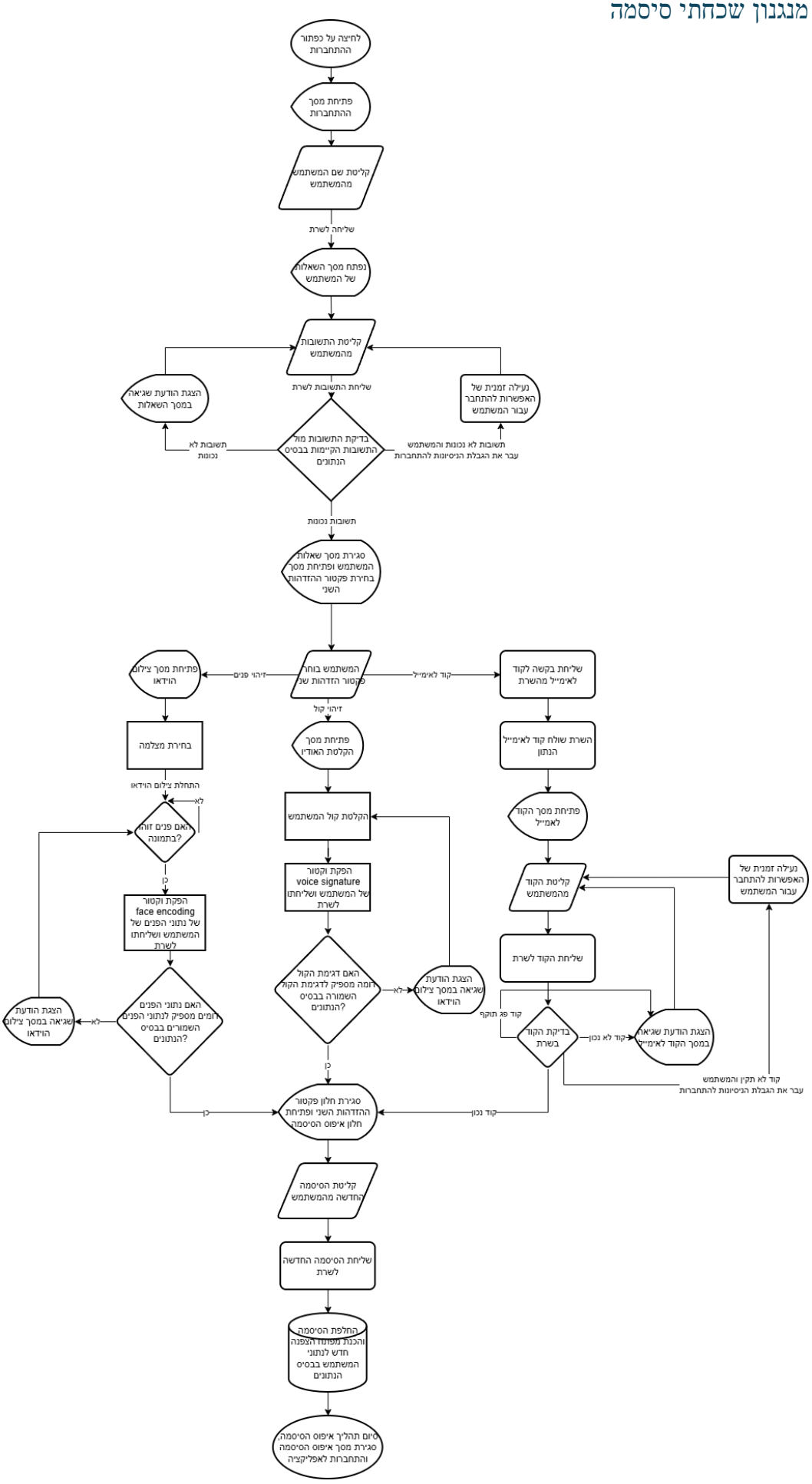
## צפייה במידע השמור על האתרים



## שינוי סיסמה לאתרים



## מנגנון שכחתי סיסמה



## אלגוריתמים מרכזיים בפרויקט

### הצפנה בתקשורת

אני מעביר בתקשורת בין השרת ללקוח מידע רגיש של המשתמשים הכולל סיסמאות ונתונים ביומטריים. כדי לשמור על האפליקציה שלי והמידע הרגיש של המשתמשים מאובטחים, אני צריך להצפין את הנתונים בתקשורת.

אני רציתי הצפנה שתהיה מהירה ולא תפגע ביעילות האפליקציה שלי, ומצאתי שהצפנה כזו היא הצפנת AES. AES זו הצפנה סימטרית, כלומר אותו מפתח המשמש להצפנה משמש גם לפענוח של המידע. הבעיה עם הצפנה סימטרית היא העברת המפתח ללקוח – אי אפשר סתם כך להעביר את המפתח ללקוח בתקשורת, כי מישהו יכול להאזין ולגנוב את המפתח, ולאחר מכן לפענח את כל המידע העובר בתקשורת.

הצפנה שפותרת את בעיית העברת המפתחות היא הצפנת RSA. הצפנת RSA היא הצפנה א-סימטרית, כלומר יש מפתח פרטי המשמש לפענוח המידע, ומפתח ציבורי המשמש להצפנה. המפתח הציבורי, כשמו הוא, יכול להיות ציבורי ולהישלח בתקשורת בלי חשש, כי גם אם מישהו מאזין ומקבל את המפתח הציבורי, הוא לא יכול לפענח את ההודעות בתקשורת. הבעיה עם הצפנת RSA היא שזו הצפנה איטית יחסית, והעברת מידע בסדר גודל גדול איתה לוקחת הרבה זמן.

הפתרון לבעיות האלו היא שימוש בהצפנה היברידית – הצפנה המשלבת גם RSA וגם AES. השרת יוצר מפתחות RSA פרטי וציבורי, ושולח ללקוח את המפתח הציבורי. הלקוח מקבל את המפתח הציבורי, יוצר מפתח AES, מצפין את מפתח ה-AES בעזרת מפתח ה-RSA הציבורי ושולח לשרת. בגלל שרק לשרת יש את מפתח ה-RSA הפרטי, רק הוא יכול לפענח את מפתח ה-AES שהלקוח יצר. השרת מפענח את מפתח ה-AES ושומר אותו בזיכרון כמפתח הייחודי של הלקוח המסוים. כך, לכל לקוח יש מפתח AES אחר והלקוחות לא יכולים לפענח את המידע אחד של השני, ומנקודה זו השרת והלקוח יכולים להצפין ולפענח עם מפתח ה-AES וכך לתקשר בחופשיות.

כדי להשתמש בהצפנות בפועל בקוד שלי, אני משתמש בספריית cryptography.
עבור הצפנת AES: Fernet מ - cryptography.fernet.
עבור הצפנת RSA: rsa מ - cryptography.hazmat.primitives.asymmetric.

Fernet (symmetric encryption) — Cryptography 48.0.0.dev1 documentation

RSA – ויקיפדיה

### הצפנה בבסיס הנתונים

ברוב המקרים, בשביל אבטחת המידע הרגיש בבסיס הנתונים היה מספיק אלגוריתם hash. Hash הוא אלגוריתם הממיר קלט בכל גודל לפלט באורך קבוע, באופן חד כיווני ודטרמיניסטי – אי אפשר לשחזר את הקלט המקורי, וכל קלט הוא ייחודי לחלוטין (קלט זהה יפיק אותו פלט, אבל קלט קצת שונה יפיק פלט שונה לחלוטין). השתמשתי בקוד שלי באלגוריתם sha-256 עבור ה- hashing של המידע הרגיש שלא צריך לשחזר עבור המשתמש (לדוגמה סיסמה לאפליקציה).

הבעיה הספציפית אצלי בפרויקט היא שאני צריך להחזיר למשתמש את המידע על האתרים השמורים אצלו באפליקציה. אני לא יכול להשתמש באלגוריתם hash כי אני לא אוכל לשחזר את

המידע המקורי שהמשתמש הכניס ואת הסיסמאות שנוצרו עבור האתרים, אבל אני גם לא יכול להשאיר את המידע הרגיש והסיסמאות חשופים בבסיס הנתונים.

הפתרון היה להצפין את המידע הזה באמצעות הצפנת AES. אך הפתרון הזה העלה בעיה אחרת – איפה לשמור את מפתח ה-AES? הבנתי שאני לא יכול פשוט לשמור את המפתח בקובץ על השרת, כי במקרה שמישהו פורץ ומצליח לקחת את המפתח, הוא יכול לפענח את כל המידע הרגיש בבסיס הנתונים.

כדי לפתור את הבעיה הזו, החלטתי להשתמש באלגוריתם KDF (Key Derivation Function). האלגוריתם מפיק מפתח הצפנה בצורה אקראית מתוך seed של סיסמה או סוד אחר. אני יוצר עבור כל משתמש מפתח AES רנדומלי להצפנת המידע שלו (data key), ולאחר מכן אני משתמש באלגוריתם PBKDF2HMAC על ה-hash של הסיסמה של המשתמש לאפליקציה שלי כדי ליצור מפתח להצפנת מפתח ה-AES הרנדומלי (key encryption key). אני מצפין את המפתח הרנדומלי ואותו אני שומר בבסיס הנתונים. מפה, בכל פעם שצריך להצפין או לפענח מידע של המשתמש בבסיס הנתונים, אני יוצר מחדש את המפתח שמצפין את המפתח הרנדומלי עם hash הסיסמה של המשתמש (ה – key encryption key), מפענח את המפתח הרנדומלי (data key), ואיתו אני מפענח או מצפין.

הפתרון הזה גם מקל עלי בתהליך איפוס הסיסמה: כשהמשתמש מחליף את הסיסמה שלו, אני צריך לשנות רק את מפתח ההצפנה של מפתח המידע (ה – key encryption key), ולא את מפתח המידע (ה – data key) ואת כל המידע של המשתמש המוצפן בבסיס הנתונים.

בשביל להשתמש בהצפנה ובאלגוריתם ה- KDF, אני משתמש בספריית cryptography. עבור ה-hashing אני משתמש בספריית hashlib.
עבור הצפנת AES: Fernet מ - cryptography.fernet.
עבור אלגוריתם KDF: PBKDF2HMAC מ - cryptography.hazmat.primitives.kdf.pbkdf2
עבור hashing: hashlib.sha256()

Fernet (symmetric encryption) — Cryptography 48.0.0.dev1 documentation

Key derivation functions — Cryptography 48.0.0.dev1 documentation

## זיהוי קול וזיהוי פנים

### זיהוי הפנים

המחשב לא יכול לזהות פנים כמו שבן אדם יכול – הוא לא יכול לזהות בן אדם לפי צבע העיניים שלו או לפי מדידת אורך האף. תהליך מציאת הפנים וחישוב הזהות של הפנים של המשתמש בספרייה שאני משתמש בה (face_recognition) הולך כך:

- כדי למצוא את הפנים של המשתמש בתמונה, משתמשים בפונקציה face_locations. הפונקציה הזו, בעזרת אלגוריתם שנקרא HOG, ממפה את הצללים בתמונה ואת הכיוון שבו הם הולכים ונהיים כהים יותר, ובעזרת מודל שאומן מראש על פנים של אנשים ויודע איך מפת הצללים של פנים בדרך כלל נראית הוא מוצא את המיקום של הפנים בתמונה.

- לאחר שיש לנו את מיקום הפנים בתמונה, נוכל לחשב את קידוד הפנים של המשתמש בעזרת הפונקציה face_encodings.
הפונקציה הזו משתמשת ב ResNet-34 – מודל שאומן על מיליוני פנים של אנשים ויודע למצוא 128 נקודות מדידה שמאפשרות לזהות בצורה מיטבית את הבן אדם, ויוצרות מהן וקטור בעל 128 מימדים.
- אחר כך, בשביל להשוות בין וקטור הפנים השמור של המשתמש לבין וקטור הפנים של מי שמנסה להתחבר, משתמשים בפונקציה compare_faces שמוצאת את המרחק בין שתי הנקודות המייצגות את וקטורי נתוני הפנים בעזרת נוסחת Euclidean distance. אם שתי הנקודות קרובות מספיק במרחב אז יש התאמה, והמשתמש זוהה.

<u>זיהוי קול</u>

אני משתמש בספרייה vosk ובמודלים של vosk עבור זיהוי הקול. תהליך זיהוי הקול עובד כך:

- טוענים את המודלים model ו spkModel בשביל שנוכל להשתמש בהם בהמשך. model הוא מודל הדיבור לטקסט, ו spkModel הוא מודל זיהוי הדובר.
- מזינים פיד של קול אל KaldiRecognizer (המוח של המערכת שמחבר בין שני המודלים), ובעזרת הפונקציה AcceptWaveForm הוא מחלק את הפיד לפריימים קטנים שיהיה יותר קל לנתח.
- בכל פריים קטן של קול, ה AI מנתח את התדירות, ויוצר לעצמו מעין תמונה של התדירות של קול המשתמש כתלות בזמן של הפריים.
- לאחר מכן, ה AI מנתח את התמונה הזו, מזהה צורות הגייה ייחודית של המשתמש עבור אותיות שונות, ובעזרת הסתכלות על ה context ועל ניתוחים סמוכים הוא מצליח לקלוט את הצורה הייחודית שבה המשתמש עובר בין ההגיות הקטנות האלו של הצלילים המרכיבים את האותיות. וכך בעצם הוא מצליח לייחד את טקסטורת הדיבור של המשתמש.
- לבסוף, שכבה נוספת של ה AI עוברת על הפלטים של טקסטורות הדיבור, עושה ממוצע שלהם ומחשבת וקטור באורך קבוע של 128 מימדים, בלי תלות באורך זמן הקלטת הדיבור של המשתמש.
- בשביל לבצע את הזיהוי, משתמשים בנוסחת cosine similarity בין וקטורי טביעות הקול של המשתמש והלקוח שמנסה להתחבר. הנוסחה נועדה למדוד את הזווית בין הוקטורים, ולא את ההבדל בין עוצמת הקול שלהם. אם התוצאה היא מעל רף מסוים, ניתן לקבוע שהלקוח שמנסה להתחבר הוא המשתמש, וזיהוי הקול הצליח.

## תיאור סביבת הפיתוח

**גרסת python**: python 3.13
**הפרויקט פותח ב**: Visual Studio Code

**הסביבה והכלים הנדרשים לבדיקות**:

- יש לוודא שהתיקיות Model ו- SpkModel קיימות בתוך התיקייה המכילה את הפרויקט. אלה המודלים לזיהוי הקול של Vosk.
  Model - vosk-model-small-en-us-0.15
  SpkModel - vosk-model-spk-0.4
- כדי להיות מסוגל להשתמש בכל יכולות האפליקציה, למשתמש צריכים להיות מיקרופון ומצלמה תקינים המחוברים למחשב.
- עבור זיהוי הפנים צריך להיות מותקן במחשב C++ compiler. ניתן להוריד את MSVC ו- CMake הדרושים לפרויקט מ Microsoft C++ Build Tools - Visual Studio. במסך ההתקנה צריך לסמן "Desktop development with C++".

## תיאור פרוטוקול התקשורת

### מבנה ההודעה בפרוטוקול התקשורת

CMD_FIELD {DELIMETER}LENGTH_FIELD {DELIMETER}DATA1{DATA_DELIMETER}DATA2

**CMD_FIELD** – שדה הפקודה בהודעה. השדה באורך 30 תווים (בתים)
**LENGTH_FIELD** – שדה שמתאר את האורך של שדה המידע בהודעה. השדה באורך 4 תווים (בתים).
**DATA** – שדה של מידע הרלוונטי לפקודה הניתנת בהודעה. לאור העובדה ש LENGTH_FIELD הוא באורך 4, האורך של שדה המידע יכול להגיע ל-9999 תווים (בתים). בפועל, אני מגביל את אורך ההודעה בתקשורת ל-4096 בתים.
**DELIMETER** – תו מפריד בין שדות ההודעה. התו הוא התו בעל ערך ה- ascii של 0 (chr(0)).
**DATA_DELIMETER** – תו מפריד בין הפרמטרים השונים שמועברים בהודעה. התו הוא התו בעל ערך ה- ascii של 1 (chr(1)).

### הודעות מהלקוח לשרת

נסמן ב- | את DELIMETER וב- / את DATA_DELIMETER במבנה ההודעות.

**SIGNUP**
בקשת הרשמה לאפליקציה

SIGNUP |0000|username/password/email/phone_number/voice_signature/face_encoding/questions

**LOGIN**

בקשת התחברות לאפליקציה

LOGIN |0000|username/password

**SECURITY_QUESTIONS**

בקשה לקבלת רשימת השאלות האפשריות שאפשר לענות עליהן

SECURITY_QUESTIONS |0000|

**MY_SECURITY_QUESTIONS**

בקשה לקבלת רשימת השאלות עליהן המשתמש ענה בעת ההרשמה

MY_SECURITY_QUESTIONS |0000|username

**CHECK_SECURITY_ANSWERS**

בקשה לבדיקה של התשובות לשאלות של המשתמש בעת ההתחברות

CHECK_SECURITY_ANSWERS |0000|username/answers

**RESET_PASSWORD**

בקשה לאיפוס (שינוי) הסיסמה לאפליקציה

RESET_PASSWORD |0000|username/new_password

**SAVE_PREFERENCES**

בקשה לשמירת ההעדפות של המשתמש ליצירת הסיסמה עבור אתר מסוים

SAVE_PREFERENCES |0000|site_url/preferences

**GET_PREFERENCES**

בקשה לקבלת ההעדפות של המשתמש ליצירת הסיסמה עבור אתר מסוים

GET_PREFERENCES |0000|site_url

**REGISTER_SITE**

בקשה לרישום אתר חדש עבור המשתמש

SAVE_PREFERENCES |0000|site_url/site_username/preferences

**CHANGE_PASSWORD**

בקשה לשינוי הסיסמה עבור אתר מסוים

CHANGE_PASSWORD |0000|site_url

**REGISTERED_SITES**

בקשה לקבלת רשימת כל האתרים הרשומים אצל משתמש מסוים (המחוקים או לא)

REGISTERED_SITES |0000|get_deleted

**SITE_INFO**

בקשה לקבלת המידע על אתר רשום (שם המשתמש והסיסמה)

SITE_INFO |0000|site_url

**GET_AUTH_OPTIONS**

בקשה לקבלת כל פקטורי ההזדהות שהמשתמש הקים ויכול להתחבר באמצעותם

GET_AUTH_OPTIONS |0000|username

**SEND_CODE**

בקשה לקבלת קוד חד פעמי לאימייל של המשתמש

SEND_CODE |0000|client_email

**CODE_LOGIN**

בקשה לבדיקת הקוד שהמשתמש הכניס

CODE_LOGIN |0000|code

**MY_EMAIL**

בקשה לקבלת כתובת האימייל של המשתמש

MY_EMAIL |0000|username

**VOICE_AUTH**

בקשה לבדיקת דגימת הקול של המשתמש

VOICE_AUTH |0000|username/voice_signature

**CHECK_FACE_RECOGNITION**

בקשה לבדיקת זיהוי הפנים של המשתמש

CHECK_FACE_RECOGNITION |0000|username/face_encoding

**SETUP_VOICE_AUTH**

בקשה להקמת זיהוי הקול (לאחר ההרשמה)

SETUP_VOICE_AUTH |0000|voice_signature

**SETUP_FACE_RECOGNITION**

בקשה להקמת זיהוי הפנים (לאחר ההרשמה)

SETUP_FACE_RECOGNITION |0000|face_encoding

**DELETE_SITE**

בקשה למחיקת אתר מרשימת האתרים השמורים

DELETE_SITE |0000|site_url

**RECOVER_SITE**

בקשה לשחזור אתר שנמחק מרשימת האתרים השמורים

RECOVER_SITE |0000|site_url

## הודעות מהשרת ללקוח

נסמן ב- | את DELIMETER וב- / את DATA_DELIMETER במבנה ההודעות.

**SIGNUP_OK**

תשובה להודעת SIGNUP. ההרשמה בוצעה בהצלחה.

SIGNUP_OK |0000|

**USER_EXISTS**

תשובה להודעת SIGNUP. ההרשמה נכשלה כי שם המשתמש תפוס.

USER_EXISTS |0000|

**WEAK_PASSWORD**

תשובה להודעות SIGNUP, RESET_PASSWORD. ההרשמה נכשלה כי הסיסמה אינה חזקה מספיק.

WEAK_PASSWORD |0000|

**LOGIN_OK**

תשובה להודעת LOGIN. ההתחברות בוצעה בהצלחה.

LOGIN_OK |0000|

**USERNAME_INVALID**

תשובה להודעת LOGIN. ההתחברות נכשלה כי שם המשתמש לא קיים.

USERNAME_INVALID |0000|

**INCORRECT_PASSWORD**

תשובה להודעת LOGIN. ההתחברות נכשלה כי הסיסמה לא נכונה.

INCORRECT_PASSWORD |0000|

**TIMEOUT**

תשובה לבקשות התחברות. המשתמש עבר את הגבלת ניסיונות ההתחברות והוא חסום זמנית.

TIMEOUT |0000|timeout_length

**SECURITY_QUESTIONS_LIST**

תשובה להודעת SECURITY_QUESTIONS. החזרת רשימת שאלות האפשריות.

SECURITY_QUESTIONS_LIST |0000|security_questions

**YOUR_SECURITY_QUESTIONS**

תשובה להודעה MY_SECURITY_QUESTIONS. החזרת רשימת השאלות עליהן המשתמש ענה בעת ההרשמה.

YOUR_SECURITY_QUESTIONS |0000|user_questions

**SECURITY_ANSWERS_CORRECT**

תשובה להודעת CHECK_SECURITY_ANSWERS. התשובות לשאלות של המשתמש נכונות.

SECURITY_QUESTIONS_CORRECT |0000|

**SECURITY_ANSWERS_INCORRECT**

תשובה להודעת CHECK_SECURITY_ANSWERS. התשובות לשאלות של המשתמש לא נכונות.

SECURITY_QUESTIONS_INCORRECT |0000|

**RESET_OK**

תשובה להודעת RESET_PASSWORD. איפוס הסיסמה בוצע בהצלחה.

RESET_OK |0000|

**PREFERENCES_SAVED**

תשובה להודעת SAVE_PREFERENCES. שמירת ההעדפות ליצירת הסיסמה עבור האתר בוצעה בהצלחה.

PREFERENCES_SAVED |0000|

**YOUR_PREFERENCES**

תשובה להודעת GET_PREFERENCES. החזרת ההעדפות ליצירת הסיסמה של המשתמש עבור האתר.

YOUR_PREFERENCES |0000|preferences

**REGISTRATION_OK**

תשובה להודעה REGISTER_SITE. רישום האתר בוצע בהצלחה.

REGISTRATION_OK |0000|generated_password

**ALREADY_REGISTERED**

תשובה להודעה REGISTER_SITE. רישום האתר נכשל כי האתר כבר רשום.

ALREADY_REGISTERED |0000|

**CHANGE_OK**

תשובה להודעת CHANGE_PASSWORD. החלפת הסיסמה בוצעה בהצלחה.

CHANGE_OK |0000|generated_password

**NOT_REGISTERED**

תשובה להודעות CHANGE_PASSWORD, SAVE_PREFERENCES, GET_PREFERENCES, SITE_INFO, DELETE_SITE, RECOVER_SITE. האתר אינו רשום.

NOT_REGISTERED |0000|

**YOUR_SITES**

תשובה להודעת REGISTERED_SITES. החזרת רשימת האתרים הרשומים של המשתמש.

YOUR_SITES |0000|registered_sites

**INFO**

תשובה להודעת SITE_INFO. החזרת המידע השמור על האתר.

SITE_INFO |0000|site_username/site_password

**YOUR_AUTH_OPTIONS**

תשובה להודעת GET_AUTH_OPTIONS. החזרת אפשרויות ההתחברות של הפקטור השני של המשתמש.

YOUR_AUTH_OPTIONS |0000|auth_options

**CODE_CORRECT**

תשובה להודעה CODE_LOGIN. הקוד נכון.

CODE_CORRECT |0000|

**CODE_INCORRECT**

תשובה להודעה CODE_LOGIN. הקוד שגוי.

CODE_INCORRECT |0000|

**CODE_EXPIRED**

תשובה להודעה CODE_LOGIN. פג תוקפו של הקוד.

CODE_EXPIRED |0000|

**CODE_SENT**

תשובה להודעת SEND_CODE. הקוד נשלח בהצלחה לאימייל של המשתמש.

CODE_SENT |0000|

**YOUR_EMAIL**

תשובה להודעת MY_EMAIL. החזרת כתובת האימייל של המשתמש.

YOUR_EMAIL |0000|user_email

**NO_EMAIL_REGISTERED**

תשובה להודעת MY_EMAIL. למשתמש אין כתובת אימייל שמורה.

NO_EMAIL_REGISTERED |0000|

**VOICE_VERIFIED**

תשובה להודעת VOICE_AUTH. זיהוי הקול עבר בהצלחה.

VOICE_VERIFIED |0000|

**VOICE_NOT_VERIFIED**

תשובה להודעת VOICE_AUTH. זיהוי הקול נכשל.

VOICE_NOT_VERIFIED |0000|

**FACE_MATCHES**

תשובה להודעת CHECK_FACE_RECOGNITION. זיהוי הפנים עבר בהצלחה.

FACE_MATCHES |0000|

**FACE_NOT_MATCHES**

תשובה להודעת CHECK_FACE_RECOGNITION. זיהוי הפנים נכשל.

FACE_NOT_MATCHES |0000|

**VOICE_AUTH_SET**

תשובה להודעת SETUP_VOICE_AUTH. זיהוי הקול הוקם בהצלחה.

VOICE_AUTH_SET |0000|

**FACE_RECOGNITION_SET**

תשובה להודעת SETUP_FACE_RECOGNITION. זיהוי הפנים הוקם בהצלחה.

FACE_RECOGNITION_SET |0000|

**SITE_DELETED**

תשובה להודעת DELETE_SITE. האתר נמחק בהצלחה.

SITE_DELETED |0000|

**SITE_RECOVERED**

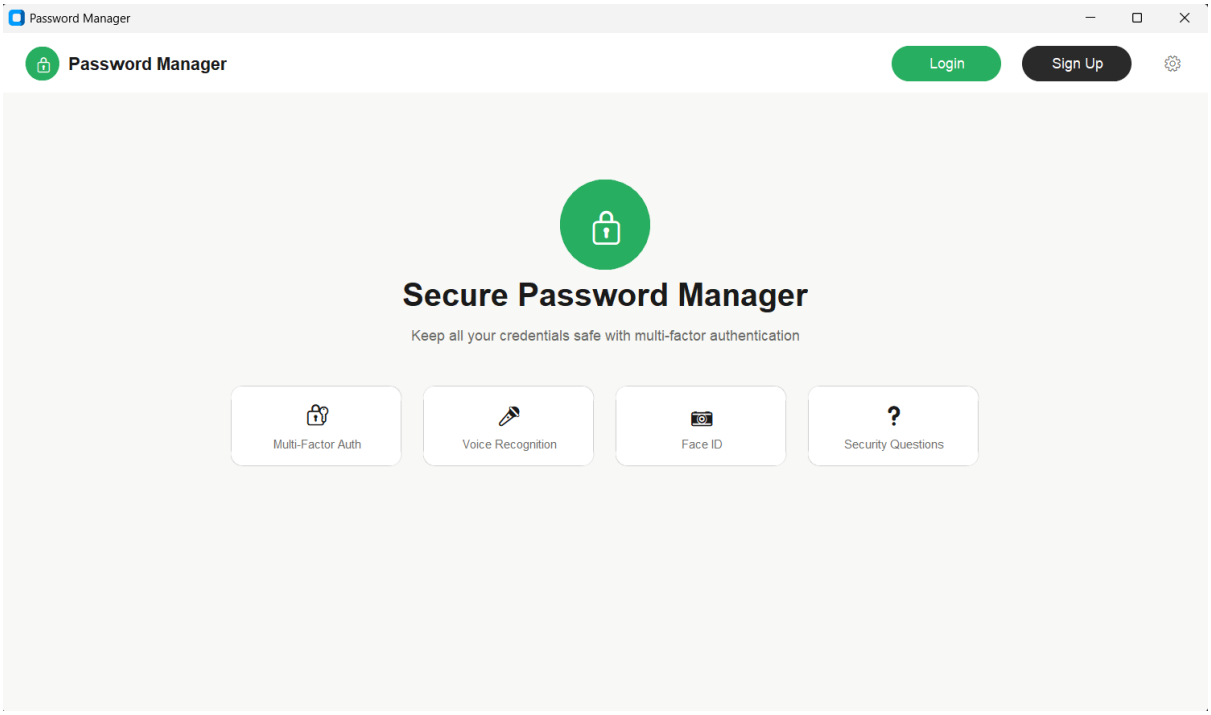
תשובה להודעת RECOVER_SITE. האתר שוחזר בהצלחה.

SITE_RECOVERED |0000|

## מסכי המערכת

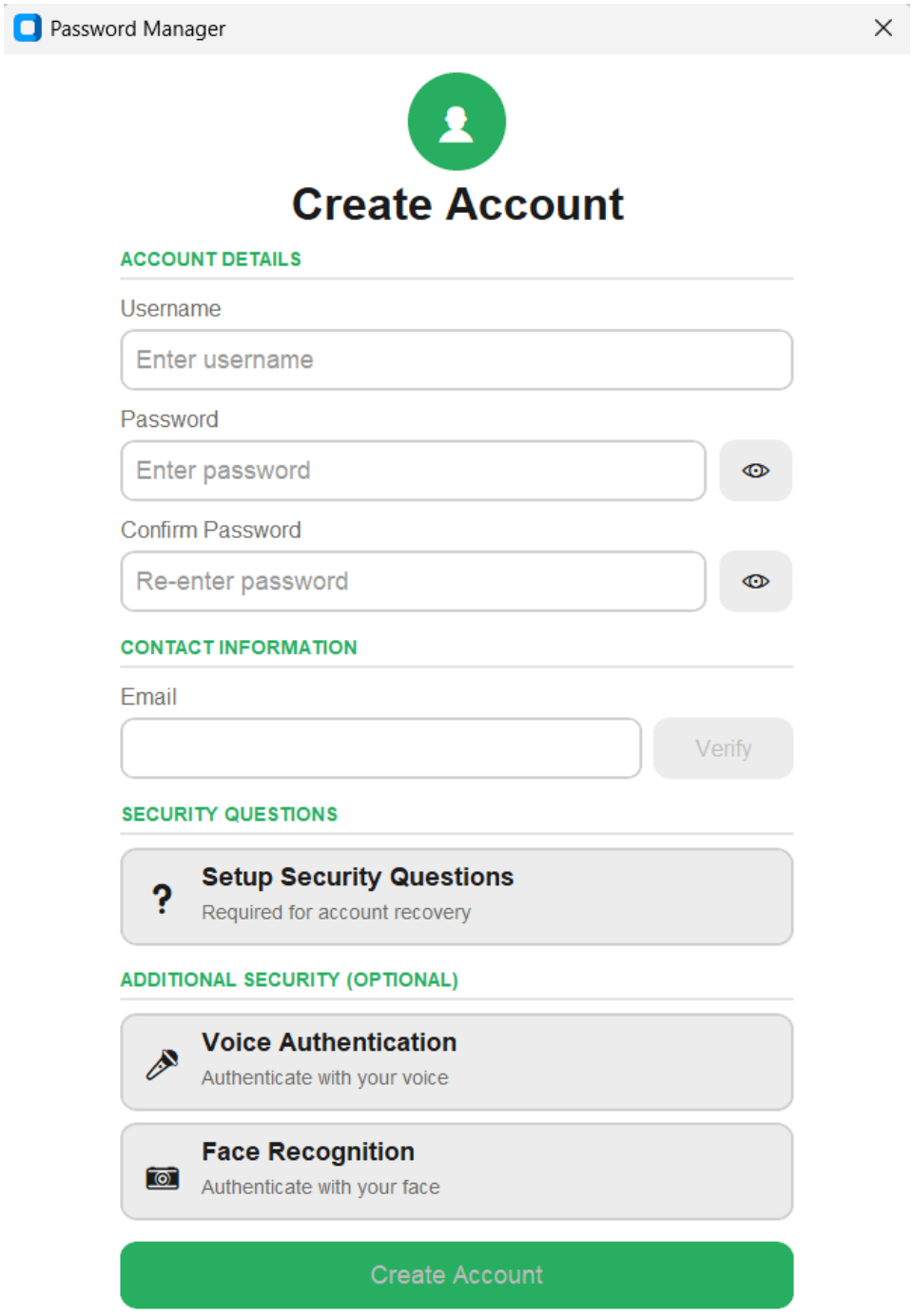
### מסך הבית

**תיאור המסך**: מסך הבית של האפליקציה. המסך הראשון שהמשתמש רואה לאחר שהוא פותח את האפליקציה. ממסך הבית, המשתמש יכול להתחיל את תהליך הרישום לאפליקציה, להתחיל את תהליך ההתחברות או לשנות את תצוגת האפליקציה ל – dark mode בחלונית ההגדרות.



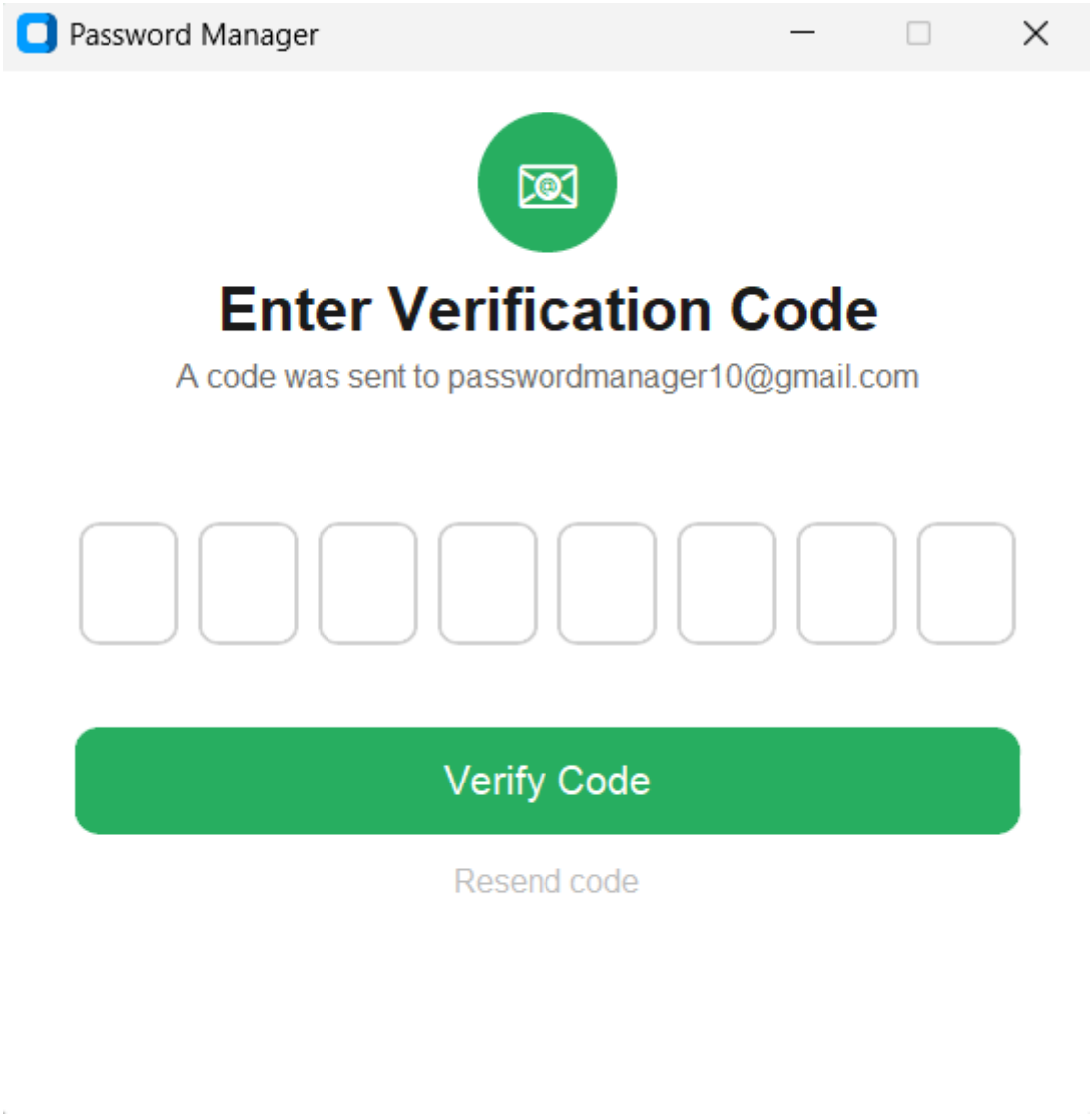
## מסך הרשמה

**תיאור המסך**: מסך ההרשמה של משתמש חדש לאפליקציה. המשתמש מכניס את הפרטים שלו (שם משתמש, סיסמה לאפליקציה, אימייל) כדי לבצע את הרשמת המשתמש.
המשתמש בנוסף מחויב להגדיר את שאלות האבטחה שלו כאמצעי הזדהות חלופי אם הוא שכח את הסיסמה שלו לאפליקציה (אחרי לחיצה על כפתור setup security question, נפתח חלון הגדרת שאלות האבטחה של המשתמש.
המשתמש יכול להגדיר את זיהוי הקול או הפנים שלו במסך זה, או לאחר התחברות במסך הראשי (לחיצה על כפתורי הגדרת זיהוי הקול או הפנים יפתחו את מסכי הקמת זיהוי הקול או הפנים).



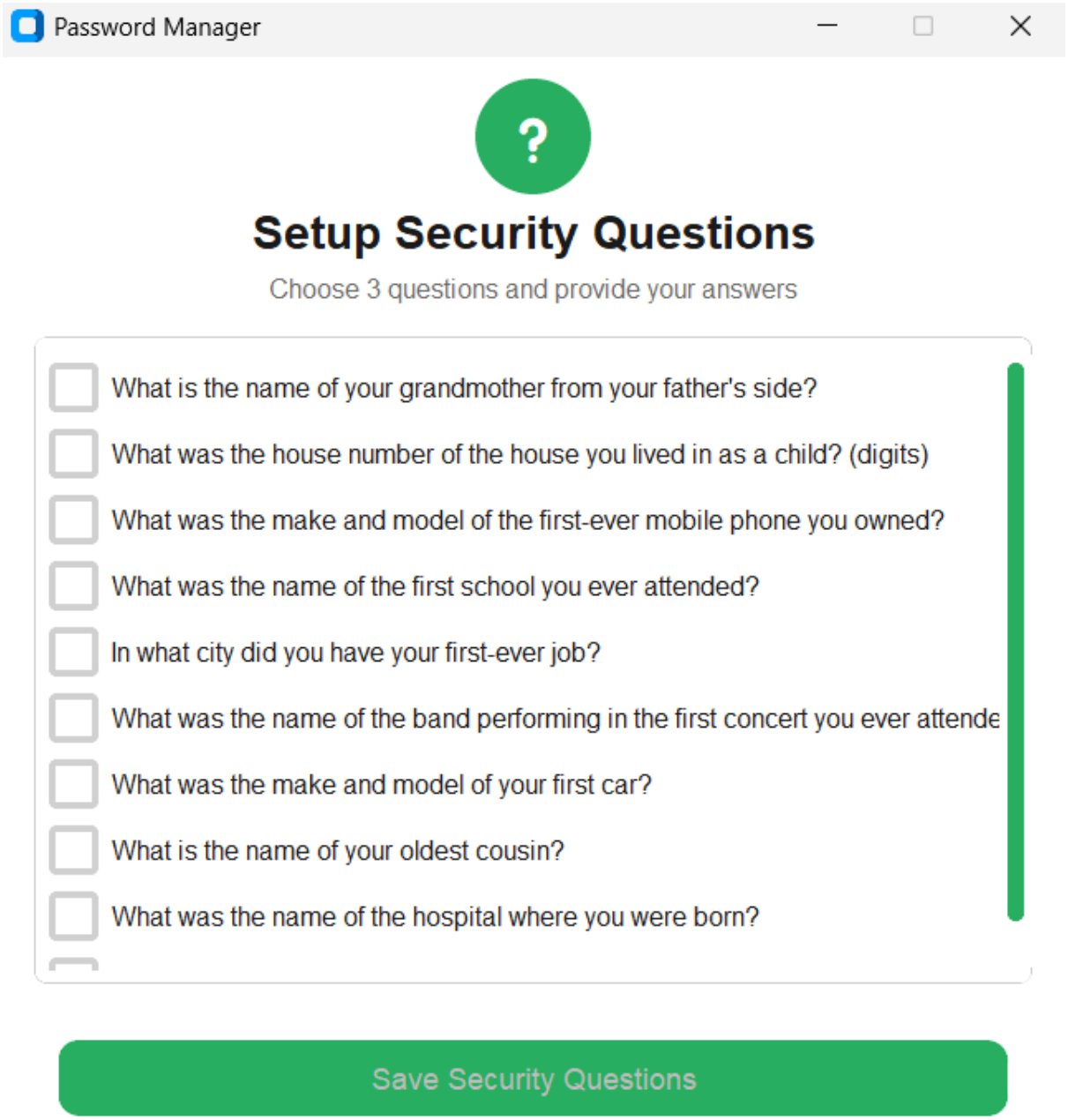
## מסך הזנת קוד לאימייל

**תיאור המסך**: המסך נפתח אחרי שנשלח קוד חד פעמי לאימייל של המשתמש. המשתמש נדרש להכניס את הקוד שהתקבל באימייל שלו על מנת לאמת את כתובת האימייל שלו או להזדהות. אם עבר מספיק זמן והקוד פג תוקף, המשתמש יכול לבקש שליחה מחדש של קוד נוסף.



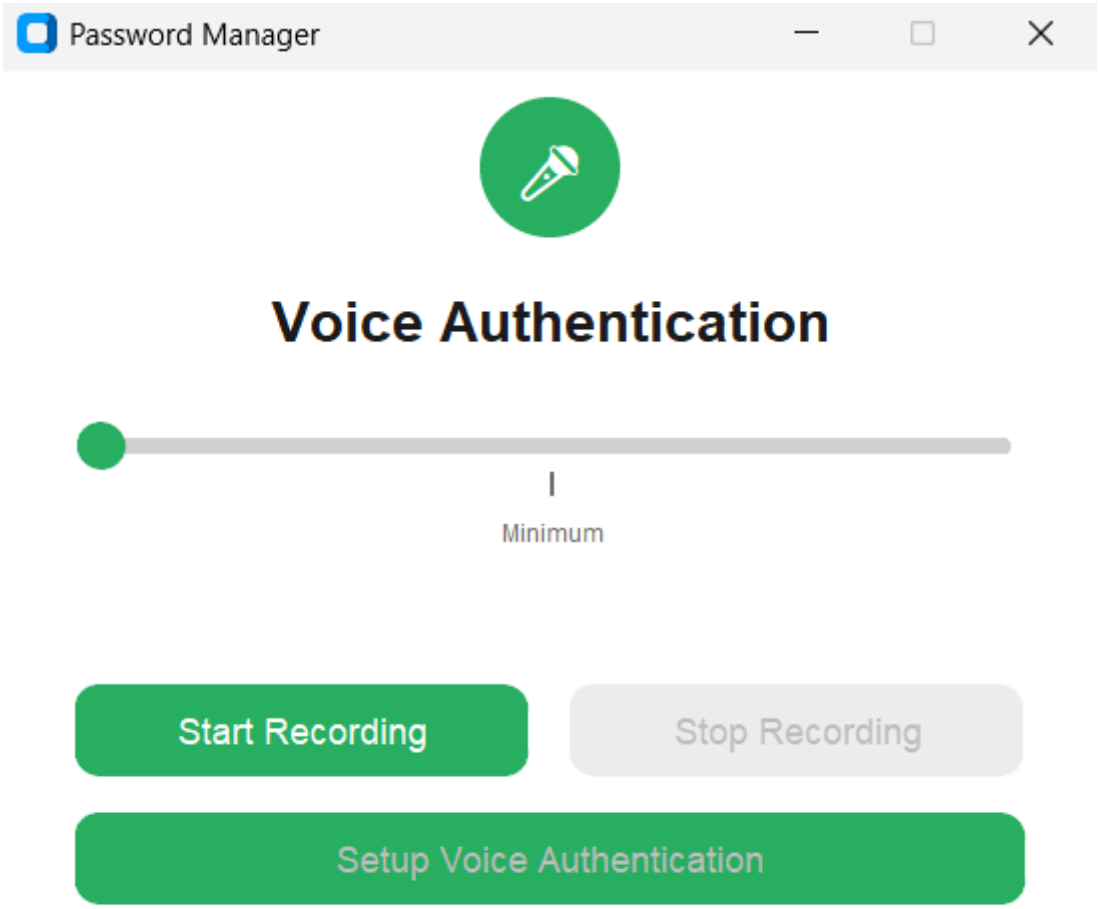
## מסך הגדרת השאלות של המשתמש

**תיאור המסך**: מסך הגדרת שאלות האבטחה של המשתמש (חלק חובה בתהליך ההרשמה).
המשתמש צריך לבחור שלוש שאלות מתוך הרשימה ולענות עליהן על עצמו.



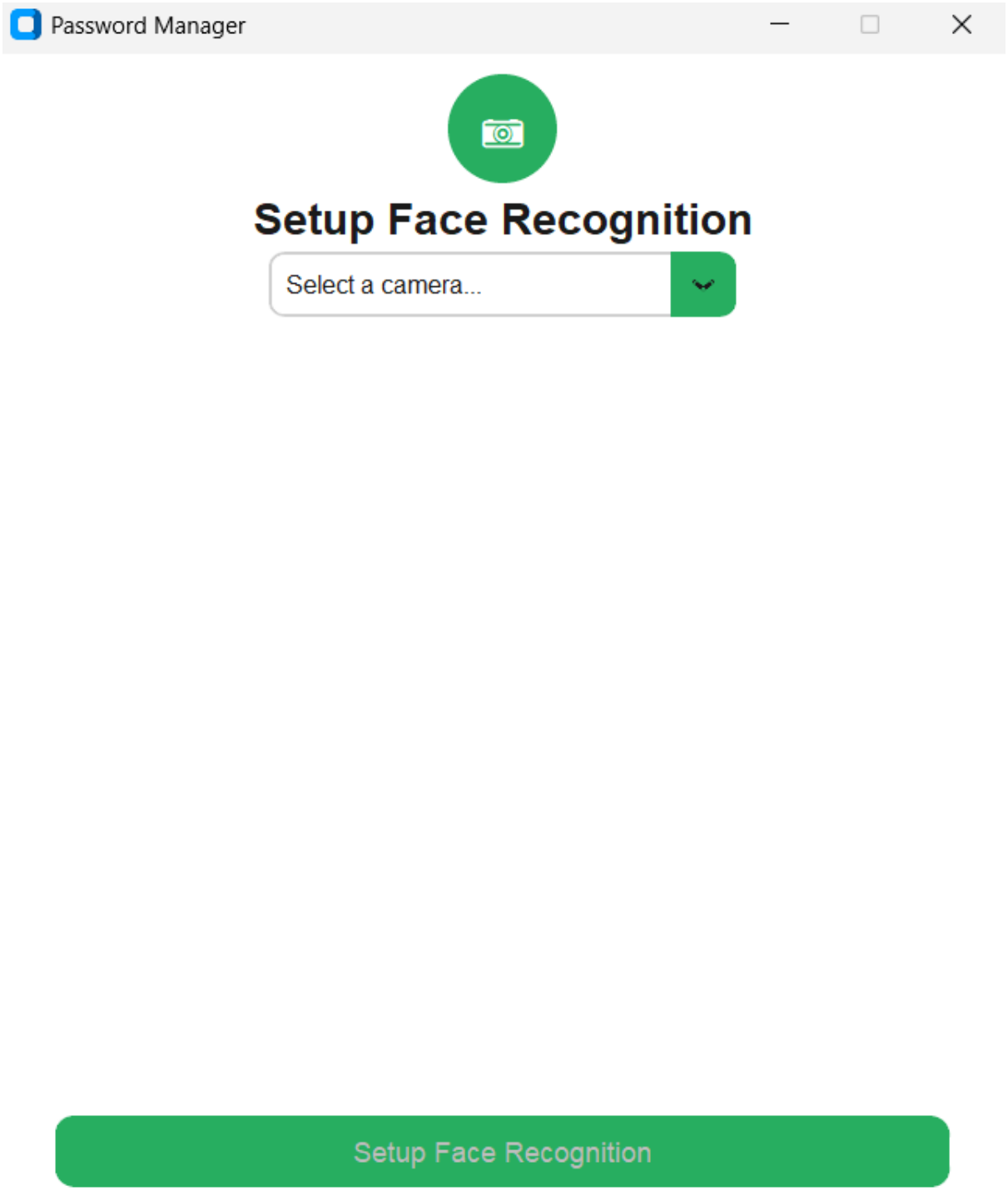
## מסך הקמת זיהוי הקול

**תיאור המסך**: מסך הקמת זיהוי הקול (חלק רשות בתהליך ההרשמה או מתוך חלונית ההגדרות). המשתמש לוחץ על כפתור start recording ומקליט את עצמו מדבר במשך כמה שניות. אם המערכת הצליחה להפיק את חתימת הקול של המשתמש, הוא יכול ללחוץ על כפתור setup voice authentication בשביל להקים את זיהוי הקול. אם הפקת חתימת הקול של המשתמש נכשלה, מוצגת הודעת שגיאה והמשתמש נדרש להקליט את עצמו שוב.



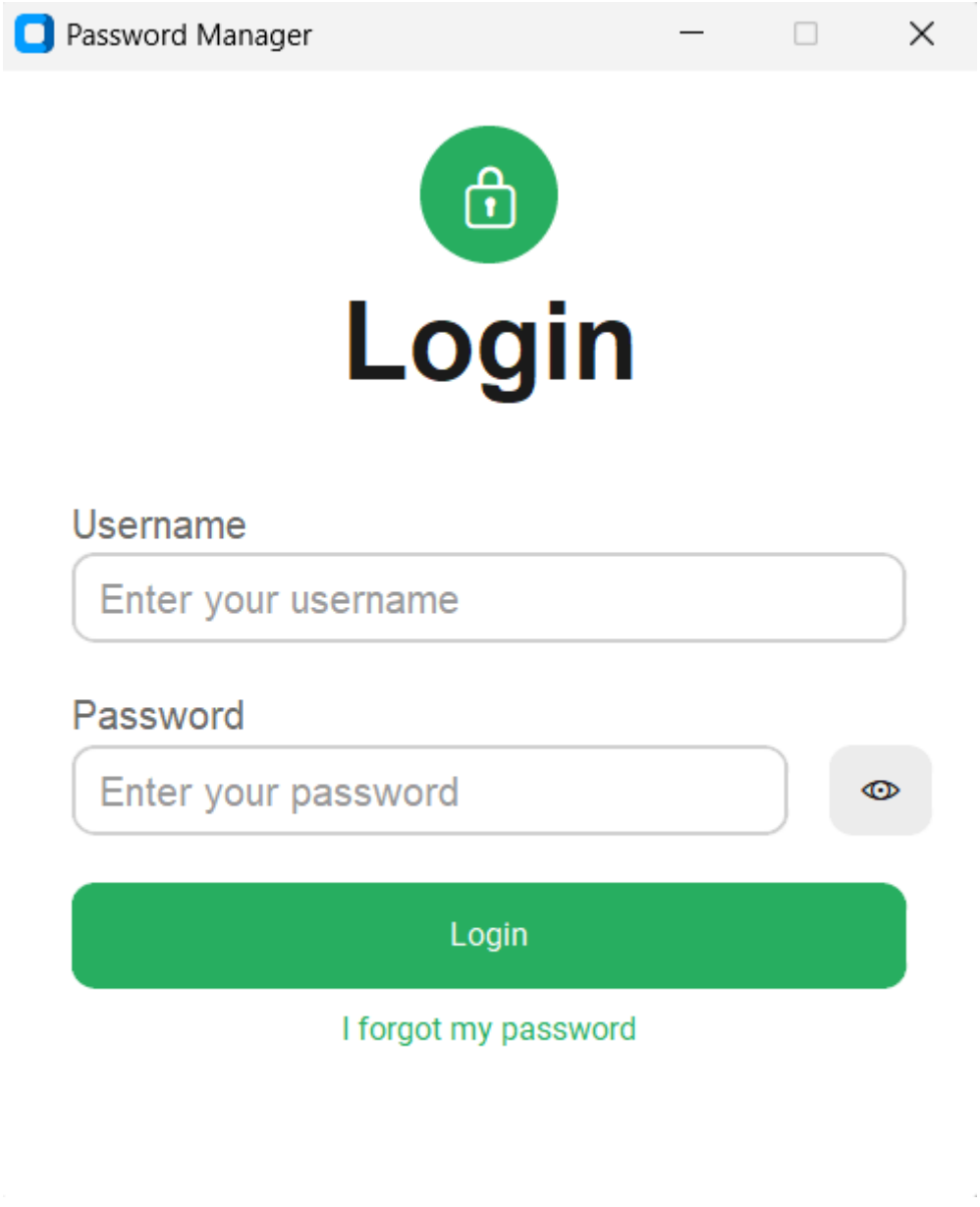
## מסך הקמת זיהוי הפנים

**תיאור המסך**: מסך הקמת זיהוי הפנים (חלק רשות בתהליך ההרשמה או מתוך חלונית ההגדרות).
המשתמש בוחר מצלמה מהמצלמות המחוברות לו למחשב, ומופיע במסך הוידאו שהמצלמה קולטת.
כאשר מערכת זיהוי הפנים מזהה פנים בתמונה ומצליחה להפיק את נתוני הפנים, כפתור setup face recognition מופעל והמשתמש יכול ללחוץ עליו כדי להשלים את הקמת זיהוי הפנים.



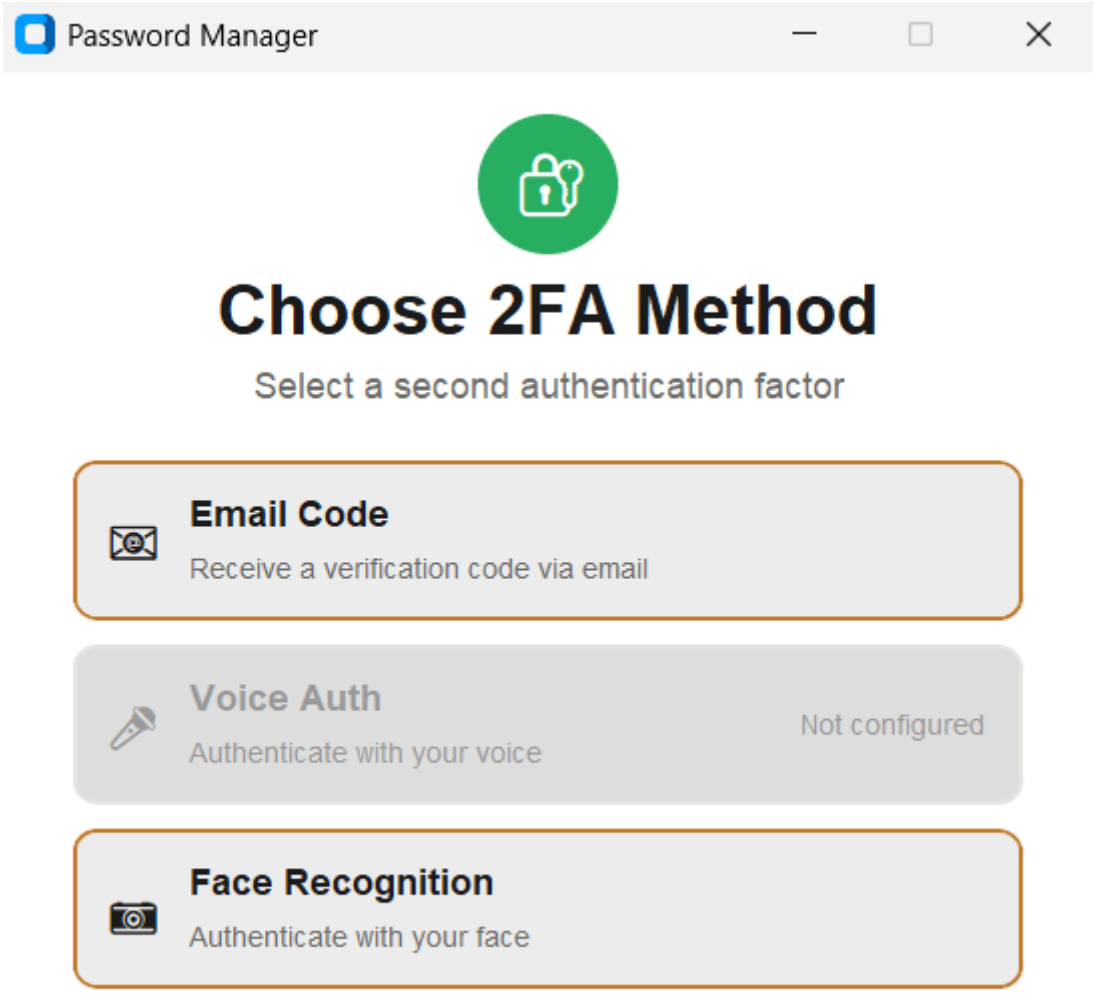
## מסך התחברות

**תיאור המסך**: מסך ההתחברות לאפליקציה עבור משתמשים שכבר נרשמו בעבר. המשתמש מכניס את שם המשתמש שלו ואת הסיסמה שלו לאפליקציה. הסיסמה מוצגת בצורה נסתרת, עם אפשרות להציגה עם הכפתור. אם המשתמש הכניס פרטי התחברות לא נכונים, מוצגת הודעת שגיאה. אם המשתמש שכח את הסיסמה שלו לאפליקציה, הוא יכול להכניס את שם המשתמש שלו וללחוץ על כפתור שכחתי סיסמה, וכך להתחיל את תהליך איפוס הסיסמה.



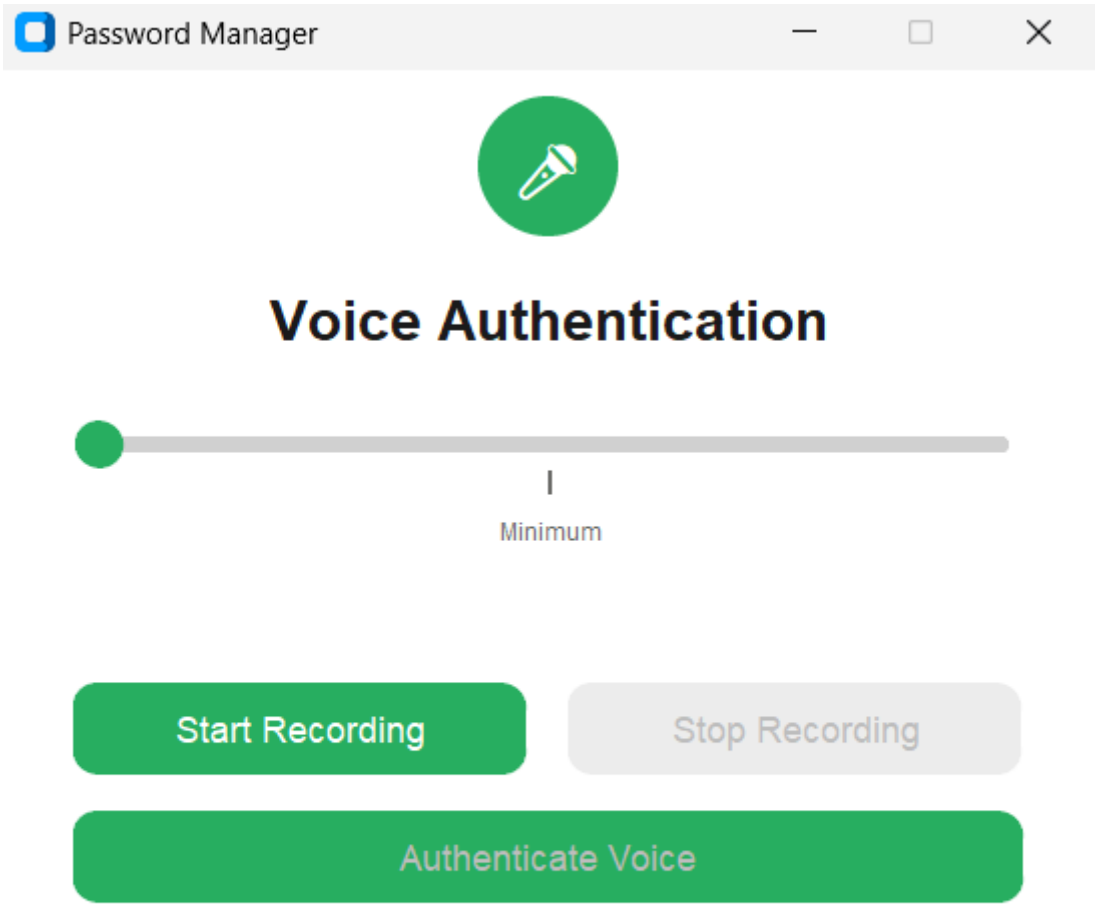
### מסך בחירת פקטור ההזדהות השני

**תיאור המסך**: לאחר התחברות עם שם משתמש וסיסמה, נפתח מסך בחרית פקטור ההזדהות השני. במסך זה, המשתמש יכול לבחור בין אפשרויות הפקטור השני להתחברות שמוגדרים אצלו במשתמש ולהתקדם לפקטור ההזדהות השני שבחר (כמו שרואים בתמונה, במשתמש מוגדרים ההתחברות עם קוד לאימייל וזיהוי פנים, איתם הוא יכול להתחבר. זיהוי קול לא מוגדר במשתמש ולכן הוא לא יכול לבחור אותו).



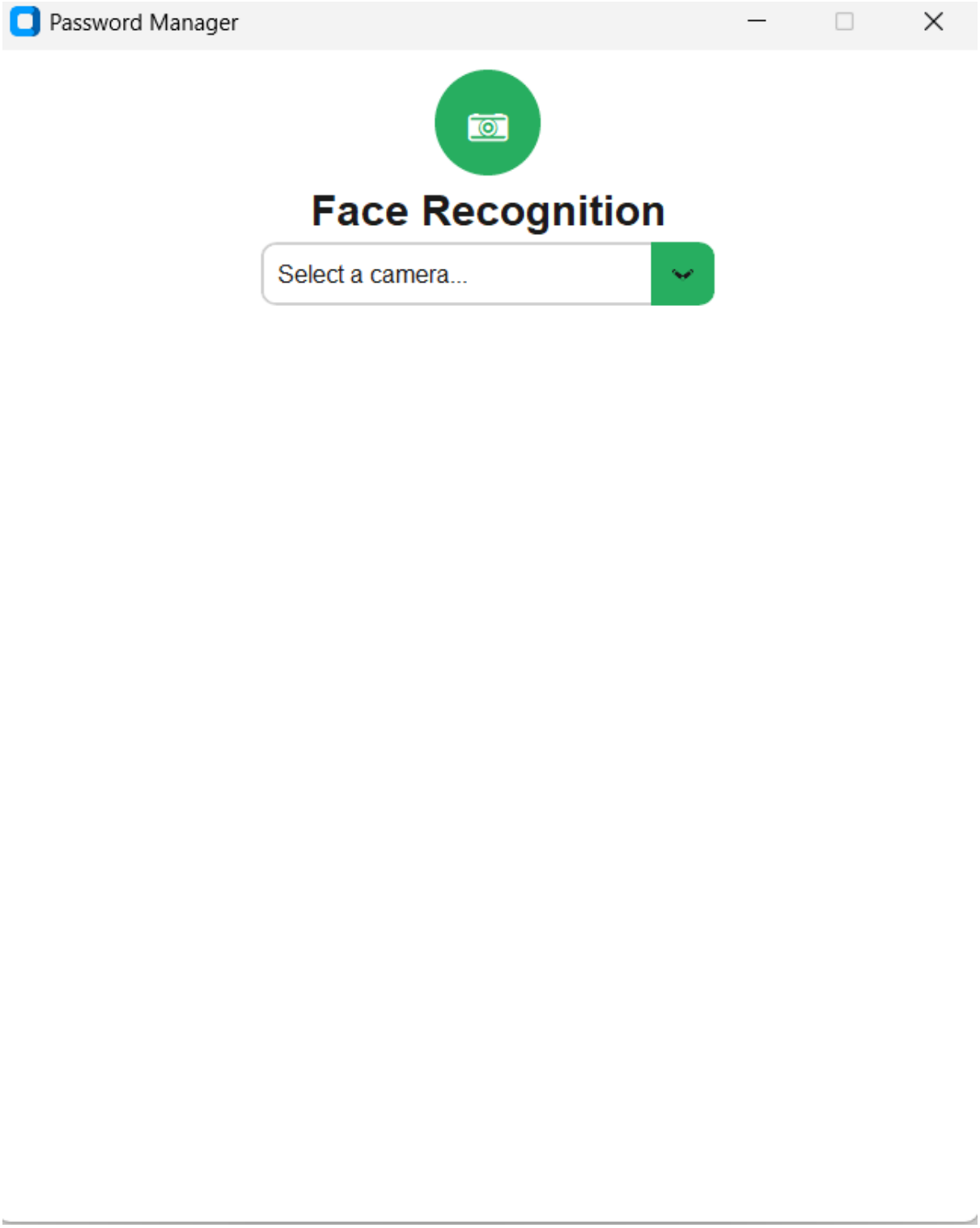
## מסך הזדהות עם זיהוי קול

**תיאור המסך**: מסך ההזדהות עם זיהוי הקול כפקטור שני בתהליך ההתחברות לאפליקציה. המשתמש לוחץ על כפתור start recording ומקליט את עצמו מדבר במשך כמה שניות. אם המערכת הצליחה להפיק את חתימת הקול של המשתמש, הוא יכול ללחוץ על כפתור authenticate voice בשביל לנסות להזדהות עם חתימת הקול שהופקה. אם חתימת הקול הנוכחית דומה מספיק לחתימת הקול השמורה למשתמש, המשתמש זוהה וההתחברות צלחה. אם הפקת חתימת הקול של המשתמש נכשלה או שחתימת הקול שהופקה לא דומה מספיק לחתימת הקול השמורה, מוצגת הודעת שגיאה והמשתמש נדרש להקליט את עצמו שוב.



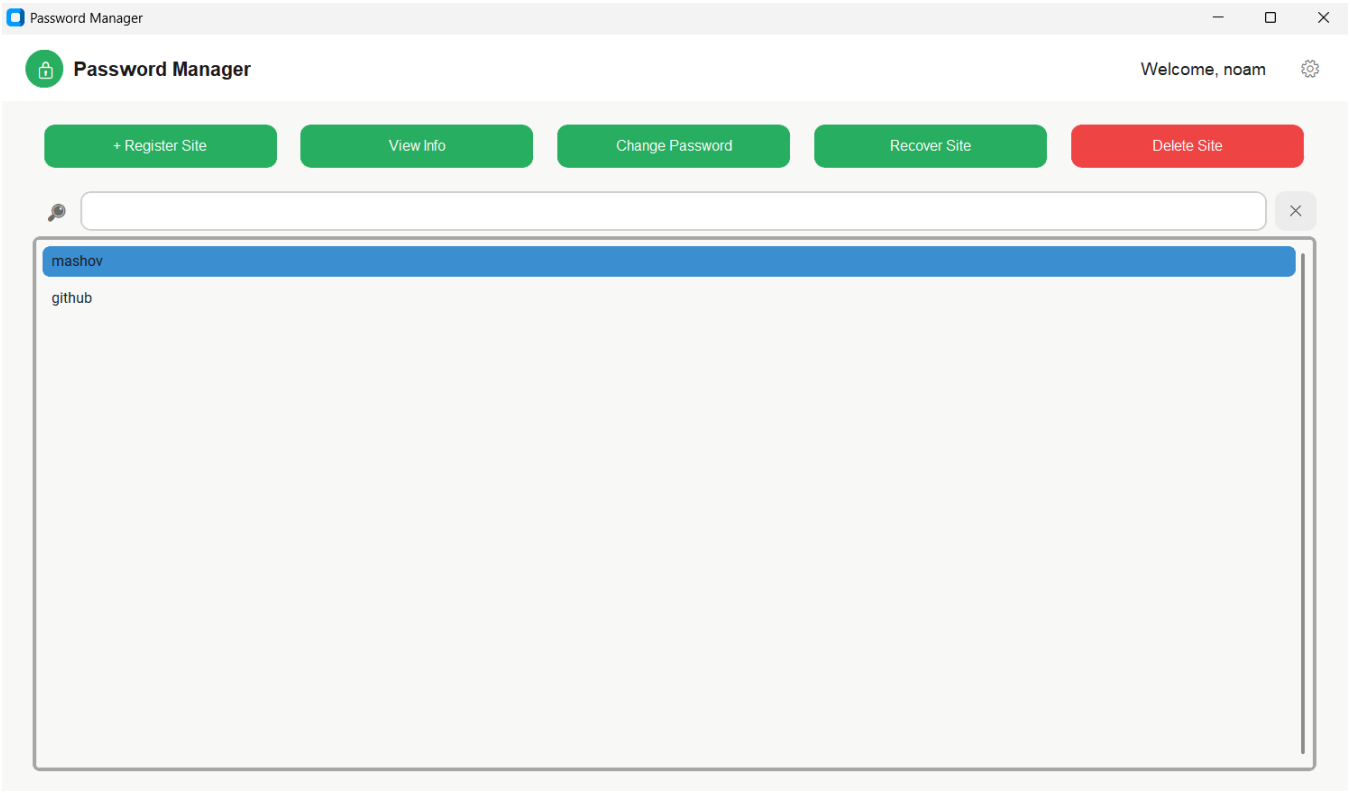
## מסך הזדהות עם זיהוי פנים

**תיאור המסך**: מסך הזדהות עם זיהוי פנים כפקטור שני בתהליך ההתחברות לאפליקציה.
המשתמש בוחר מצלמה מהמצלמות המחוברות לו למחשב, ומופיע במסך הוידאו שהמצלמה קולטת.
כאשר מערכת זיהוי הפנים מזהה פנים בתמונה ומצליחה להפיק את נתוני הפנים, היא משווה את נתוני הפנים שהופקו לנתוני הפנים השמורים למשתמש. אם נתוני הפנים שהופקו דומים מספיק, המשתמש זוהה וההתחברות צלחה. אם נתוני הפנים שהופקו לא דומים מספיק, מוצגת הודעת שגיאה שאומרת שהפנים לא תואמות את הפנים השמורות למשתמש.



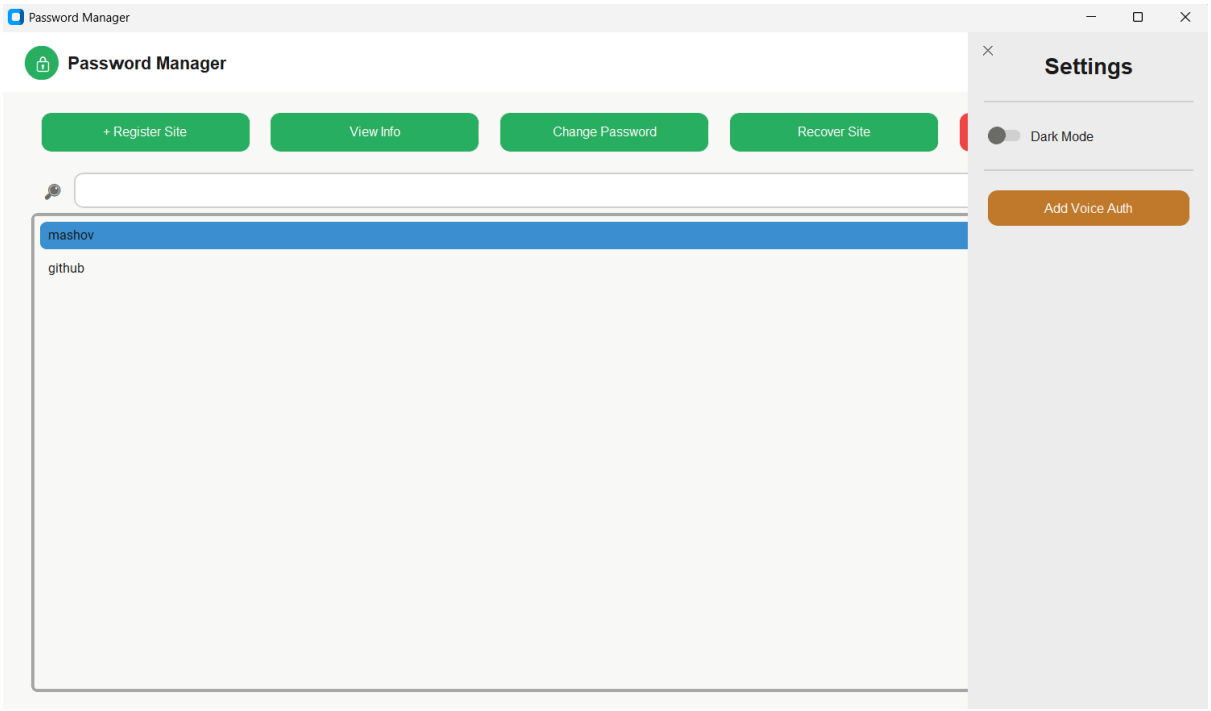
## המסך הראשי (לאחר התחברות)

**תיאור המסך**: לאחר התחברות, הרשמה או איפוס סיסמה, נפתח המסך הראשי של האפליקציה.
במסך הראשי ניתן לראות את רשימת האתרים השמורים של המשתמש, ולעשות עליהם פעולות.
ממסך זה המשתמש יכול להוסיף אתר חדש, לצפות במידע על האתרים הרשומים, לשנות סיסמה לאתר רשום, למחוק אתר מרשימת האתרים השמורים ולשחזר אתר מחוק.
בנוסף, המשתמש להיעזר בשדה החיפוש כדי למצוא אתר מסוים מרשימת האתרים השמורים.



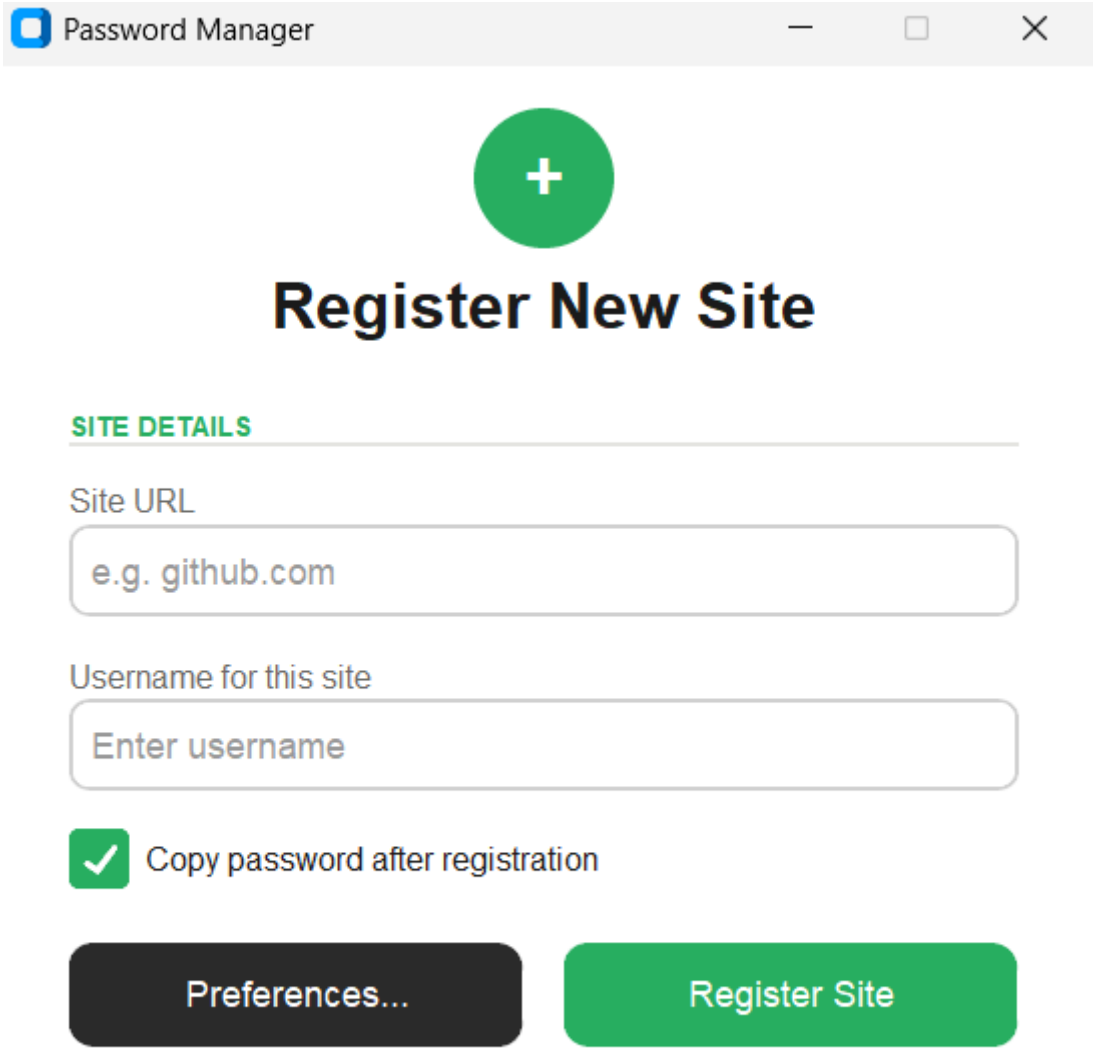
## חלונית הגדרות במסך הראשי

**תיאור המסך**: כאשר לוחצים על כפתור ההגדרות במסך הראשי, נפתחת חלונית ההגדרות. בחלונית ההגדרות המשתמש יכול לבחור מצב dark mode לתצוגת האפליקציה, והגדיר פקטורי הזדהות שעדיין לא הוגדרו עבור המשתמש באפליקציה (להוסיף זיהוי קול או פנים).



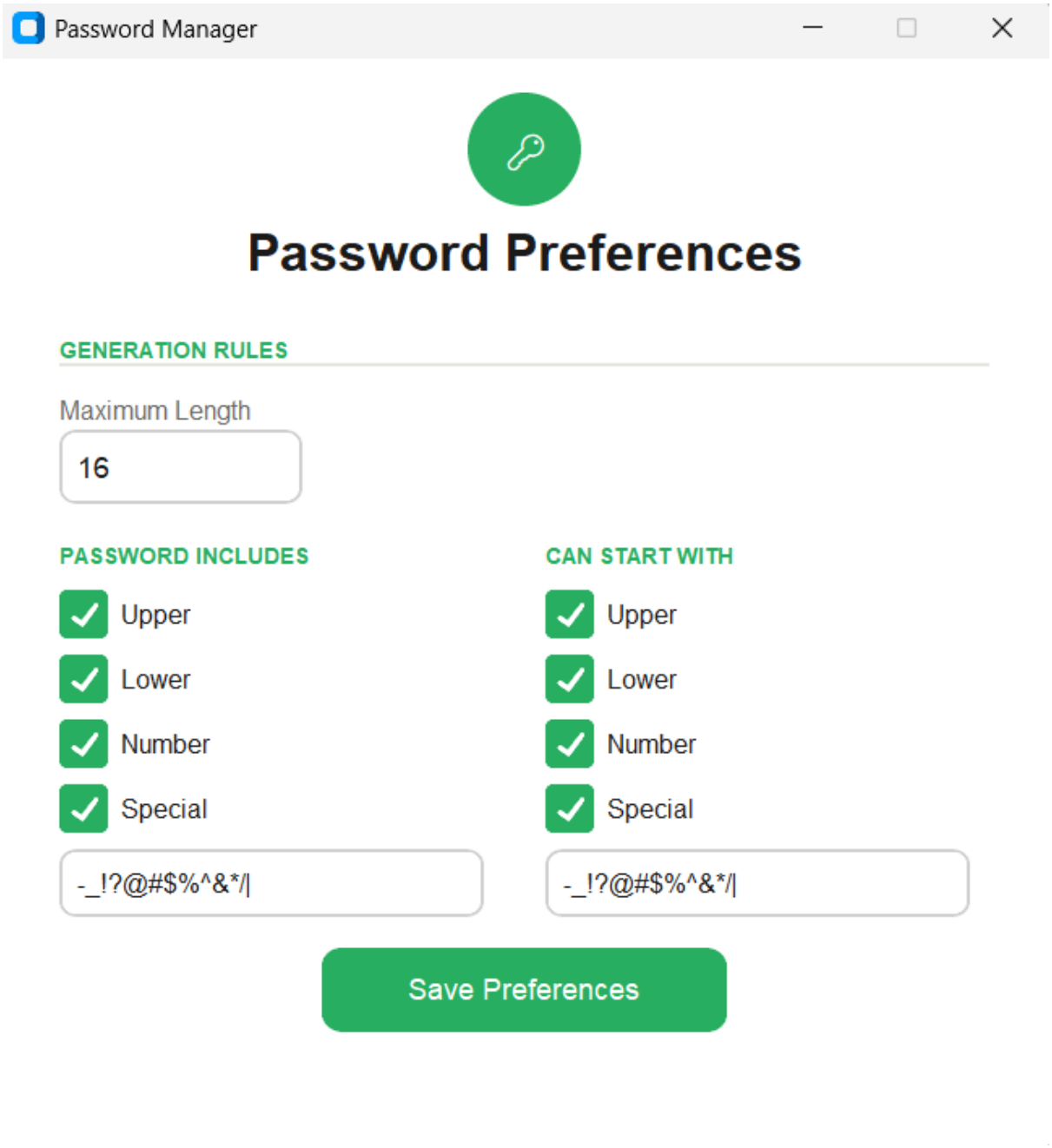
## מסך רישום אתר

**תיאור מסך**: מסך רישום אתר חדש למשתמש.
המשתמש מכניס את כתובת האתר החדש, ושם משתמש לאתר. המשתמש יכול לבחור לפתוח את מסך העדפות יצירת הסיסמה כדי לשנות את הסיסמה שתיווצר עבור האתר ברישום, והוא יכול לבחור האם להעתיק את הסיסמה של האתר החדש ל – clipboard של המחשב לאחר יצירתה.



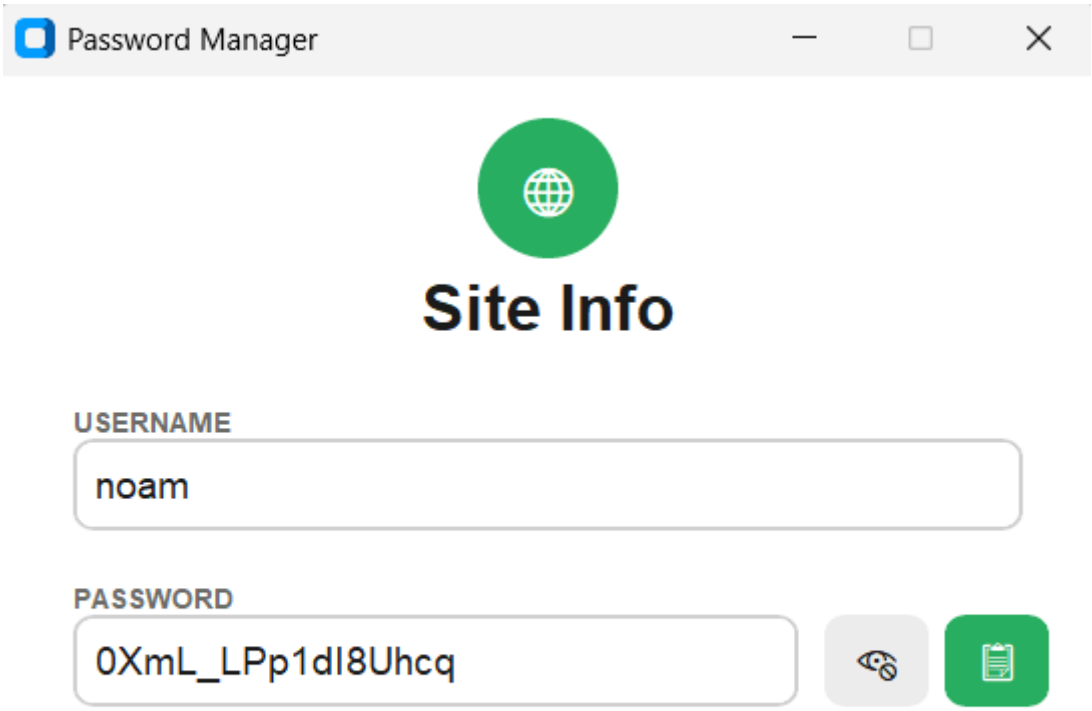
## מסך העדפות ליצירת הסיסמה

**תיאור המסך**: מסך קביעת העדפות ליצירה / החלפה של סיסמה עבור אתר מסוים.
במסך זה, המשתמש יכול לקבוע את האורך המקסימלי של הסיסמה שתיווצר, איזה תווים יכללו בה ובאילו תווים היא יכולה להתחיל. אחרי שמירת ההעדפות, הסיסמה שתיווצר תהיה לפי ההעדפות שנקבעו.



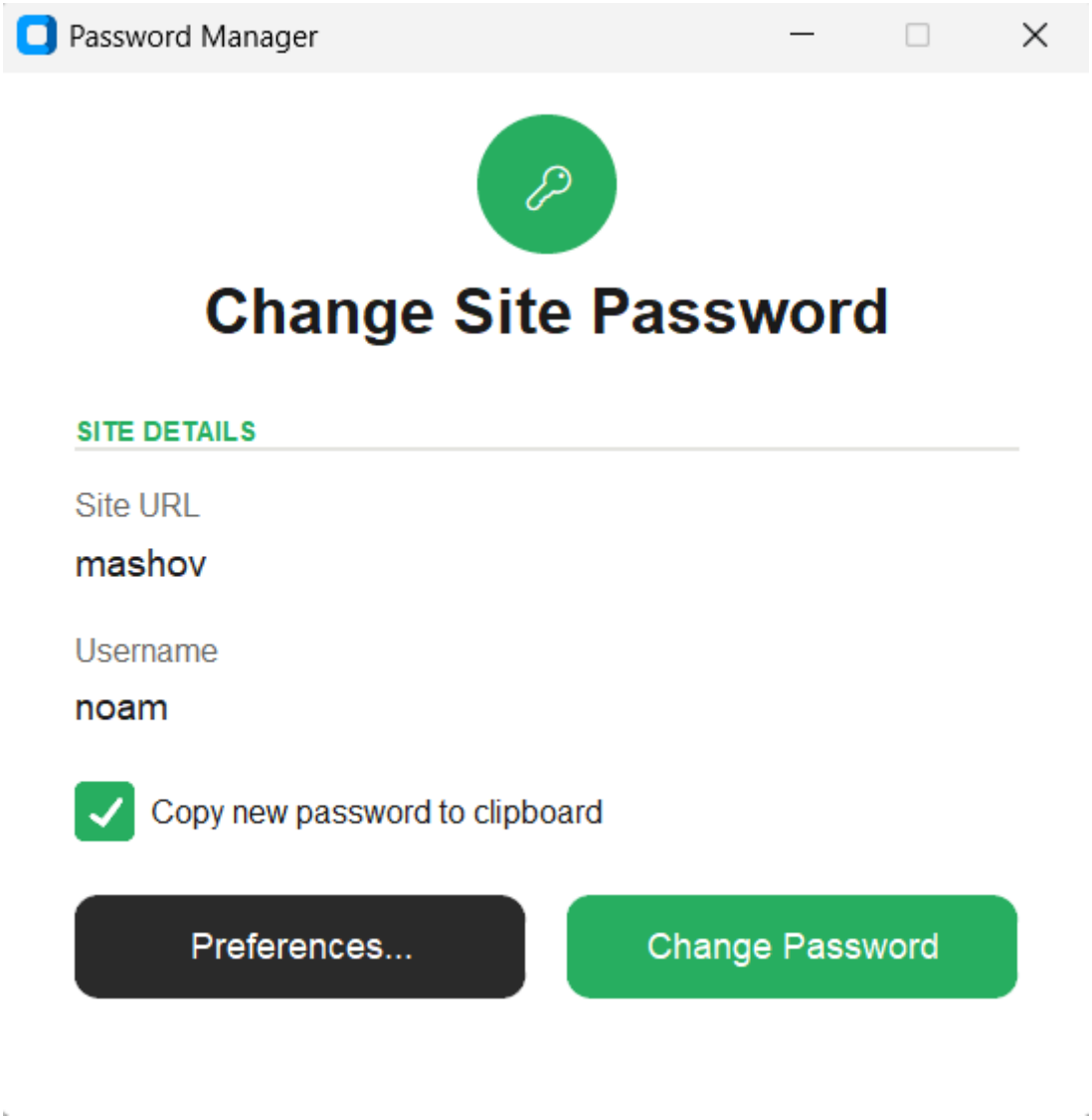
## מסך מידע על אתר רשום

**תיאור המסך**: מסך צפייה במידע על אתר רשום.
במסך מוצג שם המשתמש השמור לאתר המסוים, והסיסמה השמורה עבורו. הסיסמה מוצגת בצורה נסתרת, עם אפשרות להציגה עם הכפתור. בנוסף, ניתן להעתיק את הסיסמה ל – clipboard של המחשב באמצעות לחיצה על כפתור ההעתקה (הכפתור הירוק).



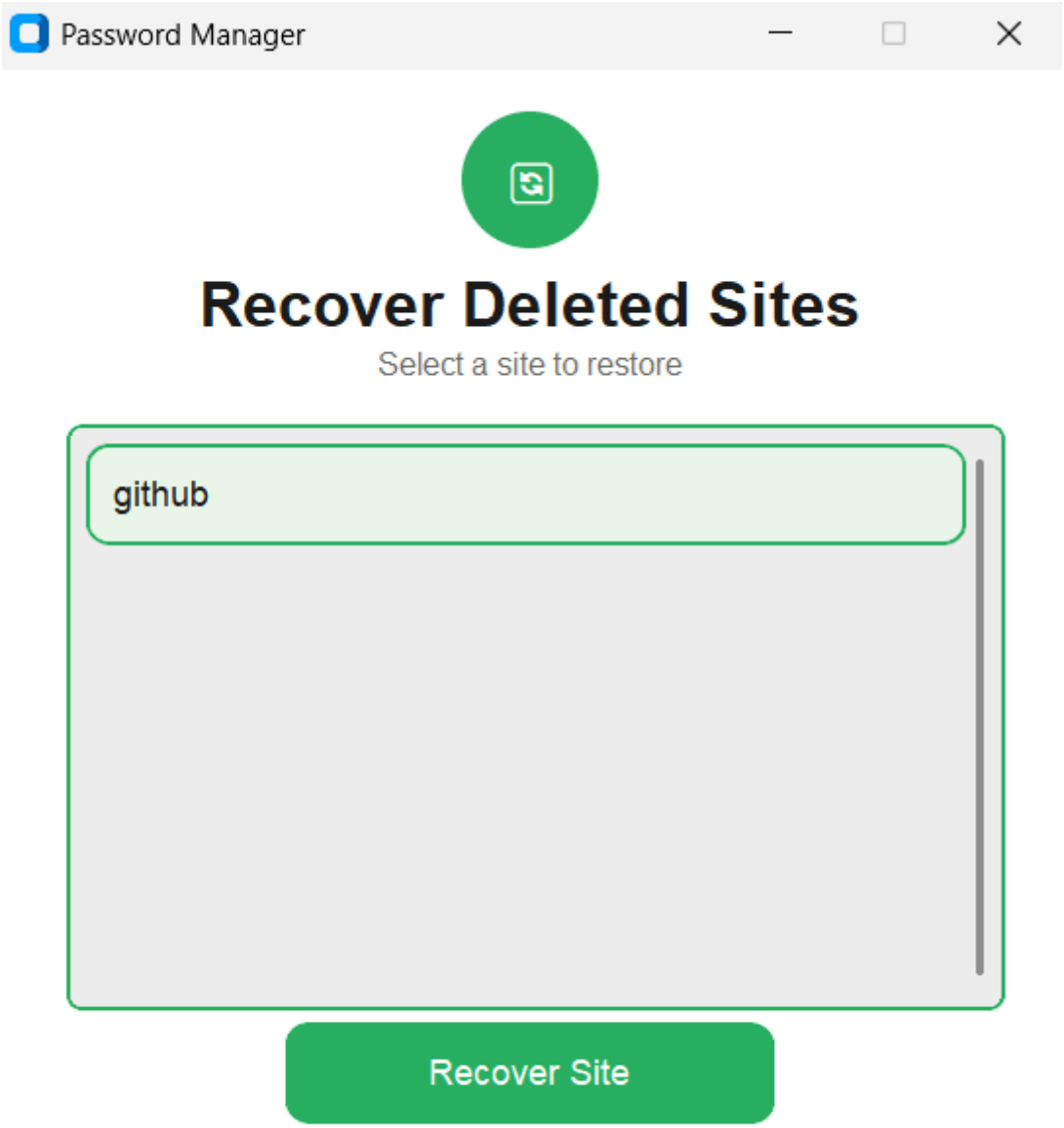
## מסך שינוי סיסמה לאתר רשום

**תיאור המסך**: מסך שינוי הסיסמה השמורה לאתר רשום.
במסך מוצג כתובת האתר הרשום ושם המשתמש לאתר. המשתמש יכול לפתוח את מסך ההעדפות ליצירת הסיסמה ולשנות אותן, והוא יכול לבחור האם להעתיק את הסיסמה ל – clipboard אחרי יצירתה. כשהמשתמש לוחץ על change password, נוצרת הסיסמה החדשה לפי ההעדפות של המשתמש והיא נשמרת עבור האתר.



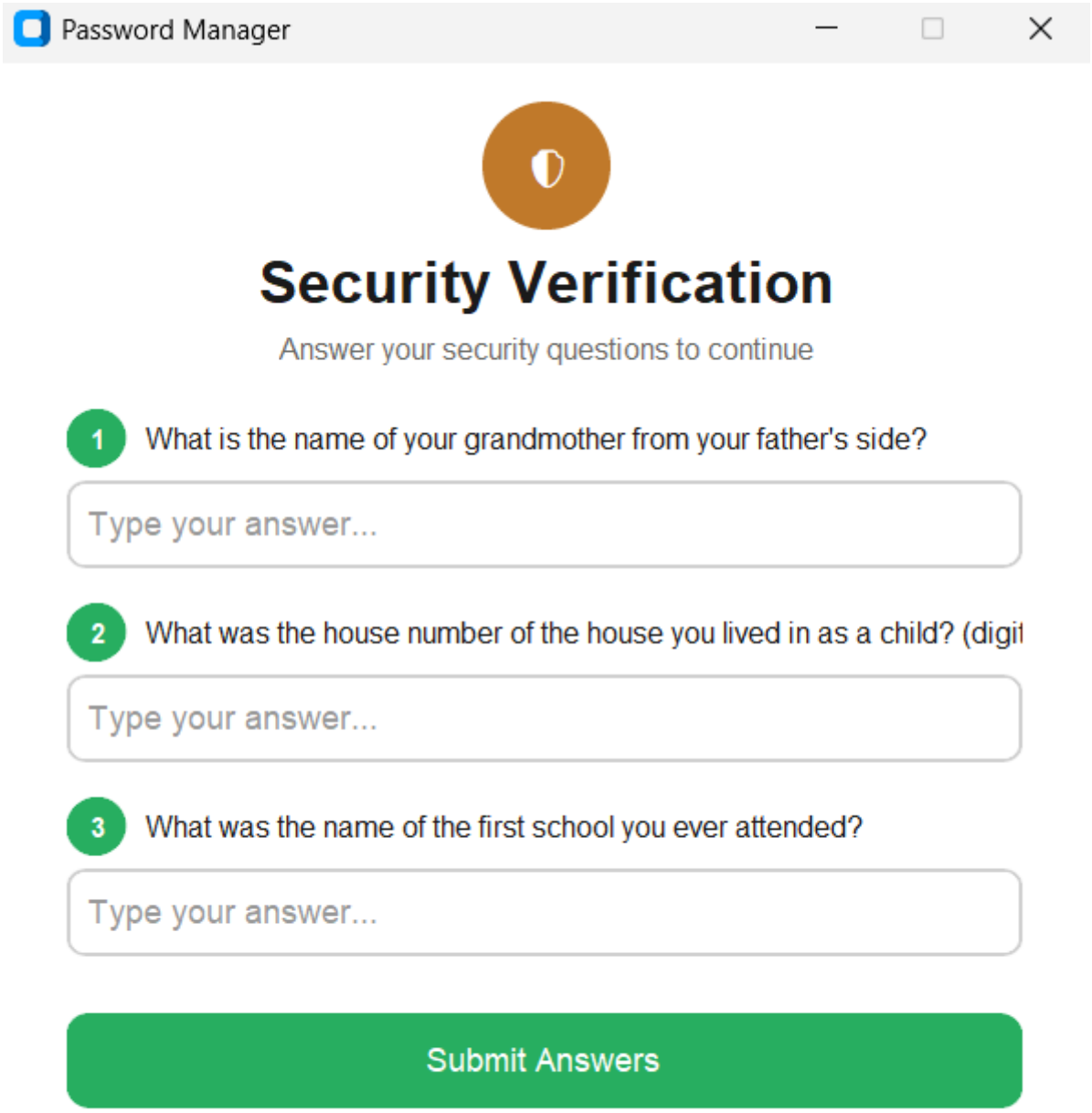
## מסך שחזור אתר

**תיאור המסך**: מסך שחזור אתרים שנמחקו מרשימת האתרים הרשומים.
המשתמש בוחר אתר לשחזור מתוך רשימת האתרים שנמחקו, ואחרי שהוא לוחץ על כפתור recover site, האתר משוחזר לרשימת האתרים הרשומים במסך הראשי.



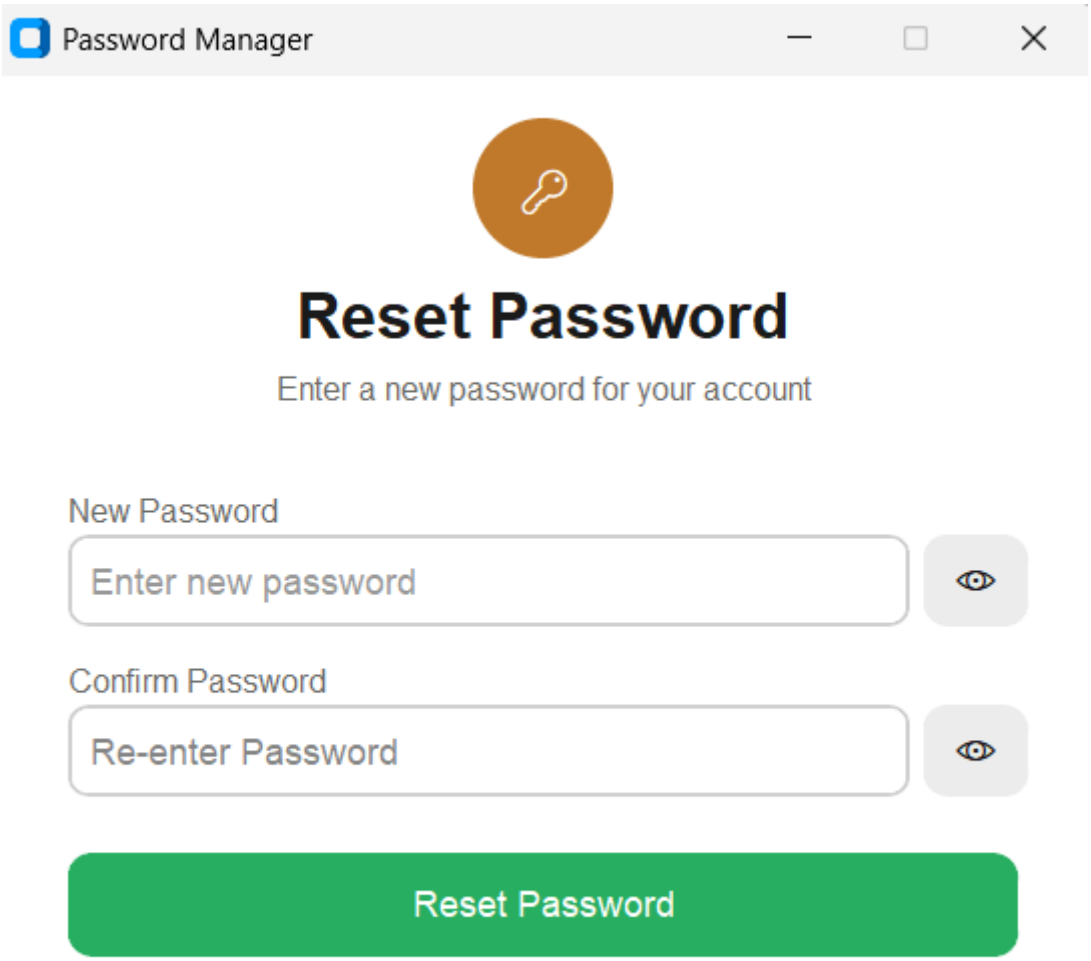
## מסך הזנת תשובות לשאלות המשתמש

**תיאור המסך**: מסך הזנת תשובות לשאלות האבטחה של המשתמש – השלב הראשון בתהליך איפוס הסיסמה כאשר המשתמש שכח את הסיסמה שלו לאפליקציה.
המשתמש עונה על השאלות שהוא בחר ונתן תשובה בשבילן בתהליך ההרשמה. אם התשובות של המשתמש תואמות את התשובות השמורות מתהליך ההרשמה, המשתמש יכול לעבור לפקטור הזדהות שני ולהמשיך את תהליך איפוס הסיסמה. אם התשובות לא תואמות, מתקבלת הודעת שגיאה והמשתמש יכול לנסות שוב לענות על השאלות.



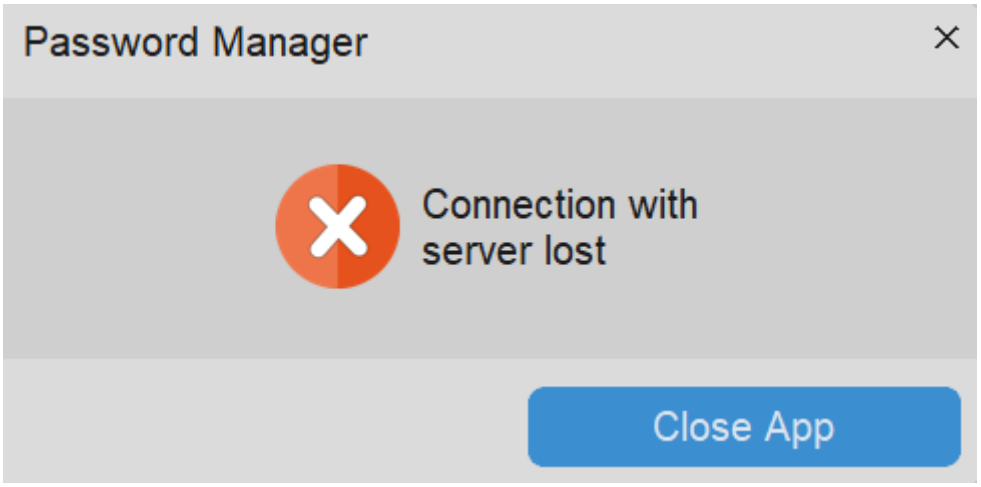
## מסך איפוס סיסמה

**תיאור המסך**: מסך איפוס הסיסמה של המשתמש לאפליקציה.
המשתמש מכניס סיסמה חדשה לאפליקציה. אם הסיסמה החדשה חזקה מספיק, הסיסמה של המשתמש לאפליקציה מוחלפת בסיסמה החדשה שהוזנה, והמשתמש מחובר אוטומטית לאפליקציה. אם הסיסמה לא חזקה מספיק, מוצגת הודעת שגיאה והמשתמש מתבקש להכניס סיסמה אחרת.

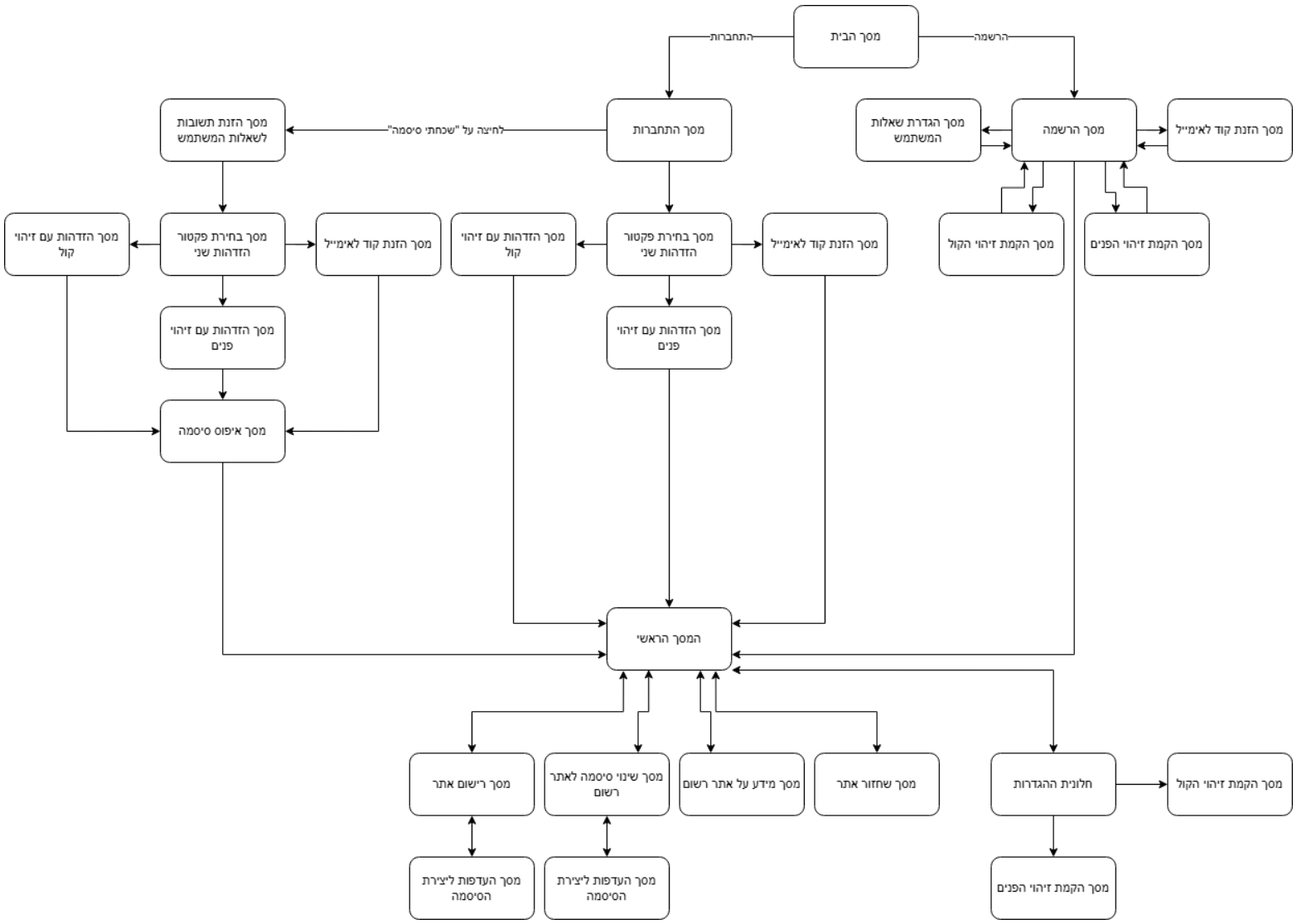


## מסך ניתוק מהשרת

**תיאור המסך**: מסך שמופיע כאשר אבד החיבור של הלקוח לשרת. המסך מציג הודעה המדווחת על איבוד החיבור עם השרת. איבוד חיבור יכול לקרות גם אם משתמש יוצר עומס על השרת שמוביל לניתוק התקשורת.
לחיצה על כפתור Close App תסגור את כל מסכי האפליקציה עבור המשתמש.



## תרשים מסכי המערכת



## מבני הנתונים

בפרויקט מנהל הסיסמאות יש מסד נתונים, ומבני נתונים בזיכרון (כגון רשימות ומילונים) החשובים לכמה יכולות שונות.

### מסד הנתונים

**שם מסד הנתונים**: PasswordManager.db

**טבלת USERS**

| שם השדה | טיפוס | תיאור השדה | דוגמה לערך אפשרי |
|---|---|---|---|
| user_id | INTEGER | המזהה המספרי של המשתמש | 1 |
| password | TEXT | הסיסמה של המשתמש לאפליקציה (hashed) | 1649df9c9cdaf64... |
| username | TEXT | שם המשתמש של המשתמש לאפליקציה | noam |
| password_salt | BLOB | ה – salt של hash הסיסמה של המשתמש | c5602ae1531c48... |
| email | BLOB | כתובת האימייל של המשתמש (מוצפן) | gAAAAAABp0R4u... |
| phone_number | BLOB | מספר הטלפון של המשתמש (מוצפן) | gAAAAAABp0R4u... |
| user_dk | BLOB | מפתח הצפנת המידע של המשתמש בבסיס הנתונים (מוצפן על ידי מפתח אחר) | gAAAAAABp0R4u... |
| key_salt | BLOB | ה – salt של יצירת מפתח הצפנת ה dk | a3bd42a890963f... |
| voice_signature | BLOB | הקוד הבינארי של הוקטור בעל ה 128 מימדים של טביעת הקול (מוצפן) | NULL / gAAAAAA... |
| face_encoding | BLOB | הקוד הבינארי של הוקטור בעל ה 128 מימדים של נתוני הפנים (מוצפן) | NULL / gAAAAAA... |
| question_id_1 | INTEGER | המזהה המספרי של השאלה הראשונה שעליה המשתמש בחר לענות | 1 |
| answer_1 | BLOB | התשובה של המשתמש לשאלה הראשונה שהוא בחר (hashed) | a530ff8722... |

| | | | |
|---|---|---|---|
| question_id_2 | INTEGER | המזהה המספרי של השאלה השנייה שעליה המשתמש בחר לענות | 2 |
| answer_2 | BLOB | התשובה של המשתמש לשאלה השנייה שהוא בחר (hashed) | a530ff8722... |
| question_id_3 | INTEGER | המזהה המספרי של השאלה השלישית שעליה המשתמש בחר לענות | 3 |
| answer_3 | BLOB | התשובה של המשתמש לשאלה השלישית שהוא בחר (hashed) | a530ff8722... |

המפתח הראשי של הטבלה – user_id

**טבלת PASSWORD_PREFERENCES**

| שם השדה | טיפוס | תיאור השדה | דוגמה לערך אפשרי |
|---|---|---|---|
| preferences_id | INTEGER | המזהה המספרי של ההעדפה | 1 |
| max_length | INTEGER | האורך המקסימלי של הסיסמה | 16 |
| include_upper | BOOLEAN | האם לכלול אותיות גדולות בסיסמה? | 0/1 |
| include_lower | BOOLEAN | האם לכלול אותיות קטנות בסיסמה? | 0/1 |
| include_num | BOOLEAN | האם לכלול מספרים בסיסמה? | 0/1 |
| include_special | BOOLEAN | האם לכלול תווים מיוחדים בסיסמה? | 0/1 |
| special_allowed | TEXT | איזה תווים מיוחדים ניתן לכלול בסיסמה? | -_!@#$%^&*/\| |
| can_start_with_upper | BOOLEAN | האם הסיסמה יכולה להתחיל עם אות גדולה? | 0/1 |
| can_start_with_lower | BOOLEAN | האם הסיסמה יכולה להתחיל עם אות קטנה? | 0/1 |
| can_start_with_num | BOOLEAN | האם הסיסמה יכולה להתחיל עם מספר? | 0/1 |
| can_start_with_special | BOOLEAN | האם הסיסמה יכולה להתחיל עם תו מיוחד? | 0/1 |

| | | | |
|---|---|---|---|
| special_allowed_start | TEXT | עם אילו תווים מיוחדים הסיסמה יכולה להתחיל? | -_!@#$%^&*/\| |

המפתח הראשי של הטבלה - preferences_id.

**טבלת USER_SITE**

| שם השדה | טיפוס | תיאור השדה | דוגמה לערך אפשרי |
|---|---|---|---|
| user_id | INTEGER | המזהה המספרי של המשתמש | 1 |
| site_id | INTEGER | המזהה המספרי של האתר | 0 |
| site_url | BLOB | כתובת האתר (מוצפן) | gAAAAAABp... |
| site_username | BLOB | שם המשתמש עבור האתר (מוצפן) | gAAAAAABp... |
| site_password | BLOB | הסיסמה עבור האתר (מוצפן) | gAAAAAABp... |
| preference_id | INTEGER | המזהה המספרי של העדפות יצירת הסיסמה עבור האתר | 1 |
| is_deleted | BOOLEAN | האם האתר מחוק? | 0/1 |

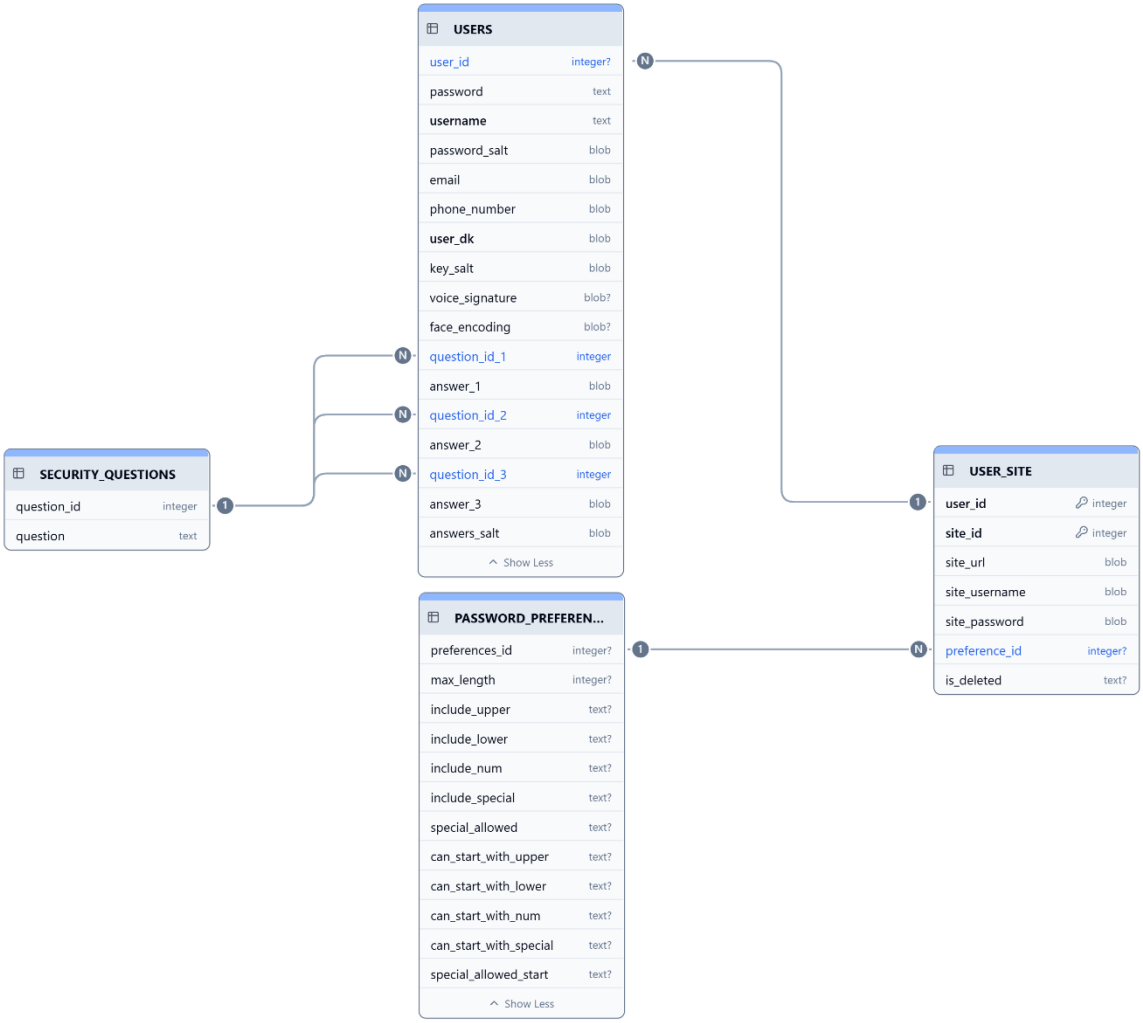
לטבלה יש מפתח ראשי משולב – (user_id, site_id)

**טבלת SECURITY_QUESTIONS**

| שם השדה | טיפוס | תיאור השדה | דוגמה לערך אפשרי |
|---|---|---|---|
| question_id | INTEGER | המזהה המספרי של השאלה | 1 |
| question | TEXT | השאלה עצמה | What is the name of your grandmother from your father's side? |

המפתח הראשי של הטבלה – question_id

## תרשים של טבלאות מסד הנתונים




## מבני הנתונים בזיכרון

ב – server.py

- conn_code – מילון ששומר את הקוד החד פעמי הרנדומלי שנשלח לאימייל של המשתמש (המפתח הוא הסוקט של הלקוח, והערך הוא הקוד הרנדומלי).

ב – user_manager.py

- logged_users – מילון ששומר את ה – user_id בבסיס הנתונים של המשתמשים המחוברים לאפליקציה ברגע נתון (המפתח הוא הסוקט של הלקוח, והערך הוא ה- user_id של הלקוח המחובר)
- user_login_attempts – מילון ששומר רשימה של זמני התחברויות כושלות עבור כל משתמש (המפתח הוא שם המשתמש, והערך הוא רשימה של זמנים של התחברויות כושלות)

- users_in_timeout – מילון ששומר את זמני הכניסה לפסק זמן עבור כל משתמש שבפסק זמן (המפתח הוא שם המשתמש, והערך הוא זמן הכניסה של המשתמש לפסק זמן)

ב – server_conn_manager.py

- client_sockets – רשימה של כל הסוקטים של הלקוחות
- sockets_to_keys – מילון השומר את מפתחות ה – AES של הלקוחות (המפתח הוא הסוקט, הערך הוא מפתח ה – AES של הלקוח)
- client_message_count – מילון ששומר את כמות ההודעות שלקוח שלח בזמן מסוים – נועד למניעת DoS (המפתח הוא הסוקט של הלקוח, הערך הוא מספר ההודעות שהלקוח שלח בפרק זמן מסוים

ב – database.py

- users_keys – מילון ששומר את מפתח ההצפנה של מפתח המידע עבור כל משתמש במקום ליצור אותו כל פעם מחדש (המפתח הוא ה- user_id של המשתמש בבסיס הנתונים, והערך הוא מפתח ההצפנה של מפתח המידע של המשתמש)

## סקירת חולשות ואיומים

למערכת של מנהל סיסמאות, שכל המטרה שלו הוא לאסוף סיסמאות רבות מהמשתמשים שלו, צפויות סכנות רבות. נקטתי בכמה אמצעי הגנה ומניעה כדי לאבטח כמה שיותר את האפליקציה ואת פרטי המשתמשים.

### שכבת האפליקציה

- **Sql injection** – התקפה שקורית כאשר משתמש מנסה להכניס קוד sql לתוך שדות קלט, וכך לקבל, לשנות או למחוק מידע מבסיס הנתונים. אני התגוננתי מההתקפה באמצעות כתיבת השאילתות לבסיס הנתונים עם פרמטרים, ולא הכנסתי את הקלט מהמשתמש ישר למחרוזת של השאילתה.
- **תהליך login אימות ווידוא** – אחת המטרות המרכזיות של האפליקציה שלי היא להבטיח תהליך הזדהות עם פקטורים מרובים ומתוחכמים שקשה לרמות. אני מצפין ומאבטח עם hash את המידע הרגיש ופרטי ההזדהות של המשתמשים שלי בבסיס הנתונים וכך מקשה על גניבת זהות ומשתמשים בידי אנשים אחרים. בנוסף, אני מונע brute force בתהליך ההזדהות באמצעות נעילה זמנית של האפליקציה בפני משתמשים שנכשלו באמצעי הזדהות מספר רב של פעמים בזמן מסוים.
- **MITM** – סכנה שמישהו מאזין בתקשורת בין השרת ללקוח וגונב מידע. כדי למנוע את זה, הצפנתי את התקשורת שלי בין השרת ללקוח בהצפנה היברידית חזקה.
- **DoS / DDos** – התקפה שהמטרה שלה היא ליצור עומס יתר על השרת כך שיקרוס או לא יוכל להעניק שירות ללקוחות. אני מונע התקפת DoS בכך שאני מגביל את כמות הבקשות שהלקוח יכול לשלוח לשרת בשנייה, ואם הוא עבר את אותה כמות, אני מנתק את התקשורת בינו לבין השרת כך שהוא לא יוכל להמשיך להכביד עליו. DDoS לעומת זאת, יכול עדיין לעבוד – אם למישהו יש אלפי בוטים מכתובות ip שונות שמתחברים בו זמנית לשרת ושולחים רק בקשה אחת כל אחד, זה יכול עדיין לפגוע בתפקוד השרת ואף להקריס אותו. השרת ינתק את הבוטים רק אם הם יעברו את מגבלת הבקשות לשנייה.

### שכבת התעבורה

- השתמשתי בפרוטוקול TCP. תהליך ההתחברות בפרוטוקול TCP בנוי עם לחיצת יד משולשת שנועדה להבטיח תקשורת אמינה:
  - הלקוח שולח בהתחלה לשרת בקשת התחברות SYN.
  - השרת מקבל את הבקשה ומשיב ללקוח בהודעת SYN-ACK. ההודעה מאשרת את בקשת ההתחברות של הלקוח ושולחת בקשת התחברות גם מצד השרת.
  - לבסוף, הלקוח שולח הודעת ACK לשרת כדי לאשר את בקשת ההתחברות שלו, ומכאן החיבור הוקם וניתן להתחיל בהעברת הנתונים.
- לפרוטוקול TCP אין הצפנה מובנית, ולכן פיתחתי הצפנה היברידית משלי בתקשורת.

# מימוש הפרויקט

## מודולים מיובאים

- base64 – ספרייה שימושית לקידוד מידע לתווי ASCII בשביל העברה בטוחה בפרוטוקולים מבוססי טקסט.
- cryptography – ספרייה המספקת כלים להצפנה ופענוח מאובטחים של נתונים. ייבאתי מהספרייה את FERNET עבור הצפנה סימטרית ו rsa עבור הצפנה א-סימטרית.
- CTkListBox – ווידג'ט מוכן של רשימה עבור ספריית customtkinter.
- CTkMessageBox – תיבת דיאלוג קופצת הניתנת להתאמה אישית עבור ספריית customtkinter.
- customtkinter – ספריית ממשק משתמש מודרנית המבוססת על Tkinter.
- cv2 – ספרייה לצילום, עיבוד וניתוח של תמונות.
- Datetime – ספרייה לניהול זמנים ותאריכים הכוללת תמיכה באזורי זמן.
- enum – ספרייה המאפשרת שימוש של Enum בפייתון (שימושי לקישור ערכים קבועים מרובים למשתנים).
- face_recognition – ספריית מציאת וזיהוי פנים בתמונות. קלה ונוחה לשימוש, אבל גם בעלת דיוק גבוה.
- hashlib – ספרייה המציעה הרבה שיטות hash שונות.
- json – ספרייה הממירה נתוני json לאובייקטים של פייתון ולהפך.
- numpy – ספרייה לחישובים מדעיים ועיבוד נתונים ומערכים. מהירה ויעילה.
- os – ספרייה המספקת שימוש לפונקציות של מערכת ההפעלה כמו עבודה עם מערכת הקבצים.
- PIL (pillow) – ספרייה לפתיחה ועיבוד של תמונות בפורמטים רבים.
- PyAudio – ספרייה להקלטה והשמעה של קבצי אודיו.
- pygrabber – ספרייה לשימוש, בחירה ושינוי הגדרות במצלמות המחוברות למחשב.
- random – ספרייה המחוללת מספרים אקראיים.
- secrets – ספרייה המייצרת מספרים אקראיים חזקים קריפטוגרפית.

- select – ספרייה המאפשרת גישה למספר ערוצי קלט-פלט (כמו סוקטים) בו זמנית.
- smtplib – ספרייה לשליחת אימיילים בעזרת פרוטוקול SMTP.
- socket – ספרייה המספקת גישה לסוקטים לצורך תקשורת בין מחשבים עם פרוטוקול UDP/TCP.
- sqlite3 – ספרייה המאפשרת ניהול מסד נתונים המאוחסן בקובץ.
- ssl – ספרייה המספקת הצפנת TLS/SSL לתקשורת כמו שליחת אימייל כדי להבטיח העברה של הנתונים בצורה מאובטחת.
- string – ספרייה המכילה קבועים ופונקציות שימושיות לעבודה עם מחרוזות.
- sys – ספרייה המספקת גישה למשתנים ולפונקציות הקשורים ל- Interpreter של פייתון.
- threading – מאפשר הרצה של מספר פעולות במקביל בתוך אותה תוכנית.
- time – ספרייה המספקת פונקציות הקשורות לזמן, כמו מדידת זמנים או השהייה של התוכנית לזמן מסוים.
- vosk – ערכה לזיהוי קול שעובדת ללא חיבור לאינטרנט.
- wave – ספרייה לקריאה וכתיבה של קבצי אודיו בפורמט WAV.

## מחלקות הפרוייקט

### ProtocolHandler

**תפקיד המחלקה**: המחלקה אחראית על פרוטוקול התקשורת שלפיו עובדים השרת והלקוח.

**תכונות המחלקה**:

- server_messages - מילון של כל הודעות הפרוטוקול בצד השרת.
- client_messages - מילון של כל הודעות הפרוטוקול בצד הלקוח.

**פעולות במחלקה**:

- build_message(cmd, data) - מקבלת פקודה ונתונים ומחזירה הודעת פרוטוקול תקינה הבנויה מהם.
- parse_message(data) - מקבלת הודעה, מפרסרת אותה לפי הפרוטוקול ובודקת את תקינותה. מחזירה את הפקודה והנתונים של ההודעה אם תקינה.
- check_validation() - פעולה מופשטת לבדיקת תקינות הפקודה והנתונים. ממומשת במחלקות היורשות.
- split_data(data) - מקבלת הודעה ומפצלת אותה לפי תו ההפרדה של הפרוטוקול.

### ServerProtocolHandler

**תפקיד המחלקה**: המחלקה אחראית על פרוטוקול התקשורת של השרת. יורשת מהמחלקה ProtocolHandler.

**תכונות המחלקה**:

אין תכונות נוספות מעבר לתכונות ProtocolHandler.

**פעולות במחלקה**:

- check_validation(cmd, data) - מקבלת פקודה ונתונים ובודקת את תקינותם לפי הפרוטוקול של השרת.

### ClientProtocolHandler

**תפקיד המחלקה**: המחלקה אחראית על פרוטוקול התקשורת של הלקוח. יורשת מהמחלקה ProtocolHandler.

**תכונות המחלקה**:

אין תכונות נוספות מעבר לתכונות ProtocolHandler.

**פעולות במחלקה**:

- check_validation(cmd, data) - מקבלת פקודה ונתונים ובודקת את תקינותם לפי הפרוטוקול של הלקוח.

### ConnectionManager

**תפקיד המחלקה**: המחלקה אחראית על המשותף בין התקשורת וההצפנות בתקשורת של השרת והלקוח.

**תכונות המחלקה**:

אין תכונות מחלקה.

**פעולות במחלקה**:

- create_socket() - יוצרת אובייקט סוקט TCP ומחזירה אותו.
- receive_msg(socket) - מקבלת סוקט ומחזירה את ההודעה המפוענחת שנתקבלה ממנו.
- send_msg(socket, msg) - מקבלת סוקט והודעה, ושולחת את ההודעה המקודדת לסוקט שהתקבל.
- encrypt_data(data, AES_key) - מקבלת נתונים ומפתח AES, ומחזירה את ההודעה המוצפנת.
- decrypt_data(data, AES_key) - מקבלת נתונים ומפתח AES, ומחזירה את הנתונים המפוענחים.

### ServerConnectionManager

**תפקיד המחלקה**: המחלקה אחראית על התקשורת וההצפנות בתקשורת בצד של השרת. יורשת מהמחלקה ConnectionManager.

**תכונות המחלקה**:

- public_pem - ה-pem של מפתח ה-RSA הציבורי של השרת.
- client_sockets - רשימת כל סוקטי הלקוחות.
- _sockets_to_keys - מילון הממפה סוקטים של לקוחות למפתחות ה-AES שלהם.
- client_message_count - מילון הממפה סוקט לקוח למספר ההודעות שהוא שלח.
- message_count_dict_lock - נעילת threading למילון מונה ההודעות.
- server_socket - הסוקט של השרת.

**פעולות במחלקה**:

- setup_socket() - מגדירה את סוקט השרת ומייצרת את מפתחות ה-RSA.
- read_from_socket() - מחזירה רשימת סוקטים של כל הלקוחות ששלחו הודעה.
- accept_client_socket(socket) - מקבלת סוקט לקוח חדש ומתחילה את תהליך חילופי המפתחות איתו.
- has_reached_max_messages(conn) - מקבלת סוקט לקוח ובודקת אם הלקוח הגיע למספר ההודעות המקסימלי המותר בפרק זמן מסוים.
- reset_client_message_count_loop() - לולאה של איפוס מונה ההודעות של כל הלקוחות.
- close_conn(socket) - מקבלת סוקט וסוגרת את החיבור איתו.
- get_private_ip() - מחזירה את כתובת ה-IP הפרטית של השרת.
- generate_RSA() - מייצרת מפתחות RSA ציבורי ופרטי. שומרת את מפתח ה-RSA הפרטי במשתנה סביבה.
- _load_and_recv_RSA() - טוענת את מפתח ה-RSA הפרטי ממשתנה הסביבה ומחזירה אותו מפוענח.
- recv_and_save_AES(client_socket) - מקבלת סוקט לקוח, מפענחת את מפתח ה-AES המוצפן שהתקבל ממנו ושומרת אותו במילון.
- handshake(client_socket) - מקבלת סוקט לקוח ומתחילה את תהליך חילופי המפתחות על ידי שליחת מפתח ה-RSA הציבורי ללקוח.
- encrypt_msg(client_socket, msg) - מקבלת סוקט לקוח והודעה, ומצפינה את ההודעה באמצעות מפתח ה-AES של הלקוח.
- decrypt_msg(client_socket, msg) - מקבלת סוקט לקוח והודעה, ומפענחת את ההודעה באמצעות מפתח ה-AES של הלקוח.

### ClientConnectionManager

**תפקיד המחלקה**: המחלקה אחראית על התקשורת וההצפנות בתקשורת בצד של הלקוח. יורשת מהמחלקה ConnectionManager.

**תכונות המחלקה**:

- socket - הסוקט של הלקוח.

**פעולות במחלקה**:

- connect(server_ip) - מקבלת את כתובת ה-IP של השרת ומחברת את סוקט הלקוח אליו.
- generate_and_send_AES() - מייצרת מפתח AES, מצפינה אותו באמצעות מפתח ה-RSA הציבורי של השרת, ושולחת אותו לשרת.
- recv_RSA() - שומרת את מפתח ה-RSA הציבורי שהתקבל מהשרת.
- get_AES() - מחזירה את מפתח ה-AES ממשתני הסביבה.
- handshake() - ממשיכה את תהליך חילופי המפתחות עם השרת.

## Server

**תפקיד המחלקה**: המחלקה מרכזת את כל יכולות השרת, ואחראית על קבלת פקודות מהלקוח והחזרת תשובות.

**תכונות המחלקה**:

- conn_manager - אובייקט ServerConnectionManager, אחראי על ניהול חיבורי הלקוחות לשרת.
- socket - סוקט השרת.
- protocol_handler - אובייקט ServerProtocolHandler, אחראי על בניית ופענוח הודעות פרוטוקול.
- messages_to_send - רשימת הודעות הממתינות לשליחה ללקוחות.
- user_manager - אובייקט UserManager, אחראי על ניהול משתמשים מחוברים, ניסיונות כניסה וזמני השהייה.
- db_manager - אובייקט DataBase, אחראי על כל פעולות מסד הנתונים.
- email_manager - אובייקט EmailManager, אחראי על שליחת קודים לדוא"ל.
- voice_authenticator - אובייקט VoiceAuthenticator, אחראי על אימות חתימת קול.
- conn_code - מילון המכיל את הקודים החד פעמיים שנשלחו ללקוחות, ממופה לפי סוקט הלקוח.

**פעולות במחלקה**:

- run() - לולאת השרת הראשית. מאזינה לחיבורים ולהודעות מלקוחות, מעבדת אותן וקוראת לפעולה המתאימה.
- handle_client_message(conn, cmd, data) - מקבלת חיבור עם לקוח, פקודה ונתונים, וקוראת לפעולה המתאימה לטיפול בפקודה.
- handle_login_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בניסיון כניסה של לקוח. תשובות אפשריות של השרת ללקוח: login_ok, username_invalid, incorrect_password, timeout.

- handle_signup_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בהרשמת לקוח חדש. שומרת את המשתמש במסד הנתונים אם כל הנתונים הנדרשים התקבלו. תשובות אפשריות של השרת ללקוח: signup_ok, user_exists, weak_password.
- handle_security_questions_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את רשימת שאלות האבטחה הזמינות.
- handle_my_security_questions_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את שאלות האבטחה שהוא בחר. אם שם המשתמש לא תקין, משיבה username_invalid.
- handle_check_security_answers_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ובודקת את תשובות האבטחה של הלקוח. תשובות אפשריות של השרת ללקוח: security_answers_correct, security_answers_incorrect, timeout.
- handle_reset_password_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בבקשת איפוס סיסמה. אם הסיסמה החדשה חזקה מספיק, מאפסת ומשיבה reset_ok. אחרת, משיבה weak_password.
- handle_get_auth_options_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את אפשרויות האימות הזמינות לו.
- handle_my_email_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את כתובת הדוא"ל הרשומה שלו. אם לא רשום דוא"ל, משיבה no_email_registered.
- handle_send_code_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת קוד חד-פעמי לדוא"ל הלקוח.
- handle_code_login_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בניסיון כניסה באמצעות קוד. תשובות אפשריות של השרת ללקוח: code_correct, code_incorrect, code_expired, timeout.
- handle_voice_auth_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בניסיון אימות קול. תשובות אפשריות של השרת ללקוח: voice_verified, voice_not_verified, timeout.
- handle_check_face_recognition_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בניסיון זיהוי פנים. תשובות אפשריות של השרת ללקוח: face_matches, face_not_matches.
- handle_setup_voice_auth_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומגדירה אימות קול עבור המשתמש.
- handle_setup_face_recognition_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומגדירה זיהוי פנים עבור המשתמש.
- handle_save_preferences_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושומרת את העדפות יצירת הסיסמה לאתר רשום. תשובות אפשריות של השרת ללקוח: preferences_saved, not_registered.
- handle_get_preferences_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את העדפות יצירת הסיסמה של אתר רשום. תשובות אפשריות של השרת ללקוח: your_preferences, not_registered.

- handle_register_site_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בבקשת רישום אתר חדש. מייצרת סיסמה לפי העדפות הלקוח או לפי ברירת מחדל. תשובות אפשריות של השרת ללקוח: registration_ok (עם הסיסמה שנוצרה), already_registered.
- handle_registered_sites_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את רשימת האתרים הרשומים שלו. אם data הוא "True", שולחת רק אתרים שנמחקו.
- handle_site_info_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ושולחת ללקוח את שם המשתמש והסיסמה של אתר רשום. תשובות אפשריות של השרת ללקוח: info, not_registered.
- handle_change_password_message(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בבקשת שינוי סיסמה לאתר רשום. מייצרת סיסמה חדשה לפי העדפות האתר. תשובות אפשריות של השרת ללקוח: change_ok (עם הסיסמה שנוצרה), not_registered.
- handle_delete_site_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בבקשת מחיקת אתר רשום. תשובות אפשריות של השרת ללקוח: site_deleted, not_registered.
- handle_recover_site_msg(conn, data) - מקבלת חיבור עם לקוח ונתונים, ומטפלת בבקשת שחזור אתר שנמחק. תשובות אפשריות של השרת ללקוח: site_recovered, not_registered.
- build_and_queue_message(conn, cmd, data) - מקבלת חיבור עם לקוח, פקודה ונתונים, בונה הודעת פרוטוקול ומוסיפה אותה לתור ההודעות היוצאות.
- send_messages_to_clients() - שולחת את כל ההודעות בתור ללקוחות בצורה מוצפנת, ומרוקנת את התור.

## DataBase

**תפקיד המחלקה**: המחלקה אחראית על יצירת בסיס הנתונים, וכתיבה ושליפה מבסיס הנתונים.

**תכונות המחלקה**:

- users_keys - מילון הממפה user_id למפתח הצפנת ה-DK שלו.
- conn - אובייקט חיבור SQLite למסד הנתונים.
- cur - אובייקט cursor של SQLite לביצוע שאילתות.

**פעולות במחלקה**:

- init_db() - יוצרת את כל טבלאות מסד הנתונים (USERS, PASSWORD_PREFERENCES, USER_SITE, SECURITY_QUESTIONS) וממלאת את טבלת שאלות האבטחה.
- derive_key(password, salt) - מקבלת סיסמה ו-salt, ומחזירה מפתח הצפנה באורך 32 בתים באמצעות PBKDF2 עם SHA256.
- encrypt_data(user_id, data) - מקבלת user_id ורשימת פריטי נתונים, ומחזירה רשימה של הפריטים מוצפנים באמצעות ה-DK של המשתמש.

- decrypt_data(user_id, data) - מקבלת user_id ורשימת פריטי נתונים מוצפנים, ומחזירה רשימה של הפריטים מפוענחים באמצעות ה-DK של המשתמש.
- create_dk_encryption_key(username, password_hex, key_salt) - מקבלת שם משתמש, סיסמה בהקסדצימלי ו-salt, ומחזירה מפתח הצפנת DK הנגזר מהם.
- get_dk_encryption_key(user_id) - מקבלת user_id ומחזירה את מפתח הצפנת ה-DK שלו. אם המפתח לא נמצא במילון, גוזרת אותו ממסד הנתונים ושומרת.
- user_signup(username, password, email, phone_number, voice_signature, face_encoding, questions_dict) - מקבלת נתוני הרשמה של משתמש ושומרת אותו במסד הנתונים. מגיבבת את הסיסמה עם salt אקראי, מייצרת DK, מצפינה את כל הנתונים הרגישים עם ה-DK, מצפינה את ה-DK עם מפתח הנגזר משם המשתמש והסיסמה, ומכניסה את המשתמש למסד הנתונים. מחזירה את ה-user_id בהצלחה, None אחרת.
- does_user_exist(username) - מקבלת שם משתמש ומחזירה True אם המשתמש קיים במסד הנתונים, False אחרת.
- user_login(username, password) - מקבלת שם משתמש וסיסמה, ומאמתת את פרטי הכניסה. מגיבבת את הסיסמה שהתקבלה עם ה-salt השמור ומשווה לגיבוב השמור. מחזירה את ה-user_id אם הפרטים נכונים, None אחרת.
- get_user_email(username) - מקבלת שם משתמש ומחזירה את כתובת הדוא"ל המפוענחת של המשתמש.
- get_user_phone_number(username) - מקבלת שם משתמש ומחזירה את מספר הטלפון המפוענח של המשתמש.
- get_voice_signature(username) - מקבלת שם משתמש ומחזירה את חתימת הקול המפוענחת של המשתמש. מחזירה None אם לא שמורה חתימת קול.
- get_face_encoding(username) - מקבלת שם משתמש ומחזירה את קידוד הפנים המפוענח של המשתמש. מחזירה None אם לא שמור קידוד פנים.
- get_user_auth_options(username) - מקבלת שם משתמש ומחזירה מילון של אפשרויות האימות הזמינות לו.
- setup_voice_auth(user_id, voice_signature) - מקבלת user_id וחתימת קול, מצפינה את חתימת הקול ושומרת אותה במסד הנתונים.
- setup_face_recognition(user_id, face_encoding) - מקבלת user_id וקידוד פנים, מצפינה את קידוד הפנים ושומרת אותו במסד הנתונים.
- get_security_questions_list() - מחזירה רשימה של כל שאלות האבטחה ממסד הנתונים.
- get_user_security_questions(username) - מקבלת שם משתמש ומחזירה רשימה של 3 שאלות האבטחה שהמשתמש בחר.
- check_user_security_answers(username, answers_list) - מקבלת שם משתמש ורשימת תשובות, מגיבבת כל תשובה עם ה-salt השמור ומשווה לגיבובים השמורים. מחזירה True אם כל התשובות נכונות, False אחרת.
- reset_user_password(username, new_password) - מקבלת שם משתמש וסיסמה חדשה, ומאפסת את סיסמת המשתמש. מפענחת את ה-DK עם מפתח ההצפנה הישן, מגיבבת את הסיסמה החדשה, גוזרת מפתח הצפנת DK חדש ומצפינה מחדש את ה-DK איתו. מחזירה את ה-user_id.

- does_site_exist(user_id, site_url) - מקבלת user_id וכתובת אתר, ומחזירה True אם האתר קיים ולא נמחק, False אחרת.
- does_deleted_site_exist(user_id, site_url) - מקבלת user_id וכתובת אתר, ומחזירה True אם האתר קיים ונמחק, False אחרת.
- get_site_id(user_id, site_url) - מקבלת user_id וכתובת אתר, ומחזירה את ה-site_id על ידי פענוח והשוואת כתובות האתרים. מחזירה None אם לא נמצא.
- count_registered_sites(user_id) - מקבלת user_id ומחזירה את מספר האתרים הרשומים למשתמש.
- set_default_password_preferences() - מכניסה שורה חדשה עם העדפות סיסמה ברירת מחדל לטבלת PASSWORD_PREFERENCES ומחזירה את ה-ID שלה.
- update_password_preferences(user_id, site_url, preferences) - מקבלת user_id, כתובת אתר ומילון העדפות, ומעדכנת את העדפות יצירת הסיסמה של האתר.
- get_password_preferences(user_id, site_url) - מקבלת user_id וכתובת אתר, ומחזירה מילון של העדפות יצירת הסיסמה של האתר.
- register_site(user_id, site_url, site_username, site_password, password_preferences) - מקבלת user_id, כתובת אתר, שם משתמש לאתר, סיסמה והעדפות סיסמה אופציונליות, מצפינה את הנתונים ומכניסה אותם למסד הנתונים עם העדפות ברירת מחדל. אם הועברו העדפות מותאמות, מעדכנת אותן לאחר ההכנסה.
- get_preference_id(user_id, site_id) - מקבלת user_id ו-site_id, ומחזירה את ה-preference_id של האתר.
- change_site_password(user_id, site_url, new_site_password) - מקבלת user_id, כתובת אתר וסיסמה חדשה, מצפינה את הסיסמה החדשה ומעדכנת אותה במסד הנתונים.
- get_registered_sites(user_id, get_deleted) - מקבלת user_id ודגל בוליאני, ומחזירה רשימה מפוענחת של כתובות האתרים הרשומים של המשתמש. אם get_deleted הוא True, מחזירה את האתרים שנמחקו. אחרת, מחזירה את האתרים הפעילים.
- get_site_info(user_id, site_url) - מקבלת user_id וכתובת אתר, ומחזירה את שם המשתמש והסיסמה המפוענחים של האתר.
- delete_site(user_id, site_url) - מקבלת user_id וכתובת אתר, ומסמנת את האתר כמחוק במסד הנתונים.
- recover_site(user_id, site_url) - מקבלת user_id וכתובת אתר, ומסמנת את האתר שנמחק בעבר כפעיל במסד הנתונים.

### UserManager

**תפקיד המחלקה**: המחלקה אחראית על ניהול המשתמשים המחוברים וניסיונות ההתחברות שלהם.

**תכונות המחלקה**:

- logged_users - מילון הממפה סוקטים של לקוחות ל-user_id שלהם.
- user_login_attempts - מילון הממפה שמות משתמשים לרשימת זמני ניסיונות כניסה כושלים שלהם.
- users_in_timeout - מילון הממפה שמות משתמשים לזמן שנכנסו להשהייה.

**פעולות במחלקה**:

- is_user_in_timeout(username) - מקבלת שם משתמש ומחזירה True אם המשתמש נמצא כרגע בהשהייה, False אחרת. אם ההשהייה פגה, מסירה את המשתמש ממילון ההשהיות.
- register_failed_login_attempt(username) - מקבלת שם משתמש, רושמת את זמן ניסיון הכניסה הכושל ומחזירה האם המשתמש חרג ממספר הניסיונות הכושלים המותרים בחלון זמן מסוים.
- successful_login_attempt(username) - מקבלת שם משתמש ומוחקת את היסטוריית ניסיונות הכניסה הכושלים שלו.

## EmailManager

**תפקיד המחלקה**: המחלקה אחראית על שליחת קוד התחברות חד פעמי לאימייל של המשתמשים.

**תכונות המחלקה**:

- _security_context - הקשר אבטחת SSL לחיבור SMTP.

**פעולות במחלקה**:

- send_gmail(recipient_gmail, msg) - מקבלת כתובת דוא"ל של נמען והודעה, ושולחת את ההודעה דרך SMTP של Gmail.
- send_login_code_gmail(recipient_gmail) - מקבלת כתובת דוא"ל של נמען, מייצרת קוד כניסה חד-פעמי, שולחת אותו בדוא"ל ומחזירה את הקוד יחד עם זמן היצירה שלו.
- _generate_login_code() - מייצרת ומחזירה קוד כניסה אקראי בן 8 ספרות.

## AudioManager

**תפקיד המחלקה**: המחלקה אחראית על הקלטה והשמעה של אודיו.

**תכונות המחלקה**:

- client - הפניה לאובייקט הלקוח של ה-UI עבור callbacks.
- chunk - גודל חתיכת ההקלטה בדגימות (1024).
- sample_format - פורמט הדגימה (16 ביט לדגימה).
- channels - מספר ערוצי השמע (מונו).
- fs - קצב הדגימה (16000 דגימות לשנייה).
- seconds - משך ההקלטה המקסימלי בשניות.
- filename - נתיב קובץ ה-WAV הפלט.
- is_recording - האם מנהל השמע מקליט כרגע.

**פעולות במחלקה**:

- start_recording() - פותחת זרם שמע ומתחילה הקלטה בתהליכון נפרד.
- _record_loop() - לולאת הקלטה שקוראת נתוני שמע מהזרם עד שנעצרת או שמגיעים למשך המקסימלי.
- stop_recording() - עוצרת את ההקלטה, שומרת את השמע שהוקלט לקובץ WAV ומודיעה ל-UI באמצעות callback.
- play_audio(filename) - מקבלת שם קובץ ומנגנת את השמע מקובץ ה-WAV.

### VoiceAuthenticator

**תפקיד המחלקה**: המחלקה אחראית על זיהוי הקול של המשתמשים.

**תכונות המחלקה**:

- model - מודל זיהוי דיבור של Vosk.
- spk_model - מודל זיהוי דובר של Vosk.

**פעולות במחלקה**:

- calc_voice_signature(voice_file) - מקבלת נתיב לקובץ קול, מעבדת את השמע ומחזירה את חתימת הקול כנתונים בינאריים. מחזירה None אם לא ניתן לחלץ חתימת קול.
- verify_user(bin_voice_signature, bin_login_signature) - מקבלת חתימת קול שמורה וחתימת קול כניסה כנתונים בינאריים, משווה בין שתי החתימות ומחזירה True אם הן תואמות, False אחרת.

### VideoManager

**תפקיד המחלקה**: המחלקה אחראית על צילום וידאו.

**תכונות המחלקה**:

אין תכונות מחלקה. כל הפעולות הן סטטיות.

**פעולות במחלקה**:

- list_cameras() - פעולה סטטית. מחזירה רשימה של התקני המצלמה הזמינים.
- get_cap(camera_index) - פעולה סטטית. מקבלת אינדקס מצלמה ומחזירה אובייקט VideoCapture עבורה. מחזירה None אם לא ניתן לפתוח את המצלמה.
- get_frame(cap) - פעולה סטטית. מקבלת אובייקט VideoCapture ומחזירה tuple המכיל דגל הצלחה, פריים RGB משוקף לתצוגה, ופריים RGB מוקטן לעיבוד זיהוי פנים. אם לא ניתן לקרוא את הפריים, משחררת את המצלמה ומחזירה (False, ).

### FaceRecognizer

**תפקיד המחלקה**: המחלקה אחראית על זיהוי הפנים של המשתמשים.

**תכונות המחלקה**:

אין תכונות מחלקה. כל הפעולות הן סטטיות.

**פעולות במחלקה**:

- get_face_encodings(rgb_frame) - פעולה סטטית. מקבלת פריים RGB, מזהה פנים ומחזירה את קידוד הפנים של הפנים הראשונות שנמצאו כנתונים בינאריים. מחזירה None אם לא זוהו פנים.
- compare_faces(stored_bin_face_encoding, bin_face_encoding) - פעולה סטטית. מקבלת קידוד פנים שמור וקידוד פנים כניסה כנתונים בינאריים, משווה בין שני קידודי הפנים ומחזירה True אם הם תואמים, False אחרת.

### PasswordHandler

**תפקיד המחלקה**: המחלקה אחראית על יצירת סיסמאות ובדיקת עוצמתן.

**תכונות המחלקה**:

אין תכונות מחלקה. כל הפעולות הן סטטיות.

**פעולות במחלקה**:

- generate_password(preferences) - פעולה סטטית. מקבלת מילון העדפות ומייצרת סיסמה אקראית העומדת בכללים הנתונים. מוודאת שהתו הראשון תואם לכללי ההתחלה ושלפחות תו אחד מכל קטגוריה שנכללה נמצא בסיסמה.
- is_strong_password(password) - פעולה סטטית. מקבלת סיסמה ומחזירה True אם היא עומדת בדרישות החוזק המינימליות: לפחות 8 תווים, עם אות גדולה, אות קטנה, ספרה ותו מיוחד.

### Client

**תפקיד המחלקה**: המחלקה מרכזת את כל יכולות הלקוח, והיא אחראית על שליחת בקשות לשרת וקבלת התשובות מהשרת.

**תכונות המחלקה**:

- protocol_handler - אובייקט ClientProtocolHandler.
- conn_manager - אובייקט ClientConnectionManager.
- ui - אובייקט UI.
- audio_manager - אובייקט AudioManager.
- voice_authenticator - אובייקט VoiceAuthenticator.

- username - שם המשתמש של המשתמש המחובר.
- is_email_verified - האם הדוא"ל של המשתמש אומת.
- bin_voice_signature - חתימת הקול של המשתמש כנתונים בינאריים.
- bin_face_encoding - קידוד הפנים של המשתמש כנתונים בינאריים.
- are_questions_setup - האם המשתמש הגדיר שאלות אבטחה.
- chosen_questions_dict - מילון של שאלות האבטחה והתשובות שהמשתמש בחר.
- auth_options - מילון של אפשרויות האימות הזמינות למשתמש.
- status - הסטטוס הנוכחי של הלקוח.

**פעולות במחלקה**:

- get_server_ip() - מחזירה את כתובת ה-IP של השרת מארגומנטי שורת הפקודה, או את כתובת ברירת המחדל אם לא סופקה.
- run() - מפעילה את ממשק המשתמש של הלקוח.
- change_status(status) - מקבלת סטטוס וקובעת אותו כסטטוס הנוכחי של הלקוח.
- can_user_signup() - מחזירה True אם המשתמש השלים את כל שלבי ההרשמה הנדרשים.
- signup(username, password, email) - מקבלת שם משתמש, סיסמה ודוא"ל, ושולחת בקשת הרשמה לשרת. מקודדת חתימת קול וקידוד פנים אם זמינים, וכוללת את שאלות האבטחה שנבחרו. מודיעה ל-UI על תגובת השרת.
- login(username, password) - מקבלת שם משתמש וסיסמה, ושולחת בקשת כניסה לשרת. מודיעה ל-UI על תגובת השרת.
- get_security_questions_list() - מבקשת מהשרת את רשימת שאלות האבטחה ומחזירה אותה.
- setup_security_questions(chosen_questions_dict) - מקבלת מילון של שאלות ותשובות אבטחה שנבחרו, שומרת אותן ומודיעה ל-UI.
- get_user_security_questions(username) - מקבלת שם משתמש ומבקשת מהשרת את שאלות האבטחה של המשתמש. מודיעה ל-UI על תגובת השרת.
- check_security_answers(answers) - מקבלת רשימת תשובות אבטחה ושולחת אותן לשרת לאימות. מודיעה ל-UI על תגובת השרת.
- reset_password(new_password) - מקבלת סיסמה חדשה ושולחת בקשת איפוס סיסמה לשרת. מודיעה ל-UI על תגובת השרת.
- get_auth_options() - מבקשת מהשרת את אפשרויות האימות הזמינות ומחזירה אותן.
- code_login(code) - מקבלת קוד חד-פעמי ושולחת אותו לשרת לאימות. מטפלת בתגובה בהתאם לסטטוס הנוכחי של הלקוח (הרשמה, כניסה או שחזור סיסמה).
- request_code(client_email) - מקבלת כתובת דוא"ל ומבקשת מהשרת לשלוח אליה קוד כניסה חד-פעמי. מחזירה True אם הקוד נשלח, False אחרת.
- get_email(username) - מקבלת שם משתמש ומבקשת מהשרת את הדוא"ל של המשתמש. מחזירה את הדוא"ל או None אם לא רשום. אם לא סופק שם משתמש, משתמשת בשם המשתמש המחובר.
- is_email_valid(email) - מקבלת כתובת דוא"ל ומחזירה True אם היא כתובת תקינה וברת-מסירה, False אחרת.

- start_voice_recording() - מתחילה הקלטת שמע באמצעות מנהל השמע.
- stop_voice_recording() - עוצרת את הקלטת השמע.
- recording_stopped() - פעולת callback שנקראת על ידי מנהל השמע כשההקלטה נעצרת. מודיעה ל-UI.
- setup_voice_auth() - מחלצת את חתימת הקול מקובץ השמע שהוקלט ומגדירה אימות קול. בזמן הרשמה, מודיעה ל-UI. אחרת, שולחת את חתימת הקול לשרת לשמירה.
- voice_auth() - מחלצת את חתימת הקול מקובץ השמע שהוקלט ושולחת אותה לשרת לאימות קול. מודיעה ל-UI על תגובת השרת.
- get_face_encoding(rgb_frame) - מקבלת פריים RGB, מחלצת את קידוד הפנים ושומרת אותו. בזמן כניסה או שחזור סיסמה, ממשיכה לזיהוי פנים מיידית. אחרת, מודיעה ל-UI האם זוהו פנים.
- setup_face_recognition() - מגדירה זיהוי פנים באמצעות קידוד הפנים השמור. בזמן הרשמה, מודיעה ל-UI. אחרת, שולחת את קידוד הפנים לשרת לשמירה.
- face_recognition() - שולחת את קידוד הפנים השמור לשרת לאימות זיהוי פנים. מודיעה ל-UI על תגובת השרת.
- register_site(site_url, site_username, password_preferences) - מקבלת כתובת אתר, שם משתמש לאתר והעדפות סיסמה אופציונליות, ושולחת בקשת רישום אתר לשרת. מודיעה ל-UI על תגובת השרת.
- get_password_preferences(site_url) - מקבלת כתובת אתר ומבקשת מהשרת את העדפות יצירת הסיסמה של האתר. מחזירה את מילון ההעדפות.
- update_password_preferences(site_url, preferences) - מקבלת כתובת אתר ומילון העדפות, ושולחת בקשה לעדכון העדפות יצירת הסיסמה של האתר.
- get_registered_sites(get_deleted) - מקבלת דגל בוליאני ומבקשת מהשרת את רשימת האתרים הרשומים. אם get_deleted הוא True, מבקשת את האתרים שנמחקו. אחרת, מבקשת את האתרים הפעילים.
- get_site_info(site_url) - מקבלת כתובת אתר ומבקשת מהשרת את שם המשתמש והסיסמה של האתר. מחזירה אותם כ-tuple.
- change_password(site_url) - מקבלת כתובת אתר ומבקשת מהשרת ליצור סיסמה חדשה לאתר. מחזירה את הסיסמה החדשה.
- delete_site(site_url) - מקבלת כתובת אתר ושולחת בקשה לשרת למחוק את האתר. מודיעה ל-UI בהצלחה.
- recover_site(site_url) - מקבלת כתובת אתר ושולחת בקשה לשרת לשחזר את האתר שנמחק. מודיעה ל-UI בהצלחה.
- build_and_send_message(cmd, data) - מקבלת פקודה ונתונים, בונה הודעת פרוטוקול, מצפינה אותה ושולחת אותה לשרת.
- receive_and_parse_message() - מפענחת הודעה מוצפנת שהתקבלה מהשרת ומנתחת אותה לפי הפרוטוקול. מחזירה את הפקודה והנתונים.

UI

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של הלקוח. היא מנהלת את החלון הראשי ואת כל המחלקות של החלונות המשניים. אחראית גם להעברת מידע בין הלקוח לממשק המשתמש ולהפך.

**תכונות המחלקה**:

- window - חלון CTk הראשי של האפליקציה.
- client - אובייקט Client המנהל את התקשורת עם השרת.
- login_window - הפניה לחלון LoginWindow הפתוח כרגע, או None.
- reset_password_window - הפניה לחלון ResetPasswordWindow הפתוח כרגע, או None.
- signup_window - הפניה לחלון SignupWindow הפתוח כרגע, או None.
- settings_menu - פאנל צד עם הגדרות אימות וחשבון.
- auth_select_window - הפניה לחלון AuthSelectWindow הפתוח כרגע, או None.
- code_login_window - הפניה לחלון CodeLoginWindow הפתוח כרגע, או None.
- voice_recording_window - הפניה לחלון VoiceRecordingWindow הפתוח כרגע, או None.
- cap - אובייקט VideoCapture של OpenCV למצלמה הפעילה, או None.
- current_camera_index - אינדקס המצלמה הפעילה כרגע (-1 אם אין).
- rgb_small_frame - פריים RGB מוקטן לעיבוד זיהוי פנים.
- video_capture_window - הפניה לחלון VideoCaptureWindow הפתוח כרגע, או None.
- security_questions_window - הפניה לחלון שאלות האבטחה הפתוח כרגע, או None.
- site_registration_window - הפניה לחלון SiteRegistrationWindow הפתוח כרגע, או None.
- site_info_window - הפניה לחלון SiteInfoWindow הפתוח כרגע, או None.
- change_password_window - הפניה לחלון ChangePasswordWindow הפתוח כרגע, או None.
- site_recovery_window - הפניה לחלון SiteRecoveryWindow הפתוח כרגע, או None.
- search_var - משתנה StringVar המקושר לתיבת החיפוש; שינויים מפעילים סינון רשימת האתרים.
- current_site_selection - כתובת האתר הנבחר כרגע ברשימה, או None.
- registered_sites_list - רשימת כתובות האתרים הרשומים של המשתמש.
- password_preferences_window - הפניה לחלון PasswordPreferencesWindow הפתוח כרגע, או None.

**פעולות במחלקה**:

- run() - מפעילה את לולאת האירועים הראשית של ה-UI.
- run_signup_window() - פותחת את חלון ההרשמה, או מתמקדת בו אם כבר פתוח.
- call_signup(username, password, email) - מקבלת שם משתמש, סיסמה ודוא"ל, ומעבירה בקשת הרשמה ללקוח.

- display_signup_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון ההרשמה.
- signup_successful(username, password) - מקבלת שם משתמש וסיסמה, סוגרת את חלון ההרשמה ומכניסה את המשתמש אוטומטית.
- run_login_window() - פותחת את חלון הכניסה, או מתמקדת בו אם כבר פתוח.
- call_login(username, password) - מקבלת שם משתמש וסיסמה, ומעבירה בקשת כניסה ללקוח.
- display_login_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון הכניסה.
- login_succesful(username, password) - מקבלת שם משתמש וסיסמה, ומטפלת בכניסה מוצלחת. אם נרשם, מגדירה את המשתמש ישירות. אם מתחבר, פותחת את חלון בחירת האימות.
- get_email() - מחזירה את הדוא"ל של המשתמש מהלקוח.
- call_get_auth_options() - מחזירה את אפשרויות האימות המוגדרות של המשתמש מהלקוח.
- run_auth_select_window() - פותחת את חלון בחירת האימות, או מתמקדת בו אם כבר פתוח.
- run_security_questions_setup_window() - פותחת את חלון הגדרת שאלות האבטחה, או מתמקדת בו אם כבר פתוח.
- call_get_security_questions_list() - מחזירה את רשימת שאלות האבטחה הזמינות מהלקוח.
- call_setup_security_questions(chosen_questions_dict) - מקבלת מילון של שאלות ותשובות אבטחה שנבחרו, ומעבירה אותו ללקוח.
- questions_setup_successful() - סוגרת את חלון שאלות האבטחה ומודיעה לחלון ההרשמה על הצלחה.
- run_security_questions_auth_window(user_security_questions) - מקבלת רשימת שאלות אבטחה, סוגרת את חלון הכניסה ופותחת את חלון אימות שאלות האבטחה.
- call_get_user_security_questions(username) - מקבלת שם משתמש ומבקשת מהשרת את שאלות האבטחה שלו.
- call_check_security_answers(answers) - מקבלת רשימת תשובות ומעבירה אותן ללקוח לאימות.
- security_answers_correct() - מטפלת בתשובות אבטחה נכונות על ידי מעבר לשחזור סיסמה ופתיחת בחירת אימות.
- security_answers_incorrect() - מציגה שגיאה בחלון שאלות האבטחה עבור תשובות שגויות.
- security_answers_timeout(minutes) - מקבלת מספר דקות ומציגה שגיאת השהייה בחלון שאלות האבטחה.
- run_reset_password_window() - פותחת את חלון איפוס הסיסמה, או מתמקדת בו אם כבר פתוח.
- call_reset_password(new_password) - מקבלת סיסמה חדשה ומעבירה בקשת איפוס ללקוח.
- password_reset_successful(username) - מקבלת שם משתמש, סוגרת את חלון איפוס הסיסמה ומגדירה את הפעלת המשתמש.

- password_reset_unsuccessful() - מציגה שגיאה בחלון איפוס הסיסמה עבור סיסמה חלשה.
- run_code_login_window(client_email) - מקבלת כתובת דוא"ל ופותחת את חלון כניסת הקוד, או מתמקדת בו אם כבר פתוח.
- request_login_code(client_email) - מקבלת כתובת דוא"ל ומבקשת קוד אימות. פותחת את חלון הקוד אם הקוד נשלח בהצלחה.
- call_is_email_valid(email) - מקבלת כתובת דוא"ל, בודקת תקינות דרך הלקוח ומודיעה לחלון ההרשמה.
- call_check_code(code) - מקבלת קוד אימות ומעבירה אותו ללקוח.
- signup_code_correct() - מטפלת בקוד נכון בזמן הרשמה. מאפשרת את טופס ההרשמה אם כל התנאים מתקיימים.
- login_code_correct() - מטפלת בקוד נכון בזמן כניסה. סוגרת את חלון הקוד ומשלימה את האימות.
- code_incorrect() - מציגה משוב קוד שגוי בחלון כניסת הקוד.
- code_expired() - מציגה הודעת קוד שפג תוקפו בחלון כניסת הקוד.
- code_login_timeout(minutes) - מקבלת מספר דקות ומציגה שגיאת השהייה בחלון כניסת הקוד.
- run_voice_recording_window() - פותחת את חלון הקלטת הקול במצב הגדרה או אימות, או מתמקדת בו אם כבר פתוח.
- call_start_recording() - מתחילה הקלטת קול דרך הלקוח.
- call_stop_recording() - עוצרת הקלטת קול דרך הלקוח.
- recording_stopped() - מודיעה לחלון הקלטת הקול שההקלטה נעצרה.
- call_setup_voice_auth() - שולחת את חתימת הקול שהוקלטה לשרת להגדרה.
- voice_auth_setup_successful() - סוגרת את חלון הקלטת הקול ומודיעה לחלון ההרשמה או לתפריט ההגדרות על הצלחה.
- signature_extraction_error() - מציגה שגיאת חילוץ חתימת קול בחלון ההקלטה.
- call_voice_auth() - שולחת את הקול שהוקלט לשרת לאימות.
- voice_auth_successful() - סוגרת את חלון הקלטת הקול ומשלימה את האימות.
- voice_auth_unsuccessful() - מציגה שגיאת אי-התאמת קול בחלון ההקלטה.
- voice_auth_timeout(minutes) - מקבלת מספר דקות ומציגה שגיאת השהייה בחלון הקלטת הקול.
- get_cameras_list() - מחזירה את רשימת המצלמות הזמינות.
- run_video_capture_window() - פותחת את חלון צילום הווידאו במצב הגדרה או אימות, או מתמקדת בו אם כבר פתוח.
- start_video_recording(camera_index) - מקבלת אינדקס מצלמה ומתחילה צילום וידאו. משחררת צילום קודם אם מחליפים מצלמה.
- get_frames() - לוכדת פריים מהמצלמה הפעילה, מעבדת אותו ומעדכנת את חלון צילום הווידאו.
- _round_image_corners(image, radius) - מקבלת תמונת PIL ורדיוס, ומחזירה את התמונה עם פינות מעוגלות.

- call_face_recognition() - מנסה לקודד פנים מהפריים המוקטן הנוכחי. משביתה כפתור הגדרה בחלון צילום הווידאו אם אין פריים זמין.
- face_located() - מאפשרת את כפתור ההגדרה/אימות בחלון צילום הווידאו כאשר זוהו פנים.
- no_face_located() - משביתה את כפתור ההגדרה/אימות בחלון צילום הווידאו כאשר לא זוהו פנים.
- call_setup_face_recognition() - שולחת את קידוד הפנים שנלכד לשרת להגדרה.
- face_recognition_setup_successful() - סוגרת את חלון צילום הווידאו ומודיעה לחלון ההרשמה או לתפריט ההגדרות על הצלחה.
- face_recognition_successful() - סוגרת את חלון צילום הווידאו ומשלימה את האימות.
- face_recognition_unsuccessful() - מציגה שגיאת אי-התאמת פנים בחלון צילום הווידאו.
- auth_successful() - מטפלת באימות מוצלח. מגדירה את המשתמש אם מתחבר, או פותחת איפוס סיסמה אם משחזר.
- get_registered_sites() - מביאה את האתרים הרשומים הפעילים של המשתמש ומרעננת את הרשימה.
- get_deleted_sites() - מחזירה את האתרים המחוקים (הניתנים לשחזור) של המשתמש מהלקוח.
- setup_user(username) - מקבלת שם משתמש ומעבירה את ה-UI ממסך הבית שלפני הכניסה ללוח המחוונים שלאחר הכניסה.
- run_site_registration_window() - פותחת את חלון רישום האתר, או מתמקדת בו אם כבר פתוח.
- call_register_site(site_url, site_username, password_preferences) - מקבלת כתובת אתר, שם משתמש והעדפות סיסמה, ומעבירה בקשת רישום ללקוח.
- display_registration_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון רישום האתר.
- registration_succesful(password) - מקבלת סיסמה, סוגרת את חלון הרישום, מרעננת את רשימת האתרים ומעתיקה את הסיסמה אם התבקש.
- run_password_preferences_window(preferences, site_url) - מקבלת העדפות וכתובת אתר, ופותחת את חלון העדפות או מתמקדת בו אם כבר פתוח.
- save_password_preferences(preferences, site_url) - מקבלת העדפות סיסמה וכתובת אתר, ושומרת מקומית בזמן רישום או מעדכנת בשרת אם האתר כבר רשום.
- load_registered_sites(*args) - מסננת ומרעננת את רשימת האתרים הרשומים לפי מונח החיפוש הנוכחי.
- enable_site_buttons(site) - מקבלת אתר שנבחר ומאפשרת את כפתורי פעולות האתר.
- disable_site_buttons() - משביתה את כפתורי פעולות האתר כאשר לא נבחר אתר.
- show_selected_site_info() - מביאה ומציגה את חלון המידע עבור האתר הנבחר כרגע.
- call_site_info(site_url) - מקבלת כתובת אתר ומחזירה את שם המשתמש והסיסמה מהלקוח.
- run_site_info_window(site_username, site_password) - מקבלת שם משתמש וסיסמה של אתר, ופותחת את חלון מידע האתר או מתמקדת בו אם כבר פתוח.

- initiate_change_password_window() - מביאה את מידע האתר הנבחר ופותחת את חלון שינוי הסיסמה.
- run_change_site_password_window(site_url, site_username) - מקבלת כתובת אתר ושם משתמש, ופותחת את חלון שינוי הסיסמה או מתמקדת בו אם כבר פתוח.
- call_change_site_password(site_url) - מקבלת כתובת אתר, מייצרת סיסמה חדשה ומעתיקה אותה ללוח אם התבקש.
- get_preferences_and_initiate_window(site_url) - מקבלת כתובת אתר, מביאה את העדפות הסיסמה ופותחת את חלון ההעדפות.
- call_update_password_preferences(site_url, preferences) - מקבלת כתובת אתר ומילון העדפות, ומעבירה אותן ללקוח.
- delete_site() - מוחקת את האתר הנבחר כרגע דרך הלקוח.
- run_site_recovery_window() - פותחת את חלון שחזור האתר עם רשימת האתרים שנמחקו, או מתמקדת בו אם כבר פתוח.
- site_recovery_successful() - סוגרת את חלון השחזור ומרעננת את רשימת האתרים הרשומים.
- call_recover_site(site_url) - מקבלת כתובת אתר ומעבירה בקשת שחזור ללקוח.
- copy_to_clipboard(text) - מקבלת טקסט ומעתיקה אותו ללוח המערכת.
- connection_lost() - מציגה תיבת הודעה על אובדן חיבור וסוגרת את האפליקציה אם המשתמש מאשר.

### SignupWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון ההרשמה.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.

**פעולות במחלקה**:

- _make_option_row(parent, h, icon, title, desc, command) - מקבלת רכיב אב, גובה, אייקון, כותרת, תיאור ופקודה, ויוצרת שורת אפשרות לחיצה.
- can_signup() - מאפשרת את כפתור ההרשמה.
- show_password(num) - מקבלת מספר שדה ומציגה את הסיסמה בשדה הקלט המתאים.
- hide_password(num) - מקבלת מספר שדה ומסתירה את הסיסמה בשדה הקלט המתאים.
- call_signup() - מאמתת את שדות הטופס ושולחת בקשת הרשמה.
- on_email_entry_changed(*args) - מפעילה בדיקת תקינות דוא"ל עם השהייה כאשר שדה הדוא"ל משתנה.
- check_email_validity() - בודקת את תקינות הדוא"ל שהוזן בתהליכון רקע.
- email_valid() - מסמנת את שדה הדוא"ל כתקין ומאפשרת את כפתור האימות.
- email_invalid() - מסמנת את שדה הדוא"ל כלא תקין.
- call_email_verification() - שולחת בקשה לשליחת קוד אימות דוא"ל.

- email_verified() - מסמנת את הדוא"ל כמאומת ומשביתה את כפתור האימות.
- call_setup_security_questions_window() - פותחת את חלון הגדרת שאלות האבטחה.
- questions_setup_successful() - מעדכנת את שורת שאלות האבטחה לאחר הגדרה מוצלחת.
- call_run_voice_recording_window() - פותחת את חלון הקלטת הקול.
- voice_auth_added() - מעדכנת את שורת אימות הקול לאחר הגדרה מוצלחת.
- call_run_video_capture_window() - פותחת את חלון צילום הווידאו.
- face_recognition_added() - מעדכנת את שורת זיהוי הפנים לאחר הגדרה מוצלחת.
- display_signup_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### AuthSelectWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון בחירת פקטור ההזדהות השני.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.

**פעולות במחלקה**:

- _make_option_row(parent, w, h, icon, title, desc, enabled, command) - מקבלת רכיב אב, רוחב, גובה, אייקון, כותרת, תיאור, דגל הפעלה ופקודה, ויוצרת שורת אפשרות אימות. אם לא מופעלת, מוצגת כמושבתת.
- call_email_login() - סוגרת את החלון ומתחילה כניסה באמצעות קוד דוא"ל.
- call_voice_auth() - סוגרת את החלון ופותחת את חלון הקלטת הקול.
- call_face_recognition() - סוגרת את החלון ופותחת את חלון צילום הווידאו.
- close_window() - משחררת את ה-grab וסוגרת את החלון.

### ChangePasswordWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון שינוי הסיסמה לאתר.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- site_url - כתובת האתר לשינוי הסיסמה.
- site_username - שם המשתמש המשויך לאתר.

**פעולות במחלקה**:

- call_password_preferences_window() - פותחת את חלון העדפות הסיסמה עבור אתר זה.
- change_site_password() - מבקשת סיסמה חדשה לאתר דרך האפליקציה.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון ומחזירה האם המשתמש רוצה שהסיסמה תועתק.

### CodeLoginWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון הזנת קוד האימות שנשלח לאימייל של המשתמש.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- client_email - כתובת הדוא"ל שאליה נשלח הקוד.

**פעולות במחלקה**:

- _handle_keypress(event, index) - מקבלת אירוע מקש ואינדקס שדה, ומטפלת בלחיצת מקשים בשדות קלט הקוד ומנווטת בין השדות.
- _focus_next(index) - מקבלת אינדקס שדה ומעבירה את הפוקוס לשדה הקלט הבא.
- _focus_back(index) - מקבלת אינדקס שדה ומעבירה את הפוקוס לשדה הקלט הקודם.
- get_full_code() - מחזירה את מחרוזת הקוד המלאה על ידי שרשור כל שדות הקלט.
- call_check_code() - שולחת את הקוד שהוזן לאפליקציה לאימות.
- incorrect_code_feedback() - מציגה שגיאת קוד שגוי ומסמנת את שדות הקלט.
- display_code_expired_msg() - מציגה שגיאת קוד שפג תוקפו, מנקה את השדות ומאפשרת את כפתור שליחה מחדש.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה.
- display_msg(msg) - מקבלת הודעה ומציגה אותה.
- call_resend_code() - מבקשת קוד אימות חדש דרך האפליקציה.
- code_resent() - מציגה הודעת אישור ומשביתה את כפתור שליחה מחדש.
- close_window() - סוגרת את החלון.

### HomeFrame

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון מסך הבית.

**תכונות המחלקה**:

אין תכונות נוספות מעבר לתכונות CTkFrame. המחלקה בונה את מסך הבית בלבד.

**פעולות במחלקה**:

אין פעולות נוספות. המחלקה בונה את ממשק מסך הבית ב-__init__ בלבד.

### LoginWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון ההתחברות.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.

**פעולות במחלקה**:

- call_login() - קוראת את שדות שם המשתמש והסיסמה ושולחת בקשת כניסה.
- show_password() - מציגה את הסיסמה בשדה הקלט.
- hide_password() - מסתירה את הסיסמה בשדה הקלט.
- call_security_questions_window() - מבקשת את שאלות האבטחה של המשתמש לשחזור סיסמה.
- display_login_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### PasswordPreferencesWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון הגדרת העדפות יצירת הסיסמה.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- site_url - כתובת האתר שההעדפות חלות עליו, או None בזמן רישום.

**פעולות במחלקה**:

- check_inclusion_selection(label) - מקבלת שם קטגוריה, מוודאת שלפחות checkbox אחד של include מסומן ומסנכרנת את מצב ה-checkbox של start.
- check_start_selection(label) - מקבלת שם קטגוריה, מוודאת שלפחות checkbox אחד של start מסומן ומחליפה את הצגת שדה התווים המיוחדים.
- save_preferences() - מאמתת את ההעדפות ושולחת אותן לאפליקציה.
- _remove_duplicates(str) - מקבלת מחרוזת ומחזירה אותה ללא תווים כפולים.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### ResetPasswordWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון איפוס הסיסמה של המשתמש.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.

**פעולות במחלקה**:

- call_reset_password() - מוודאת ששתי הסיסמאות תואמות ושולחת את בקשת האיפוס.
- show_password(num) - מקבלת מספר שדה ומציגה את הסיסמה בשדה הקלט המתאים.
- hide_password(num) - מקבלת מספר שדה ומסתירה את הסיסמה בשדה הקלט המתאים.

- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### SecurityQuestionsAuthWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון אימות המשתמש באמצעות שאלות אבטחה.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- user_security_questions - רשימת שאלות האבטחה של המשתמש.
- answers_entries - רשימת רכיבי entry עבור כל תשובה.

**פעולות במחלקה**:

- check_answers() - אוספת את כל התשובות ושולחת אותן לאפליקציה לאימות.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### SecurityQuestionsSetupWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון הגדרת שאלות האבטחה.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- security_questions - רשימת שאלות האבטחה הזמינות.
- check_vars - רשימת BooleanVar עבור כל checkbox של שאלה.
- questions_entries - רשימת רכיבי entry עבור התשובה לכל שאלה.

**פעולות במחלקה**:

- on_checkbox(i) - מקבלת אינדקס שאלה, ומציגה או מסתירה את שדה התשובה. מגבילה את הבחירה ל-3 שאלות.
- setup_security_questions() - אוספת את השאלות והתשובות שנבחרו ושולחת אותן לאפליקציה.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון.

### SettingsMenuFrame

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון תפריט ההגדרות של האפליקציה.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- is_open - האם פאנל ההגדרות מוצג כרגע.

**פעולות במחלקה**:

- toggle() - מחליקה את פאנל ההגדרות פנימה או החוצה.
- animate_in() - אנימציית כניסה של הפאנל.
- animate_out() - אנימציית יציאה של הפאנל.
- change_theme() - מחליפה בין מצב כהה לבהיר לפי מצב המתג.
- show_auth_options(auth_options) - מקבלת מילון אפשרויות אימות ומציגה כפתורים להוספת שיטות שלא הוגדרו.
- setup_voice_auth() - פותחת את חלון הקלטת הקול להגדרת אימות קול.
- voice_auth_added() - משביתה את כפתור הוספת אימות קול לאחר הגדרה.
- setup_face_recognition() - פותחת את חלון צילום הווידאו להגדרת זיהוי פנים.
- face_recognition_added() - משביתה את כפתור הוספת זיהוי פנים לאחר הגדרה.

### SiteInfoWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון הצגת פרטי הכניסה של אתר.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- site_username - שם המשתמש באתר.
- site_password - הסיסמה באתר.

**פעולות במחלקה**:

- close_window() - סוגרת את החלון.
- call_copy_password() - מעתיקה את סיסמת האתר ללוח ומציגה אישור.
- show_password() - מציגה את הסיסמה בשדה הקלט.
- hide_password() - מסתירה את הסיסמה בשדה הקלט.

### SiteRecoveryWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון שחזור אתרים שנמחקו.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.

**פעולות במחלקה**:

- on_site_selected(selected_value) - מקבלת את הערך שנבחר, מסמנת את האתר ומאפשרת את כפתור השחזור.
- recover_site() - שולחת בקשת שחזור עבור האתר הנבחר.
- display_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.

- close_window() - משחררת את ה-grab וסוגרת את החלון.

### SiteRegistrationWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון רישום אתר חדש.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- preferences - העדפות יצירת סיסמה, או None עבור ברירת מחדל.

**פעולות במחלקה**:

- call_password_preferences_window() - פותחת את חלון העדפות הסיסמה עם ההעדפות הנוכחיות.
- call_register_site() - מאמתת את השדות ושולחת בקשת רישום אתר.
- display_registration_error(msg) - מקבלת הודעה ומציגה אותה כשגיאה בחלון.
- close_window() - סוגרת את החלון ומחזירה האם המשתמש רוצה שהסיסמה תועתק.

### ToolTip

**תפקיד המחלקה**: המחלקה אחראית על הצגת חלונית הסבר קצר מעל רכיבי ממשק.

**תכונות המחלקה**:

- widget - הרכיב שה-tooltip מוצמד אליו.
- text - טקסט ה-tooltip.
- tip_window - חלון ה-tooltip הפתוח כרגע, או None.
- after_id - מזהה הטיימר להשהיית הצגת ה-tooltip, או None.

**פעולות במחלקה**:

- show_tooltip(event) - מקבלת אירוע עכבר ומתזמנת הצגת tooltip לאחר השהייה של 500 מילישניות.
- _create_tooltip(x, y) - מקבלת קואורדינטות x ו-y ויוצרת את חלון ה-tooltip במיקום הנתון.
- hide_tooltip(*args) - מבטלת את הטיימר וסוגרת את חלון ה-tooltip.

### VideoCaptureWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון צילום הוידאו עבור זיהוי פנים.

**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- cameras_list - רשימת התקני המצלמה הזמינים.

- is_loop_running - האם לולאת צילום הווידאו פועלת.

**פעולות במחלקה**:

- select_camera(choice) - מקבלת בחירת מצלמה ומתחילה צילום וידאו ממנה.
- record_loop() - לוכדת פריימים ברציפות מהמצלמה ומעדכנת את תווית הווידאו. עוצרת אם המצלמה לא זמינה.
- update_frame(img) - מקבלת תמונה ומעדכנת את תווית הווידאו עם פריים חדש.
- face_recognition_loop() - בודקת זיהוי פנים כל 500 מילישניות בזמן ההקלטה.
- enable_setup_button() - מאפשרת את כפתור ההגדרה/אימות כאשר זוהו פנים.
- disable_setup_button() - משביתה את כפתור ההגדרה/אימות כאשר לא זוהו פנים.
- setup_face_recognition() - שולחת את קידוד הפנים שנלכד לשרת להגדרה.
- display_camera_error() - מציגה שגיאת מצלמה לא נמצאה בתווית הווידאו.
- display_face_recognition_error(msg) - מקבלת הודעה ומציגה אותה כשגיאת זיהוי פנים.
- close_window() - עוצרת את לולאת ההקלטה, משחררת את המצלמה וסוגרת את החלון.

## VoiceRecordingWindow

**תפקיד המחלקה**: המחלקה אחראית על ממשק המשתמש של חלון הקלטת קול לצורך אימות או הגדרה.

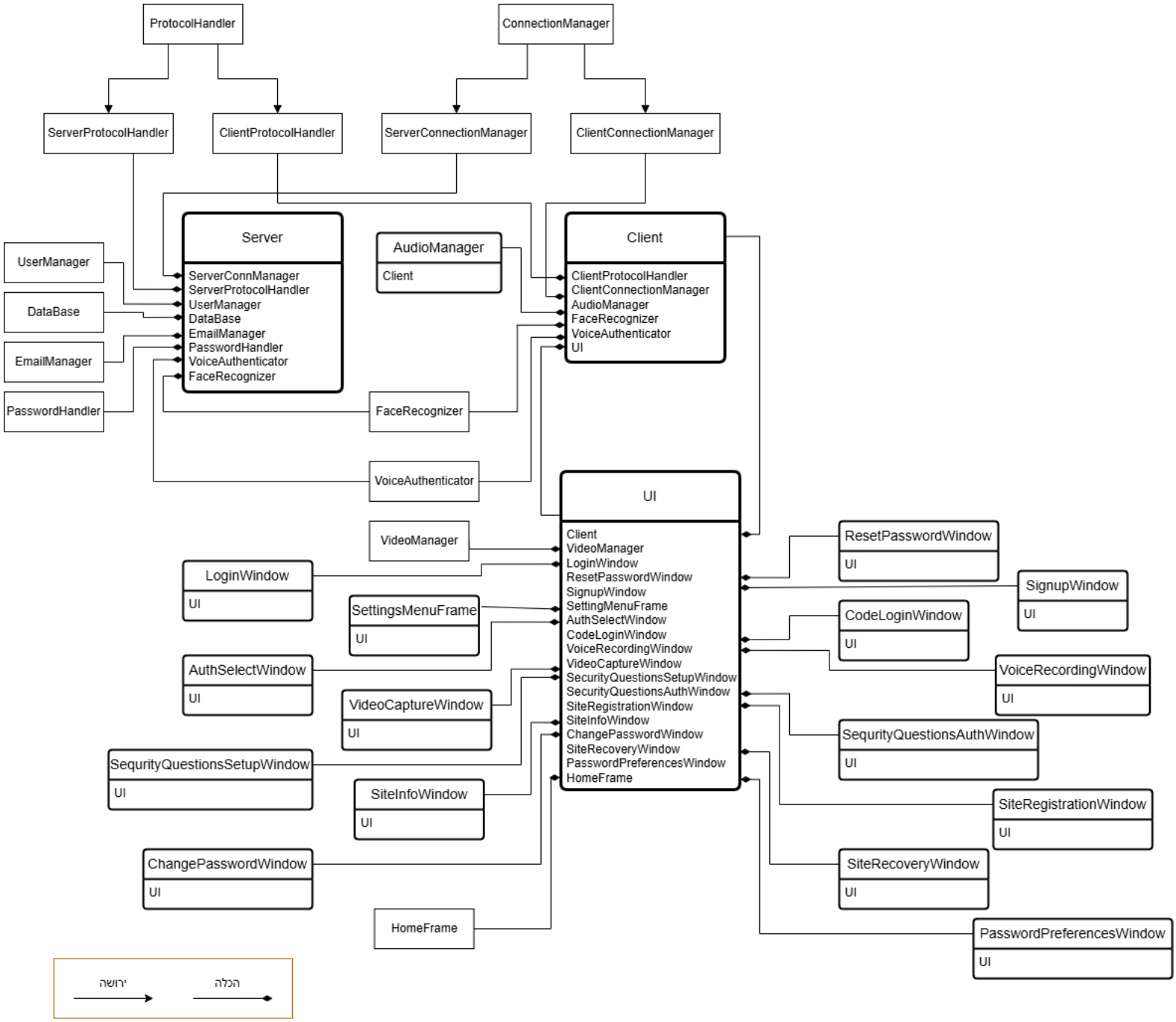
**תכונות המחלקה**:

- app - הפניה לאובייקט ה-UI.
- is_recording - האם הקלטת קול פעילה כרגע.

**פעולות במחלקה**:

- call_voice_recording() - מתחילה הקלטת קול ומפעילה את לולאת סרגל ההתקדמות.
- recording_progress_loop() - מעדכנת את סרגל ההתקדמות בזמן ההקלטה עד שמגיעים לזמן המקסימלי.
- call_stop_recording() - עוצרת את הקלטת הקול דרך האפליקציה.
- recording_stopped() - מעדכנת את ה-UI כדי להראות שההקלטה נעצרה.
- setup_voice_auth() - שולחת את הקול שהוקלט לשרת להגדרה.
- call_voice_auth() - שולחת את הקול שהוקלט לשרת לאימות.
- display_error(error_msg) - מקבלת הודעת שגיאה, מציגה אותה ומשביתה את כפתור האימות.
- close_window() - סוגרת את החלון.

## תרשים קשרים בין מחלקות הפרויקט



## אלגוריתמים וקטעי קוד מיוחדים

הסבר על האלגוריתמים נמצא בפרק אלגוריתמים מרכזיים בפרויקט.

### הצפנה בתקשורת

**ServerConnectionManager**

```
    def setup_socket(self):
        """Sets up the server socket and generates the RSA keys"""
        server_socket = self.create_socket()
        server_socket.bind(("0.0.0.0", SERVER_PORT))
        server_socket.listen()
        self.generate_RSA()
        threading.Thread(target=self.reset_client_message_count_loop).start()
        # print("Server is up and running")
        return server_socket

    def generate_RSA(self):
        """Generates public and private RSA keys. Saves the private RSA key in an environment
variable."""
        private_key = rsa.generate_private_key(public_exponent = 65537, key_size = 4096) # creates a
4096 bit private RSA key
        public_key = private_key.public_key()

        private_pem = private_key.private_bytes(encoding = serialization.Encoding.PEM,
                                                format = serialization.PrivateFormat.PKCS8,
                                                encryption_algorithm = serialization.NoEncryption())

        self.public_pem = public_key.public_bytes(encoding = serialization.Encoding.PEM,
                                                  format = serialization.PublicFormat.SubjectPublicKeyInfo)

        os.environ["RSA_PRIVATE_KEY"] = base64.b64encode(private_pem).decode("utf-8")


    def _load_and_recv_RSA(self):
        """Loads the private RSA key from the environment variable, and returns it decoded."""
        b64_string = os.environ.get("RSA_PRIVATE_KEY")
        private_pem = base64.b64decode(b64_string.encode())
        return private_pem

    def recv_and_save_AES(self, client_socket):
        """Receives an encrypted AES key from the client, decrypts it and saves it in a dictionary."""
        encrypted_AES = self.receive_msg(client_socket)
        private_key = serialization.load_pem_private_key(self._load_and_recv_RSA(), None)

        AES_key = private_key.decrypt(encrypted_AES,
                                      padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
                                      algorithm=hashes.SHA256(),
                                      label=None))
```

```
        self._sockets_to_keys[client_socket] = AES_key


    def handshake(self, client_socket):
        """Receives a client socket and starts the key exchange process by sending
        the public RSA key to the client"""
        self.send_msg(client_socket, self.public_pem)
        self.recv_and_save_AES(client_socket)


    def encrypt_msg(self, client_socket, msg):
        """Receives a message and encrypts it using the client's AES key"""
        client_AES = self._sockets_to_keys[client_socket]
        encrypted_msg = self.encrypt_data(msg, client_AES)
        return encrypted_msg

    def decrypt_msg(self, client_socket, msg):
        """Receives a message and decrypts it using the client's AES key"""
        client_AES = self._sockets_to_keys[client_socket]
        decrypted_msg = self.decrypt_data(msg, client_AES)
        return decrypted_msg
```

**ClientConnectionManager**

```
    def connect(self, server_ip):
        """Receives the server's ip and connects the client socket to it"""
        self.socket.connect((server_ip, SERVER_PORT))
        self.handshake()

    def generate_and_send_AES(self):
        """Generates an AES key, encrypts it using the server's public RSA key, and sends it to the
server."""
        AES_key = Fernet.generate_key()
        os.environ["AES_KEY"] = base64.b64encode(AES_key).decode("utf-8")
        AES_msg = self.public_RSA_key.encrypt(AES_key,
                                              padding.OAEP(mgf= padding.MGF1(algorithm=
hashes.SHA256()),
                                                                                  algorithm=
hashes.SHA256(),
                                                                                  label= None))
        self.send_msg(self.socket, AES_msg)

    def recv_RSA(self):
        """Receives the public RSA key from the server"""
        public_RSA_pem = self.receive_msg(self.socket)
        self.public_RSA_key = serialization.load_pem_public_key(public_RSA_pem.encode())
```

```
    def get_AES(self):
        """Get the AES key from the environment variables"""
        b64_string = os.environ.get("AES_KEY")
        AES_key = base64.b64decode(b64_string.encode())
        return AES_key

    def handshake(self):
        """Continues the key exchange process with the server"""
        self.recv_RSA()
        self.generate_and_send_AES()
```

**ConnectionManager – המשותף לשרת וללקוח**

```
    def encrypt_data(self, data, AES_key):
        """Method receives data and an AES key, and returns the encrypted message"""
        encryptor = Fernet(AES_key)
        msg_bytes = data.encode()
        return encryptor.encrypt(msg_bytes)

    def decrypt_data(self, data, AES_key):
        """Method receives data and an AES key, and returns the decrypted data"""
        decryptor = Fernet(AES_key)
        try:
            decrypted_msg = decryptor.decrypt(data).decode()
        except Exception as e:
            print(e)
            print("exeption in decrypt")
            return None
        else:
            return decrypted_msg
```

בקוד רואים את תהליך החלפת המפתחות בין השרת ללקוח, ובסוף את פונקציות ההצפנה והפענוח הסימטריים המשמשות את השרת והלקוח בתקשורת.

## הצפנה בבסיס הנתונים

**DataBase**

```
    def derive_key(self, password, salt):
        """Receives a password and a salt, and derives a 32-byte encryption key using PBKDF2 with
SHA256."""
        kdf = PBKDF2HMAC(
            algorithm = hashes.SHA256(),
            length = 32,
            salt = salt,
            iterations = 600000,
```

```
        )
        return kdf.derive(password)

    def encrypt_data(self, user_id, data):
        """Receives a user_id and a list of data items, and returns a list of the data items encrypted using the user's DK."""
        self.cur.execute("""SELECT user_dk FROM USERS WHERE user_id = ?""", (user_id, ))
        encrypted_dk = self.cur.fetchone()[0]
        dk_encryption_key = self.get_dk_encryption_key(user_id)
        f = Fernet(dk_encryption_key)
        dk = f.decrypt(encrypted_dk)
        data_f = Fernet(dk)
        encrypted_data = []
        for i in data:
            if type(i) != bytes:
                i = i.encode()
            encrypted_data.append(data_f.encrypt(i))
        return encrypted_data

    def decrypt_data(self, user_id, data):
        """Receives a user_id and a list of encrypted data items, and returns a list of the data items decrypted using the user's DK."""
        self.cur.execute("""SELECT user_dk FROM USERS WHERE user_id = ?""", (user_id, ))
        encrypted_dk = self.cur.fetchone()[0]
        dk_encryption_key = self.get_dk_encryption_key(user_id)
        f = Fernet(dk_encryption_key)
        dk = f.decrypt(encrypted_dk)
        data_f = Fernet(dk)
        decrypted_data = []
        for i in data:
            try:
                decrypted_data.append(data_f.decrypt(i).decode())
            except:
                decrypted_data.append(data_f.decrypt(i))
        return decrypted_data

    def create_dk_encryption_key(self, username, password_hex, key_salt):
        """Receives a username, password hex and key salt, and returns a DK encryption key derived from them."""
        raw_key = self.derive_key((username + password_hex).encode(), key_salt)
        dk_encryption_key = base64.urlsafe_b64encode(raw_key)
        return dk_encryption_key

    def get_dk_encryption_key(self, user_id):
        """Receives a user_id and returns their DK encryption key.\n
        If the key is not found in the users_keys dictionary, derives it from the database and saves it."""
        dk_encryption_key = self.users_keys.get(user_id)
```

```
        if dk_encryption_key is None:
            self.cur.execute("""SELECT username, password, key_salt FROM USERS WHERE user_id = ?""",
(user_id, ))
            username, password_hex, key_salt = self.cur.fetchone()
            dk_encryption_key = self.create_dk_encryption_key(username, password_hex, key_salt)
            self.users_keys[user_id] = dk_encryption_key
        return dk_encryption_key


    def user_signup(self, username, password, email, phone_number, voice_signature, face_encoding,
questions_dict):
        """Receives user signup data, and saves the user in the database.\n
        Hashes the password with a random salt, generates a DK, encrypts all sensitive user data with
the DK,
        encrypts the DK with a key derived from the username and password, and inserts the user into
the database.\n
        Returns the user_id if successful, None otherwise."""
        try:
            random_password_salt = os.urandom(32)
            salted_password = random_password_salt + password.encode()
            hashed_password = hashlib.sha256(salted_password)
            password_hex = hashed_password.hexdigest()

            user_dk = Fernet.generate_key()
            dk_f = Fernet(user_dk)
            encrypted_email = dk_f.encrypt(email.encode())
            encrypted_phone_number = dk_f.encrypt(phone_number.encode())

            if voice_signature:
                encrypted_voice_signature = dk_f.encrypt(voice_signature)
            else:
                encrypted_voice_signature = None

            if face_encoding:
                encrypted_face_encoding = dk_f.encrypt(face_encoding)
            else:
                encrypted_face_encoding = None

            random_key_salt = os.urandom(16)
            dk_encryption_key = self.create_dk_encryption_key(username, password_hex, random_key_salt)

            f = Fernet(dk_encryption_key)
            encrypted_user_dk = f.encrypt(user_dk)

            q_id_1, q_id_2, q_id_3 = questions_dict.keys()
            answer_1 = questions_dict[q_id_1]
            answer_2 = questions_dict[q_id_2]
            answer_3 = questions_dict[q_id_3]
            answers_salt = os.urandom(32)
```

```
            hashed_answer_1 = hashlib.sha256(answer_1.encode() + answers_salt).hexdigest()
            hashed_answer_2 = hashlib.sha256(answer_2.encode() + answers_salt).hexdigest()
            hashed_answer_3 = hashlib.sha256(answer_3.encode() + answers_salt).hexdigest()
            self.cur.execute("""INSERT INTO USERS
                                (username, password, password_salt, email, phone_number, user_dk,
key_salt, voice_signature, face_encoding,
                                question_id_1, answer_1, question_id_2, answer_2, question_id_3,
answer_3, answers_salt)
                               VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
                               (username, password_hex, random_password_salt, encrypted_email,
encrypted_phone_number, encrypted_user_dk, random_key_salt, encrypted_voice_signature,
                                encrypted_face_encoding, q_id_1, hashed_answer_1, q_id_2,
hashed_answer_2, q_id_3, hashed_answer_3, answers_salt))
        except sqlite3.IntegrityError as e:
            return None
        except AttributeError as e:
            return None

        user_id = self.cur.lastrowid

        self.conn.commit()

        self.users_keys[user_id] = dk_encryption_key
        return user_id

    def reset_user_password(self, username, new_password):
        """Receives a username and a new password, and resets the user's password.\n
        Decrypts the user's DK with the old DK encryption key, hashes the new password,
        derives a new DK encryption key and re-encrypts the DK with it.\n
        Returns the user_id."""
        self.cur.execute("""SELECT password, password_salt, user_id, key_salt, user_dk FROM USERS
WHERE username = ?""", (username, ))
        user_password, password_salt, user_id, key_salt, user_dk = self.cur.fetchone()

        dk_encryption_key = self.create_dk_encryption_key(username, user_password, key_salt)

        f = Fernet(dk_encryption_key)
        decrypted_user_dk = f.decrypt(user_dk)


        new_salted_password = password_salt + new_password.encode()
        new_hashed_password = hashlib.sha256(new_salted_password)
        new_password_hex = new_hashed_password.hexdigest()

        new_dk_encryption_key = self.create_dk_encryption_key(username, new_password_hex, key_salt)

        new_f = Fernet(new_dk_encryption_key)
        reencrypted_user_dk = new_f.encrypt(decrypted_user_dk)
```

```
        self.cur.execute("""UPDATE USERS
                        SET password = ?, user_dk = ?
                        WHERE user_id = ?""",
                        (new_password_hex, reencrypted_user_dk, user_id))

        self.conn.commit()

        self.users_keys[user_id] = new_dk_encryption_key

        return user_id
```

בקוד רואים את פונקציות ההצפנה, פענוח, וה – KDF, ואת פונקציות ההרשמה ואיפוס הסיסמה שקובעות את מפתחות ההצפנה ופענוח של המשתמשים בבסיס הנתונים.

## זיהוי פנים

```
class FaceRecognizer:

    @staticmethod
    def get_face_encodings(rgb_frame):
        """Receives an RGB frame, detects faces and returns the face encoding of the first face found
as binary data.
        Returns None if no face was detected."""
        face_locations = face_recognition.face_locations(rgb_frame, number_of_times_to_upsample= 2,
model= "hog")
        face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

        if not face_encodings:
            return None
        else:
            encoding_array = face_encodings[0]
            bin_face_encoding = encoding_array.astype(np.float64).tobytes()
            return bin_face_encoding


    @staticmethod
    def compare_faces(stored_bin_face_encoding, bin_face_encoding):
        """Receives a stored face encoding and a login face encoding as binary data,
        compares the two face encodings and returns True if they match, False otherwise."""
        if not bin_face_encoding or not stored_bin_face_encoding:
            return False

        stored_face_encoding = np.frombuffer(stored_bin_face_encoding, dtype= np.float64)
        face_encoding = np.frombuffer(bin_face_encoding, dtype= np.float64)

        matches = face_recognition.compare_faces([stored_face_encoding], face_encoding)
```

```
        if True in matches:
            return True
        else:
            return False
```

בקוד רואים את פונקציות הפקת נתוני הפנים מתוך תמונה והשוואת וקטורי נתוני פנים לצורך זיהוי פני המשתמש.

## זיהוי קול

```
BASE_PATH = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_PATH, "Model")
SPK_MODEL_PATH = os.path.join(BASE_PATH, "SpkModel")
SAMPLES_PER_SECOND = 16000
THRESHOLD = 0.7

class VoiceAuthenticator:
    def __init__(self):
        SetLogLevel(-1) # no vosk logs
        self.model = Model(MODEL_PATH)    # Vosk speech recognition model
        self.spk_model = SpkModel(SPK_MODEL_PATH)    # Vosk speaker recognition model

    def calc_voice_signature(self, voice_file):
        """Receives a voice file path, processes the audio and returns the voice signature as binary
data.
        Returns None if no voice signature could be extracted."""
        rec = KaldiRecognizer(self.model, SAMPLES_PER_SECOND, self.spk_model)
        wf = wave.open(voice_file, "rb")
        data = wf.readframes(4000)
        while data:
            rec.AcceptWaveform(data)
            data = wf.readframes(4000)

        result = json.loads(rec.FinalResult())
        voice_signature = result.get("spk")
        if voice_signature is None:
            return None
        bin_voice_signature = np.array(voice_signature).astype(np.float32).tobytes()
        return bin_voice_signature

    def verify_user(self, bin_voice_signature, bin_login_signature):
        """Receives a stored voice signature and a login voice signature as binary data,
        compares the two signatures and returns True if they match, False otherwise."""
        v1 = np.frombuffer(bin_voice_signature, dtype = np.float32)
        v2 = np.frombuffer(bin_login_signature, dtype = np.float32)

        similarity = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
```

```
        print(f"similarity: {similarity}")
        return similarity >= THRESHOLD
```

בקוד רואים את פונקציות חישוב חתימת הקול של המשתמש, ואת פונקציית אימות המשתמש לפי זיהוי הקול.

# מסמך בדיקות למערכת

## בדיקת הרשמה לאפליקציה

**מטרת הבדיקה**: בדיקה שמשתמש חדש יכול להירשם לאפליקציה, עם כל חלקי ההרשמה (שם משתמש, סיסמה, אימייל ווידוא של האימייל, הגדרת שאלות אבטחה של המשתמש, הקמת זיהוי קול ופנים בהצלחה), ובדיקה של ההתחברות האוטומטית לאפליקציה לאחר ההרשמה.

**מה בוצע**: נכנסתי למסך ההרשמה, מילאתי את כל טופס ההרשמה, והקמתי את זיהוי הקול והפנים בהצלחה. בדקתי גם מה קורה במקרי קצה (סיסמה לא חזקה מספיק, שם משתמש קיים, הלקוח לא הכניס שם משתמש, הלקוח לא הגדיר שאלות אבטחה).

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. קיבלתי הודעות שגיאה בממשק המשתמש עבור מקרי הקצה, וכשמילאתי את הטופס באופן תקין ההרשמה עבדה בהצלחה, והמערכת התחברה אוטומטית למשתמש שיצרתי.

## בדיקת התחברות לאפליקציה

**מטרת הבדיקה**: בדיקה שהלקוח יכול להתחבר למשתמש קיים באפליקציה עם שם משתמש וסיסמה, וכל פעם עם פקטור הדהות שני אחר. כוללת גם בדיקת התחברות עם הזנת פרטי התחברות לא נכונים ובדיקת הודעות השגיאה המוצגות למשתמש.

**מה בוצע**: נכנסתי למסך ההתחברות, וניסיתי קודם להתחבר עם פרטי הזדהות לא נכונים. לאחר מכן, ניסיתי להתחבר לאפליקציה שלוש פעמים: פעם אחת עם קוד לאימייל, פעם שנייה עם זיהוי קול, ופעם שלישית עם זיהוי פנים.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. קיבלתי הודעות שגיאה בממשק המשתמש עבור פרטי ההזדהות השגויים, וכשמילאתי פרטי התחברות נכונים הצלחתי להתחבר בכל אחת משלושת האפשרויות לפקטור שני להזדהות.

## בדיקת רישום אתרים חדשים עבור משתמש

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לרשום אתרים חדשים לאפליקציה, והם יופיעו ברשימת האתרים הרשומים של המשתמש.

**מה בוצע**: רשמתי שלושה אתרים חדשים למשתמש באפליקציה ובדקתי שהם מופיעים ברשימת האתרים הרשומים. בנוסף בדקתי גם את פונקציית ההעתקה ל – clipboard של הסיסמה.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. האתרים נרשמו, וניתן היה לראות אותם ברשימת האתרים הרשומים. ההעתקה ל – clipboard עבדה כשסימנתי תיבת הקלט.

## בדיקת צפייה במידע על אתר רשום

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לבחור אתר מרשימת האתרים הרשומים ולצפות במידע הרשום על האתר, ושהמידע שיופיע יהיה נכון ומדויק.

**מה בוצע**: בחרתי אתר מרשימת האתרים השמורים, ולחצתי על כפתור הצפייה במידע על האתרים כדי לראות את המידע השמור עליו. בנוסף בדקתי גם את פונקציית ההעתקה ל – clipboard של הסיסמה.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. המידע על האתר הוצג במסך והיה מדויק ונכון. גם פונקציית העתקת הסיסמה ל – clipboard עבדה.

### בדיקת הגדרת העדפות ליצירת סיסמה לאתר

**מטרת הבדיקה**: בדיקה שהמשתמש יכול להגדיר העדפות ליצירת הסיסמה, ושהסיסמה שתיווצר לאתר באמת תתאים להעדפות שהמשתמש קבע.

**מה בוצע**: במסך רישום האתרים שיניתי את ההעדפות ליצירת הסיסמה לאתר, בדקתי העדפות שונות, ולבסוף בדקתי שהסיסמה שנוצרה עבור האתר תאמה את ההעדפות שקבעתי.

**תוצאות הבדיקה**: היו כמה בעיות קטנות שבהן נתקלתי – ממשק המשתמש לא עבד כמו שצריך (היה אפשר שלא יהיה מסומן כלום באפשרויות שאיתן הסיסמה יכולה להתחיל – וזה היה מקריס את האלגוריתם), היה אפשר להכניס אורך סיסמה בלי הגבלה ואז אם הכנסתי אורך סיסמה גדול מידי זה היה מקריס את התוכנית, וההעדפות שנקבעו לא הוצגו בממשק המשתמש אם נכנסתי ויצאתי ממסך הגדרת ההעדפות.

**כיצד נפתרה הבעיה**: תיקנתי את הבעיה בממשק המשתמש על ידי תיקון אלגוריתם סימון האפשרויות במסך, הגבלתי את אורך הסיסמה שאפשר להכניס, ושמרתי את ההעדפות במשתנה זמני כדי שהמסך יתעדכן אם יוצאים ונכנסים אליו לפני השליחה של ההעדפות לשרת.

### בדיקת שינוי סיסמה לאתר רשום

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לבחור אתר מרשימת האתרים הרשומים ולשנות את הסיסמה השמורה אליו.

**מה בוצע**: בחרתי אתר מרשימת האתרים הרשומים, לחצתי על כפתור שינוי הסיסמה ובדקתי שהסיסמה שונתה.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. הסיסמה שונתה, וראיתי שהיא שונתה לפי ההעדפות ליצירת הסיסמה.

### בדיקת מחיקת אתר רשום

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לבחור אתר ולמחוק אותו מרשימת האתרים הרשומים.

**מה בוצע**: בחרתי אתר מרשימת האתרים הרשומים ולחצתי על כפתור מחיקת האתר כדי למחוק אותו.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. האתר נמחק מרשימת האתרים הרשומים.

### בדיקת שחזור אתר שנמחק

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לשחזר אתר שנמחק מרשימת האתרים הרשומים.

**מה בוצע**: לחצתי על כפתור שחזור אתר, בחרתי אתר מחוק מרשימת האתרים שנמחקו, ולחצתי על כפתור השחזור.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. האתר שבחרתי שוחזר לרשימת האתרים הרשומים.

### בדיקת שדה חיפוש האתרים הרשומים

**מטרת הבדיקה**: בדיקה שהמשתמש יכול לחפש אתר מסוים בשדה החיפוש במסך הראשי ושהאתר / אתרים המתאימים ורק הם יופיעו ברשימת האתרים הרשומים.

**מה בוצע**: הקלדתי שם של אתר אחד מתוך רשימת האתרים הרשומים בשדה חיפוש האתרים.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. רק האתר שחיפשתי הופיע ברשימת האתרים הרשומים.

### בדיקת הוספת אמצעי ההתחברות הביומטריים מחלונית ההגדרות

**מטרת הבדיקה**: בדיקה שהמשתמש יכול להוסיף זיהוי קול / זיהוי פנים מחלונית ההגדרות במסך הראשי אם הוא לא הגדיר אותם בזמן הרישום לאפליקציה. צריך לבדוק גם שאמצעי ההתחברות שנוספו באמת תקינים והמשתמש יכול להתחבר באמצעותם לאפליקציה.

**מה בוצע**: רשמתי משתמש נוסף, אך בזמן הרישום לא הקמתי את זיהוי הקול והפנים. לאחר התחברות, הוספתי את זיהוי הקול והפנים מחלונית ההגדרות במסך הראשי. אחר כך ניסיתי להתחבר לאפליקציה ממשתמש זה בעזרת זיהוי הקול והפנים שהוספתי.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. הקמת ההזדהויות הביומטריות עבדה וגם ההתחברות איתן לאחר מכן. בנוסף, האפשרות להוסיף את ההזדהויות ירדה מחלונית ההגדרות מכיוון שהן כבר הוגדרו.

### בדיקת ממשק המשתמש במצב dark mode

**מטרת הבדיקה**: בדיקה שממשק המשתמש נראה תקין ועובד כרגיל גם כאשר הוא מוצג ב – dark mode.

**מה בוצע**: נכנסתי לאפליקציה והפעלתי את מצב dark mode בחלונית ההגדרות. עברתי על כל המסכים כדי לבדוק שממשק המשתמש נראה תקין ועובד כרגיל.

**תוצאות הבדיקה**: היו כמה כפתורים שלא נראו כמעט על הרקע הכהה.

**כיצד נפתרה הבעיה**: שיניתי את הצבע של הכפתורים במצב dark mode כדי שייראו על הרקע הכהה.

### בדיקת תהליך איפוס הסיסמה

**מטרת הבדיקה**: בדיקה שהמשתמש יכול להתחיל את תהליך איפוס הסיסמה עם כפתור שכחתי סיסמה במסך ההתחברות, שהוא יכול לענות על שאלות האבטחה שהוא הגדיר בזמן ההרשמה, ושהוא יכול לאפס את הסיסמה שלו. לבדוק גם שהסיסמה באמת שונתה ושהמשתמש יכול להתחבר באמצעות הסיסמה החדשה.

**מה בוצע**: במסך ההתחברות הכנסתי את שם המשתמש ולחצתי על שכחתי סיסמה. אחר כך עניתי על שאלות האבטחה בצורה נכונה ולא נכונה. לאחר מכן במסך איפוס הסיסמה בדקתי כמה מקרי קצה (סיסמאות לא תואמות בין השדות, סיסמה לא מספיק חזקה) ואז איפסתי את הסיסמה. לבסוף בדקתי אם המשתמש יכול להתחבר לאפליקציה עם הסיסמה החדשה ולא עם הקודמת.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. כל השגיאות שהיו אמורות להיות מוצגות בממשק המשתמש הוצגו, כל השלבים באיפוס הסיסמה עבדו והצלחתי לבסוף להתחבר לאפליקציה עם הסיסמה החדשה של המשתמש.

### בדיקת ההצפנה בתקשורת

**מטרת הבדיקה**: בדיקה שהנתונים העוברים בתקשורת מוצפנים ואי אפשר להאזין ולגנוב מידע רגיש.

**מה בוצע**: הוספתי שורה בפונקציית השליחה בתקשורת שלי, שבה הדפסתי את ההודעה, אחרי שהיא כבר הייתה אמורה להיות מוצפנת ומוכנה לשליחה, אבל רגע לפני השליחה.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. ההודעות שהודפסו היו מוצפנות ואי אפשר לפענח אותן ולגנוב מידע בתקשורת.

### בדיקת אבטחת בסיס הנתונים

**מטרת הבדיקה**: בדיקה שהנתונים הרגישים בבסיס הנתונים מוצפנים או hashed, ואי אפשר לגנוב מידע רגיש של המשתמשים מבסיס הנתונים.

**מה בוצע**: פתחתי את בסיס הנתונים באפליקציית DB Browser המאפשרת צפייה בנתונים המאוחסנים בבסיס הנתונים, ובדקתי את מצב הנתונים ורמת האבטחה שלהם.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. כל המידע הרגיש של המשתמשים בבסיס הנתונים אכן היה מוצפן או hashed, ואי אפשר לפענח אותו ולגנוב אותו.

### בדיקת הגנת DoS

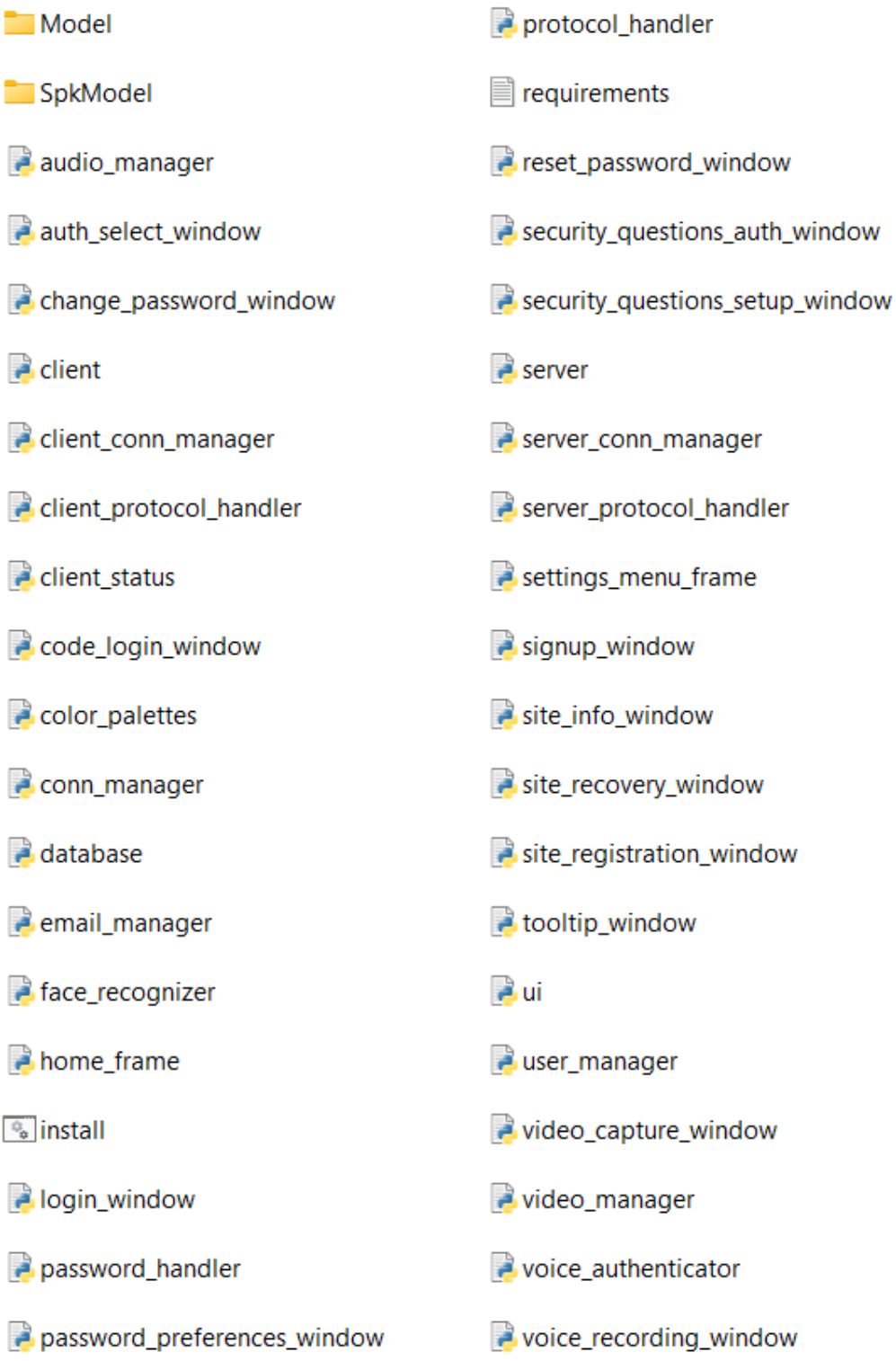
**מטרת הבדיקה**: בדיקה שהשרת מנתק את הלקוח אם הוא עבר את מגבלת הבקשות לשרת בפרק זמן קצר.

**מה בוצע**: לחצתי על כפתור הרבה פעמים בזמן קצר והעמסתי על השרת בבקשות.

**תוצאות הבדיקה**: הבדיקה עברה בהצלחה. השרת ניתק את התקשורת איתי וקיבלתי מסך המציג הודעת שגיאה מתאימה.

# מדריך למשתמש

קבצי המערכת




רוב קבצי המערכת הם קבצי פייתון המכילים מחלקה אחת מהפרויקט שלי.

הקבצים החריגים הם:

1. התיקיות Model ו- SpkModel - אלה המודלים לזיהוי הקול של Vosk.
2. client_status – קובץ פייתון שרק מגדיר enum עבור מצבי הלקוח.
3. color_palletes – קובץ פייתון המגדיר את פלטת הצבעים עבור ממשק המשתמש בפרויקט.
4. requirements.txt – קובץ המפרט את כל המודולים המיובאים שצריך להתקין עבור הפרויקט.
5. install.bat – קובץ batch שניתן להריצו בשביל התקנה קלה של כל המודולים המיובאים המפורטים ב – requirements.txt.

על מנת שהאפליקציה תרוץ ותעבוד כמו שצריך, צריך לוודא שכל קבצי המערכת מותקנים.

## התקנת המערכת

### הסביבה הנדרשת

המערכת חייבת גרסה 3.13 של פייתון בשביל לרוץ.

חובה להתקין את כל קבצי המערכת המפורטים לעיל.

### הכלים הנדרשים

Visual Studio Code – הסביבה בה נכתב קוד הפרויקט, ובה ניתן להריץ אותו.

pip – כלי חובה להתקנת המודולים המיובאים הדרושים לפרויקט.

C++ build tools – יש להתקין לפני התקנת כל המודולים המיובאים. ניתן להתקין מהקישור הבא: Microsoft C++ Build Tools Visual Studio. במסך ההתקנה צריך לסמן "Desktop development with C++".

להתקנה פשוטה של כל הספריות והמודולים המיובאים, ניתן להריץ את הקובץ install.bat ב – terminal. בלי כל הספריות והמודולים המיובאים, המערכת לא תרוץ או לא תפעל כראוי.

### נתונים התחלתיים

אין צורך בנתונים התחלתיים לבסיס הנתונים.
גם אם אני אוסיף משתמש מראש, אף אדם שהוא לא אני לא יוכל להתחבר אליו בגלל הפקטור השני הדרוש להזדהות (הנתונים הביומטריים והאימייל הם ייחודיים לי), וזה היופי בתהליך ההתחברות אל האפליקציה שלי.

### רשת

ניתן להריץ את הפרויקט על כמה מחשבים שונים באותה רשת תקשורת מקומית.
השרת, לאחר שהוא עולה, מדפיס ל – terminal את ה ip הפרטי של המחשב עליו הוא מורץ ברשת המקומית. לאחר מכן, ניתן להריץ את הלקוח במחשב אחר ולתת לו פרמטר שהוא כתובת ה ip הפרטית של השרת.
את ההרצה מהסוג הזה של הלקוח עושים דרך ה – terminal ולא בעזרת כפתור ההרצה ב – Visual Studio Code. הפקודה ב – terminal היא:

```
python client.py “(server private ip)”
```

### ארכיטקטורה מינימלית נדרשת להרצת המערכת

נדרש מחשב אחד לשרת ולפחות מחשב אחד ללקוח כדי להדגים את התקשורת בין שניהם על גבי הרשת. ניתן גם להריץ על אותו המחשב גם את השרת וגם את הלקוח בחלונות שונים.

כדי להדגים את כל יכולות המערכת, למחשב הלקוח צריך להיות מחובר גם מיקרופון וגם מצלמה שעובדים. בלית ברירה, אפשר גם להריץ את הלקוח בלי מיקרופון ומצלמה ללא בעיות.

### הפעלת המערכת

כל המשתמשים באפליקציה יכולים להתחבר למשתמש שלהם או להירשם עם משתמש חדש לאפליקציה. ההתחברות היא התחברות דו שלבית הכוללת קוד חד פעמי הנשלח לאימייל או אימות ביומטרי (זיהוי קול או פנים).

לאחר ההתחברות, המשתמשים יכולים לצפות באתרים הרשומים אצלם באפליקציה, לרשום אתרים חדשים, לשנות סיסמה שמורה לאתר, למחוק אתר ולשחזר אתר.

כל מסכי המערכת בנויים גם ל – light mode וגם ל – dark mode. המשתמש יכול לשנות את מצב המסכים דרך חלונית ההגדרות במסך הראשי או במסך הבית.

צילומים של מסכי המערכת, פירוט על כל אחד מהם ותרשים זרימה המציג את המעברים בין המסכים נמצאים תחת הכותרת "מסכי המערכת" בפרק מבנה/ארכיטקטורה של הפרויקט.

# רפלקציה

בתחילת השנה, התבקשנו לחשוב על נושא לפרויקט בסייבר. חשבתי איזה פרויקט אני יכול לעשות שהוא לא יהיה אפליקציה סתמית, אלא יהיה פרויקט "סייברי" עם דרישות אבטחה גבוהות. משם יצאתי עם המסקנה שהפרויקט שלי הולך להיות מנהל סיסמאות (וחשבתי שאולי זה גם יהיה פרויקט שבאמת אני אוכל להשתמש בו בעצמי ולקבל ממנו תועלת).

בתהליך תכנון הפרויקט, החלטתי לאתגר את עצמי עם multi factor authentication הכולל קוד חד פעמי לאימייל או לסמס, והזדהות ביומטרית עם זיהוי קול וזיהוי פנים. באותו הזמן לא היה לי מושג איך אפילו מתחילים לפתח אף אחד מהדברים האלו.

התחלתי לפתח את הפרויקט מהמוכר: פיתחתי שרת ולקוח עם ממשק משתמש בסיסי מאוד, עם פונקציית התחברות, רישום אתרים ופעולות עליהם. לאורך השנה למדתי מהמורים במגמה על בסיסי נתונים, הצפנות, פונקציות hash, threading ועוד דברים שימושיים שמימשתי בפרויקט והיוו את הבסיס לפרויקט הסייברי המאובטח שרציתי.
משלב מסוים, התחלנו לעבוד על הפרויקטים יותר במרץ באופן עצמאי, והתחלתי לחקור על אפשרויות פקטורי ההזדהות. התחלתי ממימוש קוד האימות החד פעמי לאימייל, ונתקלתי בהרבה אתגרים בשילוב ההזדהות מרובה השלבים בפרויקט שהיה אז. הייתי צריך לעשות שינויים לא קטנים והרבה debugging בשביל ליישם את זה, ובסוף הצלחתי וזה היה מספק מאוד.
לאחר מכן, עברתי לזיהוי הקול. הייתי צריך ללמוד מאפס איך להקליט קול עוד לפני זיהוי הקול, וזה ממש לא היה קל. ולבסוף עברתי לזיהוי הפנים, שדווקא הרגשתי שהלך לי יותר קל מפקטורי ההתחברות האחרים שפיתחתי, וזה מראה שלמדתי מהתהליך ובניתי קוד יותר גמיש וניתן לשינוי.

אני מרגיש שאחרי כשנה של השקעה מטורפת, יצאתי עם פרויקט גמר סייבר מרשים מאוד, עם ממשק משתמש אסתטי ואינטראקטיבי, הצפנות מתוחכמות וחזקות, תהליך הזדהות ארוך ומאובטח, ואני מאוד מרוצה ממה שהצלחתי ליצור בעצמי. אני לא חושב שיש חלק שבראייה לאחור הייתי מממש אחרת בפרויקט.

עם זאת, חשבתי על כמה דרכים שאיתן היה אפשר לשפר את הפרויקט: תכננתי לפתח הזדהות עם קוד אימות גם לסמס, אבל גיליתי שזה תהליך שדורש לדעת מה ספק הסלולר של המשתמשים והאפליקציות המציעות שירותים אלו הם בתשלום. בנוסף, שקלתי לפתח מילוי אוטומטי של סיסמאות בשדות הזדהות של אתרים באינטרנט, אבל בשביל דבר כזה צריך לפתח browser extension, שייכתב ב – javascript, ואני לא יודע לתכנת בשפה הזאת. אם היה לי תקציב לשירותי גילוי ספק הסלולר או אם הייתי יודע לתכנת ב – javascript, כנראה שהייתי יכול לשפר עוד את הפרויקט.

תודות לסיכום:
מורי המגמה – עידן, ליאת, עודי וקובי.
ההורים שלי, שתמכו בי לאורך התהליך הארוך והמתיש, עזרו לי לפתח את הרעיון של האפליקציה, וספציפית לאבא שלי שעזר לי לפתור כמה בעיות בקוד.
חברי דור רימברג, יהב בן סימון ורואי משה שעזרו לי בהרבה נקודות בפרויקט והיו שם תמיד להתייעצויות ופתרון בעיות.

# ביבליוגרפיה

GeeksforGeeks. (2025, 23 Jul). *Euclidean distance*. https://www.geeksforgeeks.org/euclidean-distance/

GeeksforGeeks. (2025, 15 Jul). *Cosine similarity*. https://www.geeksforgeeks.org/cosine-similarity/

Google. (n.d.). *Google Gemini* [Large language model]. https://gemini.google.com/

Schimansky, T. (n.d.). *CustomTkinter documentation*. CustomTkinter. Documentation Introduction | CustomTkinter

The Cryptography Developers. (n.d.). *Fernet (symmetric encryption)*. Cryptography 48.0.0.dev1 documentation. https://cryptography.io/en/latest/fernet/

The Cryptography Developers. (n.d.). *Key derivation functions*. Cryptography 48.0.0.dev1 documentation. https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/

Wikipedia contributors. (n.d.). *RSA*. Wikipedia, The Free Encyclopedia. https://he.wikipedia.org/wiki/RSA

רונזבוים ע. (2026, אפריל). *רשתות מחשבים* [ספר אלקטרוני]. https://data.cyber.org.il/networks/networks.pdf

# נספחים

## הסבר על הצפנת RSA

בוחרים באקראי שני מספרים ראשוניים גדולים – p ו- q.

מחשבים את המודולוס על ידי n = pq

מחשבים את פונקציית אוילר: $\phi(n) = (p-1)(q-1)$

בוחרים מספר e שהוא בין 1 ל $\phi(n)$ וזר ל $\phi(n)$ (כלומר אסור שיהיה להם מחלק משותף מלבד 1). **e יהיה המפתח הפומבי המשמש להצפנה.**

מחשבים מספר d המקיים $de \bmod \phi(n) = 1$. **d יהיה המפתח הפרטי המשמש להצפנה והוא צריך להישמר בסוד.**

**להצפנת המסר m**: $c = m^e \bmod n$

**לפיענוח המסר המוצפן c**: $m = c^d \bmod n$

## הסבר על אלגוריתם PBKDF2HMAC

האלגוריתם לוקח סיסמה כערך התחלתי, ויוצר ממנה פלט ארוך וחזק שיכול גם לשמש כמפתח הצפנה.

האלגוריתם יוצר את הפלט החזק על ידי hashing של הסיסמה מספר גדול מאוד של פעמים.

אצלי בפרויקט, אני מעביר את הסיסמה של המשתמש דרך אלגוריתם sha256 600,000 פעמים, ובסוף מוסיף לו salt כדי ליצור פלט באורך 32 בתים שניתן להשתמש בו כמפתח של הצפנת AES-256. בגלל הגיבוב החוזר של הסיסמה, כמעט בלתי אפשרי לנסות לשער את הסיסמה המקורית על ידי bruteforce.

## הסבר על cosine similarity

הנוסחה נועדה למדוד את הדמיון בין הזוויות של הווקטורים ולא את הגודל שלהם.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

מחשבים את הדמיון הקוסינוסי על פי סכום מכפלת כל האיברים של הווקטורים חלקי מכפלת האורכים של הווקטורים.

## קוד הפרויקט

### Protocol Handler

```
# Protocol Constants

CMD_FIELD_LENGTH = 30   # Exact length of cmd field (in bytes)
LENGTH_FIELD_LENGTH = 4   # Exact length of length field (in bytes)
MAX_DATA_LENGTH = 10**LENGTH_FIELD_LENGTH-1  # Max size of data field according to protocol
MSG_HEADER_LENGTH = CMD_FIELD_LENGTH + 1 + LENGTH_FIELD_LENGTH + 1  # Exact size of header
(CMD+LENGTH fields)
MAX_MSG_LENGTH = MSG_HEADER_LENGTH + MAX_DATA_LENGTH  # Max size of total message
DELIMITER = chr(0)  # Delimiter character in protocol
DATA_DELIMITER = chr(1)  # Delimiter in the data part of the message

ERROR_RETURN = None

PROTOCOL_SERVER = {
    "signup_ok_msg" : "SIGNUP_OK",
    "user_exists_msg" : "USER_EXISTS",
    "weak_password_msg" : "WEAK_PASSWORD",
    "login_ok_msg" : "LOGIN_OK",
    "username_invalid_msg" : "USERNAME_INVALID",
    "incorrect_password_msg" : "INCORRECT_PASSWORD",
    "timeout_msg" : "TIMEOUT",
    "security_questions_list_msg" : "SECURITY_QUESTIONS_LIST",
    "your_security_questions_msg" : "YOUR_SECURITY_QUESTIONS",
    "security_answers_correct_msg" : "SECURITY_ANSWERS_CORRECT",
    "security_answers_incorrect_msg" : "SECURITY_ANSWERS_INCORRECT",
    "reset_ok_msg" : "RESET_OK",
    "preferences_saved_msg" : "PREFERENCES_SAVED",
    "your_preferences_msg" : "YOUR_PREFERENCES",
    "registration_ok_msg" : "REGISTRATION_OK",
    "already_registered_msg" : "ALREADY_REGISTERED",
    "change_ok_msg" : "CHANGE_OK",
    "not_registered_msg" : "NOT_REGISTERED",
    "your_sites_msg" : "YOUR_SITES",
    "info_msg" : "INFO",
    "your_auth_options_msg" : "YOUR_AUTH_OPTIONS",
    "code_correct_msg" : "CODE_CORRECT",
    "code_incorrect_msg" : "CODE_INCORRECT",
    "code_expired_msg" : "CODE_EXPIRED",
    "code_sent_msg" : "CODE_SENT",
    "your_email_msg" : "YOUR_EMAIL",
    "no_email_registered_msg" : "NO_EMAIL_REGISTERED",
    "voice_verified_msg" : "VOICE_VERIFIED",
    "voice_not_verified_msg" : "VOICE_NOT_VERIFIED",
    "face_matches_msg" : "FACE_MATCHES",
    "face_not_matches_msg" : "FACE_NOT_MATCHES",
```

```
    "voice_auth_set_msg" : "VOICE_AUTH_SET",
    "face_recognition_set_msg" : "FACE_RECOGNITION_SET",
    "site_deleted_msg" : "SITE_DELETED",
    "site_recovered_msg" : "SITE_RECOVERED"
}

PROTOCOL_CLIENT = {
    "signup_msg" : "SIGNUP",
    "login_msg" : "LOGIN",
    "security_questions_msg" : "SECURITY_QUESTIONS",
    "my_security_questions_msg" : "MY_SECURITY_QUESTIONS",
    "check_security_answers_msg" : "CHECK_SECURITY_ANSWERS",
    "reset_password_msg" : "RESET_PASSWORD",
    "save_preferences_msg" : "SAVE_PREFERENCES",
    "get_preferences_msg" : "GET_PREFERENCES",
    "register_site_msg" : "REGISTER_SITE",
    "change_password_msg" : "CHANGE_PASSWORD",
    "registered_sites_msg" : "REGISTERED_SITES",
    "site_info_msg" : "SITE_INFO",
    "get_auth_options_msg" : "GET_AUTH_OPTIONS",
    "code_login_msg" : "CODE_LOGIN",
    "send_code_msg" : "SEND_CODE",
    "my_email_msg" : "MY_EMAIL",
    "voice_auth_msg" : "VOICE_AUTH",
    "check_face_recognition_msg" : "CHECK_FACE_RECOGNITION",
    "setup_voice_auth_msg" : "SETUP_VOICE_AUTH",
    "setup_face_recognition_msg" : "SETUP_FACE_RECOGNITION",
    "delete_site_msg" : "DELETE_SITE",
    "recover_site_msg" : "RECOVER_SITE"
}

class ProtocolHandler:

    def __init__(self):
        self.server_messages = PROTOCOL_SERVER # Dictionary of all server side protocol
messages
        self.client_messages = PROTOCOL_CLIENT # Dictionary of all client side protocol
messages


    def build_message(self, cmd, data):
        """Method receives command and data and returns a valid protocol message built from
them"""
        full_msg = ""
        cmd_len = len(cmd)
        data_len = len(data)
        full_msg += cmd + " " * (CMD_FIELD_LENGTH - cmd_len) + DELIMITER + "0" *
(LENGTH_FIELD_LENGTH - len(str(data_len))) + str(data_len) + DELIMITER + data
        return full_msg
```

```
    def parse_message(self, data):
        """Method receives a message, parses it according to protocol and checks its
validation.
        Returns the cmd and data of the message if valid"""
        data_list = data.split(DELIMITER)
        if len(data_list) != 3:
            return ERROR_RETURN, ERROR_RETURN
        cmd = data_list[0].replace(" ", "")
        msg_len = data_list[1]
        msg = data_list[2]
        if not self.check_validation(cmd, msg):
            return ERROR_RETURN, ERROR_RETURN
        else:
            return cmd, msg

    def check_validation(self):
        return

    def split_data(self, data):
        """Method receives a message and splits it according to protocol"""
        return data.split(DATA_DELIMITER)
```

ServerProtocolHandler

```
from protocol_handler import *


class ServerProtocolHandler(ProtocolHandler):

    def __init__(self):
        super(ServerProtocolHandler, self).__init__()

    def check_validation(self, cmd, data):
        """Method receives cmd and data and checks their validation according to server's
protocol"""
        print(cmd, data)
        cmd_len = len(cmd)
        data_len = len(data)
        if cmd not in self.client_messages.values() or cmd_len > CMD_FIELD_LENGTH or
data_len > MAX_DATA_LENGTH:
            return False
        else:
            return True
```

## ClientProtocolHandler

```
from protocol_handler import *



class ClientProtocolHandler(ProtocolHandler):

    def __init__(self):
        super(ClientProtocolHandler, self).__init__()


    def check_validation(self, cmd, data):
        """Method receives cmd and data and checks their validation according to client's
protocol"""
        cmd_len = len(cmd)
        data_len = len(data)
        if cmd not in self.server_messages.values() or cmd_len > CMD_FIELD_LENGTH or
data_len > MAX_DATA_LENGTH:
            return False
        else:
            return True
```

## ConnectionManager

```
import socket
from cryptography.fernet import Fernet

SERVER_PORT = 5678
MAX_MESSAGE_LENGTH = 4096

class ConnectionManager():

    def __init__(self):
        pass

    def create_socket(self):
        """Method creates a TCP socket object and returns it"""
        return socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    def receive_msg(self, socket):
        """Method receives a message from given socket and returns the decoded message"""
        msg = socket.recv(MAX_MESSAGE_LENGTH)
        try:
            decoded_msg = msg.decode()
        except UnicodeDecodeError as e:
            return msg
        else:
            return decoded_msg
```

```
    def send_msg(self, socket, msg):
        """Method receives a socket and a message, and sends the encoded message to the
received socket"""
        if isinstance(msg, str):
            socket.send(msg.encode())
        elif isinstance(msg, bytes):
            socket.send(msg)
        else:
            print("msg couldn't be sent")


    def encrypt_data(self, data, AES_key):
        """Method receives data and an AES key, and returns the encrypted message"""
        encryptor = Fernet(AES_key)
        msg_bytes = data.encode()
        return encryptor.encrypt(msg_bytes)

    def decrypt_data(self, data, AES_key):
        """Method receives data and an AES key, and returns the decrypted data"""
        decryptor = Fernet(AES_key)
        try:
            decrypted_msg = decryptor.decrypt(data).decode()
        except Exception as e:
            print(e)
            print("exeption in decrypt")
            return None
        else:
            return decrypted_msg
```

ServerConnectionManager

```
import socket
import select
from conn_manager import ConnectionManager, SERVER_PORT
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
import os
import base64
import threading
import time

MAX_MESSAGES_PER_CYCLE = 4
CYCLE_LENGTH = 1

class ServerConnectionManager(ConnectionManager):
```

```
    def __init__(self):
        super(ServerConnectionManager, self).__init__()
        self.public_pem = None    # pem of server's public RSA key
        self.client_sockets = []    # list of all client sockets
        self._sockets_to_keys = {}    # dictionary matching client sockets to their AES
keys
        self.client_message_count = {}    # dictionary matching a client socket to the
number of messages they have sent
        self.message_count_dict_lock = threading.Lock()
        self.server_socket = self.setup_socket()    # the socket of the server


    def setup_socket(self):
        """Sets up the server socket and generates the RSA keys"""
        server_socket = self.create_socket()
        server_socket.bind(("0.0.0.0", SERVER_PORT))
        server_socket.listen()
        self.generate_RSA()
        threading.Thread(target = self.reset_client_message_count_loop).start()
        # print("Server is up and running")
        return server_socket


    def read_from_socket(self):
        """returns a list of sockets of all clients who sent a message"""
        ready_to_read, ready_to_write, in_error = select.select([self.server_socket] +
self.client_sockets, self.client_sockets, [])
        return ready_to_read


    def accept_client_socket(self, socket):
        """Accepts a new client socket, and starts the key exchange process with it"""
        (client_socket, client_address) = socket.accept()
        print("New client joined!", client_address)
        self.client_sockets.append(client_socket)
        self.handshake(client_socket)

    def has_reached_max_messages(self, conn):
        """Receives a client socket and checks if the client has reached the max messages
        Allowed in a certain period of time"""
        with self.message_count_dict_lock:
            if conn not in self.client_message_count.keys():
                self.client_message_count[conn] = 1
            else:
                self.client_message_count[conn] += 1


            return self.client_message_count[conn] >= MAX_MESSAGES_PER_CYCLE
```

```
    def reset_client_message_count_loop(self):
        """Loop of reseting the message counter of all clients"""
        while True:
            with self.message_count_dict_lock:
                self.client_message_count.clear()
            time.sleep(CYCLE_LENGTH)


    def close_conn(self, socket):
        """Receives a socket and closes the connection with it"""
        socket.close()
        try:
            self.client_sockets.remove(socket)
        except ValueError:
            pass
        print("Connection closed")


    def get_private_ip(self):
        """Returns the private ip address of the server"""
        hostname = socket.gethostname()
        private_ip = socket.gethostbyname(hostname)
        return private_ip


    def generate_RSA(self):
        """Generates public and private RSA keys. Saves the private RSA key in an
environment variable."""
        private_key = rsa.generate_private_key(public_exponent = 65537, key_size = 4096) #
creates a 4096 bit private RSA key
        public_key = private_key.public_key()

        private_pem = private_key.private_bytes(encoding = serialization.Encoding.PEM,
                                                format = serialization.PrivateFormat.PKCS8,
                                                encryption_algorithm =
serialization.NoEncryption())

        self.public_pem = public_key.public_bytes(encoding = serialization.Encoding.PEM,
                                                  format =
serialization.PublicFormat.SubjectPublicKeyInfo)

        os.environ["RSA_PRIVATE_KEY"] = base64.b64encode(private_pem).decode("utf-8")


    def _load_and_recv_RSA(self):
        """Loads the private RSA key from the environment variable, and returns it
decoded."""
        b64_string = os.environ.get("RSA_PRIVATE_KEY")
```

```
        private_pem = base64.b64decode(b64_string.encode())
        return private_pem

    def recv_and_save_AES(self, client_socket):
        """Receives an encrypted AES key from the client, decrypts it and saves it in a
dictionary."""
        encrypted_AES = self.receive_msg(client_socket)
        private_key = serialization.load_pem_private_key(self._load_and_recv_RSA(), None)

        AES_key = private_key.decrypt(encrypted_AES,
                                      padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256
()),
                                      algorithm=hashes.SHA256(),
                                      label=None))

        self._sockets_to_keys[client_socket] = AES_key


    def handshake(self, client_socket):
        """Receives a client socket and starts the key exchange process by sending
        the public RSA key to the client"""
        self.send_msg(client_socket, self.public_pem)
        self.recv_and_save_AES(client_socket)


    def encrypt_msg(self, client_socket, msg):
        """Receives a message and encrypts it using the client's AES key"""
        client_AES = self._sockets_to_keys[client_socket]
        encrypted_msg = self.encrypt_data(msg, client_AES)
        return encrypted_msg

    def decrypt_msg(self, client_socket, msg):
        """Receives a message and decrypts it using the client's AES key"""
        client_AES = self._sockets_to_keys[client_socket]
        decrypted_msg = self.decrypt_data(msg, client_AES)
        return decrypted_msg
```

ClientConnectionManager

```
from conn_manager import ConnectionManager, SERVER_PORT
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
import os
import base64

DEFAULT_SERVER_IP = "127.0.0.1"
```

```
class ClientConnectionManager(ConnectionManager):

    def __init__(self):
        super(ClientConnectionManager, self).__init__()
        self.socket = self.create_socket()  # the socket of the client

    def connect(self, server_ip):
        """Receives the server's ip and connects the client socket to it"""
        self.socket.connect((server_ip, SERVER_PORT))
        self.handshake()

    def generate_and_send_AES(self):
        """Generates an AES key, encrypts it using the server's public RSA key, and sends
it to the server."""
        AES_key = Fernet.generate_key()
        os.environ["AES_KEY"] = base64.b64encode(AES_key).decode("utf-8")
        AES_msg = self.public_RSA_key.encrypt(AES_key,
                                              padding.OAEP(mgf= padding.MGF1(algorithm=
hashes.SHA256()),
                                                                                  algorithm=
hashes.SHA256(),
                                                                                  label= None))
        self.send_msg(self.socket, AES_msg)

    def recv_RSA(self):
        """Receives the public RSA key from the server"""
        public_RSA_pem = self.receive_msg(self.socket)
        self.public_RSA_key = serialization.load_pem_public_key(public_RSA_pem.encode())

    def get_AES(self):
        """Get the AES key from the environment variables"""
        b64_string = os.environ.get("AES_KEY")
        AES_key = base64.b64decode(b64_string.encode())
        return AES_key

    def handshake(self):
        """Continues the key exchange process with the server"""
        self.recv_RSA()
        self.generate_and_send_AES()
```

Server

```
from server_protocol_handler import ServerProtocolHandler, DATA_DELIMITER
from server_conn_manager import ServerConnectionManager
```

```
from user_manager import UserManager, TIMEOUT_LENGTH_MINUTES
from database import DataBase
from password_handler import PasswordHandler, DEFAULT_SETTINGS
from email_manager import EmailManager, EMAIL_CODE_EXPIRATION_TIME
from voice_authenticator import VoiceAuthenticator
from face_recognizer import FaceRecognizer
import datetime
import base64
import json

class Server:


    def __init__(self):
        self.conn_manager = ServerConnectionManager()   # ServerConnectionManager object
        self.socket = self.conn_manager.server_socket   # The server socket
        self.protocol_handler = ServerProtocolHandler()    # ServerProtocolHandler object
        self.messages_to_send = []    # list of scheduled messages to send
        self.user_manager = UserManager()    # UserManager object
        self.db_manager = DataBase()    # DataBase object
        self.email_manager = EmailManager()    # EmailManager object
        self.voice_authenticator = VoiceAuthenticator()    # VoiceAuthenticator object
        self.conn_code = {}    # dictionary of the codes sent to the clients' emails by the
clients' socket

    def run(self):
        """Server main loop"""
        print(f"Server is running on private IP {self.conn_manager.get_private_ip()}")
        while True:
            ready_to_read = self.conn_manager.read_from_socket()
            for current_socket in ready_to_read:
                if current_socket is self.socket:
                    self.conn_manager.accept_client_socket(self.socket)
                else:
                    if self.conn_manager.has_reached_max_messages(current_socket):
                        self.conn_manager.close_conn(current_socket)
                    else:
                        print("New data from client")
                        data = ""
                        cmd = ""
                        try:
                            encrypted_msg = self.conn_manager.receive_msg(current_socket)
                            full_msg = self.conn_manager.decrypt_msg(current_socket,
encrypted_msg)
                            print(f"[SERVER]" + full_msg)
                            cmd, data = self.protocol_handler.parse_message(full_msg)
                            if cmd is None or data is None:
                                self.conn_manager.close_conn(current_socket)
                        except ConnectionResetError as e:
```

```
                        print(e)
                        self.conn_manager.close_conn(current_socket)
                    except BaseException as baseEx:
                        print(baseEx)
                        self.conn_manager.close_conn(current_socket)
                    if cmd == "":
                        self.conn_manager.close_conn(current_socket)
                    else:
                        print((cmd, data))
                        self.handle_client_message(current_socket, cmd, data)
            self.send_messages_to_clients()

    def handle_client_message(self, conn, cmd, data):
        """Receives conn, cmd and data, and calls the right method that handles the cmd."""
        if cmd == self.protocol_handler.client_messages["login_msg"]:
            self.handle_login_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["signup_msg"]:
            self.handle_signup_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["security_questions_msg"]:
            self.handle_security_questions_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["my_security_questions_msg"]:
            self.handle_my_security_questions_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["check_security_answers_msg"]:
            self.handle_check_security_answers_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["reset_password_msg"]:
            self.handle_reset_password_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["get_auth_options_msg"]:
            self.handle_get_auth_options_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["my_email_msg"]:
            self.handle_my_email_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["send_code_msg"]:
            self.handle_send_code_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["code_login_msg"]:
            self.handle_code_login_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["voice_auth_msg"]:
            self.handle_voice_auth_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["check_face_recognition_msg"]:
            self.handle_check_face_recognition_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["setup_voice_auth_msg"]:
            self.handle_setup_voice_auth_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["setup_face_recognition_msg"]:
            self.handle_setup_face_recognition_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["register_site_msg"]:
            self.handle_register_site_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["registered_sites_msg"]:
            self.handle_registered_sites_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["get_preferences_msg"]:
            self.handle_get_preferences_message(conn, data)
```

```
        elif cmd == self.protocol_handler.client_messages["save_preferences_msg"]:
            self.handle_save_preferences_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["site_info_msg"]:
            self.handle_site_info_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["change_password_msg"]:
            self.handle_change_password_message(conn, data)
        elif cmd == self.protocol_handler.client_messages["delete_site_msg"]:
            self.handle_delete_site_msg(conn, data)
        elif cmd == self.protocol_handler.client_messages["recover_site_msg"]:
            self.handle_recover_site_msg(conn, data)


    def handle_login_message(self, conn, data):
        """Receives conn and data, and handles the client's login attempt.\n
        Sends an answer to the client. Possible Answers:\n
        login_ok, username_invalid, incorrect_password, timeout."""
        data_list = self.protocol_handler.split_data(data)
        username = data_list[0]
        password = data_list[1]
        if self.user_manager.is_user_in_timeout(username):
            self.build_and_queue_message(conn, "timeout_msg", f"{TIMEOUT_LENGTH_MINUTES}")
        else:
            if self.db_manager.does_user_exist(username):
                result = self.db_manager.user_login(username, password)
                if result == None:
                    in_timeout = self.user_manager.register_failed_login_attempt(username)
                    if in_timeout:
                        self.build_and_queue_message(conn, "timeout_msg",
f"{TIMEOUT_LENGTH_MINUTES}")
                    else:
                        self.build_and_queue_message(conn, "incorrect_password_msg", "")
                else:
                    self.user_manager.logged_users[conn] = result
                    self.user_manager.successful_login_attempt(username)
                    self.build_and_queue_message(conn, "login_ok_msg", "")
            else:
                in_timeout = self.user_manager.register_failed_login_attempt(username)
                if in_timeout:
                    self.build_and_queue_message(conn, "timeout_msg",
f"{TIMEOUT_LENGTH_MINUTES}")
                else:
                    self.build_and_queue_message(conn, "username_invalid_msg", "")


    def handle_signup_message(self, conn, data):
        """Receives conn and data, and handles the client's signup.\n
        If all the needed signup data was received, Saves the user in the database.\n
        Sends an answer to the client. Possible Answers:\n
        signup_ok, user_exists, weak_password."""
```

```
        data_list = self.protocol_handler.split_data(data)
        username = data_list[0]
        password = data_list[1]
        email = data_list[2]
        phone_number = data_list[3]
        voice_signature = data_list[4]
        face_encoding = data_list[5]
        questions_dict = json.loads(data_list[6])

        if voice_signature != "":
            bin_voice_signature = base64.b64decode(voice_signature)
        else:
            bin_voice_signature = None

        if face_encoding != "":
            bin_face_encoding = base64.b64decode(face_encoding)
        else:
            bin_face_encoding = None

        if self.db_manager.does_user_exist(username):
            self.build_and_queue_message(conn, "user_exists_msg", "")
        else:
            if not PasswordHandler.is_strong_password(password):
                self.build_and_queue_message(conn, "weak_password_msg", "")
            else:
                result = self.db_manager.user_signup(username, password, email,
phone_number, bin_voice_signature, bin_face_encoding, questions_dict)
                if result == None:
                    self.build_and_queue_message(conn, "user_exists_msg", "")
                else:
                    self.build_and_queue_message(conn, "signup_ok_msg", "")


    def handle_security_questions_msg(self, conn, data):
        """Receives conn and data, and sends the user the list of security questions they
can answer."""
        security_questions_list = self.db_manager.get_security_questions_list()
        security_questions_str = DATA_DELIMITER.join(security_questions_list)
        self.build_and_queue_message(conn, "security_questions_list_msg",
security_questions_str)

    def handle_my_security_questions_msg(self, conn, data):
        """Receives conn and data, and sends the list of the user's security question to
the client.
        If the username is invalid, responds with username_invalid instead."""
        if self.db_manager.does_user_exist(data):
            user_questions_list = self.db_manager.get_user_security_questions(data)
            user_questions_str = DATA_DELIMITER.join(user_questions_list)
```

```
            self.build_and_queue_message(conn, "your_security_questions_msg",
user_questions_str)
        else:
            self.build_and_queue_message(conn, "username_invalid_msg", "")

    def handle_check_security_answers_msg(self, conn, data):
        """Receives conn and data, and handles the request to check their security
answers.\n
        Sends an answer to the client. Possible Answers:\n
        security_answers_correct, security_answers_incorrect, timeout"""
        data_list = self.protocol_handler.split_data(data)
        username, *answers_list = data_list
        if self.user_manager.is_user_in_timeout(username):
            self.build_and_queue_message(conn, "timeout_msg", f"{TIMEOUT_LENGTH_MINUTES}")
        else:
            if self.db_manager.check_user_security_answers(username, answers_list):
                self.user_manager.successful_login_attempt(username)
                self.build_and_queue_message(conn, "security_answers_correct_msg", "")
            else:
                in_timeout = self.user_manager.register_failed_login_attempt(username)
                if in_timeout:
                    self.build_and_queue_message(conn, "timeout_msg",
f"{TIMEOUT_LENGTH_MINUTES}")
                else:
                    self.build_and_queue_message(conn, "security_answers_incorrect_msg",
"")

    def handle_reset_password_msg(self, conn, data):
        """Receives conn and data, and handles the client's request to reset the their
password to the app.\n
        If the received password is strong enough, the server resets the user's password
and responds with reset_ok.
        Otherwise, the server responds with weak_password."""
        data_list = self.protocol_handler.split_data(data)
        username = data_list[0]
        new_password = data_list[1]
        if not PasswordHandler.is_strong_password(new_password):
            self.build_and_queue_message(conn, "weak_password_msg", "")
        else:
            user_id = self.db_manager.reset_user_password(username, new_password)
            self.user_manager.logged_users[conn] = user_id
            self.build_and_queue_message(conn, "reset_ok_msg", "")

    def handle_get_auth_options_msg(self, conn, data):
        """Receives conn and data, and sends the client the authentication options
available to them."""
        username = data
        auth_options = self.db_manager.get_user_auth_options(username)
```

```
        auth_options_str = json.dumps(auth_options)
        self.build_and_queue_message(conn, "your_auth_options_msg", auth_options_str)

    def handle_my_email_msg(self, conn, data):
        """Receives conn and data, and sends the client their registered email address. If
no email registered - responds with no_email_registered."""
        try:
            user_email = self.db_manager.get_user_email(data)
            self.build_and_queue_message(conn, "your_email_msg", user_email)
        except TypeError as e:
            self.build_and_queue_message(conn, "no_email_registered_msg", "")

    def handle_send_code_msg(self, conn, data):
        """Receives conn and data, and sends a one-time code to the client's email."""
        if data != "":
            code, code_creation_time = self.email_manager.send_login_code_gmail(data)
            self.conn_code[conn] = (code, code_creation_time)
            self.build_and_queue_message(conn, "code_sent_msg", "")

    def handle_code_login_msg(self, conn, data):
        """Receives conn and data, and handles the client's code login attempt.\n
        Possible responds: code_correct, code_incorrect, code_expired, timeout."""
        sent_code, code_creation_time = self.conn_code[conn]
        current_time = datetime.datetime.now()
        time_diff = current_time - code_creation_time
        username = self.user_manager.logged_users.get(conn, "")
        if self.user_manager.is_user_in_timeout(username):
            self.build_and_queue_message(conn, "timeout_msg", f"{TIMEOUT_LENGTH_MINUTES}")
        else:
            if time_diff > datetime.timedelta(seconds = EMAIL_CODE_EXPIRATION_TIME):
                self.build_and_queue_message(conn, "code_expired_msg", "")
            elif sent_code == data:
                if username != "":
                    self.user_manager.successful_login_attempt(username)
                self.build_and_queue_message(conn, "code_correct_msg", "")
            else:
                if username != "":
                    in_timeout = self.user_manager.register_failed_login_attempt(username)
                    if in_timeout:
                        self.build_and_queue_message(conn, "timeout_msg",
f"{TIMEOUT_LENGTH_MINUTES}")
                    else:
                        self.build_and_queue_message(conn, "code_incorrect_msg", "")
                else:
                    self.build_and_queue_message(conn, "code_incorrect_msg", "")


    def handle_voice_auth_msg(self, conn, data):
        """Receives conn and data, and handles the client's voice authentication attempt.\n
```

```
        Possible responds: voice_verified, voice_not_verified, timeout."""
        data_list = self.protocol_handler.split_data(data)
        username = data_list[0]
        login_voice_signature = data_list[1]
        bin_login_voice_signature = base64.b64decode(login_voice_signature)
        bin_stored_voice_signature = self.db_manager.get_voice_signature(username)

        if self.user_manager.is_user_in_timeout(username):
            self.build_and_queue_message(conn, "timeout_msg", f"{TIMEOUT_LENGTH_MINUTES}")
        else:
            if self.voice_authenticator.verify_user(bin_stored_voice_signature,
bin_login_voice_signature):
                self.user_manager.successful_login_attempt(username)
                self.build_and_queue_message(conn, "voice_verified_msg", "")
            else:
                in_timeout = self.user_manager.register_failed_login_attempt(username)
                if in_timeout:
                    self.build_and_queue_message(conn, "timeout_msg",
f"{TIMEOUT_LENGTH_MINUTES}")
                else:
                    self.build_and_queue_message(conn, "voice_not_verified_msg", "")

    def handle_check_face_recognition_msg(self, conn, data):
        """Receives conn and data, and handles the client's face recognition attempt.
        Possible responds: face_matches, face_not_matches"""
        data_list = self.protocol_handler.split_data(data)
        username = data_list[0]
        face_encoding_str = data_list[1]
        bin_face_encoding = base64.b64decode(face_encoding_str)
        bin_stored_face_encoding = self.db_manager.get_face_encoding(username)
        if FaceRecognizer.compare_faces(bin_stored_face_encoding, bin_face_encoding):
            self.build_and_queue_message(conn, "face_matches_msg", "")
        else:
            self.build_and_queue_message(conn, "face_not_matches_msg", "")


    def handle_setup_voice_auth_msg(self, conn, data):
        """Receives conn and data, and sets up voice authentication for the user."""
        user_id = self.user_manager.logged_users[conn]
        bin_voice_signature = base64.b64decode(data)
        self.db_manager.setup_voice_auth(user_id, bin_voice_signature)
        self.build_and_queue_message(conn, "voice_auth_set_msg", "")

    def handle_setup_face_recognition_msg(self, conn, data):
        """Receives conn and data, and sets up face recognition for the user."""
        user_id = self.user_manager.logged_users[conn]
        bin_face_encoding = base64.b64decode(data)
        self.db_manager.setup_face_recognition(user_id, bin_face_encoding)
        self.build_and_queue_message(conn, "face_recognition_set_msg", "")
```

```
    def handle_save_preferences_message(self, conn, data):
        """Receives conn and data, and handles the client's request to save password
preferences for a registered site.\n
        Possible responds: preferences_saved, not_registered."""
        data_list = self.protocol_handler.split_data(data)
        site_url = data_list[0]
        preferences_json = data_list[1]
        password_preferences = json.loads(preferences_json)
        user_id = self.user_manager.logged_users[conn]
        if not self.db_manager.does_site_exist(user_id, site_url):
            self.build_and_queue_message(conn, "not_registered_msg", "")
        else:
            self.db_manager.update_password_preferences(user_id, site_url,
password_preferences)
            self.build_and_queue_message(conn, "preferences_saved_msg", "")

    def handle_get_preferences_message(self, conn, data):
        """Receives conn and data, and sends the client their password preferences for a
registered site.\n
        Possible responds: your_preferences, not_registered."""
        user_id = self.user_manager.logged_users[conn]
        if not self.db_manager.does_site_exist(user_id, data):
            self.build_and_queue_message(conn, "not_registered_msg", "")
        else:
            preferences = self.db_manager.get_password_preferences(user_id, data)
            preferences_json = json.dumps(preferences)
            self.build_and_queue_message(conn, "your_preferences_msg", preferences_json)


    def handle_register_site_message(self, conn, data):
        """Receives conn and data, and handles the client's request to register a new
site.\n
        Generates a password using the client's custom preferences if provided, otherwise
uses DEFAULT_SETTINGS.\n
        Possible responds: registration_ok (with generated password),
already_registered."""
        data_list = self.protocol_handler.split_data(data)
        site_url = data_list[0]
        site_username = data_list[1]
        user_id = self.user_manager.logged_users[conn]

        if self.db_manager.does_site_exist(user_id, site_url):
            self.build_and_queue_message(conn, "already_registered_msg", "")
        else:
            if data_list[2] != "":
                password_preferences = json.loads(data_list[2])
                generated_password =
PasswordHandler.generate_password(password_preferences)
```

```
            else:
                password_preferences = None
                generated_password = PasswordHandler.generate_password(DEFAULT_SETTINGS)
            self.db_manager.register_site(user_id, site_url, site_username,
generated_password, password_preferences)
            self.build_and_queue_message(conn, "registration_ok_msg", generated_password)


    def handle_registered_sites_message(self, conn, data):
        """Receives conn and data, and sends the client a list of their registered sites.\n
        If data is "True", sends only a list of deleted sites."""
        user_id = self.user_manager.logged_users[conn]
        if data == "True":
            get_deleted_sites = True
        else:
            get_deleted_sites = False
        registered_sites_list = self.db_manager.get_registered_sites(user_id,
get_deleted_sites)
        registered_sites_msg = DATA_DELIMITER.join(registered_sites_list)
        self.build_and_queue_message(conn, "your_sites_msg", registered_sites_msg)


    def handle_site_info_message(self, conn, data):
        """Receives conn and data, and sends the client the username and password of a
registered site.\n
        Possible responds: info, not_registered."""
        site_url = data
        user_id = self.user_manager.logged_users[conn]

        if self.db_manager.does_site_exist(user_id, site_url):
            site_username, site_password = self.db_manager.get_site_info(user_id, site_url)
            self.build_and_queue_message(conn, "info_msg",
f"{site_username}{DATA_DELIMITER}{site_password}")
        else:
            self.build_and_queue_message(conn, "not_registered_msg", "")


    def handle_change_password_message(self, conn, data):
        """Receives conn and data, and handles the client's request to change the password
for a registered site.\n
        Generates a new password based on the site's stored password preferences.\n
        Possible responds: change_ok (with generated password), not_registered."""
        site_url = data
        user_id = self.user_manager.logged_users[conn]

        if self.db_manager.does_site_exist(user_id, site_url):
            password_preference = self.db_manager.get_password_preferences(user_id,
site_url)
            generated_password = PasswordHandler.generate_password(password_preference)
```

```
            self.db_manager.change_site_password(user_id, site_url, generated_password)
            self.build_and_queue_message(conn, "change_ok_msg", generated_password)
        else:
            self.build_and_queue_message(conn, "not_registered_msg", "")



    def handle_delete_site_msg(self, conn, data):
        """Receives conn and data, and handles the client's request to delete a registered
site.\n
        Possible responds: site_deleted, not_registered."""
        site_url = data
        user_id = self.user_manager.logged_users[conn]

        if self.db_manager.does_site_exist(user_id, site_url):
            self.db_manager.delete_site(user_id, site_url)
            self.build_and_queue_message(conn, "site_deleted_msg", "")
        else:
            self.build_and_queue_message(conn, "not_registered_msg", "")


    def handle_recover_site_msg(self, conn, data):
        """Receives conn and data, and handles the client's request to recover a previously
deleted site.\n
        Possible responds: site_recovered, not_registered."""
        site_url = data
        user_id = self.user_manager.logged_users[conn]

        if self.db_manager.does_deleted_site_exist(user_id, site_url):
            self.db_manager.recover_site(user_id, site_url)
            self.build_and_queue_message(conn, "site_recovered_msg", "")
        else:
            self.build_and_queue_message(conn, "not_registered_msg", "")


    def build_and_queue_message(self, conn, cmd, data):
        """Receives conn, cmd and data, builds a protocol message and adds it to the
outgoing messages queue."""
        msg =
self.protocol_handler.build_message(self.protocol_handler.server_messages[cmd], data)
        self.messages_to_send.append((conn, msg))


    def send_messages_to_clients(self):
        """Sends all queued messages to clients encrypted, and clears the outgoing messages
queue."""
        for conn, msg in self.messages_to_send:
            encrypted_msg = self.conn_manager.encrypt_msg(conn, msg)
            self.conn_manager.send_msg(conn, encrypted_msg)
```

```
        self.messages_to_send.clear()

server = Server()
server.run()
```

## DataBase

```
import sqlite3
import os
import hashlib
from dotenv import load_dotenv
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
import base64


load_dotenv()

DEFAULT_DB_PATH = r".\Database\PasswordManager.db"
SECURITY_QUESTIONS_LIST = ["What is the name of your grandmother from your father's side?",
                           "What was the house number of the house you lived in as a child?
(digits)",
                           "What was the make and model of the first-ever mobile phone you
owned?",
                           "What was the name of the first school you ever attended?",
                           "In what city did you have your first-ever job?",
                           "What was the name of the band performing in the first concert
you ever attended?",
                           "What was the make and model of your first car?",
                           "What is the name of your oldest cousin?",
                           "What was the name of the hospital where you were born?",
                           "What was the name of the first company you ever worked for?",
                          ]

class DataBase:

    def __init__(self):
        self.users_keys = {}    # dictionary matching user_id to their DK encryption key

        init_db = False
        db_path = os.getenv("DB_PATH", DEFAULT_DB_PATH)
        db_dir = os.path.dirname(db_path)
        if not os.path.exists(db_path):
            if db_dir and not os.path.exists(db_dir):
                os.makedirs(db_dir)
            init_db = True
        self.conn = sqlite3.connect(db_path)    # SQLite connection object
```

```
        self.cur = self.conn.cursor()    # SQLite cursor object

        if init_db:
            self.init_db()



    def init_db(self):
        """Creates all database tables (USERS, PASSWORD_PREFERENCES, USER_SITE,
SECURITY_QUESTIONS) and populates the security questions."""

        # create USERS table
        self.cur.execute("""CREATE TABLE "USERS" (
                            user_id INTEGER,
                            password TEXT NOT NULL,
                            username TEXT NOT NULL UNIQUE,
                            password_salt BLOB NOT NULL,
                            email BLOB NOT NULL,
                            phone_number BLOB NOT NULL,
                            user_dk BLOB NOT NULL UNIQUE,
                            key_salt BLOB NOT NULL,
                            voice_signature BLOB,
                            face_encoding BLOB,
                            question_id_1 INTEGER NOT NULL,
                            answer_1 BLOB NOT NULL,
                            question_id_2 INTEGER NOT NULL,
                            answer_2 BLOB NOT NULL,
                            question_id_3 INTEGER NOT NULL,
                            answer_3 BLOB NOT NULL,
                            answers_salt BLOB NOT NULL,
                            FOREIGN KEY("question_id_1") REFERENCES
"SECURITY_QUESTIONS"("question_id"),
                            FOREIGN KEY("question_id_2") REFERENCES
"SECURITY_QUESTIONS"("question_id"),
                            FOREIGN KEY("question_id_3") REFERENCES
"SECURITY_QUESTIONS"("question_id"),
                            PRIMARY KEY("user_id" AUTOINCREMENT)
                        )""")

        # create PASSWORD_PREFERENCES table
        self.cur.execute("""CREATE TABLE "PASSWORD_PREFERENCES" (
                            "preferences_id"    INTEGER,
                            "max_length"    INTEGER DEFAULT 16,
                            "include_upper" BOOLEAN DEFAULT 1,
                            "include_lower" BOOLEAN DEFAULT 1,
                            "include_num"   BOOLEAN DEFAULT 1,
                            "include_special"   BOOLEAN DEFAULT 1,
                            "special_allowed"   TEXT DEFAULT "-_!@#$%^&*/|",
                            "can_start_with_upper"  BOOLEAN DEFAULT 1,
```

```
                            "can_start_with_lower"  BOOLEAN DEFAULT 1,
                            "can_start_with_num"    BOOLEAN DEFAULT 1,
                            "can_start_with_special"    BOOLEAN DEFAULT 1,
                            "special_allowed_start" TEXT DEFAULT "-_!@#$%^&*/|",
                            PRIMARY KEY("preferences_id" AUTOINCREMENT)
                        )""")

        # create USER_SITE table
        self.cur.execute("""CREATE TABLE "USER_SITE" (
                            "user_id"   INTEGER NOT NULL,
                            "site_id" INTEGER NOT NULL,
                            "site_url"  BLOB NOT NULL,
                            "site_username" BLOB NOT NULL,
                            "site_password" BLOB NOT NULL,
                            "preference_id" INTEGER,
                            "is_deleted" BOOLEAN DEFAULT 0,
                            PRIMARY KEY("user_id","site_id"),
                            FOREIGN KEY("preference_id") REFERENCES
"PASSWORD_PREFERENCES"("preferences_id"),
                            FOREIGN KEY("user_id") REFERENCES "USERS"("user_id")
                        )""")

        # create SECURITY_QUESTIONS table
        self.cur.execute("""CREATE TABLE "SECURITY_QUESTIONS" (
                            "question_id" INTEGER NOT NULL,
                            "question" TEXT NOT NULL,
                            PRIMARY KEY("question_id" AUTOINCREMENT)
                         )""")

        for question in SECURITY_QUESTIONS_LIST:
            self.cur.execute("""INSERT INTO SECURITY_QUESTIONS (question) VALUES(?)""",
(question, ))

        self.conn.commit()

    def derive_key(self, password, salt):
        """Receives a password and a salt, and derives a 32-byte encryption key using
PBKDF2 with SHA256."""
        kdf = PBKDF2HMAC(
            algorithm = hashes.SHA256(),
            length = 32,
            salt = salt,
            iterations = 600000,
        )
        return kdf.derive(password)

    def encrypt_data(self, user_id, data):
```

```
        """Receives a user_id and a list of data items, and returns a list of the data
items encrypted using the user's DK."""
        self.cur.execute("""SELECT user_dk FROM USERS WHERE user_id = ?""", (user_id, ))
        encrypted_dk = self.cur.fetchone()[0]
        dk_encryption_key = self.get_dk_encryption_key(user_id)
        f = Fernet(dk_encryption_key)
        dk = f.decrypt(encrypted_dk)
        data_f = Fernet(dk)
        encrypted_data = []
        for i in data:
            if type(i) != bytes:
                i = i.encode()
            encrypted_data.append(data_f.encrypt(i))
        return encrypted_data

    def decrypt_data(self, user_id, data):
        """Receives a user_id and a list of encrypted data items, and returns a list of the
data items decrypted using the user's DK."""
        self.cur.execute("""SELECT user_dk FROM USERS WHERE user_id = ?""", (user_id, ))
        encrypted_dk = self.cur.fetchone()[0]
        dk_encryption_key = self.get_dk_encryption_key(user_id)
        f = Fernet(dk_encryption_key)
        dk = f.decrypt(encrypted_dk)
        data_f = Fernet(dk)
        decrypted_data = []
        for i in data:
            try:
                decrypted_data.append(data_f.decrypt(i).decode())
            except:
                decrypted_data.append(data_f.decrypt(i))
        return decrypted_data

    def create_dk_encryption_key(self, username, password_hex, key_salt):
        """Receives a username, password hex and key salt, and returns a DK encryption key
derived from them."""
        raw_key = self.derive_key((username + password_hex).encode(), key_salt)
        dk_encryption_key = base64.urlsafe_b64encode(raw_key)
        return dk_encryption_key

    def get_dk_encryption_key(self, user_id):
        """Receives a user_id and returns their DK encryption key.\n
        If the key is not found in the users_keys dictionary, derives it from the database
and saves it."""
        dk_encryption_key = self.users_keys.get(user_id)
        if dk_encryption_key is None:
            self.cur.execute("""SELECT username, password, key_salt FROM USERS WHERE
user_id = ?""", (user_id, ))
            username, password_hex, key_salt = self.cur.fetchone()
```

```
            dk_encryption_key = self.create_dk_encryption_key(username, password_hex,
key_salt)
            self.users_keys[user_id] = dk_encryption_key
        return dk_encryption_key


    def user_signup(self, username, password, email, phone_number, voice_signature,
face_encoding, questions_dict):
        """Receives user signup data, and saves the user in the database.\n
        Hashes the password with a random salt, generates a DK, encrypts all sensitive user
data with the DK,
        encrypts the DK with a key derived from the username and password, and inserts the
user into the database.\n
        Returns the user_id if successful, None otherwise."""
        try:
            random_password_salt = os.urandom(32)
            salted_password = random_password_salt + password.encode()
            hashed_password = hashlib.sha256(salted_password)
            password_hex = hashed_password.hexdigest()

            user_dk = Fernet.generate_key()
            dk_f = Fernet(user_dk)
            encrypted_email = dk_f.encrypt(email.encode())
            encrypted_phone_number = dk_f.encrypt(phone_number.encode())

            if voice_signature:
                encrypted_voice_signature = dk_f.encrypt(voice_signature)
            else:
                encrypted_voice_signature = None

            if face_encoding:
                encrypted_face_encoding = dk_f.encrypt(face_encoding)
            else:
                encrypted_face_encoding = None

            random_key_salt = os.urandom(16)
            dk_encryption_key = self.create_dk_encryption_key(username, password_hex,
random_key_salt)

            f = Fernet(dk_encryption_key)
            encrypted_user_dk = f.encrypt(user_dk)

            q_id_1, q_id_2, q_id_3 = questions_dict.keys()
            answer_1 = questions_dict[q_id_1]
            answer_2 = questions_dict[q_id_2]
            answer_3 = questions_dict[q_id_3]
            answers_salt = os.urandom(32)
            hashed_answer_1 = hashlib.sha256(answer_1.encode() + answers_salt).hexdigest()
            hashed_answer_2 = hashlib.sha256(answer_2.encode() + answers_salt).hexdigest()
```

```
            hashed_answer_3 = hashlib.sha256(answer_3.encode() + answers_salt).hexdigest()
            self.cur.execute("""INSERT INTO USERS
                                (username, password, password_salt, email, phone_number,
user_dk, key_salt, voice_signature, face_encoding,
                                question_id_1, answer_1, question_id_2, answer_2,
question_id_3, answer_3, answers_salt)
                               VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
                               (username, password_hex, random_password_salt, encrypted_email,
encrypted_phone_number, encrypted_user_dk, random_key_salt, encrypted_voice_signature,
                                encrypted_face_encoding, q_id_1, hashed_answer_1, q_id_2,
hashed_answer_2, q_id_3, hashed_answer_3, answers_salt))
        except sqlite3.IntegrityError as e:
            return None
        except AttributeError as e:
            return None

        user_id = self.cur.lastrowid

        self.conn.commit()

        self.users_keys[user_id] = dk_encryption_key
        return user_id

    def does_user_exist(self, username):
        """Receives a username and returns True if the user exists in the database, False
otherwise."""
        self.cur.execute("""SELECT EXISTS(SELECT 1 FROM USERS WHERE username = ? LIMIT
1)""", (username, ))
        result = bool(self.cur.fetchone()[0])
        return result


    def user_login(self, username, password):
        """Receives a username and password, and verifies the credentials.\n
        Hashes the received password with the stored salt and compares it to the stored
hash.\n
        Returns the user_id if the credentials are correct, None otherwise."""
        self.cur.execute("""SELECT password, user_id, password_salt, key_salt FROM USERS
WHERE username = ?""", (username, ))
        user_password, user_id, password_salt, key_salt = self.cur.fetchone()

        try:
            salted_password = password_salt + password.encode()
            hashed_password = hashlib.sha256(salted_password)
            password_hex = hashed_password.hexdigest()
        except AttributeError as e:
            return None
```

```
        if user_password == password_hex:
            dk_encryption_key = self.create_dk_encryption_key(username, password_hex,
key_salt)
            self.users_keys[user_id] = dk_encryption_key
            return user_id

        return None

    def get_user_email(self, username):
        """Receives a username and returns the user's decrypted email address."""
        self.cur.execute("""SELECT user_id, email FROM USERS WHERE username = ?""",
(username, ))
        user_id, email = self.cur.fetchone()
        decrypted_email = self.decrypt_data(user_id, [email])[0]
        return decrypted_email

    def get_user_phone_number(self, username):
        """Receives a username and returns the user's decrypted phone number."""
        self.cur.execute("""SELECT user_id, phone_number FROM USERS WHERE username = ?""",
(username, ))
        user_id, phone_num = self.cur.fetchone()
        decrypted_phone_num = self.decrypt_data(user_id, [phone_num])[0]
        return decrypted_phone_num

    def get_voice_signature(self, username):
        """Receives a username and returns the user's decrypted voice signature. Returns
None if no voice signature is stored."""
        self.cur.execute("""SELECT user_id, voice_signature FROM USERS WHERE username =
?""", (username, ))
        user_id, voice_signature = self.cur.fetchone()
        if voice_signature is None:
            return None
        decrypted_voice_signature = self.decrypt_data(user_id, [voice_signature])[0]
        return decrypted_voice_signature

    def get_face_encoding(self, username):
        """Receives a username and returns the user's decrypted face encoding. Returns None
if no face encoding is stored."""
        self.cur.execute("""SELECT user_id, face_encoding FROM USERS WHERE username = ?""",
(username, ))
        user_id, face_encoding = self.cur.fetchone()
        if face_encoding is None:
            return None
        decrypted_face_encoding = self.decrypt_data(user_id, [face_encoding])[0]
        return decrypted_face_encoding

    def get_user_auth_options(self, username):
```

```
        """Receives a username and returns a dictionary of the authentication options
available to them."""
        auth_options = {}
        if self.get_voice_signature(username):
            auth_options["voice_auth"] = True
        else:
            auth_options["voice_auth"] = False

        if self.get_face_encoding(username):
            auth_options["face_auth"] = True
        else:
            auth_options["face_auth"] = False

        return auth_options

    def setup_voice_auth(self, user_id, voice_signature):
        """Receives a user_id and a voice signature, encrypts the voice signature and
stores it in the database."""
        encrypted_voice_signature = self.encrypt_data(user_id, [voice_signature])[0]
        self.cur.execute("""UPDATE USERS
                        SET voice_signature = ?
                        WHERE user_id = ?""",
                        (encrypted_voice_signature, user_id))

        self.conn.commit()

    def setup_face_recognition(self, user_id, face_encoding):
        """Receives a user_id and a face encoding, encrypts the face encoding and stores it
in the database."""
        encrypted_face_encoding = self.encrypt_data(user_id, [face_encoding])[0]
        self.cur.execute("""UPDATE USERS
                        SET face_encoding = ?
                        WHERE user_id = ?""",
                        (encrypted_face_encoding, user_id))

        self.conn.commit()

    def get_security_questions_list(self):
        """Returns a list of all security questions from the database."""
        self.cur.execute("""SELECT question FROM SECURITY_QUESTIONS""")
        security_questions_list = [i[0] for i in self.cur.fetchall()]
        return security_questions_list

    def get_user_security_questions(self, username):
        """Receives a username and returns a list of the 3 security questions the user
chose."""
        self.cur.execute("""SELECT question_id_1, question_id_2, question_id_3 FROM USERS
WHERE username = ?""", (username, ))
```

```
        q_id_1, q_id_2, q_id_3 = self.cur.fetchone()
        self.cur.execute("""SELECT question FROM SECURITY_QUESTIONS WHERE question_id IN
(?,?,?)""", (q_id_1, q_id_2, q_id_3))
        user_questions_list = [row[0] for row in self.cur.fetchall()]
        return user_questions_list


    def check_user_security_answers(self, username, answers_list):
        """Receives a username and a list of answers, hashes each answer with the stored
salt
        and compares them to the stored hashes. Returns True if all answers are correct,
False otherwise."""
        self.cur.execute("""SELECT answer_1, answer_2, answer_3, answers_salt FROM USERS
WHERE username = ?""", (username, ))
        *user_answers_list, answers_salt = self.cur.fetchone()
        for i in range(len(user_answers_list)):
            answer = answers_list[i]
            hashed_answer = hashlib.sha256(answer.encode() + answers_salt).hexdigest()
            if hashed_answer != user_answers_list[i]:
                return False
        return True


    def reset_user_password(self, username, new_password):
        """Receives a username and a new password, and resets the user's password.\n
        Decrypts the user's DK with the old DK encryption key, hashes the new password,
        derives a new DK encryption key and re-encrypts the DK with it.\n
        Returns the user_id."""
        self.cur.execute("""SELECT password, password_salt, user_id, key_salt, user_dk FROM
USERS WHERE username = ?""", (username, ))
        user_password, password_salt, user_id, key_salt, user_dk = self.cur.fetchone()

        dk_encryption_key = self.create_dk_encryption_key(username, user_password,
key_salt)

        f = Fernet(dk_encryption_key)
        decrypted_user_dk = f.decrypt(user_dk)


        new_salted_password = password_salt + new_password.encode()
        new_hashed_password = hashlib.sha256(new_salted_password)
        new_password_hex = new_hashed_password.hexdigest()

        new_dk_encryption_key = self.create_dk_encryption_key(username, new_password_hex,
key_salt)

        new_f = Fernet(new_dk_encryption_key)
        reencrypted_user_dk = new_f.encrypt(decrypted_user_dk)
```

```
        self.cur.execute("""UPDATE USERS
                        SET password = ?, user_dk = ?
                        WHERE user_id = ?""",
                        (new_password_hex, reencrypted_user_dk, user_id))

        self.conn.commit()

        self.users_keys[user_id] = new_dk_encryption_key

        return user_id


    def does_site_exist(self, user_id, site_url):
        """Receives a user_id and a site URL, and returns True if the site exists and is
not deleted, False otherwise."""
        registered_sites = self.get_registered_sites(user_id, False)
        return site_url in registered_sites

    def does_deleted_site_exist(self, user_id, site_url):
        """Receives a user_id and a site URL, and returns True if the site exists and is
deleted, False otherwise."""
        registered_sites = self.get_registered_sites(user_id, True)
        return site_url in registered_sites


    def get_site_id(self, user_id, site_url):
        """Receives a user_id and a site URL, and returns the site_id by decrypting and
comparing site URLs. Returns None if not found."""
        self.cur.execute("""SELECT site_id, site_url FROM USER_SITE WHERE user_id = ?""",
(user_id, ))
        lst = self.cur.fetchall()
        for i in lst:
            encrypted_site_url = i[1]
            if self.decrypt_data(user_id, [encrypted_site_url])[0] == site_url:
                return i[0]
        return None

    def count_registered_sites(self, user_id):
        """Receives a user_id and returns the number of sites registered to the user."""
        self.cur.execute("""SELECT COUNT(*) FROM USER_SITE WHERE user_id = ?""", (user_id,
))
        count = self.cur.fetchone()[0]
        return count

    def set_default_password_preferences(self):
        """Inserts a new row with default password preferences into the
PASSWORD_PREFERENCES table and returns its ID."""
        self.cur.execute("""INSERT INTO PASSWORD_PREFERENCES DEFAULT VALUES""")
        preference_id = self.cur.lastrowid
```

```
        self.conn.commit()
        return preference_id

    def update_password_preferences(self, user_id, site_url, preferences):
        """Receives a user_id, site URL and a preferences dictionary, and updates the
password preferences for the site."""
        site_id = self.get_site_id(user_id, site_url)
        preference_id = self.get_preference_id(user_id, site_id)
        max_length = preferences["max_length"]
        include = preferences["include"]
        special_allowed = preferences["special_allowed"]
        start = preferences["start"]
        special_allowed_start = preferences["special_allowed_start"]
        self.cur.execute("""UPDATE PASSWORD_PREFERENCES
                            SET max_length = ?, include_upper = ?, include_lower = ?,
include_num = ?, include_special = ?, special_allowed = ?,
                            can_start_with_upper = ?, can_start_with_lower = ?,
can_start_with_num = ?, can_start_with_special = ?, special_allowed_start = ?
                            WHERE preferences_id = ?""",
                            (max_length, include["Upper"], include["Lower"],
include["Number"], include["Special"], special_allowed,
                            start["Upper"], start["Lower"], start["Number"], start["Special"],
special_allowed_start, preference_id))

        self.conn.commit()

    def get_password_preferences(self, user_id, site_url):
        """Receives a user_id and a site URL, and returns a dictionary of the password
preferences for the site."""
        site_id = self.get_site_id(user_id, site_url)
        preference_id = self.get_preference_id(user_id, site_id)
        self.cur.execute("""SELECT max_length, include_upper, include_lower, include_num,
include_special, special_allowed,
                            can_start_with_upper, can_start_with_lower, can_start_with_num,
can_start_with_special, special_allowed_start
                            FROM PASSWORD_PREFERENCES WHERE preferences_id = ?""",
                            (preference_id, ))

        preferences_lst = self.cur.fetchone()
        preferences = {}
        preferences["max_length"] = preferences_lst[0]
        preferences["include"] = {"Upper" : preferences_lst[1], "Lower" :
preferences_lst[2], "Number" : preferences_lst[3], "Special" : preferences_lst[4]}
        preferences["special_allowed"] = preferences_lst[5]
        preferences["start"] = {"Upper" : preferences_lst[6], "Lower" : preferences_lst[7],
"Number" : preferences_lst[8], "Special" : preferences_lst[9]}
        preferences["special_allowed_start"] = preferences_lst[10]
        return preferences
```

```
    def register_site(self, user_id, site_url, site_username, site_password,
password_preferences = None):
        """Receives a user_id, site URL, site username, site password and optional password
preferences,
        encrypts the site data and inserts it into the database with default password
preferences.
        If custom password preferences are provided, updates them after insertion."""
        site_id = self.count_registered_sites(user_id)
        preference_id = self.set_default_password_preferences()
        encrypted_site_url, encrypted_site_username, encrypted_site_password =
self.encrypt_data(user_id, [site_url, site_username, site_password])
        self.cur.execute("""INSERT INTO USER_SITE (user_id, site_id, site_url,
site_username, site_password, preference_id)
                        VALUES(?,?,?,?,?,?)""",
                        (user_id, site_id, encrypted_site_url, encrypted_site_username,
encrypted_site_password, preference_id))

        if password_preferences is not None:
            self.update_password_preferences(user_id, site_url, password_preferences)

        self.conn.commit()

    def get_preference_id(self, user_id, site_id):
        """Receives a user_id and a site_id, and returns the preference_id for the site."""
        self.cur.execute("""SELECT preference_id FROM USER_SITE WHERE user_id = ? AND
site_id = ?""",
                         (user_id, site_id))
        preference_id = self.cur.fetchone()[0]
        return preference_id

    def change_site_password(self, user_id, site_url, new_site_password):
        """Receives a user_id, site URL and a new password, encrypts the new password and
updates it in the database."""
        site_id = self.get_site_id(user_id, site_url)
        encrypted_site_password = self.encrypt_data(user_id, [new_site_password])[0]
        self.cur.execute("""UPDATE USER_SITE
                        SET site_password = ?
                        WHERE user_id = ? AND site_id = ?""",
                        (encrypted_site_password, user_id, site_id))

        self.conn.commit()


    def get_registered_sites(self, user_id, get_deleted):
        """Receives a user_id and a boolean, and returns a decrypted list of the user's
registered site URLs.\n
```

```
        If get_deleted is True, returns the deleted sites. Otherwise, returns the active
sites."""
        self.cur.execute("""SELECT site_url FROM USER_SITE WHERE user_id = ? AND is_deleted
= ?""", (user_id, int(get_deleted)))

        lst = self.cur.fetchall()
        sites = [i[0] for i in lst]

        decrypted_sites = self.decrypt_data(user_id, sites)
        return decrypted_sites

    def get_site_info(self, user_id, site_url):
        """Receives a user_id and a site URL, and returns the decrypted site username and
site password."""
        site_id = self.get_site_id(user_id, site_url)
        self.cur.execute("""SELECT site_username, site_password FROM USER_SITE WHERE
user_id = ? AND site_id = ?""",
                         (user_id, site_id))

        site_info = self.cur.fetchone()
        decrypted_site_info = self.decrypt_data(user_id, site_info)
        return decrypted_site_info

    def delete_site(self, user_id, site_url):
        """Receives a user_id and a site URL, and marks the site as deleted in the
database."""
        site_id = self.get_site_id(user_id, site_url)
        self.cur.execute("""UPDATE USER_SITE
                         SET is_deleted = 1
                         WHERE user_id = ? AND site_id = ?""",
                         (user_id, site_id))

        self.conn.commit()

    def recover_site(self, user_id, site_url):
        """Receives a user_id and a site URL, and marks the previously deleted site as
active in the database."""
        site_id = self.get_site_id(user_id, site_url)
        self.cur.execute("""UPDATE USER_SITE
                         SET is_deleted = 0
                         WHERE user_id = ? AND site_id = ?""",
                         (user_id, site_id))

        self.conn.commit()
```

UserManager

```
import datetime
```

```
MAX_FAILED_LOGIN_ATTEMPTS_ALLOWED = 5
FAILED_LOGIN_ATTEMPTS_MINUTES_WINDOW = 1
TIMEOUT_LENGTH_MINUTES = 2


class UserManager:

    def __init__(self):
        self.logged_users = {}    # dictionary matching client sockets to their user_id
        self.user_login_attempts = {}    # dictionary matching usernames to a list of their
failed login attempt times
        self.users_in_timeout = {}    # dictionary matching usernames to the time they
entered timeout


    def is_user_in_timeout(self, username):
        """Receives a username and returns True if the user is currently in timeout, False
otherwise.\n
        If the timeout has expired, removes the user from the timeout dictionary."""
        if username in self.users_in_timeout.keys():
            timeout_entry_time = self.users_in_timeout[username]
            current_time = datetime.datetime.now()
            time_diff = current_time - timeout_entry_time
            if time_diff > datetime.timedelta(minutes = TIMEOUT_LENGTH_MINUTES):
                self.users_in_timeout.pop(username)
                return False
            else:
                return True
        else:
            return False

    def register_failed_login_attempt(self, username):
        """registers the time of the failed login attempt of the user and return whether or
not the user has exceeded
        the max ammount of failed attempts allowed in a certain amount of time"""

        current_time = datetime.datetime.now()
        if username not in self.user_login_attempts.keys():
            self.user_login_attempts[username] = [current_time]
            return False
        else:
            attempts_list = self.user_login_attempts[username]
            failed_login_attempts_count = 1
            for i in range(len(attempts_list) - 1, -1, -1):
                login_time = attempts_list[i]
                time_diff = current_time - login_time
```

```
                if time_diff > datetime.timedelta(minutes =
FAILED_LOGIN_ATTEMPTS_MINUTES_WINDOW):
                    attempts_list.append(current_time)
                    self.user_login_attempts[username] = attempts_list[i + 1::]
                    return False
                else:
                    failed_login_attempts_count += 1
                    if failed_login_attempts_count > MAX_FAILED_LOGIN_ATTEMPTS_ALLOWED:
                        self.users_in_timeout[username] = current_time
                        return True

            self.user_login_attempts[username].append(current_time)
            return False

    def successful_login_attempt(self, username):
        """Receives a username and clears their failed login attempts history."""
        if username in self.user_login_attempts.keys():
            self.user_login_attempts.pop(username)
```

## EmailManager

```
import smtplib
import ssl
import secrets
import string
import datetime

PORT = 465
SMTP_SERVER = "smtp.gmail.com"
SENDER_GMAIL = "mypassmanager10@gmail.com"
SENDER_GMAIL_PASSWORD = "jhbjotmuqxailhkt"
EMAIL_CODE_EXPIRATION_TIME = 45
LOGIN_CODE_MSG = """\
Subject: Password Manager Test

Your login code is {code}."""

class EmailManager:
    def __init__(self):
        self._security_context = ssl.create_default_context()    # SSL security context for
SMTP connection

    def send_gmail(self, recipient_gmail, msg):
        """Receives a recipient email address and a message, and sends the message via
Gmail SMTP."""
        with smtplib.SMTP_SSL(SMTP_SERVER, PORT, context = self._security_context) as
server:
```

```
            server.login(SENDER_GMAIL, SENDER_GMAIL_PASSWORD)
            server.sendmail(SENDER_GMAIL, recipient_gmail, msg)

    def send_login_code_gmail(self, recipient_gmail):
        """Receives a recipient email address, generates a one-time login code, sends it
via Gmail
        and returns the code along with the time it was created."""
        login_code = self._generate_login_code()
        login_msg_with_code = LOGIN_CODE_MSG.format(code = login_code)
        self.send_gmail(recipient_gmail, login_msg_with_code)
        code_creation_time = datetime.datetime.now()
        return (login_code, code_creation_time)

    def _generate_login_code(self):
        """Generates and returns a random 8-digit login code."""
        login_code = "".join(secrets.choice(string.digits) for _ in range(8))
        return login_code
```

## AudioManager

```
import pyaudio
import wave
import threading
import time
import os

MAX_RECORD_SECONDS = 10

class AudioManager:

    def __init__(self, client):
        self.client = client    # reference to the UI client for callbacks
        self.chunk = 1024    # record in chunks of 1024 samples
        self.sample_format = pyaudio.paInt16    # 16 bits per sample
        self.channels = 1    # mono audio
        self.fs = 16000    # record at 16000 samples per second
        self.seconds = MAX_RECORD_SECONDS    # maximum recording duration in seconds
        base_dir = os.path.dirname(os.path.abspath(__file__))
        self.filename = os.path.join(base_dir, "output.wav")    # path to the output WAV
file
        self.is_recording = False    # whether the audio manager is currently recording

    def start_recording(self):
        """Opens an audio stream and starts recording in a separate thread."""
        self.is_recording = True

        self.p = pyaudio.PyAudio()
        self.frames = []  # Initialize array to store frames
```

```
        self.stream = self.p.open(format=self.sample_format,
                channels=self.channels,
                rate=self.fs,
                frames_per_buffer=self.chunk,
                input=True)

        threading.Thread(target = self._record_loop, daemon= True).start()

    def _record_loop(self):
        """Recording loop that reads audio data from the stream until stopped or the max
duration is reached."""
        start_time = time.time()
        while self.is_recording and (time.time() - start_time) < self.seconds:
            data = self.stream.read(self.chunk)
            self.frames.append(data)

        if self.is_recording:
            self.stop_recording()

    def stop_recording(self):
        """Stops the recording, saves the recorded audio to a WAV file and notifies the UI
via a callback."""
        self.is_recording = False

        # Stop and close the stream
        self.stream.stop_stream()
        self.stream.close()

        # Save the recorded data as a WAV file
        wf = wave.open(self.filename, 'wb')
        wf.setnchannels(self.channels)
        wf.setsampwidth(self.p.get_sample_size(self.sample_format))
        wf.setframerate(self.fs)
        wf.writeframes(b''.join(self.frames))
        wf.close()

        # Terminate the PortAudio interface
        self.p.terminate()

        try:
            self.client.recording_stopped()
        except Exception as e:
            print(e)


    def play_audio(self, filename):
        """Receives a filename and plays the audio from the WAV file."""
        wf = wave.open(filename, 'rb')
```

```
        p = pyaudio.PyAudio()

        stream = p.open(format=p.get_format_from_width(wf.getsampwidth()),
                        channels=wf.getnchannels(),
                        rate=wf.getframerate(),
                        output=True)

        data = wf.readframes(1024)
        while data:
            stream.write(data)
            data = wf.readframes(1024)

        stream.stop_stream()
        stream.close()
        p.terminate()
```

## VoiceAuthenticator

```
from vosk import Model, SpkModel, KaldiRecognizer, SetLogLevel
import os
import wave
import json
import numpy as np

BASE_PATH = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_PATH, "Model")
SPK_MODEL_PATH = os.path.join(BASE_PATH, "SpkModel")
SAMPLES_PER_SECOND = 16000
THRESHOLD = 0.7

class VoiceAuthenticator:
    def __init__(self):
        SetLogLevel(-1) # no vosk logs
        self.model = Model(MODEL_PATH)    # Vosk speech recognition model
        self.spk_model = SpkModel(SPK_MODEL_PATH)    # Vosk speaker recognition model

    def calc_voice_signature(self, voice_file):
        """Receives a voice file path, processes the audio and returns the voice signature
as binary data.
        Returns None if no voice signature could be extracted."""
        rec = KaldiRecognizer(self.model, SAMPLES_PER_SECOND, self.spk_model)
        wf = wave.open(voice_file, "rb")
        data = wf.readframes(4000)
        while data:
            rec.AcceptWaveform(data)
            data = wf.readframes(4000)
```

```
        result = json.loads(rec.FinalResult())
        voice_signature = result.get("spk")
        if voice_signature is None:
            return None
        bin_voice_signature = np.array(voice_signature).astype(np.float32).tobytes()
        return bin_voice_signature

    def verify_user(self, bin_voice_signature, bin_login_signature):
        """Receives a stored voice signature and a login voice signature as binary data,
        compares the two signatures and returns True if they match, False otherwise."""
        v1 = np.frombuffer(bin_voice_signature, dtype = np.float32)
        v2 = np.frombuffer(bin_login_signature, dtype = np.float32)

        similarity = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
        print(f"similarity: {similarity}")
        return similarity >= THRESHOLD
```

## VideoManager

```
import cv2
from pygrabber.dshow_graph import FilterGraph


class VideoManager:

    @staticmethod
    def list_cameras():
        """Returns a list of available camera devices."""
        graph = FilterGraph()
        cameras = graph.get_input_devices()
        return cameras

    @staticmethod
    def get_cap(camera_index):
        """Receives a camera index and returns a VideoCapture object for it. Returns None
if the camera could not be opened."""
        cap = cv2.VideoCapture(camera_index)
        if not cap.isOpened():
            return None
        else:
            return cap

    @staticmethod
    def get_frame(cap : cv2.VideoCapture):
        """Receives a VideoCapture object and returns a tuple containing a success flag,
```

```
        a mirrored RGB frame for display, and a small RGB frame for face recognition
processing.\n
        If the frame could not be read, releases the capture and returns (False, )."""
        ret, frame = cap.read()
        if not ret:
            cap.release()
            return (False, )

        mirrored_frame = cv2.flip(frame, 1)
        rgb_mirrored_frame = cv2.cvtColor(mirrored_frame, cv2.COLOR_BGR2RGB)

        small_frame = cv2.resize(frame, (0, 0), fx= 0.25, fy= 0.25)
        rgb_small_frame = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

        return (True, rgb_mirrored_frame, rgb_small_frame)
```

## FaceRecognizer

```
import face_recognition
import dlib
import numpy as np

class FaceRecognizer:

    @staticmethod
    def get_face_encodings(rgb_frame):
        """Receives an RGB frame, detects faces and returns the face encoding of the first
face found as binary data.
        Returns None if no face was detected."""
        face_locations = face_recognition.face_locations(rgb_frame,
number_of_times_to_upsample= 2, model= "hog")
        face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

        if not face_encodings:
            return None
        else:
            encoding_array = face_encodings[0]
            bin_face_encoding = encoding_array.astype(np.float64).tobytes()
            return bin_face_encoding


    @staticmethod
    def compare_faces(stored_bin_face_encoding, bin_face_encoding):
        """Receives a stored face encoding and a login face encoding as binary data,
        compares the two face encodings and returns True if they match, False otherwise."""
        if not bin_face_encoding or not stored_bin_face_encoding:
            return False
```

```
        stored_face_encoding = np.frombuffer(stored_bin_face_encoding, dtype= np.float64)
        face_encoding = np.frombuffer(bin_face_encoding, dtype= np.float64)

        matches = face_recognition.compare_faces([stored_face_encoding], face_encoding)

        if True in matches:
            return True
        else:
            return False
```

## PasswordHandler

```
import random

MINIMUM_PASSWORD_LENGTH = 8    # Minimum length required for a password to be considered
strong
MAXIMUM_PASSWORD_LENGTH = 64    # Maximum allowed password length
DEFAULT_LENGTH = 16    # Default generated password length
DEFAULT_SPECIAL = "-_!?@#$%^&*/|"    # Default set of allowed special characters
DEFAULT_SETTINGS = {"max_length" : DEFAULT_LENGTH,
                    "include" : {"Upper" : True, "Lower" : True, "Number" : True, "Special"
: True},
                    "special_allowed" : DEFAULT_SPECIAL,
                    "start" : {"Upper" : True, "Lower" : True, "Number" : True, "Special" :
True},
                    "special_allowed_start" : DEFAULT_SPECIAL}

UPPER_SET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
LOWER_SET = "abcdefghijklmnopqrstuvwxyz"
NUM_SET = "0123456789"


class PasswordHandler:

    @staticmethod
    def generate_password(preferences):
        """Receives a preferences dict and generates a random password that satisfies the
given rules.
        Ensures the first character matches the start rules and that at least one character
from each
        included category is present."""
        length = preferences["max_length"]
        include_rules = preferences["include"]
        start_rules = preferences["start"]
        password = ""
        start_set = ""
        if start_rules["Upper"]:
            start_set += UPPER_SET
```

```
        if start_rules["Lower"]:
            start_set += LOWER_SET
        if start_rules["Number"]:
            start_set += NUM_SET
        if start_rules["Special"]:
            start_set += preferences["special_allowed_start"]
        password = random.choice(start_set)

        full_password_set = ""
        mandatory_chars = ""
        if include_rules["Upper"]:
            full_password_set += UPPER_SET
            mandatory_chars += random.choice(UPPER_SET)
        if include_rules["Lower"]:
            full_password_set += LOWER_SET
            mandatory_chars += random.choice(LOWER_SET)
        if include_rules["Number"]:
            full_password_set += NUM_SET
            mandatory_chars += random.choice(NUM_SET)
        if include_rules["Special"]:
            full_password_set += preferences["special_allowed"]
            mandatory_chars += random.choice(preferences["special_allowed"])

        remaining_password = list(mandatory_chars)

        for i in range(length - len(remaining_password) - 1):
            remaining_password.append(random.choice(full_password_set))
        random.shuffle(remaining_password)
        shuffled_password = "".join(remaining_password)
        password += shuffled_password
        return password


    @staticmethod
    def is_strong_password(password):
        """Receives a password and returns True if it meets the minimum strength
requirements:
        at least 8 characters, with uppercase, lowercase, digit, and special character."""
        if len(password) >= MINIMUM_PASSWORD_LENGTH and any(char.islower() for char in
password) and any(char.isupper() for char in password) and any(char.isdigit() for char in
password) and any(not char.isalnum() for char in password):
            return True
        else:
            return False
```

## Client

```
import sys
import os
import base64
import json
from client_protocol_handler import ClientProtocolHandler, DATA_DELIMITER
from client_conn_manager import ClientConnectionManager, DEFAULT_SERVER_IP
from ui import UI
from email_validator import validate_email, EmailNotValidError
from client_status import Status
from audio_manager import AudioManager
from voice_authenticator import VoiceAuthenticator
from face_recognizer import FaceRecognizer

class Client:

    def __init__(self):
        self.protocol_handler = ClientProtocolHandler()    # ClientProtocolHandler object
        self.conn_manager = ClientConnectionManager()    # ClientConnectionManager object
        self.conn_manager.connect(self.get_server_ip())
        self.ui = UI(self)    # UI object
        self.audio_manager = AudioManager(self)    # AudioManager object
        self.voice_authenticator = VoiceAuthenticator()    # VoiceAuthenticator object
        self.username = ""    # the username of the logged-in user
        self.is_email_verified = False    # whether the user's email has been verified
        self.bin_voice_signature = None    # the user's voice signature as binary data
        self.bin_face_encoding = None    # the user's face encoding as binary data
        self.are_questions_setup = False    # whether the user has set up security
questions
        self.chosen_questions_dict = {}    # dictionary of the user's chosen security
questions and answers
        self.auth_options = None    # dictionary of the authentication options available to
the user
        self.status = Status.NORMAL    # the current status of the client

    def get_server_ip(self):
        """Returns the server IP from the command line arguments, or the default server IP
if not provided."""
        if len(sys.argv) > 1:
            return sys.argv[1]
        else:
            return DEFAULT_SERVER_IP

    def run(self):
        """Starts the client UI."""
        self.ui.run()

    def change_status(self, status):
```

```
        """Receives a status and sets it as the client's current status."""
        self.status = status

    def can_user_signup(self):
        """Returns True if the user has completed all required signup steps."""
        if self.is_email_verified:
            return True

    def signup(self, username, password, email):
        """Receives username, password and email, and sends a signup request to the
server.\n
        Encodes voice signature and face encoding if available, and includes the chosen
security questions.\n
        Notifies the UI of the server's response."""
        phone_number = "0" # Placeholder for phone number, not used in current
implementation

        self.auth_options = {}
        if self.bin_voice_signature:
            voice_signature = base64.b64encode(self.bin_voice_signature).decode()
            self.auth_options["voice_auth"] = True
        else:
            voice_signature = ""
            self.auth_options["voice_auth"] = False

        if self.bin_face_encoding:
            face_encoding = base64.b64encode(self.bin_face_encoding).decode()
            self.auth_options["face_auth"] = True
        else:
            face_encoding = ""
            self.auth_options["face_auth"] = False

        if self.are_questions_setup:
            questions_str = json.dumps(self.chosen_questions_dict)
        else:
            self.ui.display_signup_error("Security questions are not set")
            return
        msg_data =
f"{username}{DATA_DELIMITER}{password}{DATA_DELIMITER}{email}{DATA_DELIMITER}{phone_number}
{DATA_DELIMITER}{voice_signature}{DATA_DELIMITER}{face_encoding}{DATA_DELIMITER}{questions_
str}"
        self.build_and_send_message(self.protocol_handler.client_messages["signup_msg"],
msg_data)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["user_exists_msg"]:
            self.ui.display_signup_error("Username already exists")
        elif cmd == self.protocol_handler.server_messages["weak_password_msg"]:
            self.ui.display_signup_error("Password is not strong enough")
```

```
        elif cmd == self.protocol_handler.server_messages["signup_ok_msg"]:
            print("signup successful")
            self.ui.signup_successful(username, password)


    def login(self, username, password):
        """Receives username and password, and sends a login request to the server.
        Notifies the UI of the server's response."""
        self.build_and_send_message(self.protocol_handler.client_messages["login_msg"],
f"{username}{DATA_DELIMITER}{password}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["login_ok_msg"]:
            print("login successful")
            self.username = username
            self.ui.login_succesful(username, password)
        elif cmd == self.protocol_handler.server_messages["username_invalid_msg"]:
            self.ui.display_login_error("Username does not exist")
        elif cmd == self.protocol_handler.server_messages["incorrect_password_msg"]:
            self.ui.display_login_error("Password is incorrect")
        elif cmd == self.protocol_handler.server_messages["timeout_msg"]:
            self.ui.display_login_error(f"Too many login attempts. Timeout for {data}
minutes")

    def get_security_questions_list(self):
        """Requests the list of security questions from the server and returns it."""
        self.build_and_send_message(self.protocol_handler.client_messages["security_questio
ns_msg"], "")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["security_questions_list_msg"]:
            security_questions_list = data.split(DATA_DELIMITER)
            return security_questions_list

    def setup_security_questions(self, chosen_questions_dict):
        """Receives a dictionary of chosen security questions and answers, saves them and
notifies the UI."""
        self.chosen_questions_dict = chosen_questions_dict
        self.are_questions_setup = True
        self.ui.questions_setup_successful()

    def get_user_security_questions(self, username):
        """Receives a username and requests the user's security questions from the server.
        Notifies the UI of the server's response."""
        self.build_and_send_message(self.protocol_handler.client_messages["my_security_ques
tions_msg"], username)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["your_security_questions_msg"]:
            user_security_questions = data.split(DATA_DELIMITER)
            self.username = username
            self.ui.run_security_questions_auth_window(user_security_questions)
```

```
        elif cmd == self.protocol_handler.server_messages["username_invalid_msg"]:
            self.ui.display_login_error("Username does not exist")

    def check_security_answers(self, answers):
        """Receives a list of security answers and sends them to the server for
verification.
        Notifies the UI of the server's response."""
        answers_str = DATA_DELIMITER.join(answers)
        self.build_and_send_message(self.protocol_handler.client_messages["check_security_a
nswers_msg"], f"{self.username}{DATA_DELIMITER}{answers_str}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["security_answers_correct_msg"]:
            self.ui.security_answers_correct()
        elif cmd ==
self.protocol_handler.server_messages["security_answers_incorrect_msg"]:
            self.ui.security_answers_incorrect()
        elif cmd == self.protocol_handler.server_messages["timeout_msg"]:
            self.ui.security_answers_timeout(data)

    def reset_password(self, new_password):
        """Receives a new password and sends a password reset request to the server.
        Notifies the UI of the server's response."""
        self.build_and_send_message(self.protocol_handler.client_messages["reset_password_m
sg"], f"{self.username}{DATA_DELIMITER}{new_password}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["reset_ok_msg"]:
            self.ui.password_reset_successful(self.username)
        elif cmd == self.protocol_handler.server_messages["weak_password_msg"]:
            self.ui.password_reset_unsuccessful()

    def get_auth_options(self):
        """Requests the available authentication options from the server and returns
them."""
        self.build_and_send_message(self.protocol_handler.client_messages["get_auth_options
_msg"], f"{self.username}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["your_auth_options_msg"]:
            self.auth_options = json.loads(data)
            return self.auth_options

    def code_login(self, code):
        """Receives a one-time code and sends it to the server for verification.\n
        Handles the response differently depending on the client's current status (signing
in, logging in, or recovering password)."""
        self.build_and_send_message(self.protocol_handler.client_messages["code_login_msg"]
, code)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["code_correct_msg"]:
```

```
            self.is_email_verified = True
            if self.status == Status.SIGNING_IN:
                self.ui.signup_code_correct()
            elif self.status == Status.LOGGING_IN or self.status ==
Status.RECOVERING_PASSWORD:
                self.ui.login_code_correct()
        elif cmd == self.protocol_handler.server_messages["code_incorrect_msg"]:
            self.ui.code_incorrect()
        elif cmd == self.protocol_handler.server_messages["code_expired_msg"]:
            self.ui.code_expired()
        elif cmd == self.protocol_handler.server_messages["timeout_msg"]:
            self.ui.code_login_timeout(data)

    def request_code(self, client_email):
        """Receives an email address and requests the server to send a one-time login code
to it.
        Returns True if the code was sent, False otherwise."""
        self.build_and_send_message(self.protocol_handler.client_messages["send_code_msg"],
f"{client_email}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["code_sent_msg"]:
            return True
        else:
            return False

    def get_email(self, username = ""):
        """Receives a username and requests the user's email from the server. Returns the
email or None if not registered.\n
        If no username is provided, uses the logged-in user's username."""
        if username == "":
            username = self.username
        self.build_and_send_message(self.protocol_handler.client_messages["my_email_msg"],
f"{username}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["your_email_msg"]:
            return data
        elif cmd == self.protocol_handler.server_messages["no_email_registered_msg"]:
            return None

    def is_email_valid(self, email):
        """Receives an email address and returns True if it is a valid deliverable email,
False otherwise."""
        try:
            validate_email(email, check_deliverability=True)
            return True
        except EmailNotValidError as e:
            return False
```

```
    def start_voice_recording(self):
        """Starts recording audio using the audio manager."""
        self.audio_manager.start_recording()

    def stop_voice_recording(self):
        """Stops the audio recording."""
        self.audio_manager.stop_recording()

    def recording_stopped(self):
        """Callback method called by the audio manager when recording has stopped. Notifies
the UI."""
        self.ui.recording_stopped()

    def setup_voice_auth(self):
        """Extracts the voice signature from the recorded audio file and sets up voice
authentication.\n
        During signup, notifies the UI. Otherwise, sends the voice signature to the server
for storage."""
        self.bin_voice_signature =
self.voice_authenticator.calc_voice_signature(self.audio_manager.filename)
        if self.bin_voice_signature is not None:
            if self.status == Status.SIGNING_IN:
                self.ui.voice_auth_setup_successful()
            elif self.status == Status.NORMAL:
                voice_signature = base64.b64encode(self.bin_voice_signature).decode()
                self.build_and_send_message(self.protocol_handler.client_messages["setup_vo
ice_auth_msg"], voice_signature)
                cmd, data = self.receive_and_parse_message()
                if cmd == self.protocol_handler.server_messages["voice_auth_set_msg"]:
                    self.auth_options["voice_auth"] = True
                    self.ui.voice_auth_setup_successful()
        else:
            self.ui.signature_extraction_error()
        os.remove(self.audio_manager.filename)

    def voice_auth(self):
        """Extracts the voice signature from the recorded audio file and sends it to the
server for voice authentication.
        Notifies the UI of the server's response."""
        self.bin_voice_signature =
self.voice_authenticator.calc_voice_signature(self.audio_manager.filename)
        if self.bin_voice_signature is not None:
            voice_signature = base64.b64encode(self.bin_voice_signature).decode()
            self.build_and_send_message(self.protocol_handler.client_messages["voice_auth_m
sg"], f"{self.username}{DATA_DELIMITER}{voice_signature}")
            cmd, data = self.receive_and_parse_message()
            if cmd == self.protocol_handler.server_messages["voice_verified_msg"]:
                self.ui.voice_auth_successful()
```

```
            elif cmd == self.protocol_handler.server_messages["voice_not_verified_msg"]:
                self.ui.voice_auth_unsuccessful()
            elif cmd == self.protocol_handler.server_messages["timeout_msg"]:
                self.ui.voice_auth_timeout(data)
        else:
            self.ui.signature_extraction_error()

    def get_face_encoding(self, rgb_frame):
        """Receives an RGB frame, extracts the face encoding and stores it.\n
        During login or password recovery, proceeds to face recognition immediately.
        Otherwise, notifies the UI whether a face was located."""
        self.bin_face_encoding = FaceRecognizer.get_face_encodings(rgb_frame)
        if self.bin_face_encoding:
            if self.status == Status.LOGGING_IN or self.status ==
Status.RECOVERING_PASSWORD:
                self.face_recognition()
            else:
                self.ui.face_located()
        else:
            if self.status != Status.LOGGING_IN and self.status !=
Status.RECOVERING_PASSWORD:
                self.ui.no_face_located()

    def setup_face_recognition(self):
        """Sets up face recognition using the stored face encoding.\n
        During signup, notifies the UI. Otherwise, sends the face encoding to the server
for storage."""
        if self.bin_face_encoding:
            if self.status == Status.SIGNING_IN:
                self.ui.face_recognition_setup_successful()
            elif self.status == Status.NORMAL:
                face_encoding_str = base64.b64encode(self.bin_face_encoding).decode()
                self.build_and_send_message(self.protocol_handler.client_messages["setup_fa
ce_recognition_msg"], face_encoding_str)
                cmd, data = self.receive_and_parse_message()
                if cmd ==
self.protocol_handler.server_messages["face_recognition_set_msg"]:
                    self.auth_options["face_auth"] = True
                    self.ui.face_recognition_setup_successful()
        else:
            self.ui.no_face_located()

    def face_recognition(self):
        """Sends the stored face encoding to the server for face recognition verification.
        Notifies the UI of the server's response."""
        if not self.bin_face_encoding:
            return
        else:
```

```
            face_encoding_str = base64.b64encode(self.bin_face_encoding).decode()
        self.build_and_send_message(self.protocol_handler.client_messages["check_face_recog
nition_msg"], f"{self.username}{DATA_DELIMITER}{face_encoding_str}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["face_matches_msg"]:
            self.ui.face_recognition_successful()
        elif cmd == self.protocol_handler.server_messages["face_not_matches_msg"]:
            self.ui.face_recognition_unsuccessful()


    def register_site(self, site_url, site_username, password_preferences):
        """Receives a site URL, site username and optional password preferences, and sends
a site registration request to the server.
        Notifies the UI of the server's response."""
        if password_preferences is not None:
            preferences = json.dumps(password_preferences)
        else:
            preferences = ""
        self.build_and_send_message(self.protocol_handler.client_messages["register_site_ms
g"], f"{site_url}{DATA_DELIMITER}{site_username}{DATA_DELIMITER}{preferences}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["registration_ok_msg"]:
            self.ui.registration_succesful(data)
        elif cmd == self.protocol_handler.server_messages["already_registered_msg"]:
            self.ui.display_registration_error("site already registered")

    def get_password_preferences(self, site_url):
        """Receives a site URL and requests the password preferences for the site from the
server. Returns the preferences dictionary."""
        self.build_and_send_message(self.protocol_handler.client_messages["get_preferences_
msg"], f"{site_url}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["your_preferences_msg"]:
            password_preferences = json.loads(data)
            return password_preferences

    def update_password_preferences(self, site_url, preferences):
        """Receives a site URL and a preferences dictionary, and sends a request to update
the password preferences for the site."""
        preferences_json = json.dumps(preferences)
        self.build_and_send_message(self.protocol_handler.client_messages["save_preferences
_msg"], f"{site_url}{DATA_DELIMITER}{preferences_json}")
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["preferences_saved_msg"]:
```

```
            print(f"Password preferences for {site_url} saved")

    def get_registered_sites(self, get_deleted):
        """Receives a boolean and requests the list of registered sites from the server.\n
        If get_deleted is True, requests the deleted sites. Otherwise, requests the active
sites."""
        self.build_and_send_message(self.protocol_handler.client_messages["registered_sites
_msg"], f"{get_deleted}")
        cmd, data = self.receive_and_parse_message()
        registered_sites_list = data.split(DATA_DELIMITER) if data != "" else []
        return registered_sites_list

    def get_site_info(self, site_url):
        """Receives a site URL and requests the site's username and password from the
server. Returns them as a tuple."""
        self.build_and_send_message(self.protocol_handler.client_messages["site_info_msg"],
site_url)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["info_msg"]:
            data_list = data.split(DATA_DELIMITER)
            site_username = data_list[0]
            site_password = data_list[1]
            return site_username, site_password

    def change_password(self, site_url):
        """Receives a site URL and requests the server to generate a new password for the
site. Returns the new password."""
        self.build_and_send_message(self.protocol_handler.client_messages["change_password_
msg"], site_url)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["change_ok_msg"]:
            return data

    def delete_site(self, site_url):
        """Receives a site URL and sends a request to the server to delete the site.
        Notifies the UI on success."""
        self.build_and_send_message(self.protocol_handler.client_messages["delete_site_msg"
], site_url)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["site_deleted_msg"]:
            self.ui.get_registered_sites()
```

```
    def recover_site(self, site_url):
        """Receives a site URL and sends a request to the server to recover the previously
deleted site.
        Notifies the UI on success."""
        self.build_and_send_message(self.protocol_handler.client_messages["recover_site_msg
"], site_url)
        cmd, data = self.receive_and_parse_message()
        if cmd == self.protocol_handler.server_messages["not_registered_msg"]:
            print("site not registered")
        elif cmd == self.protocol_handler.server_messages["site_recovered_msg"]:
            self.ui.site_recovery_successful()

    def build_and_send_message(self, cmd, data):
        """Receives cmd and data, builds a protocol message, encrypts it and sends it to
the server."""
        msg = self.protocol_handler.build_message(cmd, data)
        encrypted_msg = self.conn_manager.encrypt_data(msg, self.conn_manager.get_AES())
        try:
            self.conn_manager.send_msg(self.conn_manager.socket, encrypted_msg)
        except ConnectionResetError as e:
            self.ui.connection_lost()
            return


    def receive_and_parse_message(self):
        """Receives an encrypted message from the server, decrypts it and parses it.
Returns the cmd and data."""
        try:
            encrypted_msg = self.conn_manager.receive_msg(self.conn_manager.socket)
        except ConnectionResetError as e:
            self.ui.connection_lost()
            return (None, None)
        decrypted_msg = self.conn_manager.decrypt_data(encrypted_msg,
self.conn_manager.get_AES())
        cmd, data = self.protocol_handler.parse_message(decrypted_msg)
        return cmd, data

client = Client()
client.run()
```

UI

```
import customtkinter as ctk
import threading
from PIL import Image, ImageOps, ImageDraw
from video_manager import VideoManager
from audio_manager import MAX_RECORD_SECONDS
from signup_window import SignupWindow
```

```
from login_window import LoginWindow
from auth_select_window import AuthSelectWindow
from code_login_window import CodeLoginWindow
from voice_recording_window import VoiceRecordingWindow
from video_capture_window import VideoCaptureWindow
from security_questions_setup_window import SecurityQuestionsSetupWindow
from security_questions_auth_window import SecurityQuestionsAuthWindow
from reset_password_window import ResetPasswordWindow
from site_registration_window import SiteRegistrationWindow
from password_preferences_window import PasswordPreferencesWindow
from site_info_window import SiteInfoWindow
from change_password_window import ChangePasswordWindow
from site_recovery_window import SiteRecoveryWindow
from settings_menu_frame import SettingsMenuFrame
from home_frame import HomeFrame
from color_palettes import palette_color
from CTkListbox import *
from CTkMessagebox import *
from client_status import Status

WINDOW_WIDTH = 1200
WINDOW_HEIGHT = 675

class UI:

    def __init__(self, client):
        self.window = ctk.CTk()
        self.window.wm_title("Password Manager")
        self.window.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.window.minsize(WINDOW_WIDTH, WINDOW_HEIGHT)
        self.window.resizable(True, True)
        self.window.configure(fg_color=palette_color("bg"))
        self.client = client    # The Client instance managing server communication

        # ── Top navbar ──
        self.navbar = ctk.CTkFrame(self.window, height=60,
fg_color=palette_color("surface"), corner_radius=0)
        self.navbar.pack(side="top", fill="x")
        self.navbar.pack_propagate(False)

        # ── Content area below navbar ──
        self.content_area = ctk.CTkFrame(self.window, fg_color="transparent")
        self.content_area.pack(side="top", fill="both", expand=True)

        nav_icon = ctk.CTkFrame(self.navbar, width=34, height=34, corner_radius=17,
fg_color=palette_color("accent"))
        nav_icon.place(x=24, rely=0.5, anchor="w")
```

```
        nav_icon_lbl = ctk.CTkLabel(nav_icon, width=0, height=0, text="🔒",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 14))
        nav_icon_lbl.place(relx=0.5, rely=0.5, anchor="center")

        nav_title = ctk.CTkLabel(self.navbar, text="Password Manager", font=("sans", 18,
"bold"), text_color=palette_color("text"))
        nav_title.place(x=66, rely=0.5, anchor="w")

        self.login_button = ctk.CTkButton(self.navbar, width=110, height=36,
corner_radius=18, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Login",
text_color=palette_color("surface"), font=("sans", 14), command=self.run_login_window)
        self.login_button.place(relx=1.0, x=-260, rely=0.5, anchor="center")

        self.login_window = None    # Reference to the currently open LoginWindow, or None
        self.reset_password_window = None    # Reference to the currently open
ResetPasswordWindow, or None

        self.signup_button = ctk.CTkButton(self.navbar, width=110, height=36,
corner_radius=18, fg_color=palette_color("btn_secondary"),
hover_color=palette_color("input_border"), text="Sign Up", text_color="white",
font=("sans", 14), command=self.run_signup_window)
        self.signup_button.place(relx=1.0, x=-130, rely=0.5, anchor="center")

        self.signup_window = None    # Reference to the currently open SignupWindow, or
None

        self.settings_menu = SettingsMenuFrame(self, corner_radius=0)    # Side panel with
auth and account settings
        self.settings_menu.place(relx=1.0, rely=0, relheight=1, relwidth=0.2, anchor="nw")

        self.settings_button = ctk.CTkButton(self.navbar, width=36, height=36,
corner_radius=18, text="\uE713", text_color=palette_color("text_dim"),
fg_color="transparent", hover_color=palette_color("surface2"), font=("Segoe MDL2 Assets",
16), command=self.settings_menu.toggle)
        self.settings_button.place(relx=1.0, x=-36, rely=0.5, anchor="center")

        self.auth_select_window = None    # Reference to the currently open
AuthSelectWindow, or None
        self.code_login_window = None    # Reference to the currently open CodeLoginWindow,
or None
        self.voice_recording_window = None    # Reference to the currently open
VoiceRecordingWindow, or None

        self.cap = None    # OpenCV VideoCapture object for the active camera, or None
        self.current_camera_index = -1    # Index of the currently active camera (-1 if
none)
```

```
        self.rgb_small_frame = None    # Downscaled RGB frame used for face recognition
processing
        self.video_capture_window = None    # Reference to the currently open
VideoCaptureWindow, or None

        self.security_questions_window = None    # Reference to the currently open security
questions window, or None

        # ── Home content (pre-login) ──
        self.home_frame = HomeFrame(self.content_area)
        self.home_frame.place(relx=0.5, relwidth=1, relheight=1, anchor="n")

        # ── Dashboard widgets (post-login, created but not placed) ──
        self.dashboard_frame = ctk.CTkFrame(self.content_area, fg_color="transparent")
        self.dashboard_frame.pack_propagate(False)

        btn_h = 38
        btn_r = 10
        btn_font = ("sans", 13)

        self.btn_row = ctk.CTkFrame(self.dashboard_frame, fg_color="transparent")

        self.register_site_button = ctk.CTkButton(self.btn_row, height=btn_h,
corner_radius=btn_r, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="+ Register Site",
text_color=palette_color("surface"), font=btn_font,
command=self.run_site_registration_window)
        self.username_text = ctk.CTkLabel(self.navbar, anchor="center", text="",
font=("sans", 15), text_color=palette_color("text"))
        self.site_registration_window = None    # Reference to the currently open
SiteRegistrationWindow, or None

        self.site_info_button = ctk.CTkButton(self.btn_row, height=btn_h,
corner_radius=btn_r, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="View Info",
text_color=palette_color("surface"), font=btn_font, state="disabled",
command=self.show_selected_site_info)
        self.site_info_window = None    # Reference to the currently open SiteInfoWindow,
or None

        self.change_password_button = ctk.CTkButton(self.btn_row, height=btn_h,
corner_radius=btn_r, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Change Password",
text_color=palette_color("surface"), font=btn_font, state="disabled",
command=self.initiate_change_password_window)
        self.change_password_window = None    # Reference to the currently open
ChangePasswordWindow, or None
```

```
        self.recover_site_button = ctk.CTkButton(self.btn_row, height=btn_h,
corner_radius=btn_r, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Recover Site",
text_color=palette_color("surface"), font=btn_font, command=self.run_site_recovery_window)
        self.site_recovery_window = None    # Reference to the currently open
SiteRecoveryWindow, or None

        self.delete_site_button = ctk.CTkButton(self.btn_row, height=btn_h,
corner_radius=btn_r, fg_color=palette_color("error"), hover_color=("#dc2626", "#b91c1c"),
text="Delete Site", text_color="white", font=btn_font, state="disabled",
command=self.delete_site)

        self.search_var = ctk.StringVar()    # Bound to the search box; changes trigger
site list filtering
        self.search_var.trace_add("write", self.load_registered_sites)
        self.search_frame = ctk.CTkFrame(self.dashboard_frame, fg_color="transparent")
        self.search_icon_label = ctk.CTkLabel(self.search_frame, width=36, height=36,
corner_radius=10, text="\U0001F50E", font=("Segoe UI Emoji", 16), fg_color="transparent",
text_color=palette_color("text_dim"))
        self.search_box = ctk.CTkEntry(self.search_frame, height=36, corner_radius=10,
placeholder_text="Search sites...", fg_color=palette_color("surface"),
border_color=palette_color("input_border"), text_color=palette_color("text"),
placeholder_text_color=palette_color("placeholder"), font=("sans", 13),
textvariable=self.search_var)
        self.delete_search_button = ctk.CTkButton(self.search_frame, width=36, height=36,
corner_radius=10, text="✕", font=("sans", 14), fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text_color=palette_color("text_dim"),
command=lambda: self.search_var.set(""))

        self.current_site_selection = None    # URL of the currently selected site in the
listbox, or None
        self.registered_sites_list = []    # List of the user's registered site URLs
        self.registered_sites_listbox = CTkListbox(self.dashboard_frame,
command=self.enable_site_buttons)

        self.password_preferences_window = None    # Reference to the currently open
PasswordPreferencesWindow, or None


    def run(self):
        """Starts the main UI event loop."""
        self.window.mainloop()

    def run_signup_window(self):
        """Opens the signup window, or focuses it if already open."""
        self.client.change_status(Status.SIGNING_IN)
        if self.signup_window is None or not self.signup_window.winfo_exists():
            self.signup_window = SignupWindow(self)
        else:
```

```
            self.signup_window.focus()

    def call_signup(self, username, password, email):
        """Forwards a signup request to the client."""
        self.client.signup(username, password, email)

    def display_signup_error(self, msg):
        """Displays an error message in the signup window."""
        self.signup_window.display_signup_error(msg)

    def signup_successful(self, username, password):
        """Closes the signup window and automatically logs the user in."""
        self.signup_window.close_window()
        self.call_login(username, password)

    def run_login_window(self):
        """Opens the login window, or focuses it if already open."""
        self.client.change_status(Status.LOGGING_IN)
        if self.login_window is None or not self.login_window.winfo_exists():
            self.login_window = LoginWindow(self)
        else:
            self.login_window.focus()


    def call_login(self, username, password):
        """Forwards a login request to the client."""
        self.client.login(username, password)

    def display_login_error(self, msg):
        """Displays an error message in the login window."""
        self.login_window.display_login_error(msg)

    def login_succesful(self, username, password):
        """Handles a successful login.
        If signing up, sets up the user directly. If logging in, opens the auth selection
window."""
        if not self.login_window is None:
            self.login_window.close_window()
        if self.client.status == Status.SIGNING_IN:
            self.setup_user(username)
        elif self.client.status == Status.LOGGING_IN:
            self.run_auth_select_window()


    def get_email(self):
        """Returns the user's email from the client."""
        return self.client.get_email()
```

```
    def call_get_auth_options(self):
        """Returns the user's configured authentication options from the client."""
        return self.client.get_auth_options()

    def run_auth_select_window(self):
        """Opens the authentication selection window, or focuses it if already open."""
        if self.auth_select_window is None or not self.auth_select_window.winfo_exists():
            auth_options = self.call_get_auth_options()
            self.auth_select_window = AuthSelectWindow(self, auth_options)
        else:
            self.auth_select_window.focus()

    def run_security_questions_setup_window(self, ):
        """Opens the security questions setup window, or focuses it if already open."""
        if self.security_questions_window is None or not
self.security_questions_window.winfo_exists():
            security_questions_list = self.call_get_security_questions_list()
            self.security_questions_window = SecurityQuestionsSetupWindow(self,
security_questions_list, self.client.chosen_questions_dict)
        else:
            self.security_questions_window.focus()

    def call_get_security_questions_list(self):
        """Returns the available security questions list from the client."""
        return self.client.get_security_questions_list()

    def call_setup_security_questions(self, chosen_questions_dict):
        """Forwards the chosen security questions and answers to the client."""
        self.client.setup_security_questions(chosen_questions_dict)

    def questions_setup_successful(self):
        """Closes the security questions window and notifies the signup window of
success."""
        self.security_questions_window.close_window()
        self.signup_window.questions_setup_successful()

    def run_security_questions_auth_window(self, user_security_questions):
        """Closes the login window and opens the security questions authentication
window."""
        self.login_window.close_window()
        if self.security_questions_window is None or not
self.security_questions_window.winfo_exists():
            self.security_questions_window = SecurityQuestionsAuthWindow(self,
user_security_questions)
        else:
            self.security_questions_window.focus()

    def call_get_user_security_questions(self, username):
```

```
        """Requests the user's security questions from the server."""
        self.client.get_user_security_questions(username)

    def call_check_security_answers(self, answers):
        """Forwards the user's security answers to the client for verification."""
        self.client.check_security_answers(answers)

    def security_answers_correct(self):
        """Handles correct security answers by switching to password recovery and opening
auth selection."""
        self.client.change_status(Status.RECOVERING_PASSWORD)
        self.security_questions_window.close_window()
        self.run_auth_select_window()

    def security_answers_incorrect(self):
        """Displays an error in the security questions window for incorrect answers."""
        self.security_questions_window.display_error("One or more answers are incorrect")

    def security_answers_timeout(self, minutes):
        """Displays a timeout error in the security questions window."""
        self.security_questions_window.display_error(f"Too many attempts. Timeout for
{minutes} minutes")

    def run_reset_password_window(self):
        """Opens the reset password window, or focuses it if already open."""
        if self.reset_password_window is None or not
self.reset_password_window.winfo_exists():
            self.reset_password_window = ResetPasswordWindow(self)
        else:
            self.reset_password_window.focus()

    def call_reset_password(self, new_password):
        """Forwards a password reset request to the client."""
        self.client.reset_password(new_password)

    def password_reset_successful(self, username):
        """Closes the reset password window and sets up the user session."""
        self.reset_password_window.close_window()
        self.setup_user(username)

    def password_reset_unsuccessful(self):
        """Displays an error in the reset password window for a weak password."""
        self.reset_password_window.display_error("Password is not strong enough")

    def run_code_login_window(self, client_email):
        """Opens the code login window for the given email, or focuses it if already
open."""
        if self.code_login_window is None or not self.code_login_window.winfo_exists():
```

```
            self.code_login_window = CodeLoginWindow(self, client_email)
            self.code_login_window.focus()
            self.code_login_window.lift()
        else:
            self.code_login_window.focus()
            self.code_login_window.lift()

    def request_login_code(self, client_email):
        """Requests an email verification code. Opens the code window if successful."""
        was_code_sent = self.client.request_code(client_email)
        if was_code_sent:
            if self.code_login_window is None:
                self.run_code_login_window(client_email)
            else:
                self.code_login_window.code_resent()

    def call_is_email_valid(self, email):
        """Checks email validity via the client and notifies the signup window."""
        result = self.client.is_email_valid(email)
        if result:
            self.signup_window.email_valid()
        else:
            self.signup_window.email_invalid()

    def call_check_code(self, code):
        """Forwards the entered verification code to the client."""
        self.client.code_login(code)

    def signup_code_correct(self):
        """Handles a correct code during signup. Enables the signup form if all conditions
are met."""
        self.code_login_window.close_window()
        if self.client.can_user_signup():
            self.signup_window.can_signup()
            self.signup_window.email_verified()


    def login_code_correct(self):
        """Handles a correct code during login. Closes the code window and completes
authentication."""
        self.code_login_window.close_window()
        self.auth_successful()

    def code_incorrect(self):
        """Displays incorrect code feedback in the code login window."""
        self.code_login_window.incorrect_code_feedback()

    def code_expired(self):
        """Displays a code expired message in the code login window."""
```

```
        self.code_login_window.display_code_expired_msg()

    def code_login_timeout(self, minutes):
        """Displays a timeout error in the code login window."""
        self.code_login_window.display_error(f"Too many attempts. Timeout for {minutes}
minutes")

    def run_voice_recording_window(self):
        """Opens the voice recording window in setup or auth mode, or focuses it if already
open."""
        if self.voice_recording_window is None or not
self.voice_recording_window.winfo_exists():
            if self.client.status == Status.SIGNING_IN or self.client.status ==
Status.NORMAL:
                is_setup = True
            else:
                is_setup = False
            self.voice_recording_window = VoiceRecordingWindow(self, is_setup,
MAX_RECORD_SECONDS)
            self.voice_recording_window.focus()
            self.voice_recording_window.lift()
        else:
            self.voice_recording_window.focus()
            self.voice_recording_window.lift()

    def call_start_recording(self):
        """Starts voice recording via the client."""
        self.client.start_voice_recording()

    def call_stop_recording(self):
        """Stops voice recording via the client."""
        self.client.stop_voice_recording()

    def recording_stopped(self):
        """Notifies the voice recording window that recording has stopped."""
        self.voice_recording_window.recording_stopped()

    def call_setup_voice_auth(self):
        """Sends the recorded voice signature to the server for setup."""
        self.client.setup_voice_auth()

    def voice_auth_setup_successful(self):
        """Closes the voice recording window and notifies the signup or settings menu of
success."""
        self.voice_recording_window.close_window()
        if self.client.status == Status.SIGNING_IN:
            self.signup_window.voice_auth_added()
        elif self.client.status == Status.NORMAL:
```

```
            self.settings_menu.voice_auth_added()

    def signature_extraction_error(self):
        """Displays a voice signature extraction error in the recording window."""
        self.voice_recording_window.display_error("Could not extract voice signature.
Please record again.")

    def call_voice_auth(self):
        """Sends the recorded voice to the server for authentication."""
        self.client.voice_auth()

    def voice_auth_successful(self):
        """Closes the voice recording window and completes authentication."""
        self.voice_recording_window.close_window()
        self.auth_successful()

    def voice_auth_unsuccessful(self):
        """Displays a voice mismatch error in the recording window."""
        self.voice_recording_window.display_error("Voice didn't match. Try recording
again.")

    def voice_auth_timeout(self, minutes):
        """Displays a timeout error in the voice recording window."""
        self.voice_recording_window.display_error(f"Too many attempts. Timeout for
{minutes} minutes")


    def get_cameras_list(self):
        """Returns the list of available cameras."""
        return VideoManager.list_cameras()


    def run_video_capture_window(self):
        """Opens the video capture window in setup or auth mode, or focuses it if already
open."""
        if self.video_capture_window is None or not
self.video_capture_window.winfo_exists():
            cameras_list = self.get_cameras_list()
            if self.client.status == Status.SIGNING_IN or self.client.status ==
Status.NORMAL:
                is_setup = True
            else:
                is_setup = False
            self.video_capture_window = VideoCaptureWindow(self, cameras_list, is_setup)
        else:
            self.video_capture_window.focus()

    def start_video_recording(self, camera_index):
```

```
        """Starts video capture from the given camera index. Releases any previous capture
if switching cameras."""
        if self.cap is not None:
            if self.current_camera_index == camera_index:
                return
            else:
                self.cap.release()
                self.cap = None

        self.cap = VideoManager.get_cap(camera_index)
        if self.cap is None:
            self.video_capture_window.display_camera_error()
        else:
            self.current_camera_index = camera_index
            self.video_capture_window.record_loop()
            self.video_capture_window.face_recognition_loop()

    def get_frames(self):
        """Captures a frame from the active camera, processes it, and updates the video
capture window."""
        data = VideoManager.get_frame(self.cap)
        if data[0]:
            rgb_mirrored_frame = data[1]
            self.rgb_small_frame = data[2]
            if self.video_capture_window is not None:
                image_size = (460, 380)
                pil_image = Image.fromarray(rgb_mirrored_frame)
                pil_image = ImageOps.fit(pil_image, image_size)
                pil_image = self._round_image_corners(pil_image, 10)
                image = ctk.CTkImage(light_image= pil_image, dark_image= pil_image, size=
image_size)
                self.video_capture_window.update_frame(image)
        else:
            self.video_capture_window.display_camera_error()
            self.current_camera_index = -1

    def _round_image_corners(self, image, radius):
        """Receives a PIL image and radius, and returns the image with rounded corners."""
        mask = Image.new("L", image.size, 0)
        draw = ImageDraw.Draw(mask)
        draw.rounded_rectangle([0, 0, image.size[0], image.size[1]], radius=radius,
fill=255)
        image.putalpha(mask)
        return image

    def call_face_recognition(self):
        """Attempts face encoding on the current small frame. Disables setup button in
video capture window if no frame is available."""
```

```
        if self.rgb_small_frame is not None:
            self.client.get_face_encoding(self.rgb_small_frame)
        else:
            self.video_capture_window.disable_setup_button()

    def face_located(self):
        """Enables the setup/auth button in the video capture window when a face is
detected."""
        self.video_capture_window.enable_setup_button()

    def no_face_located(self):
        """Disables the setup/auth button in the video capture window when no face is
detected."""
        self.video_capture_window.disable_setup_button()

    def call_setup_face_recognition(self):
        """Sends the captured face encoding to the server for setup."""
        self.client.setup_face_recognition()

    def face_recognition_setup_successful(self):
        """Closes the video capture window and notifies the signup or settings menu of
success."""
        self.video_capture_window.close_window()
        if self.client.status == Status.SIGNING_IN:
            self.signup_window.face_recognition_added()
        elif self.client.status == Status.NORMAL:
            self.settings_menu.face_recognition_added()

    def face_recognition_successful(self):
        """Closes the video capture window and completes authentication."""
        self.video_capture_window.close_window()
        self.auth_successful()

    def face_recognition_unsuccessful(self):
        """Displays a face mismatch error in the video capture window."""
        self.video_capture_window.display_face_recognition_error("Face doesn't match.")

    def auth_successful(self):
        """Handles successful authentication. Sets up the user if logging in, or opens
reset password if recovering."""
        username = self.client.username
        if self.client.status == Status.LOGGING_IN:
            self.setup_user(username)
        elif self.client.status == Status.RECOVERING_PASSWORD:
            self.run_reset_password_window()

    def get_registered_sites(self):
        """Fetches the user's active registered sites and refreshes the listbox."""
```

```
        self.registered_sites_list = self.client.get_registered_sites(False)
        self.load_registered_sites()

    def get_deleted_sites(self):
        """Returns the user's deleted (recoverable) sites from the client."""
        return self.client.get_registered_sites(True)

    def setup_user(self, username):
        """Transitions the UI from the pre-login home screen to the post-login
dashboard."""
        self.login_button.place_forget()
        self.signup_button.place_forget()
        self.home_frame.place_forget()

        # Show username in navbar
        self.username_text.configure(text= f"Welcome, {username}")
        self.username_text.place(relx=1.0, x=-75, rely=0.5, anchor="e")

        # Show dashboard
        self.dashboard_frame.place(x=0, y=0, relwidth=1, relheight=1)

        # Button row
        self.btn_row.pack(fill="x", padx=30, pady=(20, 10))
        for btn in (self.register_site_button, self.site_info_button,
self.change_password_button, self.recover_site_button, self.delete_site_button):
            btn.pack(side="left", padx=10, expand=True, fill="x")

        # Search bar
        self.search_frame.pack(fill="x", padx=30, pady=(10, 4))
        self.search_icon_label.pack(side="left")
        self.search_box.pack(side="left", fill="x", expand=True)
        self.delete_search_button.pack(side="left", padx=(6, 0))

        # Site list
        self.registered_sites_listbox.pack(fill="both", padx=30, pady=(0, 20), expand=True)

        self.get_registered_sites()
        self.settings_menu.show_auth_options(self.client.auth_options)
        self.client.change_status(Status.NORMAL)

    def run_site_registration_window(self):
        """Opens the site registration window, or focuses it if already open."""
        if self.site_registration_window is None or not
self.site_registration_window.winfo_exists():
            self.site_registration_window = SiteRegistrationWindow(self)
            self.client.change_status(Status.REGISTERING_SITE)
        else:
            self.site_registration_window.focus()
```

```
    def call_register_site(self, site_url, site_username, password_preferences):
        """Forwards a site registration request to the client."""
        self.client.register_site(site_url, site_username, password_preferences)

    def display_registration_error(self, msg):
        """Displays an error message in the site registration window."""
        self.site_registration_window.display_registration_error(msg)

    def registration_succesful(self, password):
        """Closes the registration window, refreshes the site list, and copies the password
if requested."""
        copy_password = self.site_registration_window.close_window()
        self.get_registered_sites()
        if copy_password:
            self.copy_to_clipboard(password)
        self.client.change_status(Status.NORMAL)

    def run_password_preferences_window(self, preferences = None, site_url = None):
        """Opens the password preferences window, or focuses it if already open."""
        if self.password_preferences_window is None or not
self.password_preferences_window.winfo_exists():
            self.password_preferences_window = PasswordPreferencesWindow(self, preferences,
site_url)
            self.password_preferences_window.focus()
            self.password_preferences_window.lift()
        else:
            self.password_preferences_window.focus()
            self.password_preferences_window.lift()

    def save_password_preferences(self, preferences, site_url):
        """Saves password preferences locally during registration, or updates them on the
server if already registered."""
        if self.client.status == Status.REGISTERING_SITE:
            self.site_registration_window.preferences = preferences
            self.password_preferences_window.close_window()
        elif self.client.status == Status.NORMAL:
            self.call_update_password_preferences(site_url, preferences)
            self.password_preferences_window.close_window()

    def load_registered_sites(self, *args):
        """Filters and refreshes the registered sites listbox based on the current search
term."""
        search_term = self.search_var.get().lower()
        new_list = [site for site in self.registered_sites_list
                if not search_term or search_term in site.lower()]
        try:
            current_content = self.registered_sites_listbox.get("all")
```

```
        except:
            current_content = []
        if new_list != current_content:
            self.registered_sites_listbox.delete("all")
            found_current_selection = False
            for site_url in self.registered_sites_list:
                if self.search_var.get() != "":
                    if self.search_var.get().lower() in site_url.lower():
                        self.registered_sites_listbox.insert("end", site_url)
                        if site_url == self.current_site_selection:
                            found_current_selection = True
                            self.registered_sites_listbox.select(-1)
                else:
                    self.registered_sites_listbox.insert("end", site_url)
                    if site_url == self.current_site_selection:
                            found_current_selection = True
                            self.registered_sites_listbox.select(-1)

            if not found_current_selection:
                self.disable_site_buttons()

    def enable_site_buttons(self, site):
        """Enables site action buttons when a site is selected in the listbox."""
        self.current_site_selection = site
        self.site_info_button.configure(state= "normal")
        self.change_password_button.configure(state= "normal")
        self.delete_site_button.configure(state= "normal")

    def disable_site_buttons(self):
        """Disables site action buttons when no site is selected."""
        self.site_info_button.configure(state= "disabled")
        self.change_password_button.configure(state= "disabled")
        self.delete_site_button.configure(state= "disabled")


    def show_selected_site_info(self):
        """Fetches and displays the info window for the currently selected site."""
        site_url = self.registered_sites_listbox.get()
        site_username, site_password = self.call_site_info(site_url)
        self.run_site_info_window(site_username, site_password)

    def call_site_info(self, site_url):
        """Returns the username and password for a given site from the client."""
        return self.client.get_site_info(site_url)

    def run_site_info_window(self, site_username, site_password):
        """Opens the site info window, or focuses it if already open."""
        if self.site_info_window is None or not self.site_info_window.winfo_exists():
            self.site_info_window = SiteInfoWindow(self, site_username, site_password)
```

```
        else:
            self.site_info_window.focus()

    def initiate_change_password_window(self):
        """Fetches the selected site's info and opens the change password window."""
        site_url = self.registered_sites_listbox.get()
        site_username, site_password = self.call_site_info(site_url)
        self.run_change_site_password_window(site_url, site_username)

    def run_change_site_password_window(self, site_url, site_username):
        """Opens the change password window, or focuses it if already open."""
        if self.change_password_window is None or not
self.change_password_window.winfo_exists():
            self.change_password_window = ChangePasswordWindow(self, site_url,
site_username)
        else:
            self.change_password_window.focus()

    def call_change_site_password(self, site_url):
        """Generates a new password for the site and copies it to clipboard if
requested."""
        generated_password = self.client.change_password(site_url)
        should_copy_password = self.change_password_window.close_window()
        if should_copy_password:
            self.copy_to_clipboard(generated_password)

    def get_preferences_and_initiate_window(self, site_url):
        """Fetches password preferences for a site and opens the preferences window."""
        password_preferences = self.client.get_password_preferences(site_url)
        self.run_password_preferences_window(password_preferences, site_url)

    def call_update_password_preferences(self, site_url, preferences):
        """Forwards updated password preferences to the client."""
        self.client.update_password_preferences(site_url, preferences)

    def delete_site(self):
        """Deletes the currently selected site via the client."""
        site_url = self.registered_sites_listbox.get()
        self.client.delete_site(site_url)

    def run_site_recovery_window(self):
        """Opens the site recovery window with the list of deleted sites, or focuses it if
already open."""
        deleted_sites = self.get_deleted_sites()
        if self.site_recovery_window is None or not
self.site_recovery_window.winfo_exists():
            self.site_recovery_window = SiteRecoveryWindow(self, deleted_sites)
        else:
```

```
            self.site_recovery_window.focus()

    def site_recovery_successful(self):
        """Closes the recovery window and refreshes the registered sites list."""
        self.site_recovery_window.close_window()
        self.get_registered_sites()

    def call_recover_site(self, site_url):
        """Forwards a site recovery request to the client."""
        self.client.recover_site(site_url)

    def copy_to_clipboard(self, text):
        """Copies the given text to the system clipboard."""
        self.window.clipboard_clear()
        self.window.clipboard_append(text)
        self.window.update()

    def connection_lost(self):
        """Displays a connection lost message box and closes the app if the user
confirms."""
        msg = CTkMessagebox(title= "Password Manager", message= "Connection with server
lost", font= ("sans", 16), option_1= "Close App", icon= "cancel", icon_size= (50, 50))
        if msg.get() == "Close App":
            self.window.destroy()
```

## SignupWindow

```
import customtkinter as ctk
import threading
from color_palettes import palette_color
from tooltip_window import ToolTip

WINDOW_WIDTH = 520
WINDOW_HEIGHT = 750

class SignupWindow(ctk.CTkToplevel):

    def __init__(self, app, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        # ── Icon circle ──
```

```
        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=38, anchor="center")
        self.icon_label = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="👤",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 22))
        self.icon_label.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Create Account",
font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=85, anchor="center")

        # ── Main content frame ──
        content = ctk.CTkFrame(self, fg_color="transparent")
        content.place(relx=0.5, y=108, anchor="n")
        content.columnconfigure(0, weight=1, minsize=WINDOW_WIDTH - 40)

        # ── ACCOUNT DETAILS section ──
        self.section_account = ctk.CTkLabel(content, height=18, anchor="w", text="ACCOUNT
DETAILS", font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_account.grid(row=0, column=0, sticky="w")
        self.divider_account = ctk.CTkFrame(content, height=2,
fg_color=palette_color("divider"))
        self.divider_account.grid(row=1, column=0, sticky="ew", pady=(1, 6))

        account_frame = ctk.CTkFrame(content, fg_color="transparent")
        account_frame.grid(row=2, column=0, sticky="ew")
        account_frame.columnconfigure(0, weight=1)

        self.username_label = ctk.CTkLabel(account_frame, height=18, anchor="w",
text="Username", font=("sans", 13), text_color=palette_color("text_dim"))
        self.username_label.grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.username_entry = ctk.CTkEntry(account_frame, height=36, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text="Enter username",
placeholder_text_color=palette_color("placeholder"), font=("sans", 14))
        self.username_entry.grid(row=1, column=0, columnspan=2, sticky="ew", pady=(0, 6))

        self.password_label = ctk.CTkLabel(account_frame, height=18, anchor="w",
text="Password", font=("sans", 13), text_color=palette_color("text_dim"))
        self.password_label.grid(row=2, column=0, sticky="w", pady=(0, 2))
        self.password_entry = ctk.CTkEntry(account_frame, height=36, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text="Enter password",
placeholder_text_color=palette_color("placeholder"), font=("sans", 14), show="•")
        self.password_entry.grid(row=3, column=0, sticky="ew", pady=(0, 6))
        ToolTip(self.password_entry, "Strong password requirements:\n- At least 8
characters\n- One uppercase letter\n- One lowercase letter\n- One digit\n- One special
character")
```

```
        self.show_password_button = ctk.CTkButton(account_frame, width=42, height=36,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), command=lambda: self.show_password(1))
        self.show_password_button.grid(row=3, column=1, padx=(6, 0), pady=(0, 6))

        self.password_label_2 = ctk.CTkLabel(account_frame, height=18, anchor="w",
text="Confirm Password", font=("sans", 13), text_color=palette_color("text_dim"))
        self.password_label_2.grid(row=4, column=0, sticky="w", pady=(0, 2))
        self.password_entry_2 = ctk.CTkEntry(account_frame, height=36, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text="Re-enter password",
placeholder_text_color=palette_color("placeholder"), font=("sans", 14), show="•")
        self.password_entry_2.grid(row=5, column=0, sticky="ew")
        self.show_password_button_2 = ctk.CTkButton(account_frame, width=42, height=36,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), command=lambda: self.show_password(2))
        self.show_password_button_2.grid(row=5, column=1, padx=(6, 0))

        # ── CONTACT INFORMATION section ──
        self.section_contact = ctk.CTkLabel(content, height=18, anchor="w", text="CONTACT
INFORMATION", font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_contact.grid(row=3, column=0, sticky="w", pady=(10, 0))
        self.divider_contact = ctk.CTkFrame(content, height=2,
fg_color=palette_color("divider"))
        self.divider_contact.grid(row=4, column=0, sticky="ew", pady=(1, 6))

        contact_frame = ctk.CTkFrame(content, fg_color="transparent")
        contact_frame.grid(row=5, column=0, sticky="ew")
        contact_frame.columnconfigure(0, weight=1)

        self.email_label = ctk.CTkLabel(contact_frame, height=18, anchor="w", text="Email",
font=("sans", 13), text_color=palette_color("text_dim"))
        self.email_label.grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.email_var = ctk.StringVar()
        self.email_var.trace_add("write", self.on_email_entry_changed)
        self.email_entry = ctk.CTkEntry(contact_frame, textvariable=self.email_var,
height=36, corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"),
placeholder_text="Enter email", placeholder_text_color=palette_color("placeholder"),
font=("sans", 14))
        self.email_entry.grid(row=1, column=0, sticky="ew")
        self.email_debounce_timer = None
        self.verify_email_button = ctk.CTkButton(contact_frame, width=80, height=36,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="Verify", text_color=palette_color("text"),
```

```
border_width=2, border_color=palette_color("surface2"), font=("sans", 13),
state="disabled", command=self.call_email_verification)
        self.verify_email_button.grid(row=1, column=1, padx=(6, 0))

        # ── SECURITY QUESTIONS section ──
        self.section_questions = ctk.CTkLabel(content, height=18, anchor="w",
text="SECURITY QUESTIONS", font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_questions.grid(row=6, column=0, sticky="w", pady=(10, 0))
        self.divider_questions = ctk.CTkFrame(content, height=2,
fg_color=palette_color("divider"))
        self.divider_questions.grid(row=7, column=0, sticky="ew", pady=(1, 6))

        self.security_questions_row = self._make_option_row(content, 56, " ? ", "Setup
Security Questions", "Required for account recovery",
self.call_setup_security_questions_window)
        self.security_questions_row.grid(row=8, column=0, sticky="ew")

        # ── ADDITIONAL SECURITY (OPTIONAL) section ──
        self.section_optional = ctk.CTkLabel(content, height=18, anchor="w",
text="ADDITIONAL SECURITY (OPTIONAL)", font=("sans", 11, "bold"),
text_color=palette_color("accent"))
        self.section_optional.grid(row=9, column=0, sticky="w", pady=(10, 0))
        self.divider_optional = ctk.CTkFrame(content, height=2,
fg_color=palette_color("divider"))
        self.divider_optional.grid(row=10, column=0, sticky="ew", pady=(1, 6))

        self.voice_auth_row = self._make_option_row(content, 56, "🎤", "Voice
Authentication", "Authenticate with your voice", self.call_run_voice_recording_window)
        self.voice_auth_row.grid(row=11, column=0, sticky="ew", pady=(0, 6))

        self.face_recognition_row = self._make_option_row(content, 56, "📷", "Face
Recognition", "Authenticate with your face", self.call_run_video_capture_window)
        self.face_recognition_row.grid(row=12, column=0, sticky="ew")

        # ── Create Account button ──
        self.signup_button = ctk.CTkButton(content, height=38, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text="Create
Account", text_color=palette_color("surface"), font=("sans", 14), state="disabled",
command=self.call_signup)
        self.signup_button.grid(row=13, column=0, sticky="ew", pady=(12, 0))

        self.error_text = ctk.CTkLabel(content, anchor="center", text="", font=("sans",
16), text_color=palette_color("error"))
        self.error_text.grid(row=14, column=0, pady=(6, 0))

        self.transient(app.window)
        self.lift()
```

```
    def _make_option_row(self, parent, h, icon, title, desc, command):
        bg = palette_color("surface2")
        border = palette_color("input_border")
        text_col = palette_color("text")
        dim_col = palette_color("text_dim")
        hover_bg = palette_color("input_border")

        frame = ctk.CTkFrame(parent, height=h, corner_radius=10, fg_color=bg,
border_color=border, border_width=2)
        icon_lbl = ctk.CTkLabel(frame, text=icon, font=("Segoe UI Emoji", 18),
text_color=text_col, fg_color="transparent")
        icon_lbl.place(x=24, rely=0.5, anchor="center")
        title_lbl = ctk.CTkLabel(frame, text=title, font=("sans", 14, "bold"),
text_color=text_col, fg_color="transparent")
        title_lbl.place(x=46, rely=0.28, anchor="w")
        desc_lbl = ctk.CTkLabel(frame, height=0, text=desc, font=("sans", 11),
text_color=dim_col, fg_color="transparent")
        desc_lbl.place(x=46, rely=0.66, anchor="w")

        for widget in [frame, icon_lbl, title_lbl, desc_lbl]:
            widget.bind("<Enter>", lambda e, f=frame: f.configure(fg_color=hover_bg))
            widget.bind("<Leave>", lambda e, f=frame: f.configure(fg_color=bg))
            widget.bind("<Button-1>", lambda e, cmd=command: cmd())

        return frame

    def can_signup(self):
        """Enables the signup button."""
        self.signup_button.configure(state="normal")

    def show_password(self, num):
        if num == 1:
            self.password_entry.configure(show="")
            self.show_password_button.configure(text="\uED1A", command=lambda:
self.hide_password(1))
        elif num == 2:
            self.password_entry_2.configure(show="")
            self.show_password_button_2.configure(text="\uED1A", command=lambda:
self.hide_password(2))

    def hide_password(self, num):
        if num == 1:
            self.password_entry.configure(show="•")
            self.show_password_button.configure(text="\uE7B3", command=lambda:
self.show_password(1))
        elif num == 2:
            self.password_entry_2.configure(show="•")
```

```
            self.show_password_button_2.configure(text="\uE7B3", command=lambda:
self.show_password(2))

    def call_signup(self):
        """Validates the form fields and sends a signup request."""
        username = self.username_entry.get().strip()
        password = self.password_entry.get()
        password_2= self.password_entry_2.get()
        email = self.email_entry.get()
        if username == "":
            self.display_signup_error("Username cannot be empty")
        elif password != password_2:
            self.display_signup_error("Passwords do not match")
        else:
            self.app.call_signup(username, password, email)

    def on_email_entry_changed(self, *args):
        """Debounces the email validity check when the email entry changes."""
        self.email_entry.configure(border_color=palette_color("input_border"))
        if self.email_debounce_timer:
            self.after_cancel(self.email_debounce_timer)
        self.email_debounce_timer = self.after(500, self.check_email_validity)

    def check_email_validity(self):
        """Checks the entered email's validity in a background thread."""
        email = self.email_entry.get()
        self.verify_email_button.configure(state="disabled",
border_color=palette_color("surface2"))
        self.email_entry.configure(border_color=palette_color("warning"))
        thread = threading.Thread(target = self.app.call_is_email_valid, args = (email, ))
        thread.start()

    def email_valid(self):
        """Highlights the email field as valid and enables the verify button."""
        self.email_entry.configure(border_color=palette_color("success"))
        self.verify_email_button.configure(state="normal")

    def email_invalid(self):
        """Highlights the email field as invalid."""
        self.email_entry.configure(border_color=palette_color("error"))

    def call_email_verification(self):
        """Sends an email verification code request."""
        email = self.email_entry.get()
        self.app.request_login_code(email)

    def email_verified(self):
        """Marks the email as verified and disables the verify button."""
```

```
        self.verify_email_button.configure(state="disabled",
border_color=palette_color("success"))

    def call_run_voice_recording_window(self):
        """Opens the voice recording window for voice auth setup."""
        self.app.run_voice_recording_window()

    def voice_auth_added(self):
        """Marks the voice auth option row as configured."""
        self.voice_auth_row.configure(border_color=palette_color("accent"))

    def call_run_video_capture_window(self):
        """Opens the video capture window for face recognition setup."""
        self.app.run_video_capture_window()

    def face_recognition_added(self):
        """Marks the face recognition option row as configured."""
        self.face_recognition_row.configure(border_color=palette_color("accent"))

    def call_setup_security_questions_window(self):
        """Opens the security questions setup window."""
        self.app.run_security_questions_setup_window()

    def questions_setup_successful(self):
        """Marks the security questions option row as configured."""
        self.security_questions_row.configure(border_color=palette_color("accent"))

    def display_signup_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

AuthSelectWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 420

class AuthSelectWindow(ctk.CTkToplevel):

    def __init__(self, app, auth_options, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance
```

```
        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, anchor="center",
text="🔐", fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI
Emoji", 24))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title = ctk.CTkLabel(self, anchor="center", text="Choose 2FA Method",
font=("sans", 28, "bold"), text_color=palette_color("text"))
        self.title.place(relx=0.5, y=95, anchor="center")
        self.subtitle = ctk.CTkLabel(self, anchor="center", text="Select a second
authentication factor", font=("sans", 14), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=125, anchor="center")

        row_w = 380
        row_h = 64

        self.options_frame = ctk.CTkFrame(self, fg_color="transparent")
        self.options_frame.place(relx=0.5, y=155, anchor="n")

        self._email_row = self._make_option_row(self.options_frame, row_w, row_h, "📧",
"Email Code", "Receive a verification code via email", True, self.call_email_login)
        self._email_row.pack(pady=(0, 5))
        self._voice_row = self._make_option_row(self.options_frame, row_w, row_h, "🎤",
"Voice Auth", "Authenticate with your voice", auth_options["voice_auth"],
self.call_voice_auth)
        self._voice_row.pack(pady=5)
        self._face_row = self._make_option_row(self.options_frame, row_w, row_h, "📷",
"Face Recognition", "Authenticate with your face", auth_options["face_auth"],
self.call_face_recognition)
        self._face_row.pack(pady=(5, 0))

        self.grab_set()

    def _make_option_row(self, parent, w, h, icon, title, desc, enabled, command):
        border = palette_color("btn_auth") if enabled else palette_color("divider")
        bg = palette_color("surface2") if enabled else palette_color("disabled_bg")
        text_col = palette_color("text") if enabled else palette_color("disabled_text")
        dim_col = palette_color("text_dim") if enabled else palette_color("disabled_text")
        hover_bg = palette_color("input_border") if enabled else
palette_color("disabled_bg")
```

```
        frame = ctk.CTkFrame(parent, width=w, height=h, corner_radius=10, fg_color=bg,
border_color=border, border_width=2)

        icon_lbl = ctk.CTkLabel(frame, text=icon, font=("Segoe UI Emoji", 18),
text_color=text_col, fg_color="transparent")
        icon_lbl.place(x=24, rely=0.5, anchor="center")
        title_lbl = ctk.CTkLabel(frame, text=title, font=("sans", 14, "bold"),
text_color=text_col, fg_color="transparent")
        title_lbl.place(x=46, rely=0.32, anchor="w")
        desc_lbl = ctk.CTkLabel(frame, text=desc, font=("sans", 11), text_color=dim_col,
fg_color="transparent")
        desc_lbl.place(x=46, rely=0.68, anchor="w")

        if enabled:
            for widget in [frame, icon_lbl, title_lbl, desc_lbl]:
                widget.bind("<Enter>", lambda e, f=frame: f.configure(fg_color=hover_bg))
                widget.bind("<Leave>", lambda e, f=frame: f.configure(fg_color=bg))
                widget.bind("<Button-1>", lambda e, cmd=command: cmd())
        else:
            nc_lbl = ctk.CTkLabel(frame, text="Not configured", font=("sans", 11),
text_color=dim_col, fg_color="transparent")
            nc_lbl.place(x=w - 16, rely=0.5, anchor="e")

        return frame

    def call_email_login(self):
        """Closes the window and initiates email code login."""
        email = self.app.get_email()
        self.close_window()
        self.app.request_login_code(email)

    def call_voice_auth(self):
        """Closes the window and opens the voice recording window."""
        self.close_window()
        self.app.run_voice_recording_window()

    def call_face_recognition(self):
        """Closes the window and opens the video capture window."""
        self.close_window()
        self.app.run_video_capture_window()

    def close_window(self):
        """Releases the grab and destroys the window."""
        self.grab_release()
        self.withdraw()
        self.destroy()
```

ChangePasswordWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 420

class ChangePasswordWindow(ctk.CTkToplevel):

    def __init__(self, app, site_url, site_username, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance
        self.site_url = site_url    # URL of the site to change the password for
        self.site_username = site_username    # Username associated with the site

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🔑",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 20))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Change Site Password",
font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=95, anchor="center")

        self.section_label = ctk.CTkLabel(self, anchor="w", text="SITE DETAILS",
font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_label.place(x=30, y=130)
        self.divider = ctk.CTkFrame(self, width=380, height=2,
fg_color=palette_color("divider"))
        self.divider.place(x=30, y=150)

        self.site_url_label = ctk.CTkLabel(self, anchor="w", text="Site URL", font=("sans",
13), text_color=palette_color("text_dim"))
        self.site_url_label.place(x=30, y=160)
        self.site_url_value = ctk.CTkLabel(self, anchor="w", text=self.site_url,
font=("sans", 15), text_color=palette_color("text"))
        self.site_url_value.place(x=30, y=183)

        self.site_username_label = ctk.CTkLabel(self, anchor="w", text="Username",
font=("sans", 13), text_color=palette_color("text_dim"))
        self.site_username_label.place(x=30, y=218)
```

```
        self.site_username_value = ctk.CTkLabel(self, anchor="w", text=self.site_username,
font=("sans", 15), text_color=palette_color("text"))
        self.site_username_value.place(x=30, y=241)

        self.copy_password_checkbox = ctk.CTkCheckBox(self, width=20, height=20,
corner_radius=4, border_color=palette_color("input_border"),
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"),
checkmark_color=palette_color("surface"), text="Copy new password to clipboard",
font=("sans", 13), text_color=palette_color("text"))
        self.copy_password_checkbox.select()
        self.copy_password_checkbox.place(x=30, y=285)

        btn_w = 182
        self.change_preferences_button = ctk.CTkButton(self, width=btn_w, height=42,
corner_radius=10, fg_color=palette_color("btn_secondary"),
hover_color=palette_color("input_border"), text="Preferences...",
text_color=palette_color("surface"), font=("sans", 14),
command=self.call_password_preferences_window)
        self.change_preferences_button.place(x=30, y=330)

        self.change_password_button = ctk.CTkButton(self, width=btn_w, height=42,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Change Password",
text_color=palette_color("surface"), font=("sans", 14), command=self.change_site_password)
        self.change_password_button.place(x=228, y=330)

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, y=390, anchor="center")

        self.grab_set()


    def call_password_preferences_window(self):
        """Opens the password preferences window for this site."""
        self.app.get_preferences_and_initiate_window(self.site_url)


    def change_site_password(self):
        """Requests a new password for this site via the app."""
        self.app.call_change_site_password(self.site_url)

    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window and returns whether the user wants the password copied."""
        copy_password = self.copy_password_checkbox.get()
```

```
        self.destroy()
        return copy_password
```

## CodeLoginWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 420
CODE_LENGTH = 8

class CodeLoginWindow(ctk.CTkToplevel):

    def __init__(self, app, email, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance
        self.client_email = email    # Email address the code was sent to

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="📧",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 22))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title = ctk.CTkLabel(self, anchor="center", text="Enter Verification Code",
font=("sans", 24, "bold"), text_color=palette_color("text"))
        self.title.place(relx=0.5, y=95, anchor="center")
        self.subtitle = ctk.CTkLabel(self, anchor="center", text=f"A code was sent to
{email}", font=("sans", 13), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=122, anchor="center")

        self.entries_frame = ctk.CTkFrame(self, fg_color="transparent")
        self.entries_frame.place(relx=0.5, y=170, anchor="n")
        self.entries = []
        for i in range(CODE_LENGTH):
            entry = ctk.CTkEntry(self.entries_frame, width=40, height=50, corner_radius=8,
justify="center", font=("sans", 20, "bold"), fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"))
            entry.grid(row=0, column=i, padx=4, pady=10)
            entry.bind("<KeyPress>", lambda event, index=i: self._handle_keypress(event,
index))
```

```
            self.entries.append(entry)

        self.enter_code_button = ctk.CTkButton(self, width=380, height=44,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Verify Code",
text_color=palette_color("surface"), font=("sans", 16), command=self.call_check_code)
        self.enter_code_button.place(relx=0.5, y=285, anchor="center")

        self.resend_code_button = ctk.CTkButton(self, width=120, height=28, text="Resend
code", text_color=palette_color("text_dim"), fg_color=palette_color("surface"),
hover_color=palette_color("surface"), font=("sans", 13), state="disabled",
command=self.call_resend_code)
        self.resend_code_button.bind("<Enter>", lambda e:
self.resend_code_button.configure(text_color=palette_color("accent")))
        self.resend_code_button.bind("<Leave>", lambda e:
self.resend_code_button.configure(text_color=palette_color("text_dim")))
        self.resend_code_button.place(relx=0.5, y=325, anchor="center")

        self.msg_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 14),
text_color=palette_color("error"))
        self.msg_text.place(relx=0.5, y=365, anchor="center")

        self.grab_set()

    def _handle_keypress(self, event, index):
        current_entry = self.entries[index]
        content = current_entry.get()
        char = event.char
        key = event.keysym

        if key == "BackSpace":
            if content != "":
                current_entry.delete(0, "end")
            else:
                self.after(1, lambda : self._focus_back(index))

        if char.isdigit():
            self.after(1, lambda : self._focus_next(index))

        else:
            current_entry.delete(0, "end")

    def _focus_next(self, index):
        current_entry = self.entries[index]
        content = current_entry.get()
        if len(content) > 1:
            current_entry.delete(0, "end")
            current_entry.insert(0, content[-1])
```

```
        if index < CODE_LENGTH - 1:
            self.entries[index + 1].focus()

    def _focus_back(self, index):
        if index > 0:
            self.entries[index - 1].focus()

    def get_full_code(self):
        """Returns the full code string by concatenating all entry fields."""
        return "".join(e.get() for e in self.entries)

    def call_check_code(self):
        """Sends the entered code to the app for verification."""
        code = self.get_full_code()
        self.app.call_check_code(code)

    def incorrect_code_feedback(self):
        """Displays an incorrect code error and highlights the entry fields."""
        self.msg_text.configure(text="Code Incorrect")
        for entry in self.entries:
            entry.configure(border_color=palette_color("error"), border_width=3)

    def display_code_expired_msg(self):
        """Displays a code expired error, clears the entries, and enables the resend
button."""
        self.display_error("Code Expired! Request another code.")
        for entry in self.entries:
            entry.delete(0, "end")
        self.resend_code_button.configure(state="normal")

    def display_error(self, msg):
        """Displays an error message."""
        self.msg_text.configure(text=msg, text_color=palette_color("error"))

    def display_msg(self, msg):
        """Displays an informational message."""
        self.msg_text.configure(text=msg, text_color=palette_color("text"))

    def call_resend_code(self):
        """Requests a new verification code via the app."""
        self.app.request_login_code(self.client_email)

    def code_resent(self):
        """Displays a confirmation message and disables the resend button."""
        self.display_msg("Code Sent")
        self.resend_code_button.configure(state="disabled")
```

```
    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

## HomeFrame

```
import customtkinter as ctk
from color_palettes import palette_color


class HomeFrame(ctk.CTkFrame):

    def __init__(self, master, *args, **kwargs):
        super().__init__(master, fg_color="transparent", *args, **kwargs)

        # Large icon circle
        home_icon = ctk.CTkFrame(self, width=90, height=90, corner_radius=45,
fg_color=palette_color("accent"))
        home_icon.place(relx=0.5, y=130, anchor="center")
        home_icon_lbl = ctk.CTkLabel(home_icon, width=0, height=0, text="🔓",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 32))
        home_icon_lbl.place(relx=0.5, rely=0.5, anchor="center")

        home_title = ctk.CTkLabel(self, text="Secure Password Manager", font=("sans", 32,
"bold"), text_color=palette_color("text"))
        home_title.place(relx=0.5, y=200, anchor="center")

        home_subtitle = ctk.CTkLabel(self, text="Keep all your credentials safe with multi-
factor authentication", font=("sans", 14), text_color=palette_color("text_dim"))
        home_subtitle.place(relx=0.5, y=240, anchor="center")

        # Feature cards
        features = [
            ("🔐", "Multi-Factor Auth"),
            ("🎤", "Voice Recognition"),
            ("📷", "Face ID"),
            ("❓", "Security Questions"),
        ]
        card_w, card_h = 170, 80

        cards_frame = ctk.CTkFrame(self, fg_color="transparent")
        cards_frame.place(relx=0.5, y=290, anchor="n")

        for icon, label in features:
            card = ctk.CTkFrame(cards_frame, width=card_w, height=card_h, corner_radius=12,
fg_color=palette_color("surface"), border_width=1,
border_color=palette_color("input_border"))
            card.pack(side="left", padx=10)
```

```
            card_icon = ctk.CTkLabel(card, width=0, height=0, text=icon,
fg_color="transparent", font=("Segoe UI Emoji", 20))
            card_icon.place(relx=0.5, rely=0.35, anchor="center")
            card_lbl = ctk.CTkLabel(card, width=0, height=0, text=label, font=("sans", 12),
text_color=palette_color("text_dim"))
            card_lbl.place(relx=0.5, rely=0.72, anchor="center")
```

## LoginWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 400
WINDOW_HEIGHT = 460

class LoginWindow(ctk.CTkToplevel):

    def __init__(self, app, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(x=200, y=50, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, anchor="center",
text="🔓", fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI
Emoji", 24))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")
        self.title = ctk.CTkLabel(self, anchor="center", text="Login", font=("sans", 44,
"bold"), text_color=palette_color("text"))
        self.title.place(x=200, y=110, anchor="center")

        self.username_text = ctk.CTkLabel(self, anchor="w", text="Username", font=("sans",
16), text_color=palette_color("text_dim"))
        self.username_text.place(x=30, y=170)

        self.username_entry = ctk.CTkEntry(self, width=340, height=38, corner_radius=10,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text_color=palette_color("placeholder"),
font=("sans", 16), placeholder_text="Enter your username")
        self.username_entry.place(x=30, y=196)
```

```
        password_text = ctk.CTkLabel(self, anchor="w", text="Password", font=("sans", 16),
text_color=palette_color("text_dim"))
        password_text.place(x=30, y=248)

        self.password_entry = ctk.CTkEntry(self, width=292, height=38, corner_radius=10,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text_color=palette_color("placeholder"),
font=("sans", 16), show="•", placeholder_text="Enter your password")
        self.password_entry.place(x=30, y=274)

        self.show_password_button = ctk.CTkButton(self, width=42, height=38,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), border_color=palette_color("input_border"),
command=self.show_password)
        self.show_password_button.place(x=338, y=274)

        login_button = ctk.CTkButton(self, width=340, height=44, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text="Login",
text_color=palette_color("surface"), border_color=palette_color("accent"),
command=self.call_login)
        login_button.place(x=30, y=330)

        reset_password_button = ctk.CTkButton(self, width=220, height=28, text="I forgot my
password", text_color=palette_color("accent"), fg_color=palette_color("surface"),
hover_color=palette_color("surface"), command=self.call_security_questions_window)
        reset_password_button.bind("<Enter>", lambda e:
reset_password_button.configure(text_color=palette_color("accent_hover")))
        reset_password_button.bind("<Leave>", lambda e:
reset_password_button.configure(text_color=palette_color("accent")))
        reset_password_button.place(x=200, y=390, anchor="center")

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 16),
text_color=palette_color("error"))
        self.error_text.place(x=200, y=420, anchor="center")

        self.grab_set()


    def call_login(self):
        """Reads the username and password entries and sends a login request."""
        username = self.username_entry.get().strip()
        password = self.password_entry.get()
        self.app.call_login(username, password)

    def show_password(self):
        self.password_entry.configure(show="")
        self.show_password_button.configure(text="\uED1A", command=self.hide_password)
```

```
    def hide_password(self):
        self.password_entry.configure(show="•")
        self.show_password_button.configure(text="\uE7B3", command=self.show_password)

    def call_security_questions_window(self):
        """Requests the user's security questions for password recovery."""
        username = self.username_entry.get()
        self.app.call_get_user_security_questions(username)

    def display_login_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text= msg)


    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

## PasswordPreferencesWindow

```
import customtkinter as ctk
from color_palettes import palette_color
from password_handler import DEFAULT_SETTINGS, MINIMUM_PASSWORD_LENGTH,
MAXIMUM_PASSWORD_LENGTH

WINDOW_WIDTH = 520
WINDOW_HEIGHT = 540

COL_LEFT = 30
COL_RIGHT = 270
CB_Y_START = 262
CB_Y_STEP = 32

class PasswordPreferencesWindow(ctk.CTkToplevel):

    def __init__(self, app, settings=None, site_url=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        if settings is None:
            settings = DEFAULT_SETTINGS
```

```
        self.site_url = site_url    # URL of the site these preferences apply to, or None
during registration

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🔑",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 20))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Password Preferences",
font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=95, anchor="center")

        self.section_label = ctk.CTkLabel(self, anchor="w", text="GENERATION RULES",
font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_label.place(x=COL_LEFT, y=130)
        self.divider = ctk.CTkFrame(self, width=460, height=2,
fg_color=palette_color("divider"))
        self.divider.place(x=COL_LEFT, y=150)

        self.maximum_length_text = ctk.CTkLabel(self, anchor="w", text="Maximum Length",
font=("sans", 13), text_color=palette_color("text_dim"))
        self.maximum_length_text.place(x=COL_LEFT, y=160)
        self.maximum_length_entry = ctk.CTkEntry(self, width=120, height=38,
corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"), font=("sans",
14))
        self.maximum_length_entry.insert(0, settings.get("max_length"))
        self.maximum_length_entry.place(x=COL_LEFT, y=183)

        self.includes_header = ctk.CTkLabel(self, anchor="w", text="PASSWORD INCLUDES",
font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.includes_header.place(x=COL_LEFT, y=232)
        self.start_header = ctk.CTkLabel(self, anchor="w", text="CAN START WITH",
font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.start_header.place(x=COL_RIGHT, y=232)

        self.include_vars = {}
        for i, (label, value) in enumerate(settings["include"].items()):
            var = ctk.BooleanVar(value=value)
            self.include_vars[label] = var
            cb = ctk.CTkCheckBox(self, width=20, height=20, corner_radius=4,
border_color=palette_color("input_border"), fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), checkmark_color=palette_color("surface"),
text=label, font=("sans", 13), text_color=palette_color("text"), variable=var,
command=lambda l=label: self.check_inclusion_selection(l))
            cb.place(x=COL_LEFT, y=CB_Y_START + (i * CB_Y_STEP))
```

```
        self.special_entry_y = CB_Y_START + (len(settings["include"]) * CB_Y_STEP)
        self.special_allowed_entry = ctk.CTkEntry(self, width=210, height=34,
corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"), font=("sans",
13))
        self.special_allowed_entry.insert(0, settings.get("special_allowed"))
        if self.include_vars["Special"].get():
            self.special_allowed_entry.place(x=COL_LEFT, y=self.special_entry_y)

        self.start_vars = {}
        self.start_cb = {}
        for i, (label, value) in enumerate(settings["start"].items()):
            var = ctk.BooleanVar(value=value)
            self.start_vars[label] = var
            cb = ctk.CTkCheckBox(self, width=20, height=20, corner_radius=4,
border_color=palette_color("input_border"), fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), checkmark_color=palette_color("surface"),
text=label, font=("sans", 13), text_color=palette_color("text"), variable=var,
command=lambda l=label: self.check_start_selection(l))
            if not self.include_vars[label].get():
                cb.configure(state="disabled")
            cb.place(x=COL_RIGHT, y=CB_Y_START + (i * CB_Y_STEP))
            self.start_cb[label] = cb

        self.special_allowed_start_entry = ctk.CTkEntry(self, width=210, height=34,
corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"), font=("sans",
13))
        self.special_allowed_start_entry.insert(0, settings.get("special_allowed_start"))
        if self.start_vars["Special"].get():
            self.special_allowed_start_entry.place(x=COL_RIGHT, y=self.special_entry_y)

        self.save_preferences_button = ctk.CTkButton(self, width=200, height=42,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Save Preferences",
text_color=palette_color("surface"), font=("sans", 14), command=self.save_preferences)
        self.save_preferences_button.place(relx=0.5, y=460, anchor="center")

        self.error_label = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_label.place(relx=0.5, y=510, anchor="center")

        self.grab_set()


    def check_inclusion_selection(self, label):
        """Enforces at least one include checkbox and syncs the start checkbox state."""
        count_active = sum(var.get() for var in self.include_vars.values())
```

```
        if count_active < 1:
            self.include_vars[label].set(True)
        elif self.include_vars[label].get() == False:
            self.start_vars[label].set(False)
            self.start_cb[label].configure(state="disabled")
            if label == "Special":
                self.special_allowed_entry.place_forget()
                self.special_allowed_start_entry.place_forget()
        else:
            self.start_cb[label].configure(state="normal")
            if label == "Special":
                self.special_allowed_entry.place(x=COL_LEFT, y=self.special_entry_y)

    def check_start_selection(self, label):
        """Enforces at least one start checkbox and toggles the special characters
entry."""
        count_active = sum(var.get() for var in self.start_vars.values())
        if count_active < 1:
            self.start_vars[label].set(True)
        elif label == "Special":
            if self.start_vars[label].get():
                self.special_allowed_start_entry.place(x=COL_RIGHT, y=self.special_entry_y)
            else:
                self.special_allowed_start_entry.place_forget()

    def save_preferences(self):
        """Validates the preferences and sends them to the app."""
        preferences = {}
        try:
            length = int(self.maximum_length_entry.get())
            if length < MINIMUM_PASSWORD_LENGTH or length > MAXIMUM_PASSWORD_LENGTH:
                raise ValueError
        except ValueError:
            self.display_error(f"Maximum length should be a number equal to or greater than
{MINIMUM_PASSWORD_LENGTH}\nAnd lower than {MAXIMUM_PASSWORD_LENGTH}")
            return

        preferences["max_length"] = length
        preferences["include"] = dict((label, var.get()) for label, var in
self.include_vars.items())
        preferences["special_allowed"] =
self._remove_duplicates(self.special_allowed_entry.get())
        preferences["start"] = dict((label, var.get()) for label, var in
self.start_vars.items())

        if sum(var for var in preferences["start"].values()) < 1:
            self.display_error("At least one 'start with' checkbox must be selected")
            return
```

```
        preferences["special_allowed_start"] =
self._remove_duplicates(self.special_allowed_start_entry.get())
        self.app.save_password_preferences(preferences, self.site_url)

    def _remove_duplicates(self, str):
        """Receives a string and returns it with duplicate characters removed."""
        filtered_set = set(str)
        filtered_string = ""
        for i in filtered_set:
            filtered_string += i
        return filtered_string

    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_label.configure(text=msg)

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

## ResetPasswordWindow

```
import customtkinter as ctk
from color_palettes import palette_color
from tooltip_window import ToolTip

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 410

class ResetPasswordWindow(ctk.CTkToplevel):

    def __init__(self, app, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("btn_auth"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
```

```
        self.icon_label = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🔑",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 20))
        self.icon_label.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Reset Password",
font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=95, anchor="center")

        self.subtitle = ctk.CTkLabel(self, anchor="center", text="Enter a new password for
your account", font=("sans", 13), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=125, anchor="center")

        self.password_label = ctk.CTkLabel(self, anchor="w", text="New Password",
font=("sans", 13), text_color=palette_color("text_dim"))
        self.password_label.place(x=30, y=160)

        self.password_entry = ctk.CTkEntry(self, width=336, height=38, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text_color=palette_color("placeholder"),
font=("sans", 14), show="•", placeholder_text="Enter new password")
        self.password_entry.place(x=30, y=183)
        ToolTip(self.password_entry, "Strong password requirements:\n- At least 8
characters\n- One uppercase letter\n- One lowercase letter\n- One digit\n- One special
character")


        self.show_password_button = ctk.CTkButton(self, width=42, height=38,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), command=lambda : self.show_password(1))
        self.show_password_button.place(x=372, y=183)

        self.password_label_2 = ctk.CTkLabel(self, anchor="w", text="Confirm Password",
font=("sans", 13), text_color=palette_color("text_dim"))
        self.password_label_2.place(x=30, y=228)

        self.password_entry_2 = ctk.CTkEntry(self, width=336, height=38, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
placeholder_text="Re-enter Password", text_color=palette_color("text"), font=("sans", 14),
show="•")
        self.password_entry_2.place(x=30, y=251)

        self.show_password_button_2 = ctk.CTkButton(self, width=42, height=38,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), command=lambda : self.show_password(2))
        self.show_password_button_2.place(x=372, y=251)
```

```
        self.reset_button = ctk.CTkButton(self, width=380, height=42, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text="Reset
Password", text_color=palette_color("surface"), font=("sans", 14),
command=self.call_reset_password)
        self.reset_button.place(x=30, y=310)

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, y=370, anchor="center")

        self.grab_set()


    def call_reset_password(self):
        """Validates that both passwords match and sends the reset request."""
        password = self.password_entry.get()
        password2 = self.password_entry_2.get()
        if password != password2:
            self.display_error("Passwords do not match")
        else:
            self.app.call_reset_password(password)

    def show_password(self, num):
        if num == 1:
            self.password_entry.configure(show="")
            self.show_password_button.configure(text="\uED1A", command=lambda:
self.hide_password(1))
        elif num == 2:
            self.password_entry_2.configure(show="")
            self.show_password_button_2.configure(text="\uED1A", command=lambda:
self.hide_password(2))

    def hide_password(self, num):
        if num == 1:
            self.password_entry.configure(show="•")
            self.show_password_button.configure(text="\uE7B3", command=lambda:
self.show_password(1))
        elif num == 2:
            self.password_entry_2.configure(show="•")
            self.show_password_button_2.configure(text="\uE7B3", command=lambda:
self.show_password(2))

    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

## SequirityQuestionsAuthWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 480
WINDOW_HEIGHT = 520


class SecurityQuestionsAuthWindow(ctk.CTkToplevel):

    def __init__(self, app, user_security_questions, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("btn_auth"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🛡",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 20))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Security
Verification", font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=95, anchor="center")

        self.subtitle = ctk.CTkLabel(self, anchor="center", text="Answer your security
questions to continue", font=("sans", 13), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=125, anchor="center")

        self.user_security_questions = user_security_questions    # List of the user's
security questions
        self.answers_entries = []    # List of entry widgets for each answer

        self.questions_frame = ctk.CTkFrame(self, fg_color="transparent")
        self.questions_frame.place(x=30, y=150, relwidth=0.87)
        self.questions_frame.columnconfigure(1, weight=1)

        for i in range(len(user_security_questions)):
            question = self.user_security_questions[i]
            row = i * 2
            num_circle = ctk.CTkFrame(self.questions_frame, width=26, height=26,
corner_radius=13, fg_color=palette_color("accent"))
            num_circle.pack_propagate(False)
```

```
            num_circle.grid(row=row, column=0, sticky="w", padx=(0, 8))
            num_label = ctk.CTkLabel(num_circle, width=0, height=0, text=str(i + 1),
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 12, "bold"))
            num_label.place(relx=0.5, rely=0.5, anchor="center")
            q_label = ctk.CTkLabel(self.questions_frame, anchor="w", text=question,
font=("sans", 13), text_color=palette_color("text"))
            q_label.grid(row=row, column=1, sticky="w")
            entry = ctk.CTkEntry(self.questions_frame, height=38, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text_color=palette_color("placeholder"),
font=("sans", 14), placeholder_text="Type your answer...")
            self.answers_entries.append(entry)
            entry.grid(row=row + 1, column=0, columnspan=2, sticky="ew", pady=(4, 14))

        btn_row = len(user_security_questions) * 2
        self.enter_answers_button = ctk.CTkButton(self.questions_frame, height=42,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Submit Answers",
text_color=palette_color("surface"), font=("sans", 14), command=self.check_answers)
        self.enter_answers_button.grid(row=btn_row, column=0, columnspan=2, sticky="ew",
pady=(10, 0))

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, rely=1.0, y=-20, anchor="center")

        self.grab_set()


    def check_answers(self):
        """Collects all answers and sends them to the app for verification."""
        answers = []
        for entry in self.answers_entries:
            answer = entry.get()
            if answer == "":
                self.display_error("Answer all of the questions before submitting them")
                return
            else:
                answers.append(answer.lower().strip())
        self.app.call_check_security_answers(answers)


    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

SequrityQuestionsSetupWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 520
WINDOW_HEIGHT = 580


class SecurityQuestionsSetupWindow(ctk.CTkToplevel):

    def __init__(self, app, security_questions, answers_dict = None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        # ── Icon circle ──
        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=38, anchor="center")
        self.icon_label = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="❓",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 22))
        self.icon_label.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Setup Security
Questions", font=("sans", 22, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=85, anchor="center")

        self.subtitle = ctk.CTkLabel(self, anchor="center", text="Choose 3 questions and
provide your answers", font=("sans", 13), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=112, anchor="center")

        # ── Scrollable questions frame ──
        self.questions_frame = ctk.CTkScrollableFrame(self, width=WINDOW_WIDTH - 60,
height=300, fg_color=palette_color("surface"), border_color=palette_color("input_border"),
border_width=1, scrollbar_button_color=palette_color("accent"),
scrollbar_button_hover_color=palette_color("accent_hover"))
        self.questions_frame.place(relx=0.5, y=135, anchor="n")
        self.questions_frame.columnconfigure(0, weight=0)
        self.questions_frame.columnconfigure(1, weight=1)

        self.security_questions = security_questions    # List of available security
questions
        self.check_vars = []    # List of BooleanVar for each question checkbox
        self.questions_entries = []    # List of entry widgets for each question's answer
```

```
        for i in range(len(security_questions)):
            question = security_questions[i]
            var = ctk.BooleanVar(value=False)
            cb = ctk.CTkCheckBox(self.questions_frame, height=28, corner_radius=4,
border_color=palette_color("input_border"), checkmark_color=palette_color("surface"),
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text=question,
font=("sans", 13), text_color=palette_color("text"), variable=var, command=lambda i=i:
self.on_checkbox(i))
            cb.grid(column=0, row=i * 2, columnspan=2, sticky="w", pady=(4, 0))
            self.check_vars.append(var)
            entry = ctk.CTkEntry(self.questions_frame, height=34, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text="Type your answer...",
placeholder_text_color=palette_color("placeholder"), font=("sans", 13))
            self.questions_entries.append(entry)
            if answers_dict is not None:
                if i in answers_dict.keys():
                    var.set(True)
                    self.on_checkbox(i)
                    entry.insert(0, answers_dict[i])

        # ── Setup button ──
        self.setup_questions_button = ctk.CTkButton(self, width=WINDOW_WIDTH - 60,
height=38, corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Save Security Questions",
text_color=palette_color("surface"), font=("sans", 14), state="disabled",
command=self.setup_security_questions)
        self.setup_questions_button.place(relx=0.5, y=495, anchor="center")

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 14),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, y=540, anchor="center")

        self.grab_set()


    def on_checkbox(self, i):
        """Shows or hides the answer entry for question i. Limits selection to 3
questions."""
        ticked_checkboxes_count = sum(var.get() for var in self.check_vars)
        entry = self.questions_entries[i]
        if self.check_vars[i].get():
            if ticked_checkboxes_count > 3:
                self.check_vars[i].set(False)
            else:
                entry.grid(column=0, row=i * 2 + 1, columnspan=2, sticky="ew", padx=(26,
0), pady=(2, 6))
                if ticked_checkboxes_count == 3:
```

```
                    self.setup_questions_button.configure(state="normal")
        else:
            entry.grid_forget()
            self.setup_questions_button.configure(state="disabled")

    def setup_security_questions(self):
        """Collects the selected questions and answers, then sends them to the app."""
        chosen_questions_dict = {}
        for i in range(len(self.check_vars)):
            if self.check_vars[i].get():
                user_answer = self.questions_entries[i].get()
                if user_answer == "":
                    self.display_error("You must answer the questions you have picked")
                    return
                else:
                    # sqlite autoincrement starts at 1
                    chosen_questions_dict[i + 1] = user_answer.lower().strip()
        self.app.call_setup_security_questions(chosen_questions_dict)


    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

## SettingsMenuFrame

```
import customtkinter as ctk
from color_palettes import palette_color


class SettingsMenuFrame(ctk.CTkFrame):

    def __init__(self, app, *args, **kwargs):
        super().__init__(master= app.window, fg_color=palette_color("surface2"), *args,
**kwargs)

        self.pack_propagate(False)

        self.app = app    # Reference to the UI instance
        self.is_open = False    # Whether the settings panel is currently visible

        self.close_button = ctk.CTkButton(self, text="✕", width=30,
text_color=palette_color("text_dim"), fg_color="transparent",
hover_color=palette_color("surface2"), font=("sans", 14), command=self.toggle)
```

```
        self.close_button.place(x=5, y=5)

        self.title = ctk.CTkLabel(self, text="Settings", font=("sans", 22, "bold"),
text_color=palette_color("text"))
        self.title.pack(pady=20)

        self.title_divider = ctk.CTkFrame(self, height=2,
fg_color=palette_color("input_border"))
        self.title_divider.pack(fill="x", padx=16, pady=(0, 10))

        self.theme_switch = ctk.CTkSwitch(self, text="Dark Mode", font=("sans", 13),
text_color=palette_color("text"), fg_color=palette_color("input_border"),
progress_color=palette_color("accent"), button_color=palette_color("text_dim"),
button_hover_color=palette_color("text"), command=self.change_theme)
        self.theme_switch.pack(pady=10, padx=20, anchor="w")


    def toggle(self):
        """Slides the settings panel in or out."""
        if not self.is_open:
            self.animate_in()
        else:
            self.animate_out()
        self.is_open = not self.is_open

    def animate_in(self):
        current_relx = float(self.place_info().get('relx', 1.0))
        if current_relx > 0.8:
            self.place(relx=current_relx - 0.02)
            self.after(10, self.animate_in)
        else:
            self.lift()

    def animate_out(self):
        current_relx = float(self.place_info().get('relx', 0.8))
        if current_relx < 1.0:
            self.place(relx=current_relx + 0.02)
            self.after(10, self.animate_out)

    def change_theme(self):
        """Toggles between dark and light mode based on the switch state."""
        mode = "dark" if self.theme_switch.get() else "light"
        ctk.set_appearance_mode(mode)

    def show_auth_options(self, auth_options):
        """Displays buttons for adding unconfigured auth methods."""
        self.auth_divider = ctk.CTkFrame(self, height=2,
fg_color=palette_color("input_border"))
        self.auth_divider.pack(fill="x", padx=16, pady=(10, 10))
```

```
        if not auth_options["voice_auth"]:
            self.add_voice_auth_button = ctk.CTkButton(self, height=36, corner_radius=10,
fg_color=palette_color("btn_auth"), hover_color=palette_color("btn_auth_hover"), text="Add
Voice Auth", text_color=palette_color("surface"), font=("sans", 13),
command=self.setup_voice_auth)
            self.add_voice_auth_button.pack(pady=8, padx=20, fill="x")

        if not auth_options["face_auth"]:
            self.add_face_recognition_button = ctk.CTkButton(self, height=36,
corner_radius=10, fg_color=palette_color("btn_auth"),
hover_color=palette_color("btn_auth_hover"), text="Add Face Recognition",
text_color=palette_color("surface"), font=("sans", 13),
command=self.setup_face_recognition)
            self.add_face_recognition_button.pack(pady=8, padx=20, fill="x")

    def setup_voice_auth(self):
        """Opens the voice recording window for voice auth setup."""
        self.app.run_voice_recording_window()

    def voice_auth_added(self):
        """Disables the add voice auth button after setup."""
        self.add_voice_auth_button.configure(state= "disabled")

    def setup_face_recognition(self):
        """Opens the video capture window for face recognition setup."""
        self.app.run_video_capture_window()

    def face_recognition_added(self):
        """Disables the add face recognition button after setup."""
        self.add_face_recognition_button.configure(state= "disabled")
```

SiteInfoWindow

```
import customtkinter as ctk
import tkinter as tk
from color_palettes import palette_color
from tooltip_window import ToolTip

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 340

class SiteInfoWindow(ctk.CTkToplevel):

    def __init__(self, app, site_username, site_password, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance
        self.site_username = site_username    # Username for the site
```

```
        self.site_password = site_password    # Password for the site

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🌐",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 20))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title = ctk.CTkLabel(self, anchor="center", text="Site Info", font=("sans",
26, "bold"), text_color=palette_color("text"))
        self.title.place(relx=0.5, y=92, anchor="center")

        self.username_label = ctk.CTkLabel(self, anchor="w", text="USERNAME", font=("sans",
11, "bold"), text_color=palette_color("text_dim"))
        self.username_label.place(x=30, y=125)
        self.username_field = ctk.CTkEntry(self, width=380, height=38, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), font=("sans", 15), textvariable=tk.StringVar(self,
self.site_username), state="disabled")
        self.username_field.place(x=30, y=145)

        self.password_label = ctk.CTkLabel(self, anchor="w", text="PASSWORD", font=("sans",
11, "bold"), text_color=palette_color("text_dim"))
        self.password_label.place(x=30, y=195)
        self.site_password_entry = ctk.CTkEntry(self, width=290, height=38,
corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"), font=("sans",
15), textvariable=tk.StringVar(self, self.site_password), show="•", state="disabled")
        self.site_password_entry.place(x=30, y=215)

        self.site_password_show_button = ctk.CTkButton(self, width=42, height=38,
corner_radius=8, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="\uE7B3", font=("Segoe MDL2 Assets", 16),
text_color=palette_color("text"), command=self.show_password)
        self.site_password_show_button.place(x=330, y=215)

        self.copy_password_button = ctk.CTkButton(self, width=42, height=38,
corner_radius=8, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="📋", font=("Segoe UI Emoji", 14),
text_color=palette_color("surface"), command=self.call_copy_password)
        self.copy_password_button.place(x=378, y=215)
        ToolTip(self.copy_password_button, "Copy Password")
```

```
        self.password_copied_text = ctk.CTkLabel(self, anchor="center", text="",
font=("sans", 13), text_color=palette_color("success"))
        self.password_copied_text.place(relx=0.5, y=275, anchor="center")

        self.grab_set()

    def close_window(self):
        """Destroys the window."""
        self.destroy()


    def call_copy_password(self):
        """Copies the site password to the clipboard and shows a confirmation."""
        self.app.copy_to_clipboard(self.site_password)
        self.password_copied_text.configure(text="Password copied to clipboard")

    def show_password(self):
        self.site_password_entry.configure(show="")
        self.site_password_show_button.configure(text="\uED1A", command=self.hide_password)

    def hide_password(self):
        self.site_password_entry.configure(show="•")
        self.site_password_show_button.configure(text="\uE7B3", command=self.show_password)
```

SiteRecoveryWindow

```
import customtkinter as ctk
from CTkListbox import *
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 480

class SiteRecoveryWindow(ctk.CTkToplevel):

    def __init__(self, app, deleted_sites, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
```

```
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🔄",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 20))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Recover Deleted
Sites", font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=95, anchor="center")

        self.subtitle = ctk.CTkLabel(self, anchor="center", text="Select a site to
restore", font=("sans", 13), text_color=palette_color("text_dim"))
        self.subtitle.place(relx=0.5, y=120, anchor="center")

        self.deleted_sites_list = CTkListbox(self, height=220, width=360,
fg_color=palette_color("surface2"), border_width=2, border_color=palette_color("accent"),
highlight_color=palette_color("selected_row"), hover_color=palette_color("surface2"),
text_color=palette_color("text"), button_color=palette_color("surface2"), font=("sans",
14), command=self.on_site_selected)
        if deleted_sites is not None:
            for site_url in deleted_sites:
                btn = self.deleted_sites_list.insert("end", site_url)
                btn.configure(corner_radius=10, border_width=2,
border_color=palette_color("input_border"), height=42)
        self.deleted_sites_list.place(x=30, y=145)

        self.recover_button = ctk.CTkButton(self, width=200, height=42, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text="Recover
Site", text_color=palette_color("surface"), font=("sans", 14), state="disabled",
command=self.recover_site)
        self.recover_button.place(relx=0.5, y=410, anchor="center")

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, y=450, anchor="center")

        self.lift()
        self.focus_force()
        self.grab_set()


    def on_site_selected(self, selected_value):
        for btn in self.deleted_sites_list.buttons.values():
            btn.configure(border_color=palette_color("input_border"))
        if self.deleted_sites_list.selected:
            self.deleted_sites_list.selected.configure(border_color=palette_color("accent")
)
        self.recover_button.configure(state="normal")

    def recover_site(self):
```

```
        """Sends a recovery request for the selected site."""
        site_url = self.deleted_sites_list.get()
        self.app.call_recover_site(site_url)

    def display_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Releases the grab and destroys the window."""
        self.grab_release()
        self.destroy()
```

## SiteRegistrationWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 460

class SiteRegistrationWindow(ctk.CTkToplevel):

    def __init__(self, app, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.resizable(False, False)
        self.wm_title("Password Manager")
        self.configure(fg_color=palette_color("surface"))

        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=45, anchor="center")
        self.icon = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="+",
fg_color="transparent", text_color=palette_color("surface"), font=("sans", 28, "bold"))
        self.icon.place(relx=0.5, rely=0.5, anchor="center")

        self.title = ctk.CTkLabel(self, anchor="center", text="Register New Site",
font=("sans", 26, "bold"), text_color=palette_color("text"))
        self.title.place(relx=0.5, y=95, anchor="center")

        self.section_label = ctk.CTkLabel(self, anchor="w", text="SITE DETAILS",
font=("sans", 11, "bold"), text_color=palette_color("accent"))
        self.section_label.place(x=30, y=130)
        self.divider = ctk.CTkFrame(self, width=380, height=2,
fg_color=palette_color("divider"))
```

```
        self.divider.place(x=30, y=150)

        self.site_url_text = ctk.CTkLabel(self, anchor="w", text="Site URL", font=("sans",
13), text_color=palette_color("text_dim"))
        self.site_url_text.place(x=30, y=160)
        self.site_url_entry = ctk.CTkEntry(self, width=380, height=38, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
text_color=palette_color("text"), placeholder_text_color=palette_color("placeholder"),
font=("sans", 14), placeholder_text="e.g. github.com")
        self.site_url_entry.place(x=30, y=183)

        self.site_username_text = ctk.CTkLabel(self, anchor="w", text="Username for this
site", font=("sans", 13), text_color=palette_color("text_dim"))
        self.site_username_text.place(x=30, y=230)
        self.site_username_entry = ctk.CTkEntry(self, width=380, height=38,
corner_radius=8, fg_color=palette_color("input_bg"),
border_color=palette_color("input_border"), text_color=palette_color("text"),
placeholder_text_color=palette_color("placeholder"), font=("sans", 14),
placeholder_text="Enter username")
        self.site_username_entry.place(x=30, y=253)

        self.copy_password_checkbox = ctk.CTkCheckBox(self, width=20, height=20,
corner_radius=4, border_color=palette_color("input_border"),
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"),
checkmark_color=palette_color("surface"), text="Copy password after registration",
font=("sans", 13), text_color=palette_color("text"))
        self.copy_password_checkbox.select()
        self.copy_password_checkbox.place(x=30, y=305)

        btn_w = 182
        self.change_preferences_button = ctk.CTkButton(self, width=btn_w, height=42,
corner_radius=10, fg_color=palette_color("btn_secondary"),
hover_color=palette_color("input_border"), text="Preferences...",
text_color=palette_color("surface"), font=("sans", 14),
command=self.call_password_preferences_window)
        self.change_preferences_button.place(x=30, y=350)

        self.register_button = ctk.CTkButton(self, width=btn_w, height=42,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Register Site",
text_color=palette_color("surface"), font=("sans", 14), command=self.call_register_site)
        self.register_button.place(x=228, y=350)

        self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("error"))
        self.error_text.place(relx=0.5, y=410, anchor="center")

        self.preferences = None    # Password generation preferences, or None for defaults
```

```
        self.grab_set()


    def call_password_preferences_window(self):
        """Opens the password preferences window with current preferences."""
        self.app.run_password_preferences_window(self.preferences)


    def call_register_site(self):
        """Validates the entries and sends a site registration request."""
        site_url = self.site_url_entry.get()
        if site_url == "":
            self.display_registration_error("You must enter a site url before
registration")
            return
        site_username = self.site_username_entry.get()
        if site_username == "":
            self.display_registration_error("You must enter a username before
registration")
            return
        self.app.call_register_site(site_url, site_username, self.preferences)

    def display_registration_error(self, msg):
        """Displays an error message in the window."""
        self.error_text.configure(text=msg)

    def close_window(self):
        """Destroys the window and returns whether the user wants the password copied."""
        copy_password = self.copy_password_checkbox.get()
        self.destroy()
        return copy_password
```

ToolTip

```
import customtkinter as ctk
from color_palettes import palette_color

class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
        self.tip_window = None
        self.after_id = None

        widget.bind("<Enter>", self.show_tooltip)
        widget.bind("<Leave>", self.hide_tooltip)
```

```
    def show_tooltip(self, event = None):
        x = event.x_root
        y = event.y_root + 10
        self.after_id = self.widget.after(500, lambda: self._create_tooltip(x, y))

    def _create_tooltip(self, x, y):
        if self.tip_window or not self.text:
            return


        self.tip_window = ctk.CTkToplevel(self.widget)
        self.tip_window.wm_overrideredirect(True)
        self.tip_window.geometry(f"+{x}+{y}")

        label = ctk.CTkLabel(self.tip_window, text=self.text, justify="left",
fg_color=palette_color("surface"), text_color=palette_color("text"), corner_radius=10,
padx=8, pady=4)
        label.pack()

    def hide_tooltip(self, *args):
        if self.after_id:
            self.widget.after_cancel(self.after_id)
            self.after_id = None
        if self.tip_window:
            self.tip_window.destroy()
            self.tip_window = None
```

## VideoCaptureWindow

```
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 520
WINDOW_HEIGHT = 620

class VideoCaptureWindow(ctk.CTkToplevel):

    def __init__(self, app, cameras_list, is_setup, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        self.protocol("WM_DELETE_WINDOW", self.close_window)
```

```
        # — Icon circle —
        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=38, anchor="center")
        self.icon_label = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="📷",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 20))
        self.icon_label.place(relx=0.5, rely=0.45, anchor="center")

        title_text = "Setup Face Recognition" if is_setup else "Face Recognition"
        self.title_label = ctk.CTkLabel(self, anchor="center", text=title_text,
font=("sans", 22, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=85, anchor="center")

        # — Camera dropdown —
        self.cameras_list = cameras_list
        self.camera_dropdown = ctk.CTkComboBox(self, width=240, height=34, corner_radius=8,
fg_color=palette_color("input_bg"), border_color=palette_color("input_border"),
button_color=palette_color("accent"), button_hover_color=palette_color("accent_hover"),
dropdown_fg_color=palette_color("surface"),
dropdown_hover_color=palette_color("selected_row"), text_color=palette_color("text"),
dropdown_text_color=palette_color("text"), font=("sans", 13), dropdown_font=("sans", 13),
values=self.cameras_list, command=self.select_camera)
        self.camera_dropdown.set("Select a camera...")
        self.camera_dropdown.place(relx=0.5, y=118, anchor="center")

        # — Video feed —
        self.video_label = ctk.CTkLabel(self, text="", fg_color="transparent", width=460,
height=380)
        self.video_label.place(relx=0.5, y=145, anchor="n")

        self.is_loop_running = False

        if is_setup:
            self.setup_face_recognition_button = ctk.CTkButton(self, width=460, height=38,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Setup Face Recognition",
text_color=palette_color("surface"), font=("sans", 14), state="disabled",
command=self.setup_face_recognition)
            self.setup_face_recognition_button.place(relx=0.5, y=565, anchor="center")
        else:
            self.error_text = ctk.CTkLabel(self, anchor="center", text="", font=("sans",
14), text_color=palette_color("error"))
            self.error_text.place(relx=0.5, y=565, anchor="center")

        self.grab_set()


    def select_camera(self, choice):
        """Starts video recording from the selected camera."""
```

```
        index = self.cameras_list.index(choice)
        self.app.start_video_recording(index)

    def record_loop(self):
        """Continuously captures frames from the camera and updates the video label. Stops
if the camera is not available."""
        if self.app.cap is None or not self.app.cap.isOpened():
            self.is_loop_running = False
            return

        self.is_loop_running = True
        self.app.get_frames()
        self.after(10, self.record_loop)

    def update_frame(self, img):
        """Updates the video label with a new frame image."""
        self.video_label.configure(text= "", image= img)

    def face_recognition_loop(self):
        """Periodically checks for face detection every 500ms while recording."""
        if self.is_loop_running:
            self.app.call_face_recognition()
            self.after(500, self.face_recognition_loop)
        else:
            return

    def enable_setup_button(self):
        """Enables the setup/auth button when a face is detected."""
        if hasattr(self, "setup_face_recognition_button"):
            self.setup_face_recognition_button.configure(state= "normal")

    def disable_setup_button(self):
        """Disables the setup/auth button when no face is detected."""
        if hasattr(self, "setup_face_recognition_button"):
            self.setup_face_recognition_button.configure(state= "disabled")

    def setup_face_recognition(self):
        """Sends the captured face encoding to the server for setup."""
        self.app.call_setup_face_recognition()


    def display_camera_error(self):
        """Displays a camera not found error in the video label."""
        self.video_label.configure(text="Camera not found", image=None,
text_color=palette_color("error"))

    def display_face_recognition_error(self, msg):
        """Displays a face recognition error message."""
```

```
        if hasattr(self, "error_text"):
            self.error_text.configure(text= msg)

    def close_window(self):
        """Stops the recording loop, releases the camera, and destroys the window."""
        self.is_loop_running = False
        if self.app.cap:
            self.app.cap.release()
        self.destroy()
```

## VoiceRecordingWindow

```
import time
import customtkinter as ctk
from color_palettes import palette_color

WINDOW_WIDTH = 440
WINDOW_HEIGHT = 420

MIN_RECORDING_TIME = 5

class VoiceRecordingWindow(ctk.CTkToplevel):

    def __init__(self, app, is_setup, max_recording_time, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.app = app    # Reference to the UI instance

        self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.wm_title("Password Manager")
        self.resizable(False, False)
        self.configure(fg_color=palette_color("surface"))

        # ── Icon circle ──
        self.icon_circle = ctk.CTkFrame(self, width=56, height=56, corner_radius=28,
fg_color=palette_color("accent"))
        self.icon_circle.place(relx=0.5, y=38, anchor="center")
        self.icon_label = ctk.CTkLabel(self.icon_circle, width=0, height=0, text="🎤",
fg_color="transparent", text_color=palette_color("surface"), font=("Segoe UI Emoji", 22))
        self.icon_label.place(relx=0.5, rely=0.5, anchor="center")

        self.title_label = ctk.CTkLabel(self, anchor="center", text="Voice Authentication",
font=("sans", 22, "bold"), text_color=palette_color("text"))
        self.title_label.place(relx=0.5, y=100, anchor="center")

        # ── Progress bar ──
        self.max_recording_time = max_recording_time
        self.progress_bar_value = ctk.DoubleVar(value=0.0)
        slider_width = 380
```

```
        slider_frame = ctk.CTkFrame(self, width=slider_width, height=50,
fg_color="transparent")
        slider_frame.place(relx=0.5, y=165, anchor="center")

        self.progress_bar = ctk.CTkSlider(slider_frame, width=slider_width, height=20,
from_=0, to=self.max_recording_time, variable=self.progress_bar_value, state="disabled",
button_length=0, fg_color=palette_color("input_border"),
progress_color=palette_color("accent"), button_color=palette_color("accent"),
button_hover_color=palette_color("accent"))
        self.progress_bar.place(relx=0.5, y=0, anchor="n")

        # ── Min recording time marker ──
        marker_relx = MIN_RECORDING_TIME / self.max_recording_time
        self.min_marker = ctk.CTkFrame(slider_frame, width=2, height=10,
fg_color=palette_color("text_dim"), corner_radius=0)
        self.min_marker.place(relx=marker_relx, y=20, anchor="n")
        self.min_label = ctk.CTkLabel(slider_frame, width=0, height=0, text="Minimum",
font=("sans", 10), text_color=palette_color("text_dim"))
        self.min_label.place(relx=marker_relx, rely=1.0, anchor="s")

        self.text_label = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 13),
text_color=palette_color("text_dim"))
        self.text_label.place(relx=0.5, y=215, anchor="center")

        # ── Recording buttons ──
        btn_w = 182
        self.start_recording_button = ctk.CTkButton(self, width=btn_w, height=38,
corner_radius=10, fg_color=palette_color("accent"),
hover_color=palette_color("accent_hover"), text="Start Recording",
text_color=palette_color("surface"), font=("sans", 14), command=self.call_voice_recording)
        self.start_recording_button.place(x=30, y=245)

        self.stop_recording_button = ctk.CTkButton(self, width=btn_w, height=38,
corner_radius=10, fg_color=palette_color("surface2"),
hover_color=palette_color("input_border"), text="Stop Recording",
text_color=palette_color("text"), font=("sans", 14), state="disabled",
command=self.call_stop_recording)
        self.stop_recording_button.place(x=WINDOW_WIDTH - 30 - btn_w, y=245)

        # ── Auth button ──
        if is_setup:
            self.auth_button = ctk.CTkButton(self, width=380, height=38, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"), text="Setup
Voice Authentication", text_color=palette_color("surface"), font=("sans", 14),
state="disabled", command=self.setup_voice_auth)
        else:
            self.auth_button = ctk.CTkButton(self, width=380, height=38, corner_radius=10,
fg_color=palette_color("accent"), hover_color=palette_color("accent_hover"),
```

```
text="Authenticate Voice", text_color=palette_color("surface"), font=("sans", 14),
state="disabled", command=self.call_voice_auth)
        self.auth_button.place(relx=0.5, y=315, anchor="center")

        self.error_label = ctk.CTkLabel(self, anchor="center", text="", font=("sans", 14),
text_color=palette_color("error"))
        self.error_label.place(relx=0.5, y=365, anchor="center")

        self.is_recording = False    # Whether voice recording is currently active
        self.grab_set()

    def call_voice_recording(self):
        """Starts voice recording and begins the progress bar loop."""
        self.app.call_start_recording()
        self.is_recording = True
        self._start_time = time.time()
        self.progress_bar_value = 0
        self.progress_bar.set(self.progress_bar_value)
        self.after(10, self.recording_progress_loop)
        self.auth_button.configure(state= "disabled")
        self.start_recording_button.configure(state= "disabled")
        self.stop_recording_button.configure(state= "disabled")
        self.text_label.configure(text= "Recording...")
        self.error_label.configure(text= "")

    def recording_progress_loop(self):
        """Updates the progress bar while recording until max time is reached."""
        if self.is_recording and self.progress_bar_value < self.max_recording_time:
            self.progress_bar_value = min(time.time() - self._start_time,
self.max_recording_time)
            self.progress_bar.set(self.progress_bar_value)
            if self.progress_bar_value >= MIN_RECORDING_TIME:
                self.stop_recording_button.configure(state= "enabled")
            self.after(10, self.recording_progress_loop)

    def call_stop_recording(self):
        """Stops voice recording via the app."""
        self.app.call_stop_recording()

    def recording_stopped(self):
        """Updates the UI to show that recording has stopped."""
        self.is_recording = False
        self.text_label.configure(text= "Done Recording")
        self.auth_button.configure(state=  "normal")
        self.start_recording_button.configure(state= "enabled")
        self.stop_recording_button.configure(state= "disabled")

    def setup_voice_auth(self):
```

```
        """Sends the recorded voice to the server for setup."""
        self.app.call_setup_voice_auth()

    def call_voice_auth(self):
        """Sends the recorded voice to the server for authentication."""
        self.app.call_voice_auth()

    def display_error(self, error_msg):
        """Displays an error message and disables the auth button."""
        self.error_label.configure(text= error_msg)
        self.auth_button.configure(state= "disabled")

    def close_window(self):
        """Destroys the window."""
        self.destroy()
```

client_status

```
from enum import Enum

class Status(Enum):
    NORMAL = 1
    SIGNING_IN = 2
    LOGGING_IN = 3
    REGISTERING_SITE = 4
    RECOVERING_PASSWORD = 5
```

color_palletes

```
DARK_PALETTE = {
    "bg": "#141414",
    "surface": "#1c1c1c",
    "surface2": "#2a2a2a",
    "accent": "#2ecc71",
    "accent_hover": "#25a35c",
    "btn_secondary": "#494747",
    "btn_auth": "#fb923c",
    "btn_auth_hover" : "#ea7b21",
    "error": "#ef4444",
    "success": "#2ecc71",
    "warning": "#fdcb6e",
    "text": "#e8e8e8",
    "text_dim": "#8a8a8a",
    "placeholder": "#555555",
    "input_bg": "#1e1e1e",
    "input_border": "#3a3a3a",
    "divider": "#2a2a2a",
    "selected_row": "#1e2e1e",
```

```
    "disabled_bg": "#222222",
    "disabled_text": "#555555",
}

LIGHT_PALETTE = {
    "bg": "#f8f8f6",
    "surface": "#ffffff",
    "surface2": "#ececec",
    "accent": "#27ae60",
    "accent_hover": "#1f8c4d",
    "btn_secondary": "#2a2a2a",
    "btn_auth": "#c0792a",
    "btn_auth_hover" : "#b86c16",
    "error": "#ef4444",
    "success": "#27ae60",
    "warning": "#fdcb6e",
    "text": "#1a1a1a",
    "text_dim": "#6b6b68",
    "placeholder": "#999999",
    "input_bg": "#ffffff",
    "input_border": "#d0d0d0",
    "divider": "#e4e4e0",
    "selected_row": "#e8f5e8",
    "disabled_bg": "#dddddd",
    "disabled_text": "#999999",
}


def palette_color(key):
    return (LIGHT_PALETTE[key], DARK_PALETTE[key])
```